



cultivation

井上卓哉(@yokozuna_57)

問題概要

- 縦 R 行、横 C 列のマス目が与えられる。
 - N マスにJOI草が自生している。
 - JOI草は風によって生息圏を広げる。
 - （風向きにひとマス拡張される。）
 - JOI草の生息圏をマス目全体に広げたい。
-
- 最小で何年で可能か。

例

- 入出力例 1
- __W__W
- ____W__
- _____

例

- 西向きの風
- WWWWW
- __WW__
- _____

例

- 南向きの風
- WWWW
- WWWW
- __WW__

例

- 南向きの風
- WWWW
- WWWW
- WWWW
- よって3年

制限

- $1 \leq N \leq 300$
- $1 \leq R \leq 1000000000 (= 10^9)$
- $1 \leq C \leq 1000000000 (= 10^9)$
- $1 \leq S_i \leq R (1 \leq i \leq N)$
- $1 \leq E_i \leq C (1 \leq i \leq N)$
- 最初に自生していないマスがある。
- $(S_i, E_i) \neq (S_j, E_j) (1 \leq i < j \leq N)$

小課題 1

- $R \leq 4$
- $C \leq 4$
- マス目ちっちゃい
- $\Rightarrow$ シミュレーションできそう
- できます

小課題 1

- なぜか？
- JOI草が自生しているひとマスに注目すると、そこから広がる生息圏は長方形になる。

小課題 1

○ 例

○ _____

○ _____ W _____

○ _____

○ _____

○ _____

小課題 1

○ 例

○ _____

○ _____W_____

○ _____W_____

○ _____

○ _____

小課題 1

○ 例

○ _____

○ _____WW_____

○ _____WW_____

○ _____

○ _____

小課題 1

○ 例

○ _____

○ _____WW_____

○ _____WW_____

○ _____WW_____

○ _____

小課題 1

- どのマスからでも $R+C-2$ 年で全体に広げられる。
- $2^{(R+C-2)}$ 通り試せば良い。
- 5点。

小課題 2

- $R \leq 40$
- $C \leq 40$
- ちょっと大きくなった。

小課題 2

- 風を吹かせる順番は関係ない。
- 東、東、東、西、西、南も
- 東、東、西、南、東、西も同じ
- 東西南北の回数を決めて調べる。
- (R^2C^2 通り)

小課題 2

- どうやって調べるか？
- 長方形の範囲に1足して、すべてのマスがせいかなどうかを調べれば良い。
- 累積和
- これで $O(R^3C^3)$

小課題 2

- これだとTLE。
- 最大でも $R+C-2$ であることを用いる。
- 3方向決めたあとに残り一方向の回数を調べるのは $O(RC)$ のでできるので $O(R^3C^2)$ でもできる。
- 15点。

小課題 3

- $R \leq 40$
- 行数が少ないので各行について考える。

小課題 3

- 南北方向の風を決めてしまえば残りの操作は各行で独立に考えられる。
- 各行について、その行いっぱいには生息圏を広げるための条件は
- 東の風が a 回以上、西の風が b 回以上、合わせて c 回以上
- という形で表される。

小課題 3

- なぜか？

- _____W_____W_____W_____W_____

小課題 3

- 南北方向の風の決め方が $O(R^2)$ 。
- 各行について $O(RN)$ で調べられる。
- 各行についての結果を合わせるのは $O(R)$ 。
- 全体で $O(R^3N)$ 。
- 30点。

小課題 4

- $N \leq 25$
- 各列ごとに考えてみる

小課題 4

- 東向きの風の回数、西向きの風の回数の組み合わせの候補を考える。
- 列の状態の集合が一致するものは省く。
- （列の状態の集合が同じなら東向きの風と西向きの風の合計が最小になるものを候補とすればよい。）

小課題 4

- 東向きの風が吹くのと西向きの風が吹くのとではあまり状態が変わらない。
- 全部東向きの風だと思って畑を平行移動させれば良い。

小課題 4

- 西向きの風をひとつ減らして東向きの風をひとつ増やすと何が起こるのか。
- 畑全体がひとつ西に平行移動したのと同じ。
- 一番東の列が消えて西に一列増える。
- 列の状態の集合の変化を考えると、西に増える列の状態が既出の間この操作をしても損しない。

小課題 4

- 西向きの風は $E_1-1, E_2-1, \dots, E_n-1$ の n 通りに絞られる。
- 西向きの風を決めると、東向きの風は $O(n^2)$ 通りに絞られる。
- (理由：列の状態は $O(n^2)$)
- 以上より東向きの風の回数、西向きの風の回数の組み合わせの候補は $O(n^3)$

小課題 4

- 東西方向の風が決まった後
- 列を西から順番に見ていくときれつの状態が変化するのとは $O(N)$ 回。
- それぞれの状態について $O(N\log N)$ あれば調べられる。
- 全体で $O(N^5\log N)$ 。
- 60点。

小課題 5

- $N \leq 100$

小課題 4（再掲）

- 東向きの風が吹くのと西向きの風が吹くのとではあまり状態が変わらない。
- 全部東向きの風だと思って畑を平行移動させれば良い。

小課題 5

- 東西方向の風の回数が確定すれば、各列について、南北の風に対するJOI草の生息圏がその列いっぱい広がるための条件が
- 「南向きの風が a 回以上、北向きの風が b 回以上、合わせて c 回以上」
- の形式で求まる。

小課題 5

- 「南向きの風が a 回以上、北向きの風が b 回以上、合わせて c 回以上」の a, b, c は東西方向の風が吹く回数の関数として表される。

小課題 5

- 西向きの風が吹かない場合を考えると、
- それぞれのJ〇I草が存在する列の一つ西側の列と一番東の列が「不利」
- 西向きの風が吹く場合も畑全体を平行移動させる事を考えれば、「不利」になる列の位置は一番東の列以外変わらない。

小課題 5

- 「不利」とは何か。
- 「不利」でない列は一列東の列よりも一年早くJOI草が生える。
- 全ての「不利」な列いっぱい JOI草の生息圏が広がれば良い。

小課題 5

- 東西方向の風のうち、西向きの風は何回にするのか。
- 西向きの風の回数を増やすほど、畑は東に平行移動していき、最も東の列のJOI草が生えるのは遅くなる。
- 不利な列が減る時の $O(N)$ 種類だけ考えれば良い。

小課題 5

- 不利な列は西向きの風の回数それぞれについて $O(N)$ 個。
- それぞれの列について、 a, b, c （の関数）はそれぞれ $O(N^2)$ でできる。
- N 個の階段状のグラフの段の数はそれぞれ $O(N)$ 個なので、各列での結果を合わせるのは $O(N^2 \log N)$ 。

小課題 5

- 全体で $O(N^4)$ 。
- 80点。

小課題 6

- 追加の制限はない。

小課題 5 （再掲）

- 西向きの風が吹かない場合を考えると、
- それぞれのJ〇I草が存在する列の一つ西側の列と一番東の列が「不利」
- 西向きの風が吹く場合も畑全体を平行移動させる事を考えれば、「不利」になる列の位置は一番東の列以外変わらない。

小課題 5（再掲）

- 西向きの風が吹かない場合を考えると、
- それぞれのJ〇I草が存在する列の一つ西側の列と一番東の列が「不利」
- 西向きの風が吹く場合も畑全体を平行移動させる事を考えれば、「不利」になる列の位置は一番東の列以外変わらない。

小課題 6

- 不利な列となりうる列は $O(N)$ 個。
- それぞれの列について、 a, b, c （の関数）はそれぞれ $O(N^2)$ で求まる。
- a, b, c の計算は全体で $O(N^3)$ 。

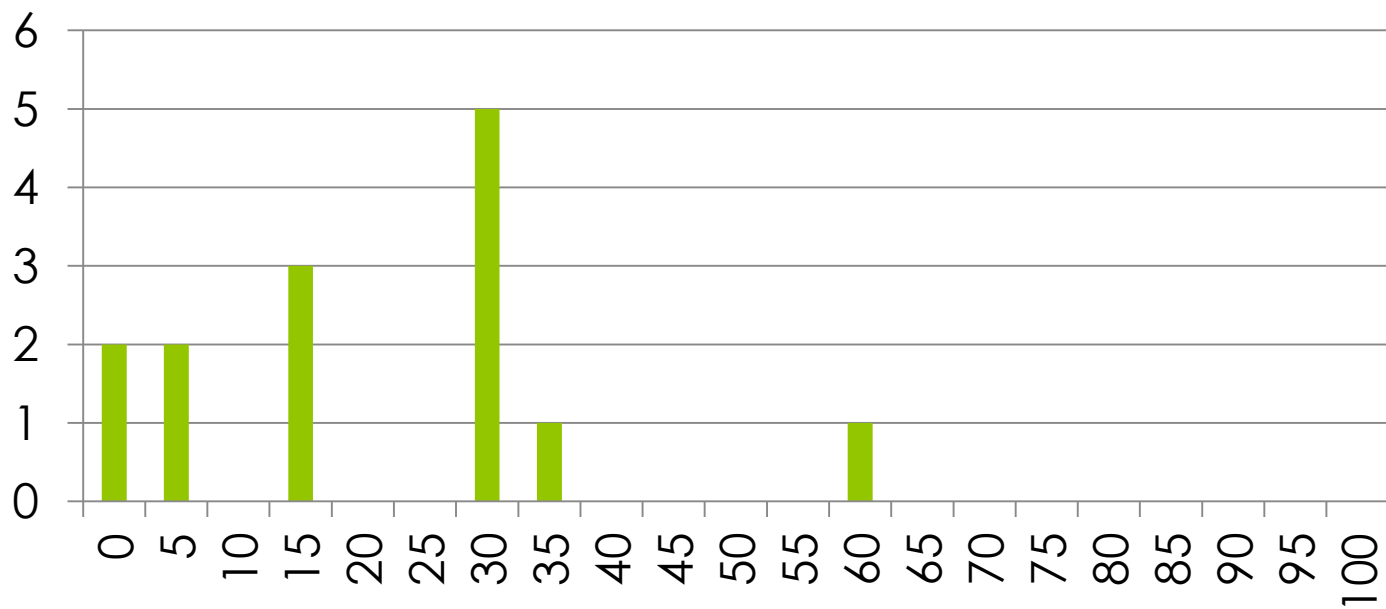
小課題 6

- 各列での結果を合わせるステップ
- $O(N^2 \log N)$ の $\log N$ は階段状のグラフの段の位置をソートしている。
- 前計算すれば前計算が $O(N^2 \log N)$ 、それぞれの西向きの風の回数に対して $O(N^2)$ で処理できる。

小課題 6

- これで全体で $O(N^3)$ 。
- 100点。

得点分布



Port Facility 解説

DEGwer

問題概要

- 荷物がN個ある
- スタックが2個ある
- i 個目の荷物は、時刻 A_i にどちらかのスタックの一番上に積まれ、時刻 B_i にどちらかのスタックの一番上から取り出されるようにしたい
- そのような積み方は何通りあるか?
- $N \leq 10^6$

小課題1

- $N \leq 20$
- $O(2^N N)$ で全探索してください
- やるだけ

ちょっと考察

- 時系列を数直線とみなし、各荷物を区間と考える
- 区間 $[A_i, B_i]$ と $[A_j, B_j]$ を同じスタックに積める条件は？

ちょっと考察

- 区間 $[A_i, B_i]$ と $[A_j, B_j]$ を同じスタックに積める条件は?
- ①二つの区間が重ならない場合



ちょっと考察

- 区間 $[A_i, B_i]$ と $[A_j, B_j]$ を同じスタックに積める条件は?

- ①二つの区間が重ならない場合

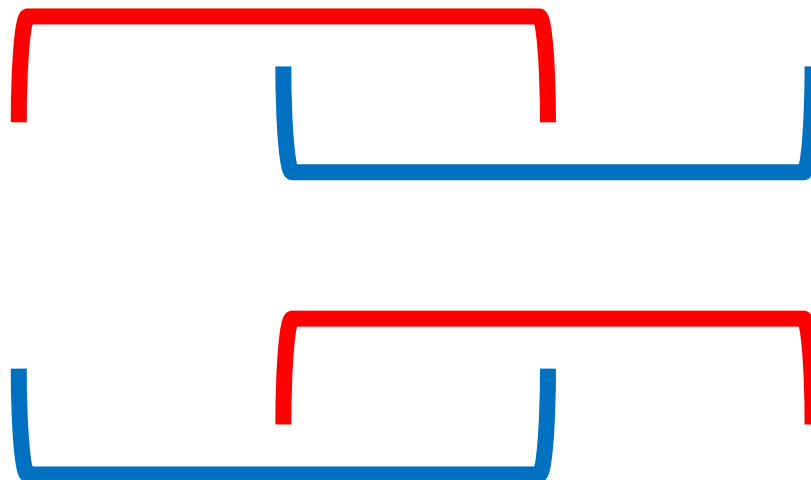


- ②片方が片方に完全に含まれる場合



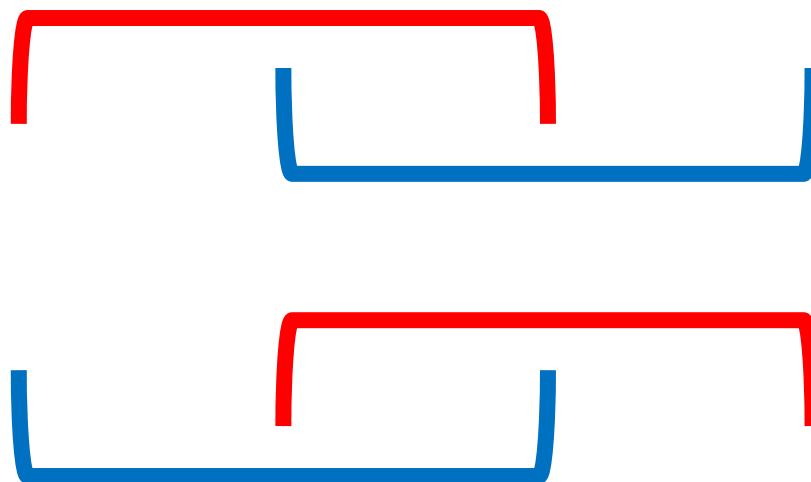
ちょっと考察

- 逆に、どちらもでもない場合は同じスタックに積めない



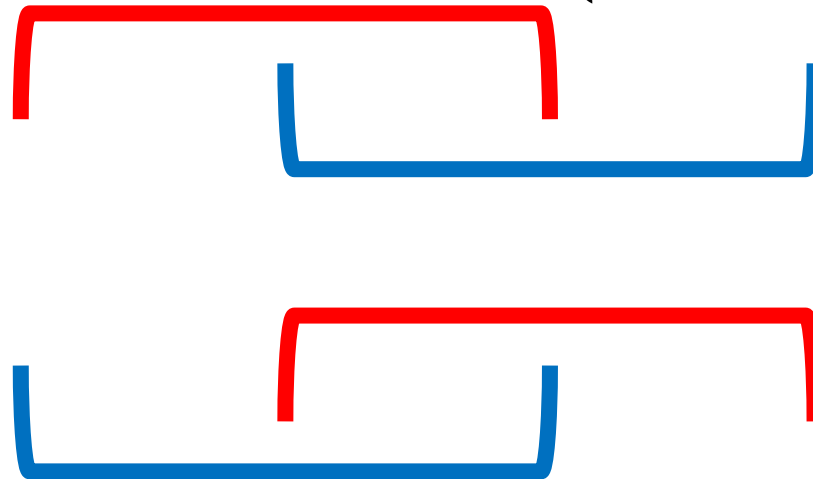
ちょっと考察

- 二つのスタックをA,Bとして、スタックAに積む荷物とスタックBに積む荷物に分けたとき、同じスタックに積む荷物が、こういう位置関係にあってはならない



ちょっと考察

- 荷物を頂点とし、こういう位置関係の二つの区間の間すべてに辺を張ったグラフの二部グラフ判定をすればよい
 - 二部グラフでないなら、答えは0
 - 二部グラフなら、答えは $2^{\text{(連結成分数)}}$



小課題2

- $N \leq 2000$
- 愚直に辺を張って二部グラフ判定をしましょう
- $O(N^2)$

小課題3,4

- $N \leq 10^5, 10^6$
- ここからは小課題2の高速化です
- いろいろやり方がある気がしますが、小課題3は $O(N \log^2 N)$ 、小課題4は $O(N \log N)$ です
- 小課題4の実装をさぼったりすると小課題3になったりします
- 満点になりうる解法をいくつか紹介しますが、それ以外にもあるかもしれません

満点解法①

- 二部グラフ判定をする幅/深さ優先探索を考える
- 頂点数 $O(N)$, 辺数 $O(N^2)$ で、全部の辺を見ると人生終了
- とりあえずまず、グラフを連結成分に分解することを考える
 - 二部グラフ判定はあとで

満点解法①

- 辺を全部見るのは無駄
 - 既に訪れた頂点に行く辺はそもそも見なくてよい
- 既に見た頂点に行く辺は、見る前に削除されているようにしたい
- とはいえ、いちいち辺を一個ずつ削除すると、それも $O(N^2)$
- 辺をまとめて消したりしたい

満点解法①

- 各区間に「隣接する」区間はどのような区間か?
- 今見ている区間を $[L,R]$ として、
 - L より前で始まって $[L,R]$ 間で終わる区間
 - $[L,R]$ 間で始まって R より後で終わる区間
- 対称なので、片方だけ考えることにする

満点解法①

- Lより前で始まって[L,R]間で終わる区間
- これをどうやって探すか？

満点解法①

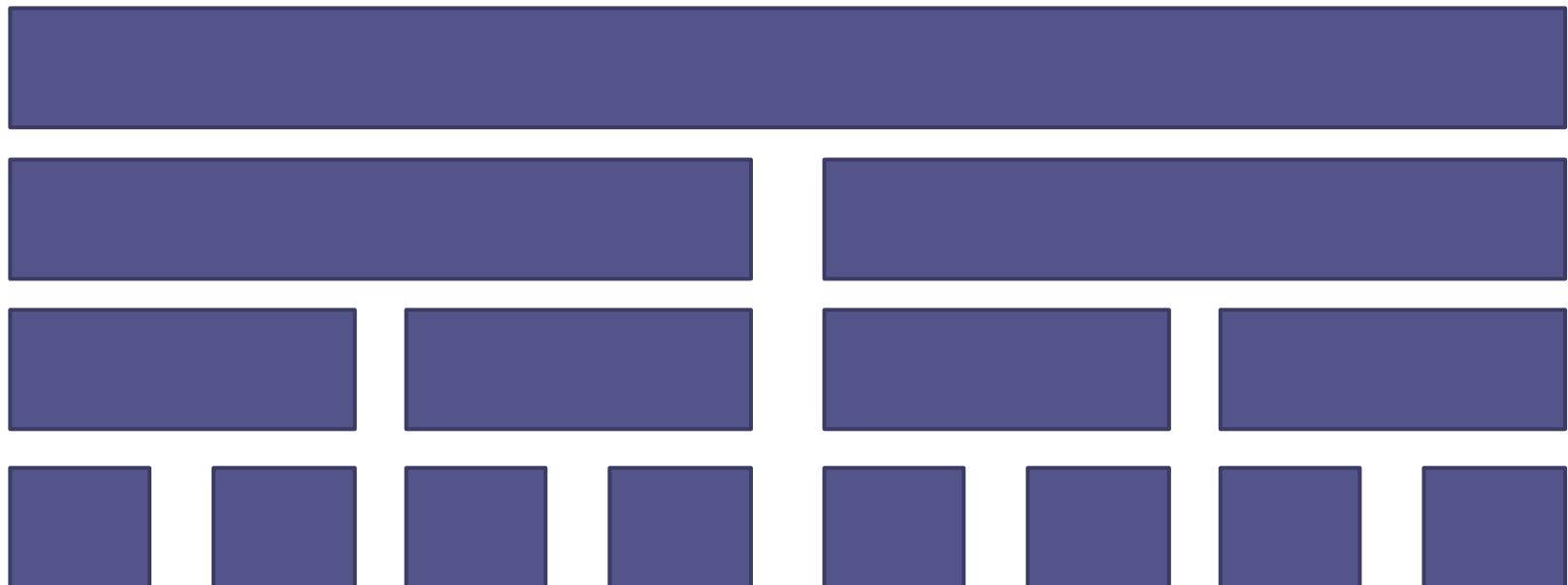
- Lより前で始まって[L,R]間で終わる区間
- これをどうやって探すか?
- segment tree

Segment tree

- Lより前で始まって[L,R]間で終わる区間
- これを高速に列挙するには?

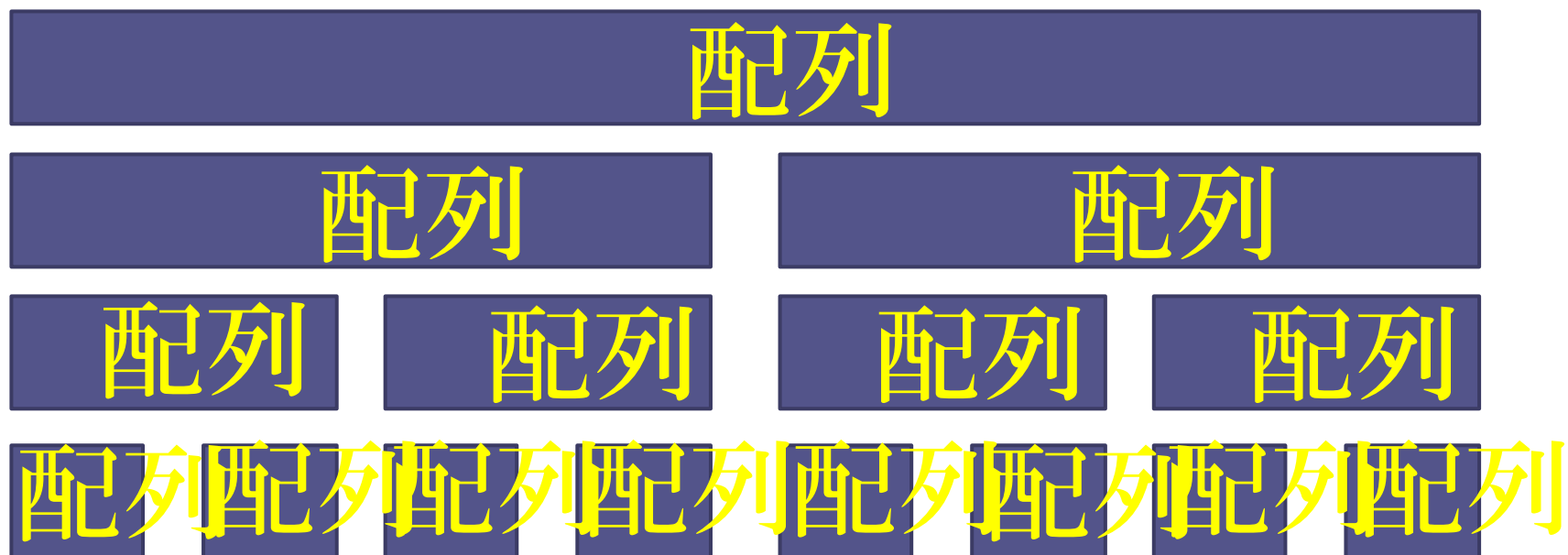
Segment tree

- 各ノードに配列を持っておく



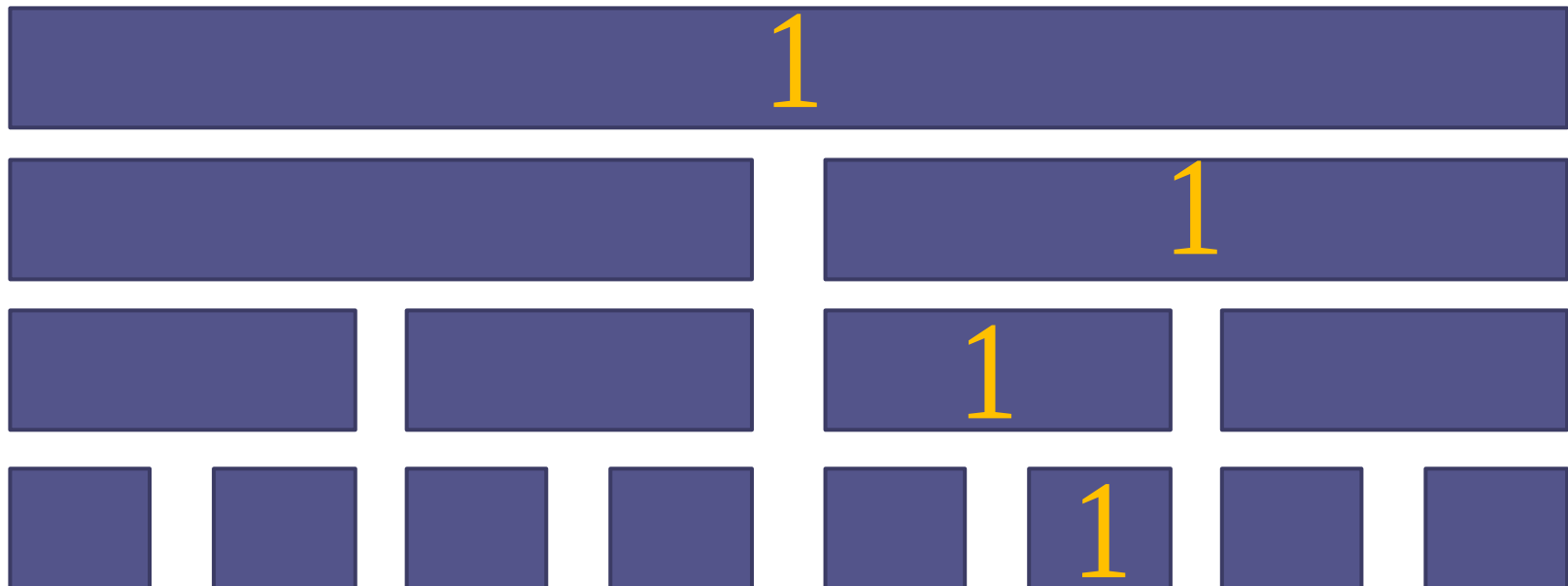
Segment tree

- 各ノードに配列を持っておく



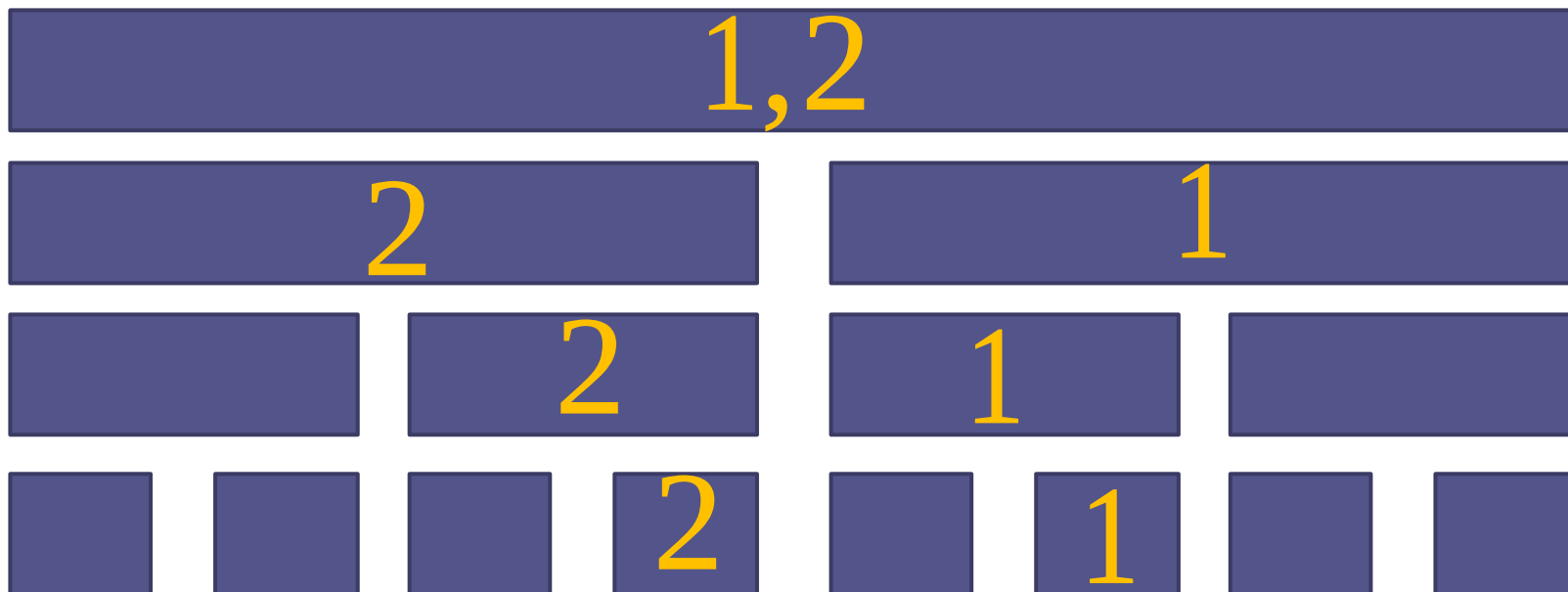
Segment tree

- 区間[L,R]に対し、Rを含む区間の配列にLを昇順に入れておく
 - 区間[1,5]を追加する場合(0-origin)



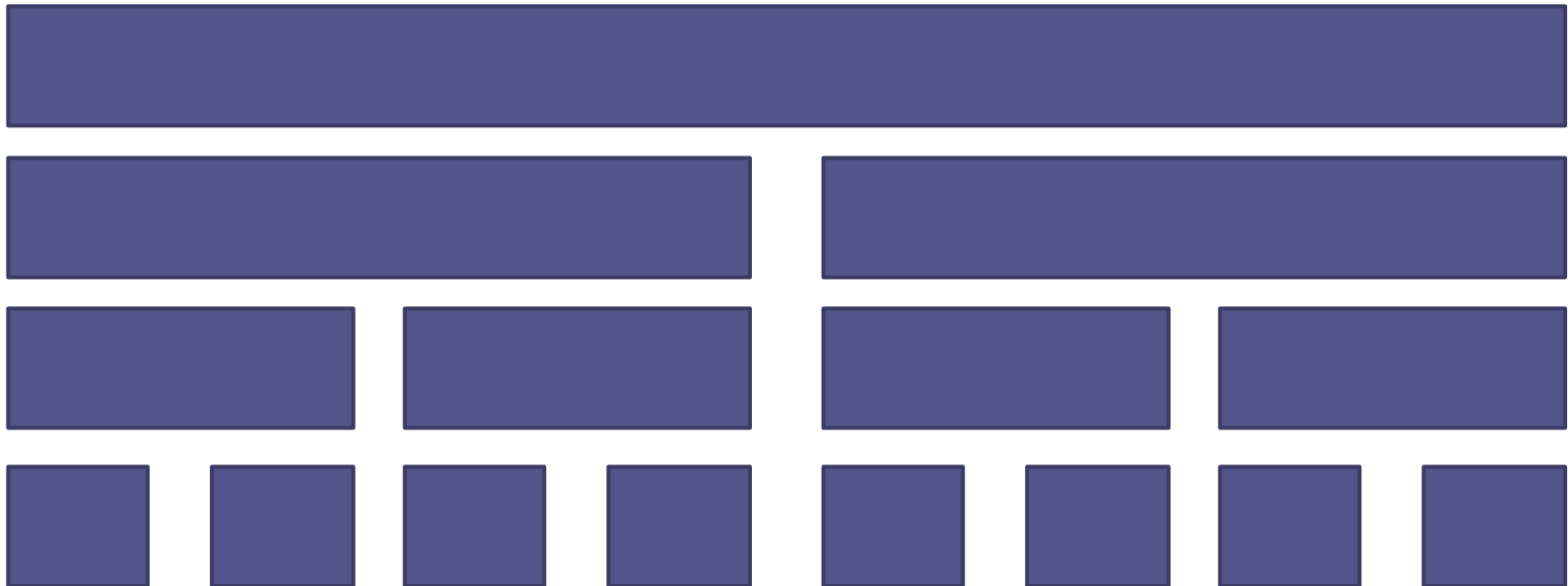
Segment tree

- 区間[L,R]に対し、Rを含む区間の配列にLを昇順に入れておく
 - 区間[2,3]を追加する場合(0-origin)



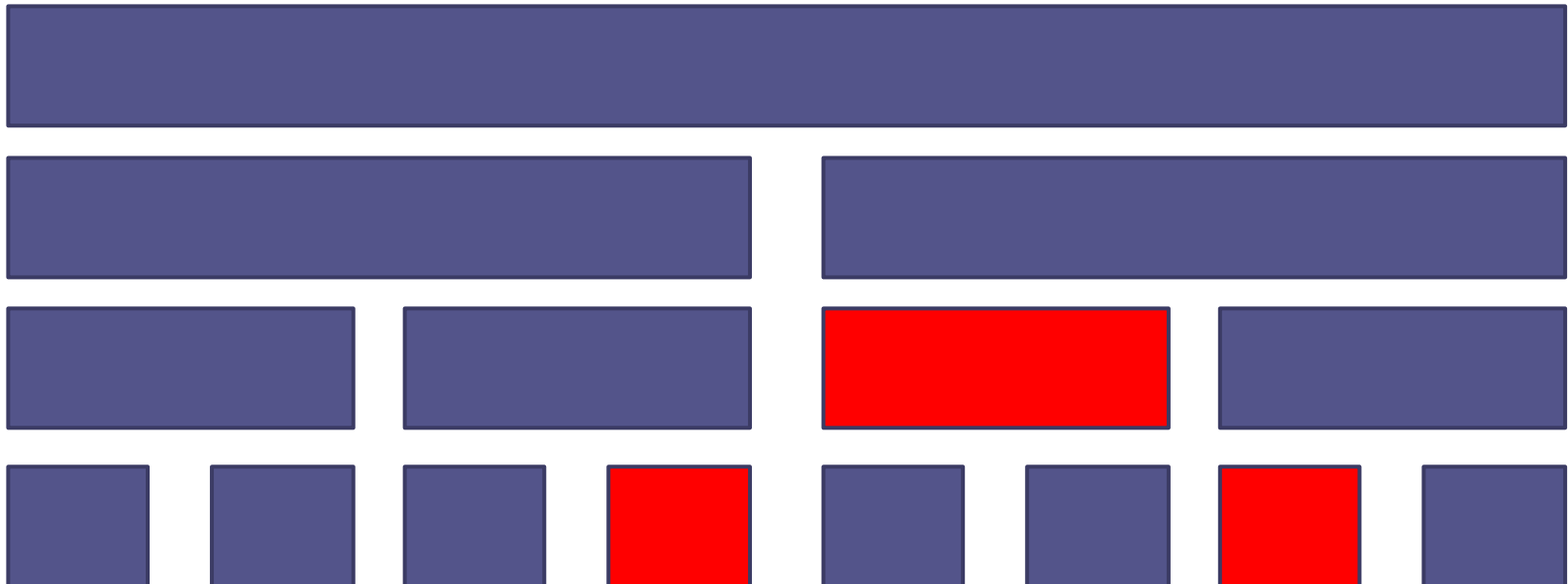
Segment tree

- 区間[3,6]に右端が含まれ、左端が3より前にある区間は?



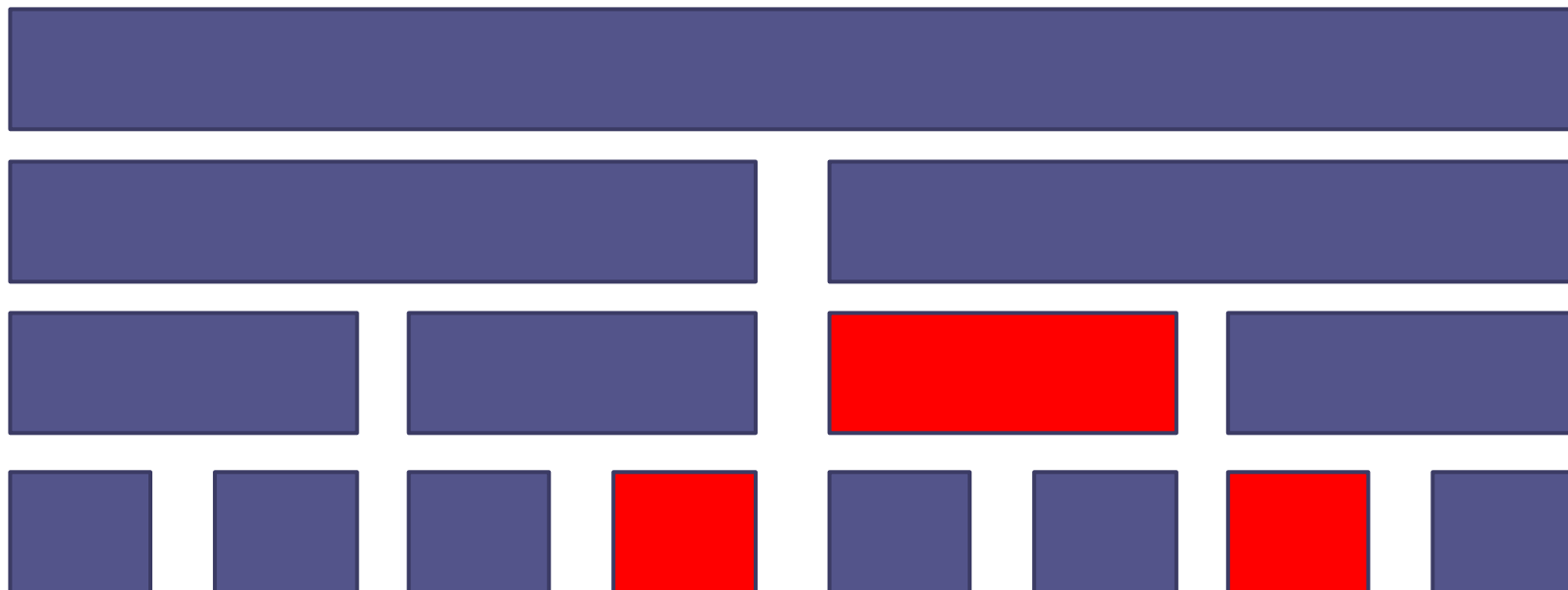
Segment tree

- 区間[3,6]に右端が含まれ、左端が3より前にある区間は?
 - ここに書かれている値のうち3未満のものに対応



Segment tree

- このようなノードで、値がある値以下のものを全部見れば、隣接点がわかる
 - 一度見たら消しておけば余計に見なくて済む



満点解法①

- $[L, R]$ 間で始まってRより後で終わる区間についても同様にsegment treeを作る
- segment treeで隣接する頂点の列挙はできるので、この上で幅優先探索等を行えば連結成分は列挙できる

満点解法①

- あとは、二部グラフ判定ができればよい
- これは、(各頂点が二部グラフのどちら側にあるかはわかるので)問題文の通り、実際にスタックに積んだりスタックから取り出したりして判定してやればいい

満点解法①

- 計算量は、
- 配列に値を追加するときは、追加を昇順に行っていけば $O(N \log N)$
- 削除も全部で $O(N \log N)$
- グラフの探索で見る辺の本数も $O(N)$
- 全部で $O(N \log N)$ でできた
- 定数倍が重いので、vectorを使わないなどの工夫をしないと通らないかもしれません

満点解法②

満点解法②

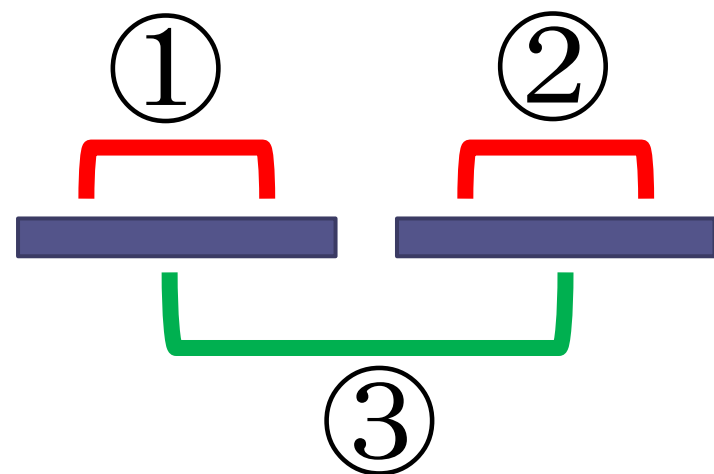
● 分割統治

満点解法②

- 分割統治法
- 区間を半分にして再帰していく
- $[L, R]$ に含まれる区間を以下の3種類に分ける
- $M = (L + R) / 2$ として、
 - ① $[L, M]$ に完全に含まれる区間
 - ② $[M + 1, R]$ に完全に含まれる区間
 - ③ $[M, M + 1]$ をまたぐ区間

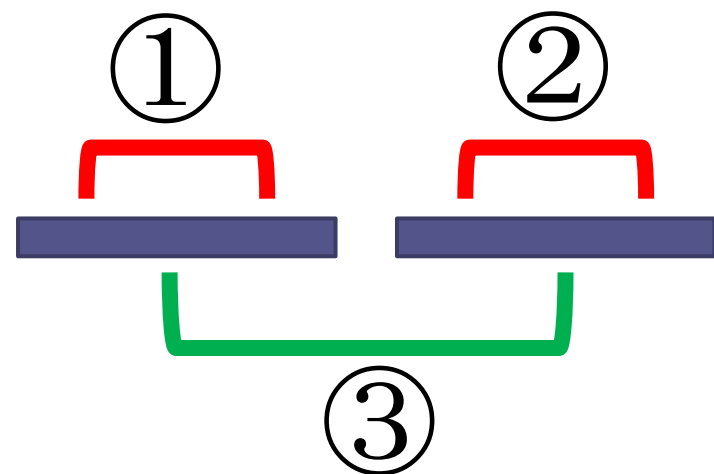
満点解法②

- 上の2つに関しては、再帰してその中で見る
- 残りの3つについて辺を張ればよい
- 3つめと4つめは対称なので、3,5個目のみ考える
 - ①の区間と①の区間の間の辺
 - ②の区間と②の区間の間の辺
 - ①の区間と③の区間の間の辺
 - ②の区間と③の区間の間の辺
 - ③の区間と③の区間の間の辺



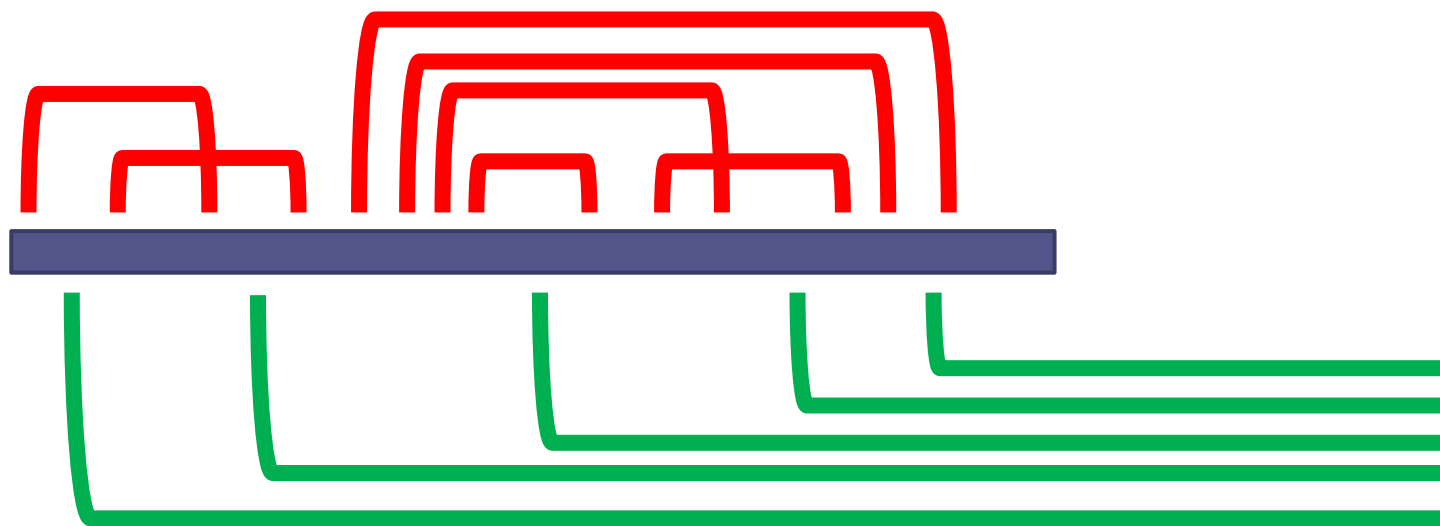
満点解法②

- ①の区間と③の区間の間の辺
- ③の区間は、左端のみ関係ある
- ③の区間を左端の座標の昇順にソートしておく



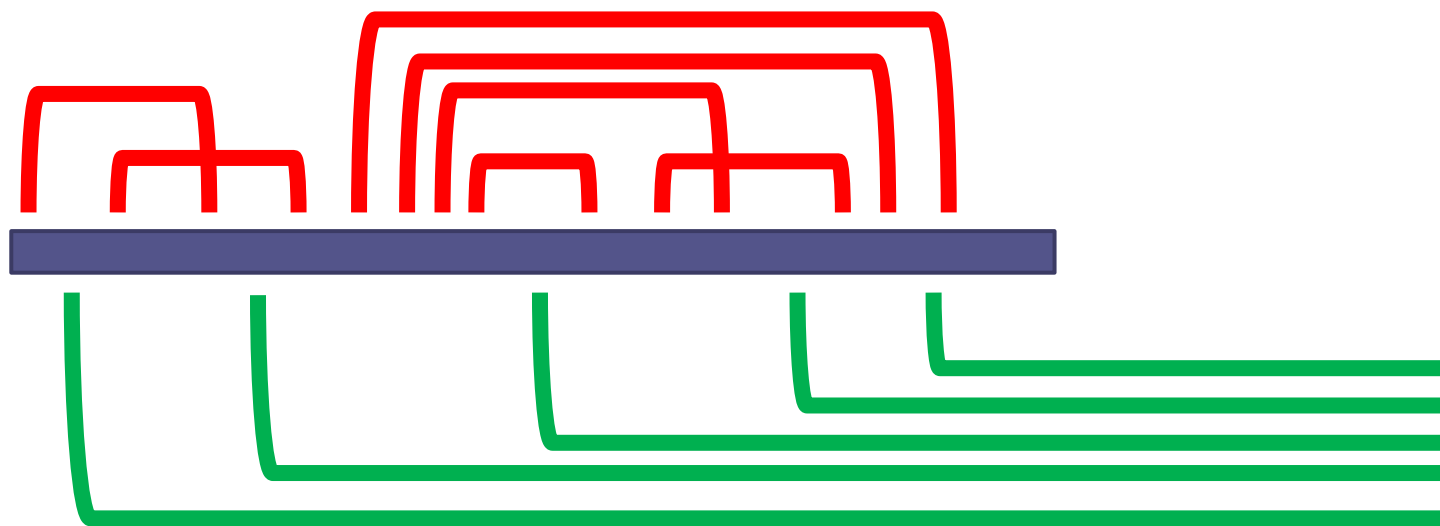
満点解法②

- ①の区間と③の区間の間の辺
- ③の区間は、左端のみ関係ある
- ③の区間を左端の座標の昇順にソートしておく



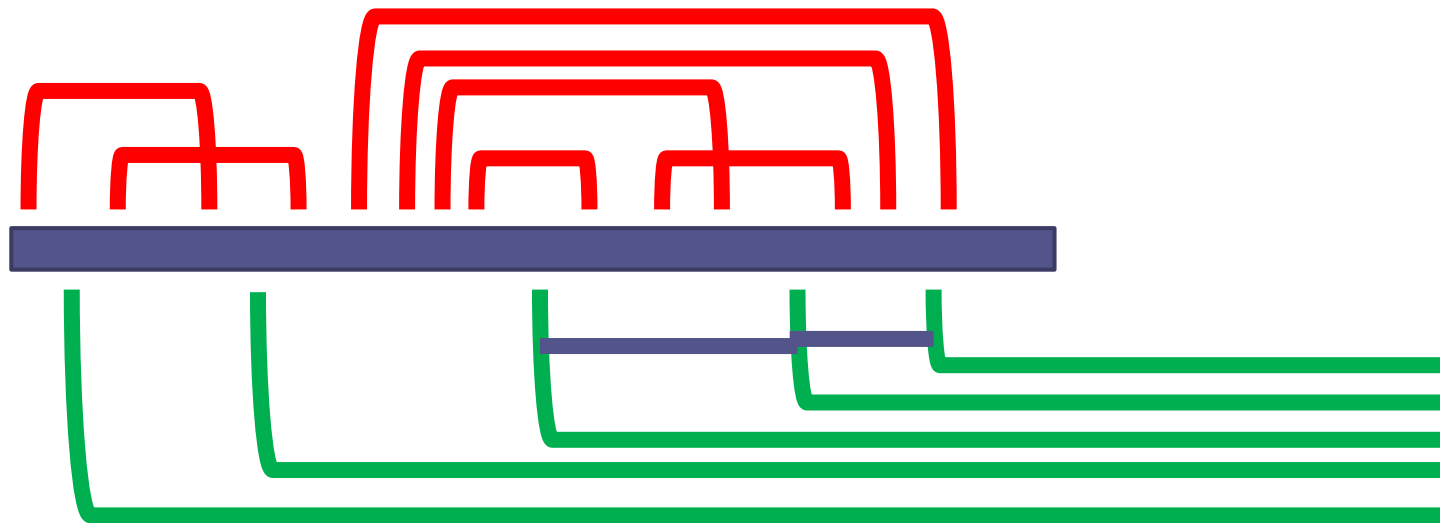
満点解法②

- 各①の区間に対し、それに左端が含まれるような「②の区間の区間」を求めておく
- この区間の区間すべてに辺を張るとまだ張りすぎ



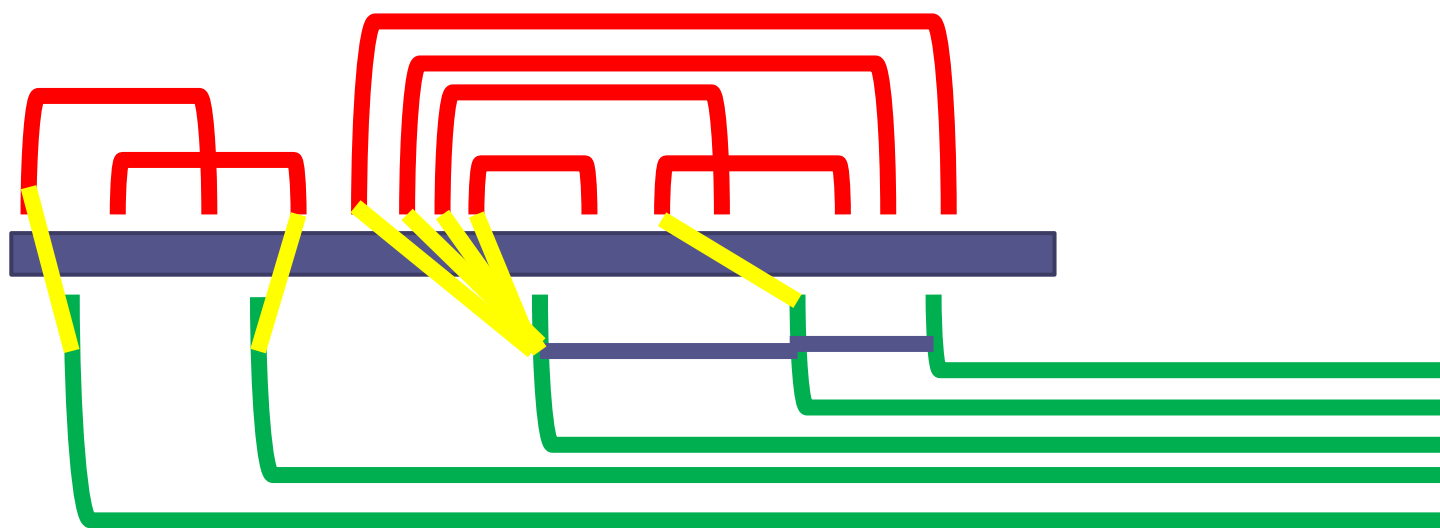
満点解法②

- となりあう2つの③の区間に対し、それを両方含む①の区間が存在するならその間にコスト0の辺を張る



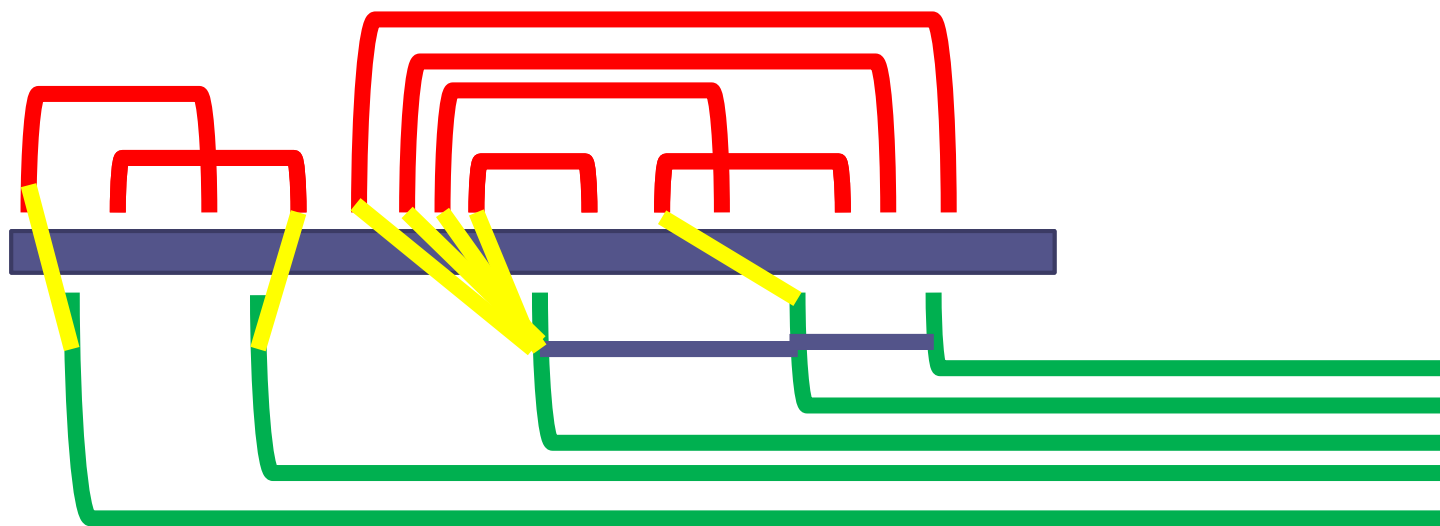
満点解法②

- さらに、各①の区間に対し、それに左端が含まれる③の区間が存在するなら、そのうち一つに辺を張る



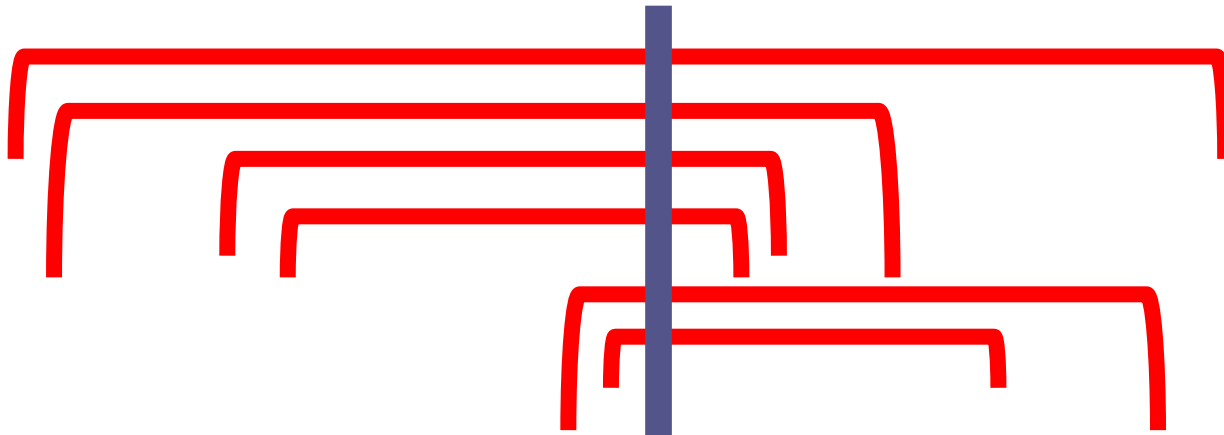
満点解法②

- こうすると、区間の数くらいの辺の数で、二部グラフ性のようなものが等価なグラフが構成できる
- 適切に実装すれば $O(\text{区間長})$ でできる



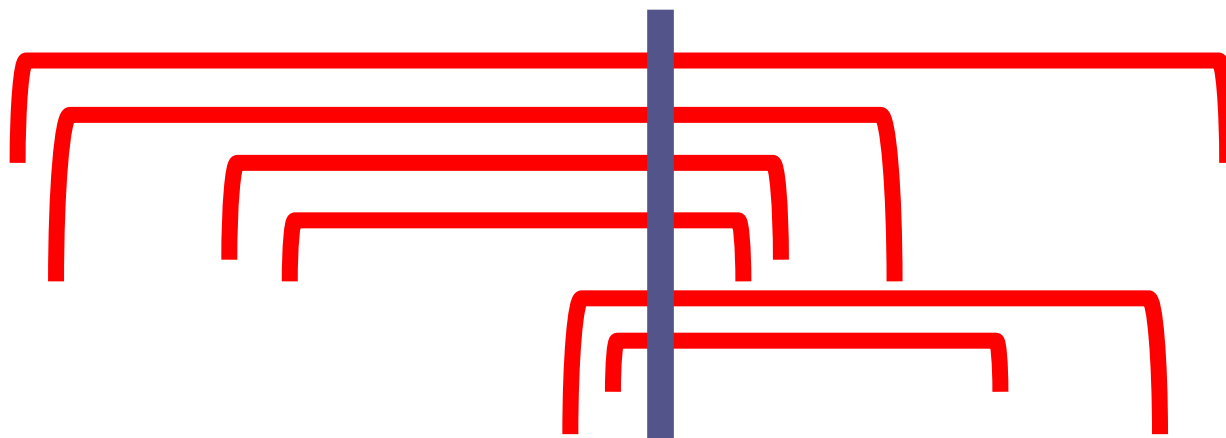
満点解法②

- ③の区間と③の区間の間の辺



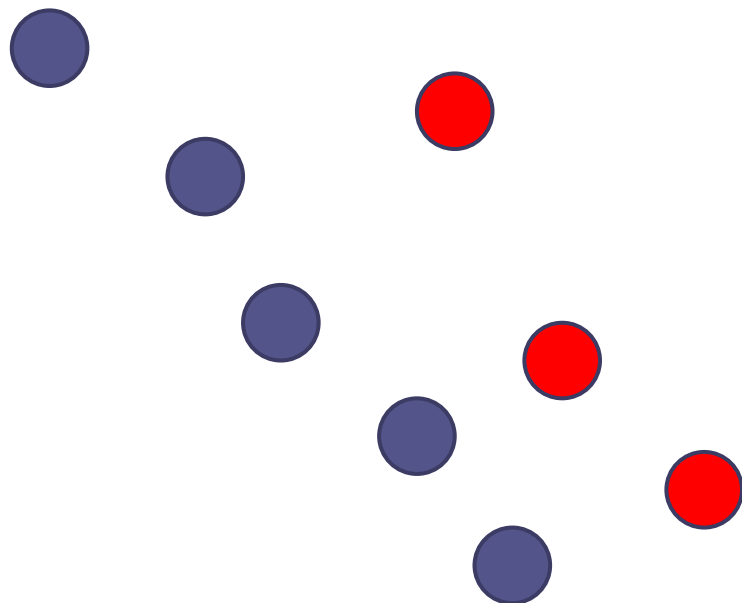
満点解法②

- (始点, 終点)のペアを平面上にプロットしておく
- 左下に点のある点と、ない点に分ける



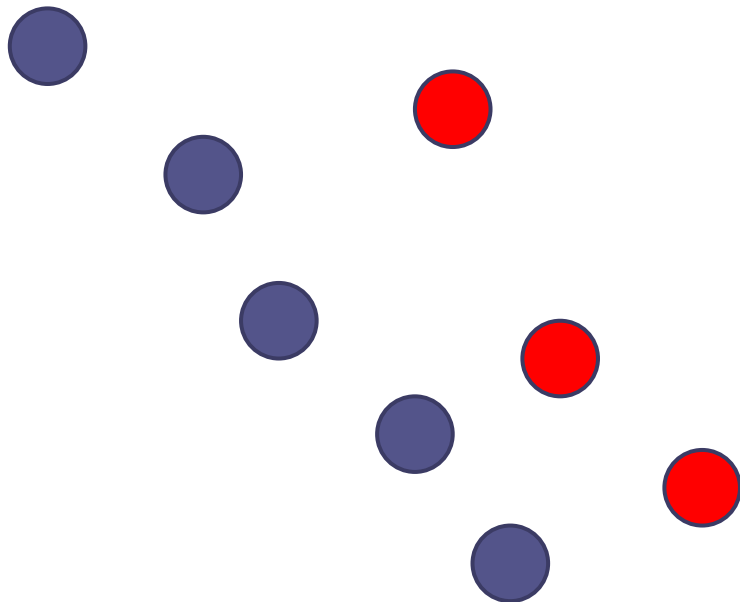
満点解法②

- (始点, 終点)のペアを平面上にプロットしておく
- 左下に点のある点と、ない点に分ける
 - 右下-左上の関係にある点対の間に辺を張りたい



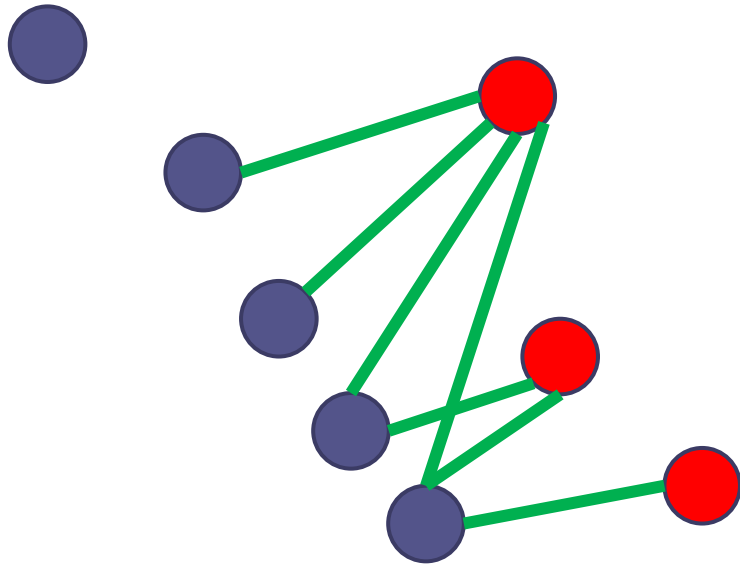
満点解法②

- まず、赤い点に右下-左上の関係にある2点があれば、二部グラフでない
 - 三角形ができるため



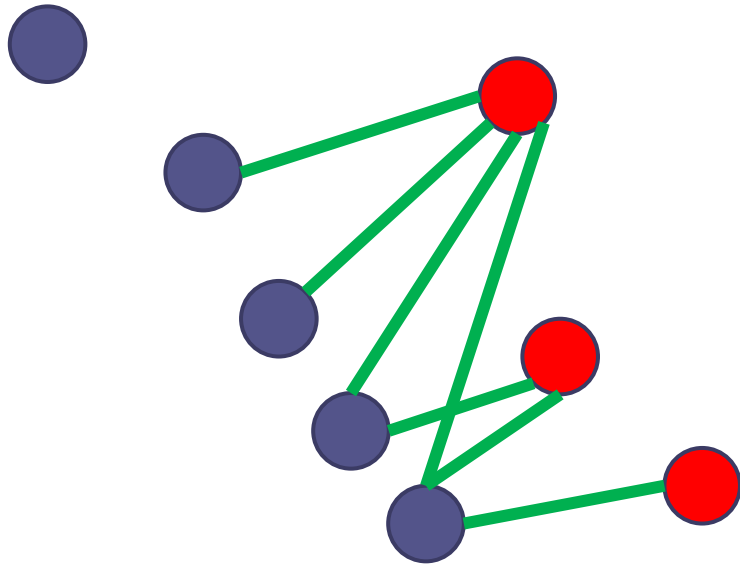
満点解法②

- (始点, 終点)のペアを平面上にプロットしておく
- 左下に点のある点と、ない点に分ける
 - 右下-左上の関係にある点対の間に辺を張りたい



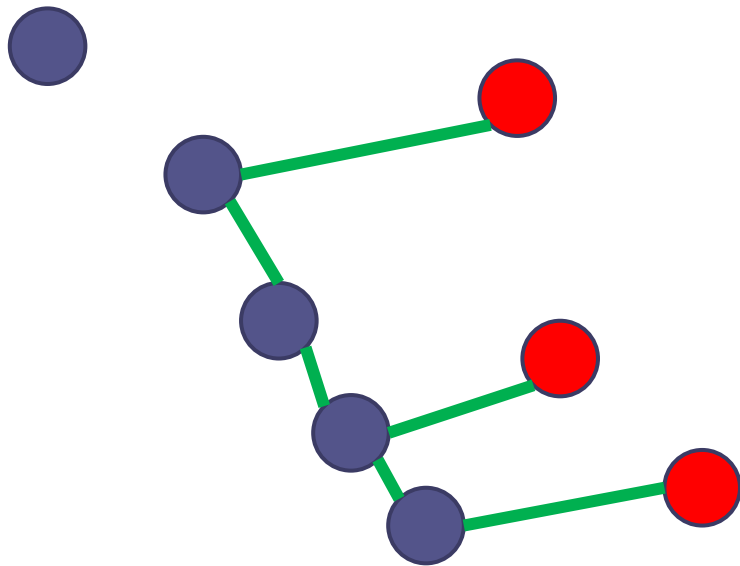
満点解法②

- このままだと辺数は $O(N^2)$



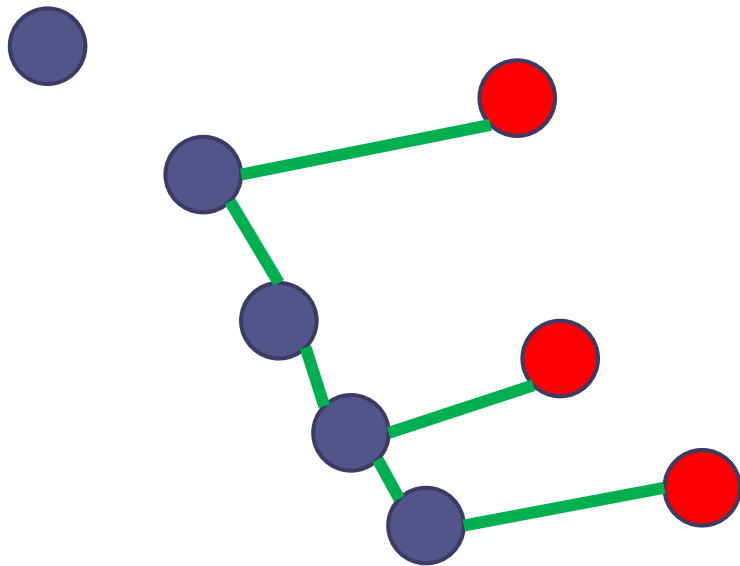
満点解法②

- これも、先ほどと同様に、隣り合う点の間にコスト0の辺を張ることで、 $O(N)$ 辺で等価なグラフが構成できる



満点解法②

- 適切に実装すれば $O((\text{点の数})\log(\text{点の数}))$ で辺が張れる



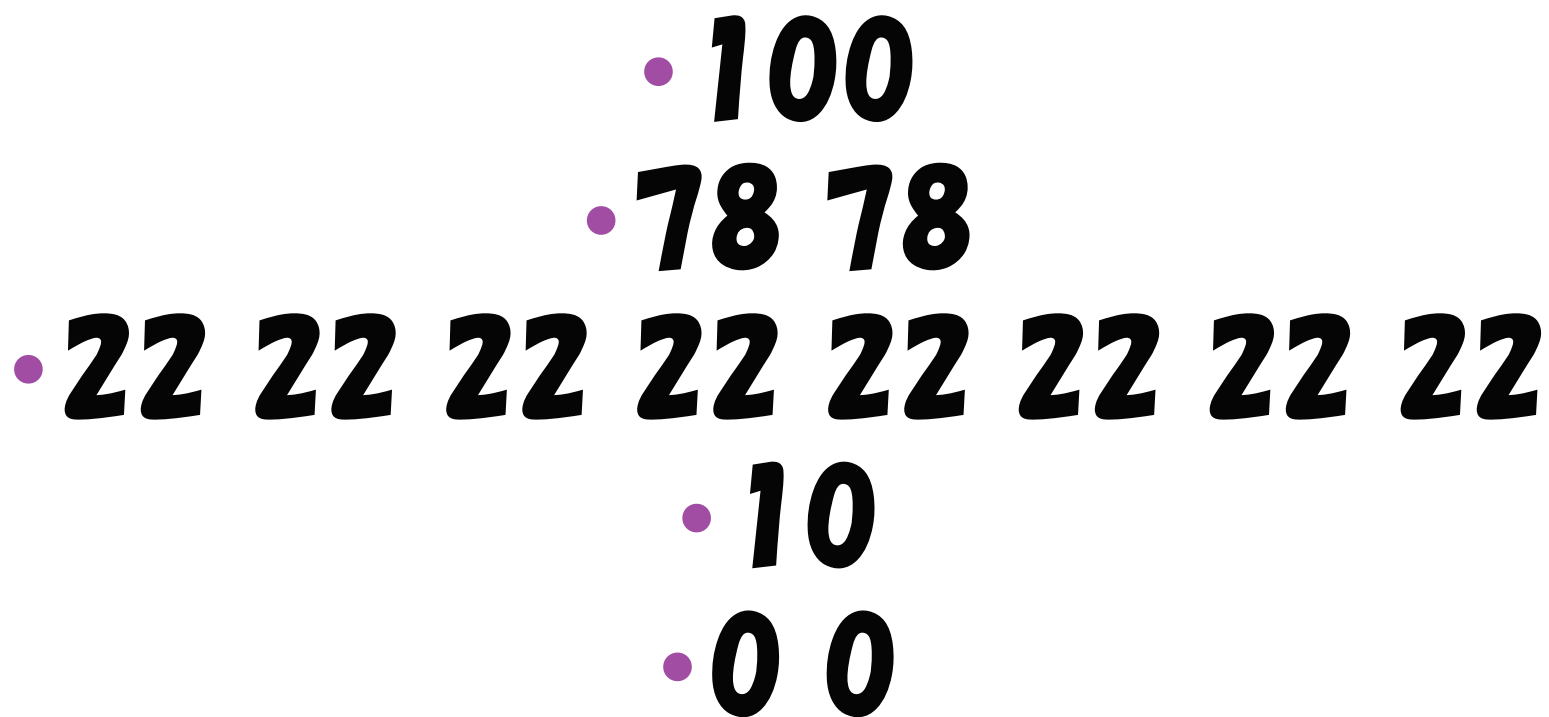
満点解法②

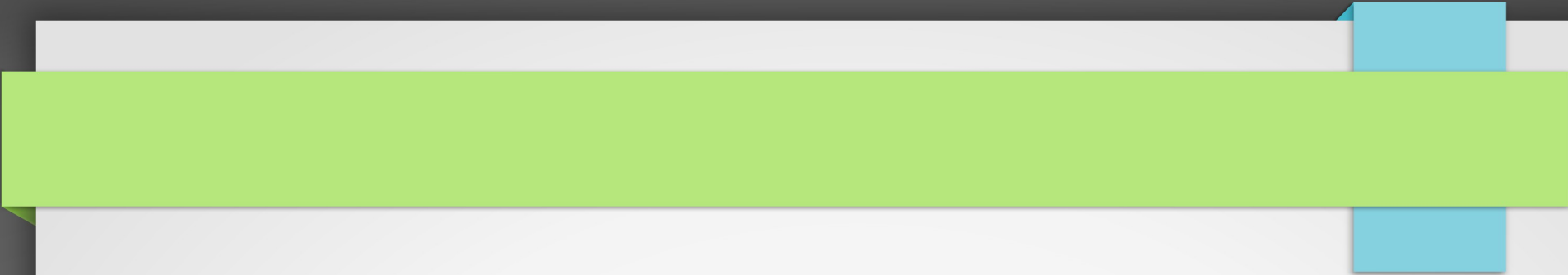
- 以上をまとめて、 $O(N \log N)$ でこの問題が解けた

満点解法③～

- ほかにいくつか解法があるそうです
- いろいろ見つけてみましょう

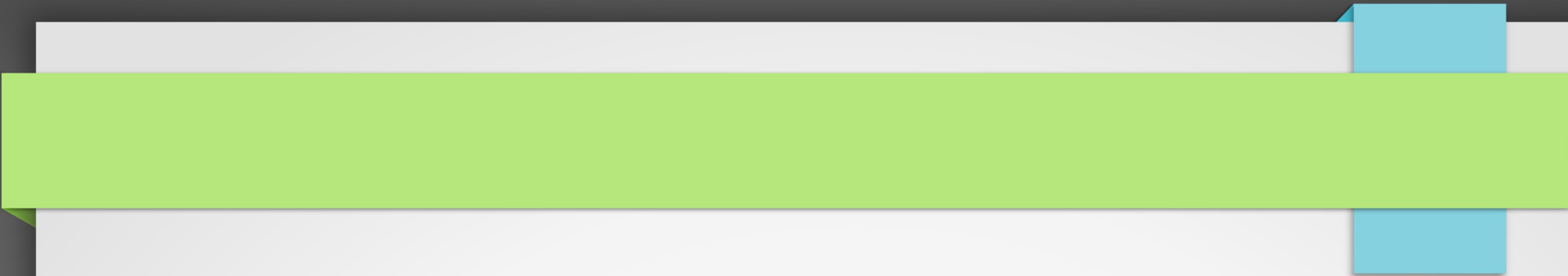
得点分布





解説





手持ち花火

sparklers

問題概要

- N人で一直線に並んで花火をする
- 花火はT秒だけ燃え続ける
- はじめJOI君の花火だけ着火する
- うまく火を移しあって全員の花火を着火するには、
どれくらい速く動けばいいか？
- 速度制限の最小値を求めよ
- $N \leq 100,000$

問題概要

- N人で一直線に並んで花火をする
- 花火はT秒だけ燃え続ける
- はじめJOI君の花火だけ着火する
- うまく火を移しあって全員の花火を着火するには、
どれくらい速く動けばいいか？
- **速度制限の最小値を求めよ**
- $N \leq 100,000$

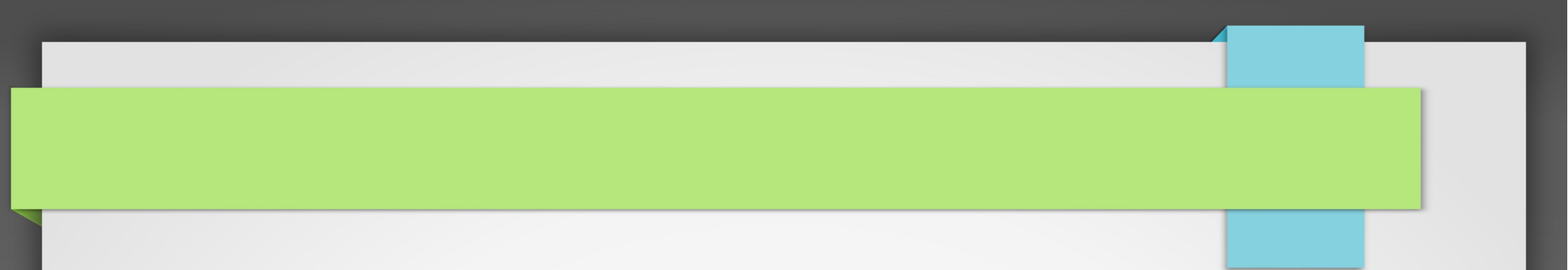
考察0

- ある速度制限 s で全員に火を移せるならば、それより s が大きい速度制限でも全員に火を移せる
- 全員に火を移せる s の中で最小値を求める
- このタイプの最適化問題で使いやすいアルゴリズムがある

考察0

- ある速度制限 s で全員に火を移せるならば、それより s が大きい速度制限でも全員に火を移せる
- 全員に火を移せる s の中で最小値を求める
- このタイプの最適化問題で使いやすいアルゴリズムがある

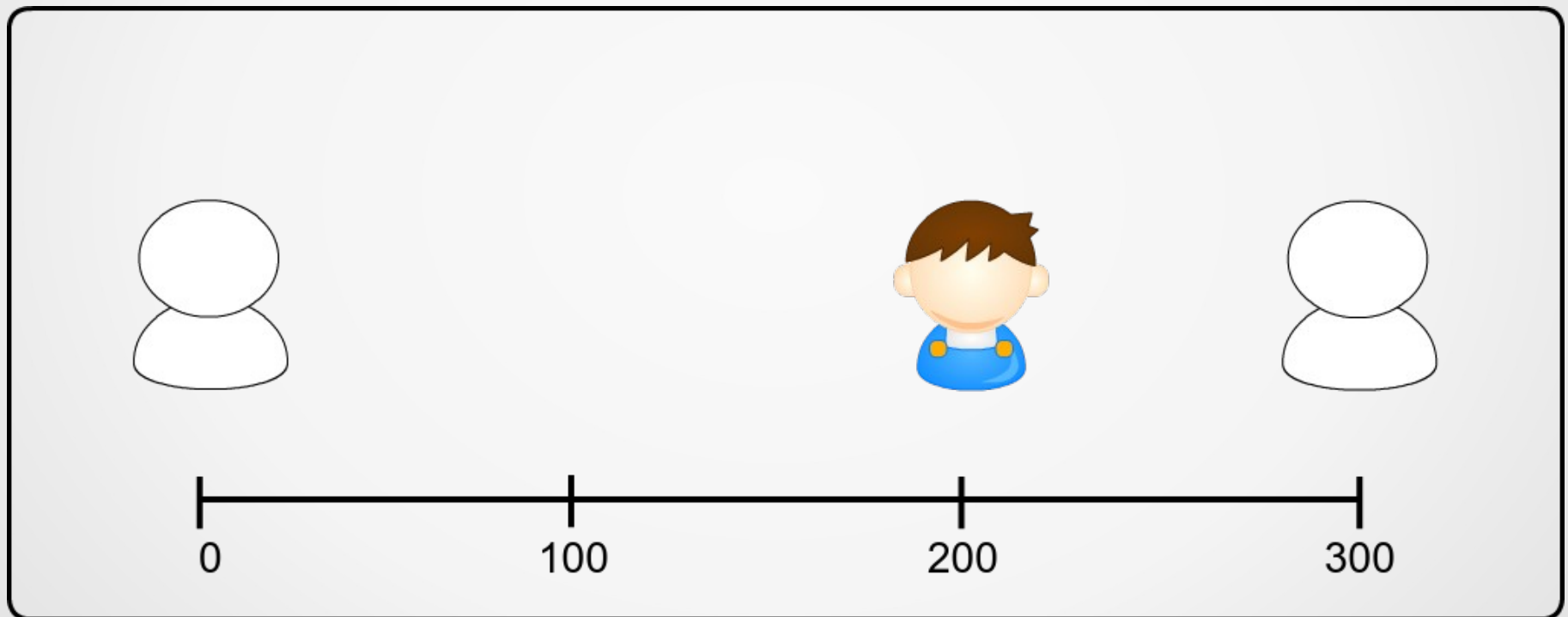
二分法



入力例

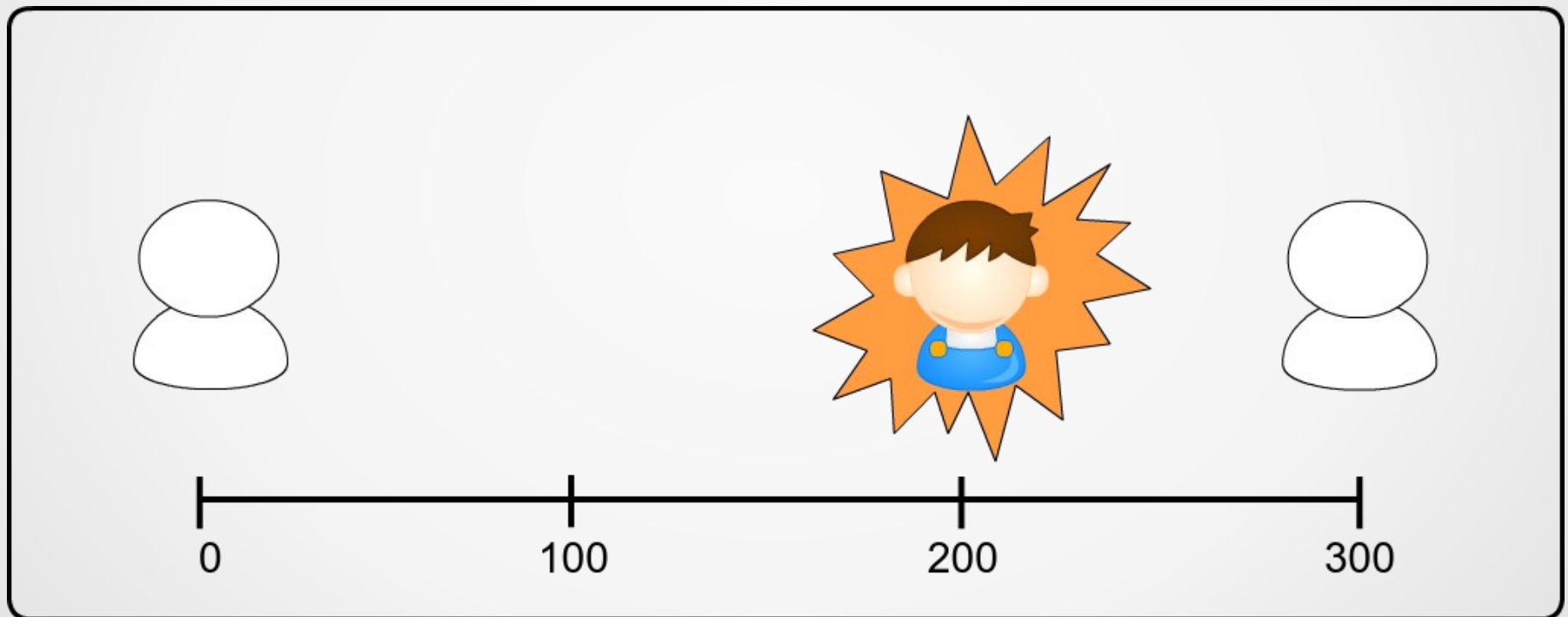
入力例1

- $N=3$, $K=2$, $T=50$



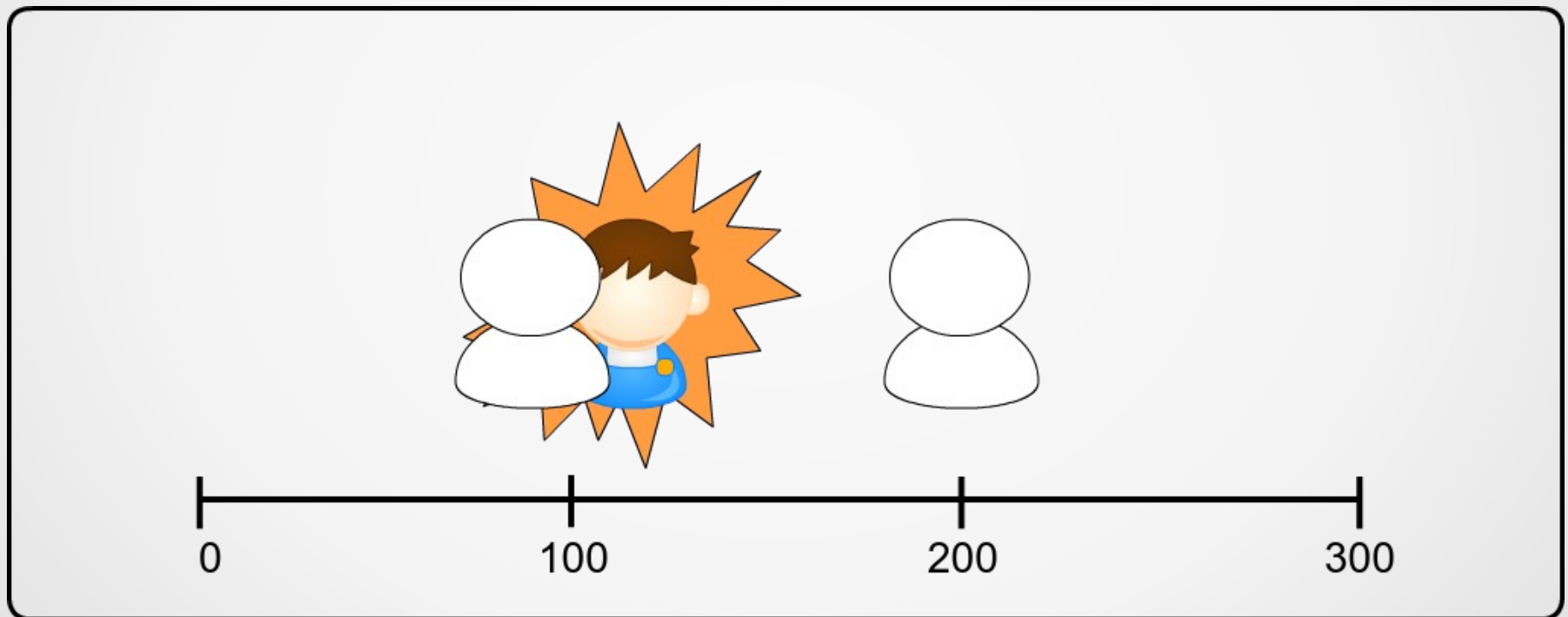
入力例1 ($s=2$)

- 時刻 0 (JOI君着火)



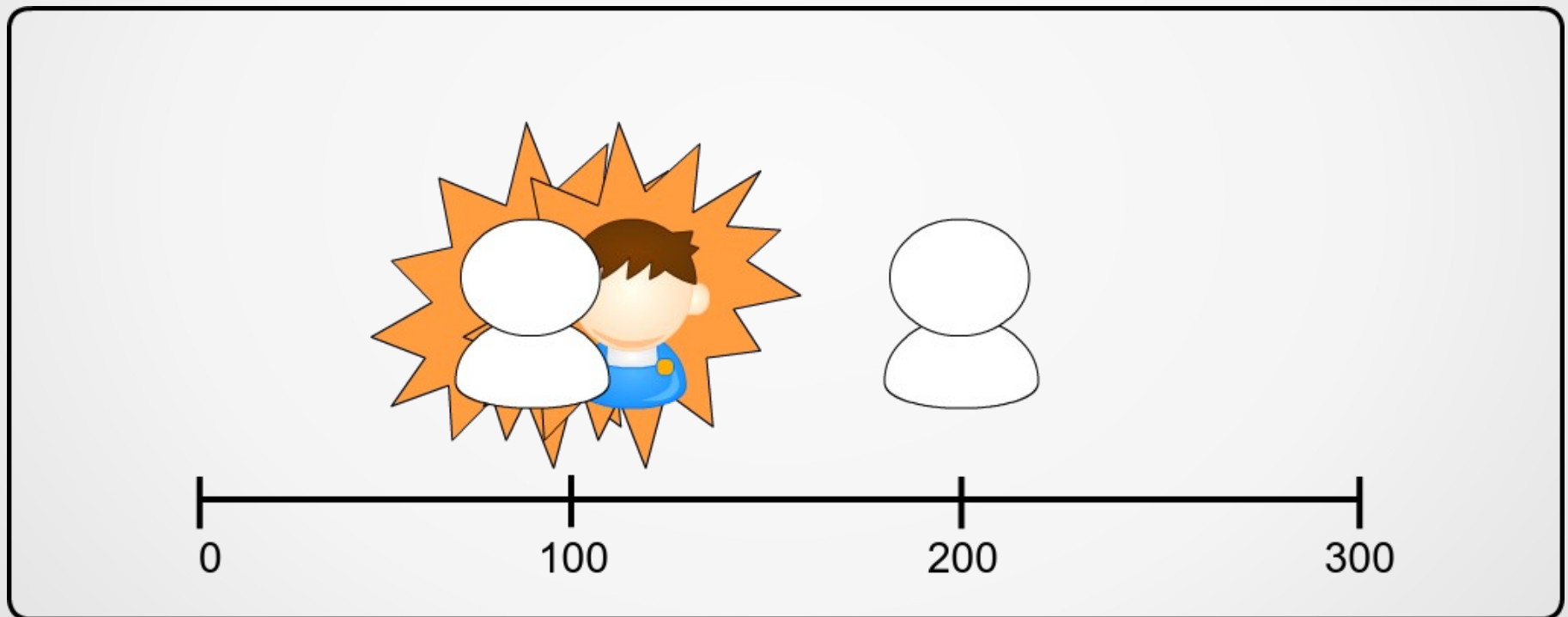
入力例1 ($s=2$)

- 時刻 50



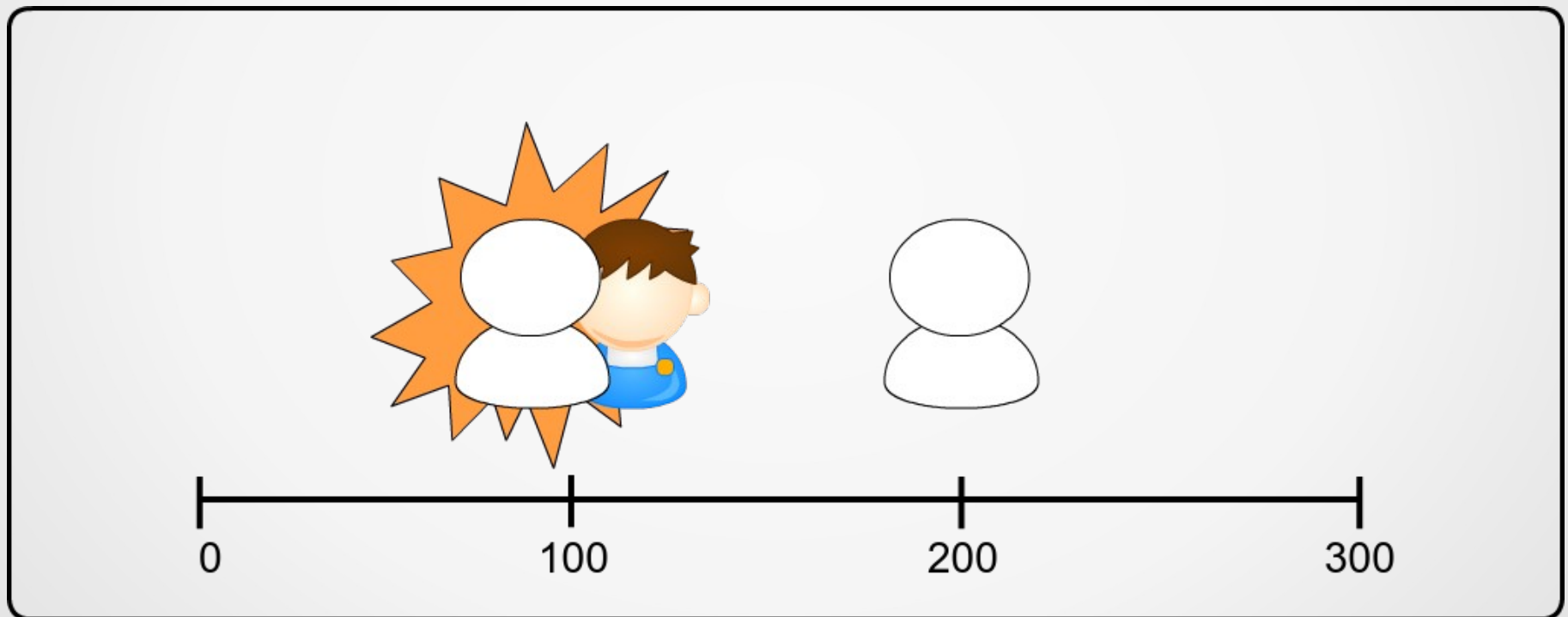
入力例1 ($s=2$)

- 時刻 50 (火を移す)



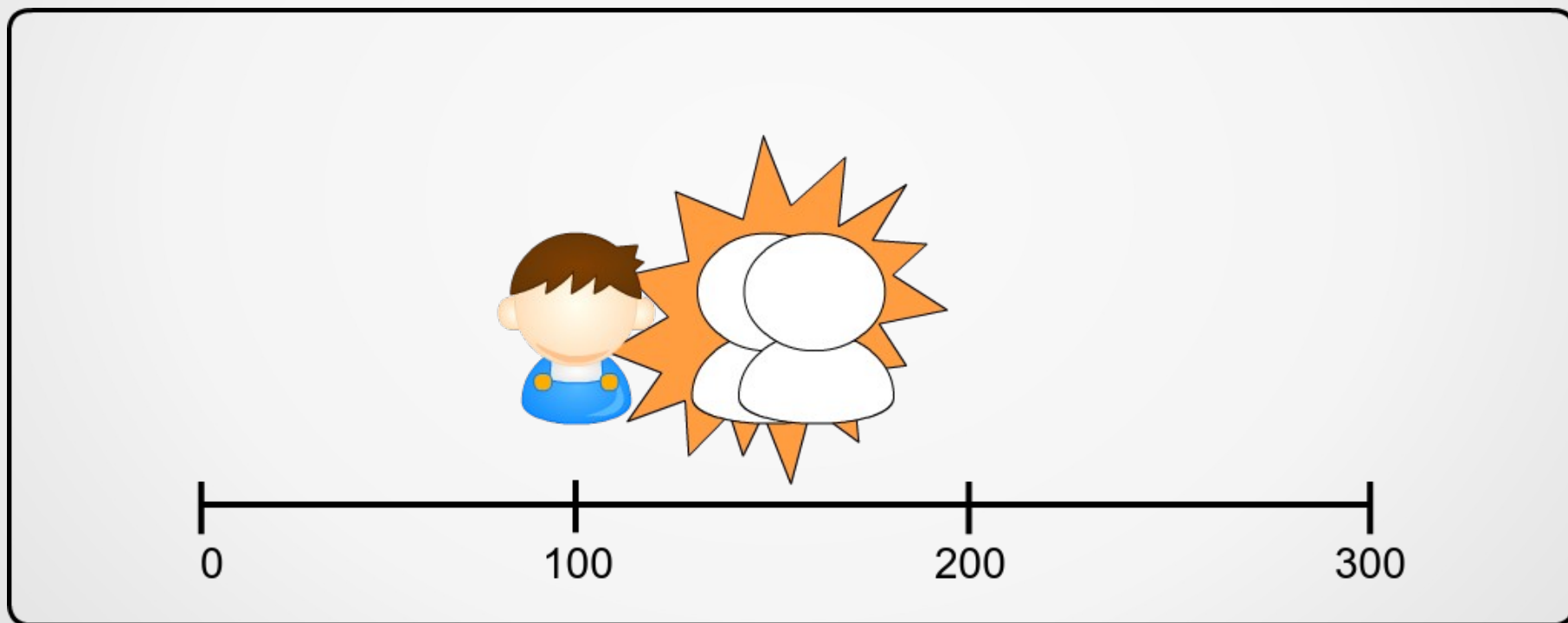
入力例1 ($s=2$)

- 時刻 50 (JOI君燃え尽きる)



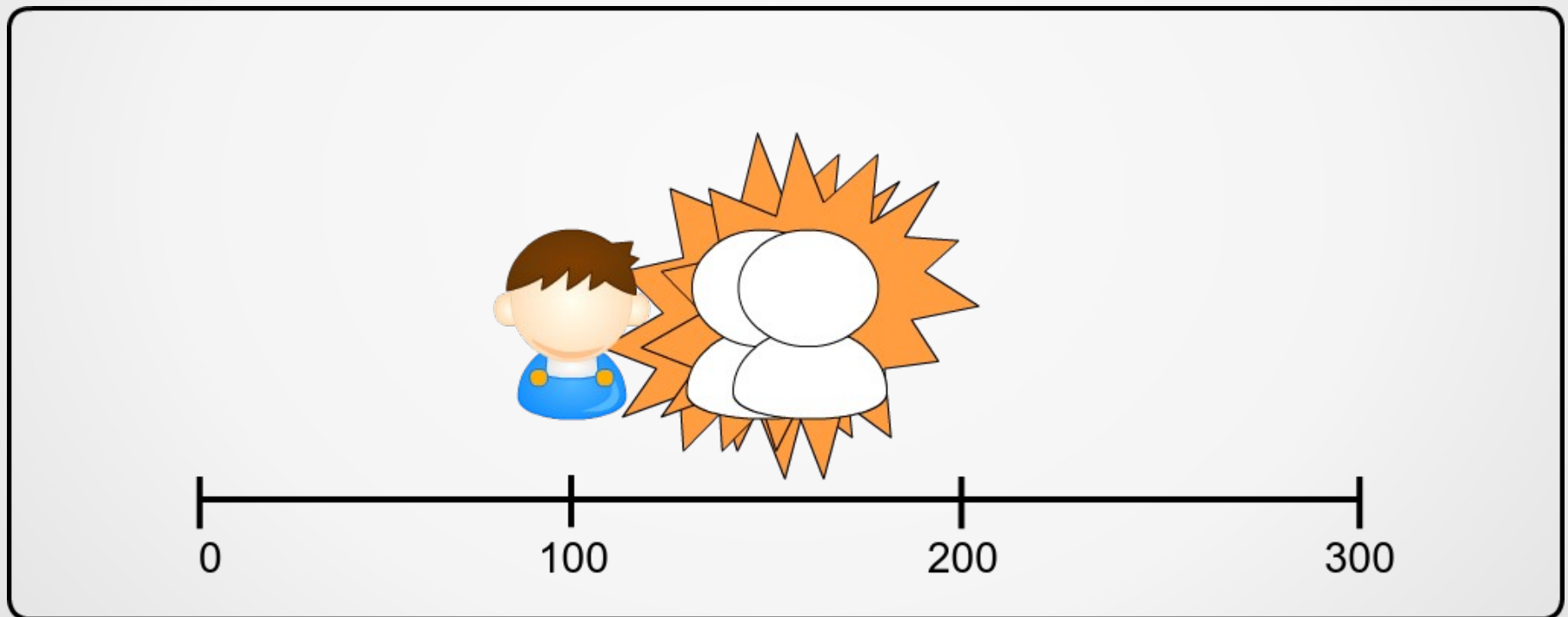
入力例1 ($s=2$)

- 時刻 75



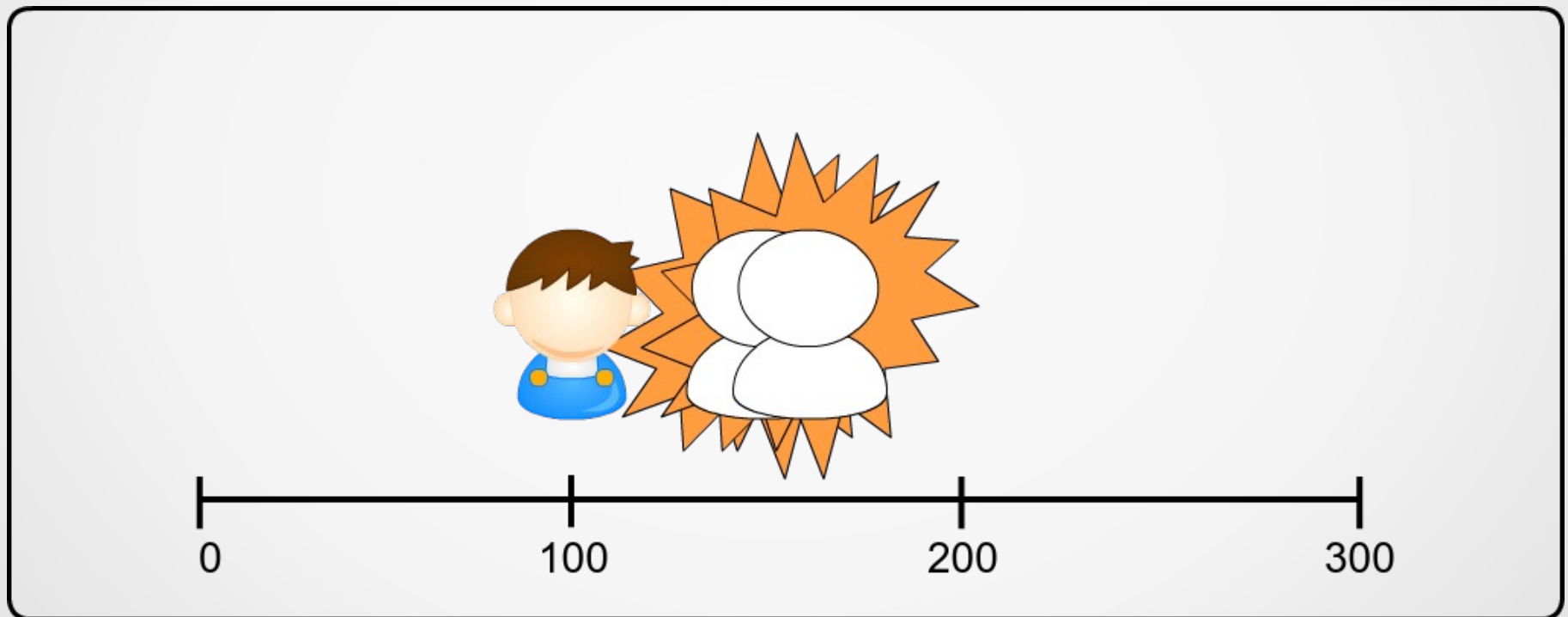
入力例1 ($s=2$)

- 時刻 75 (火を移す)



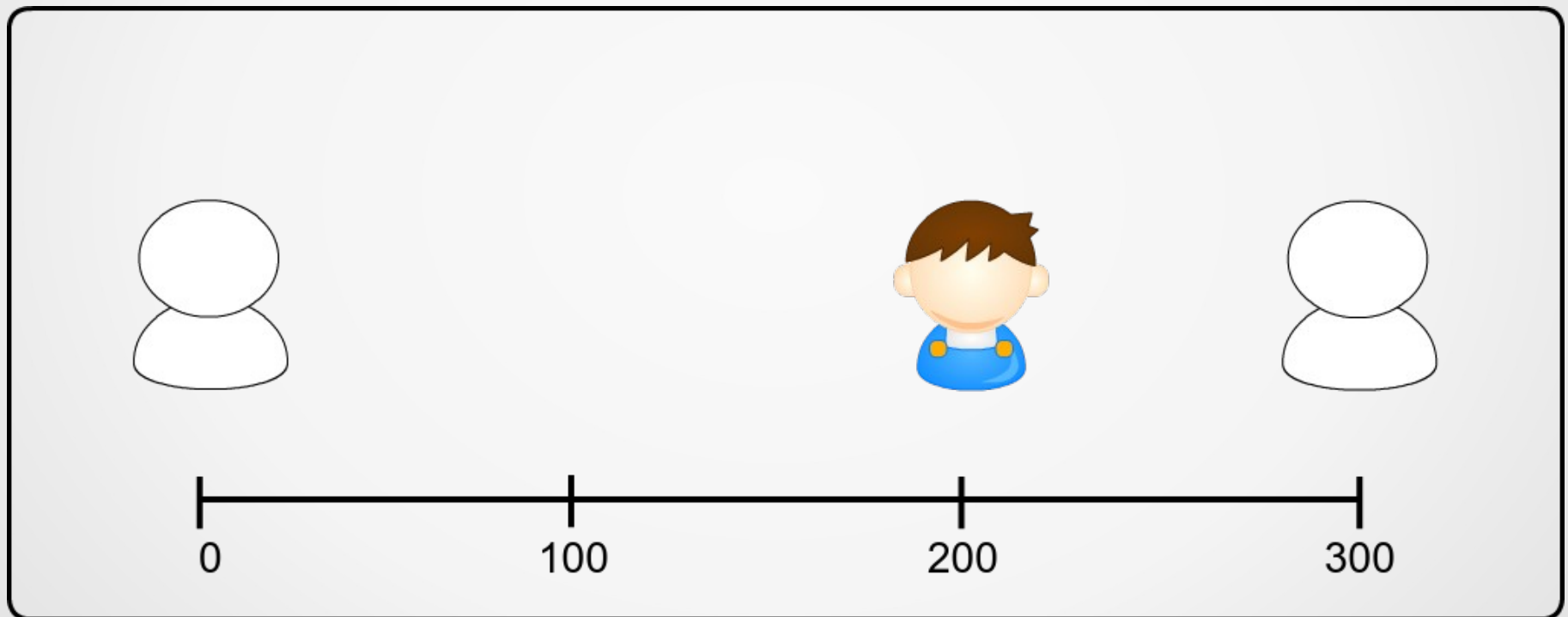
入力例1 ($s=2$)

- 時刻 75 (全員に火が移った!)



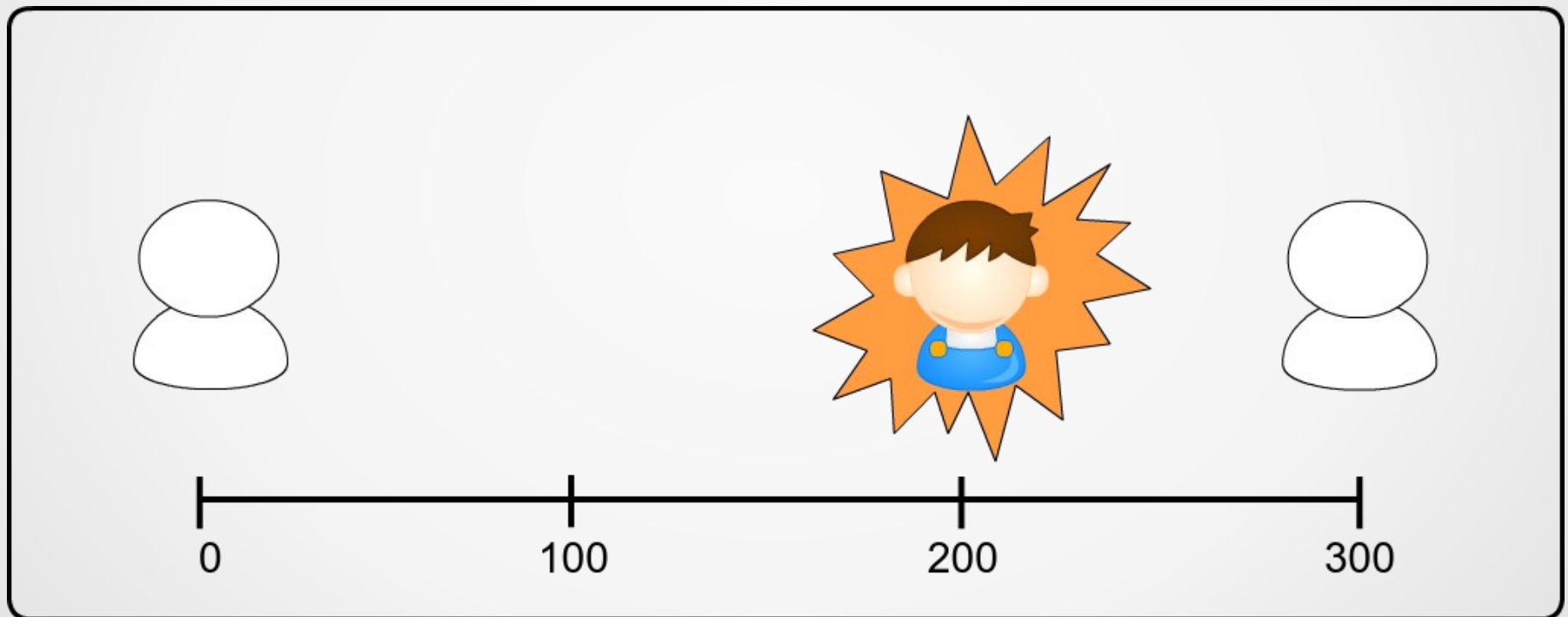
入力例2

- $N=3$, $K=2$, $T=10$



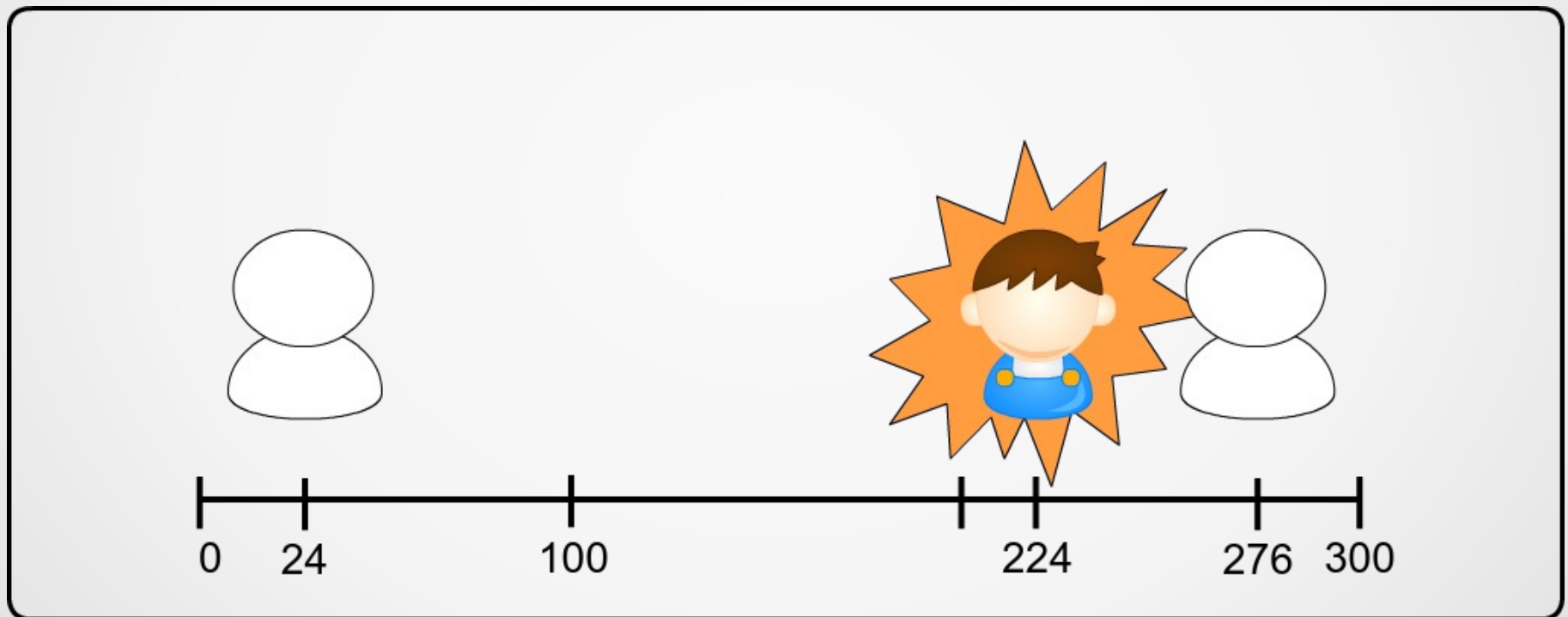
入力例2 ($s=8$)

- 時刻 0(JOI君着火)



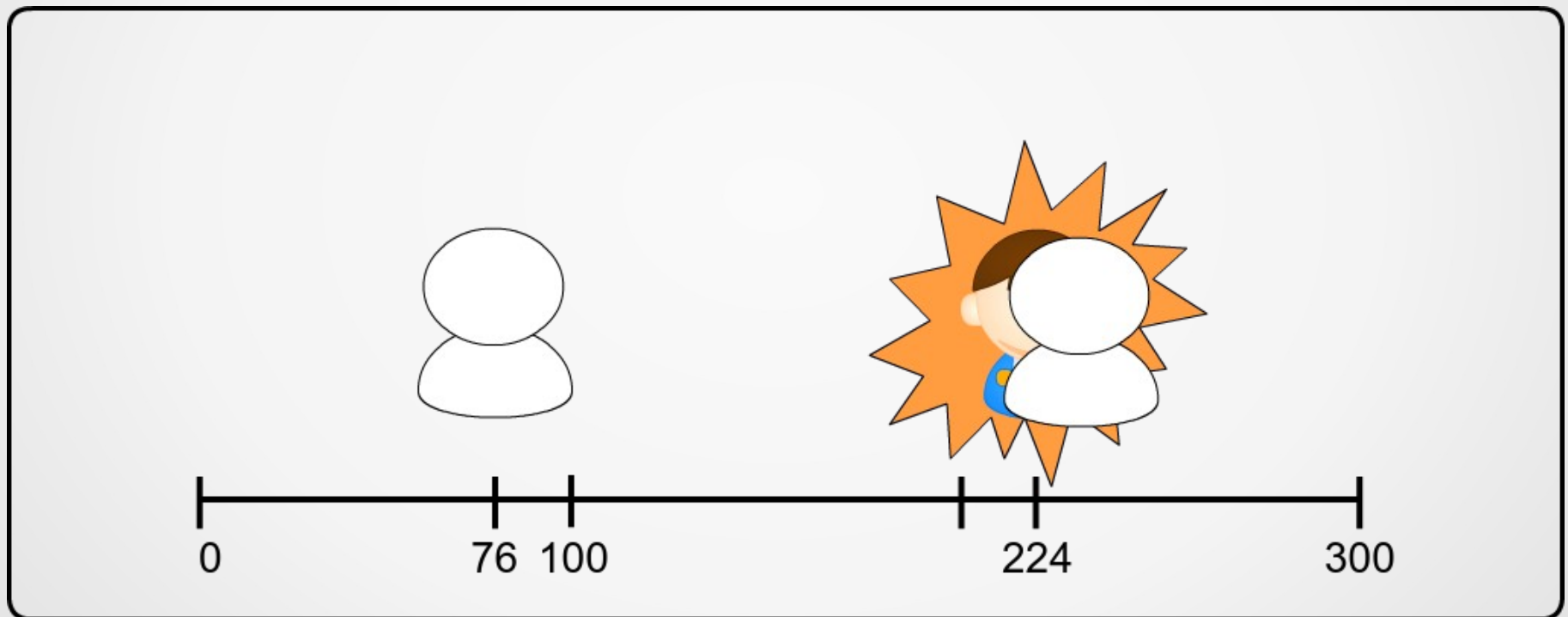
入力例2 ($s=8$)

- 時刻 3(JOI君停止)



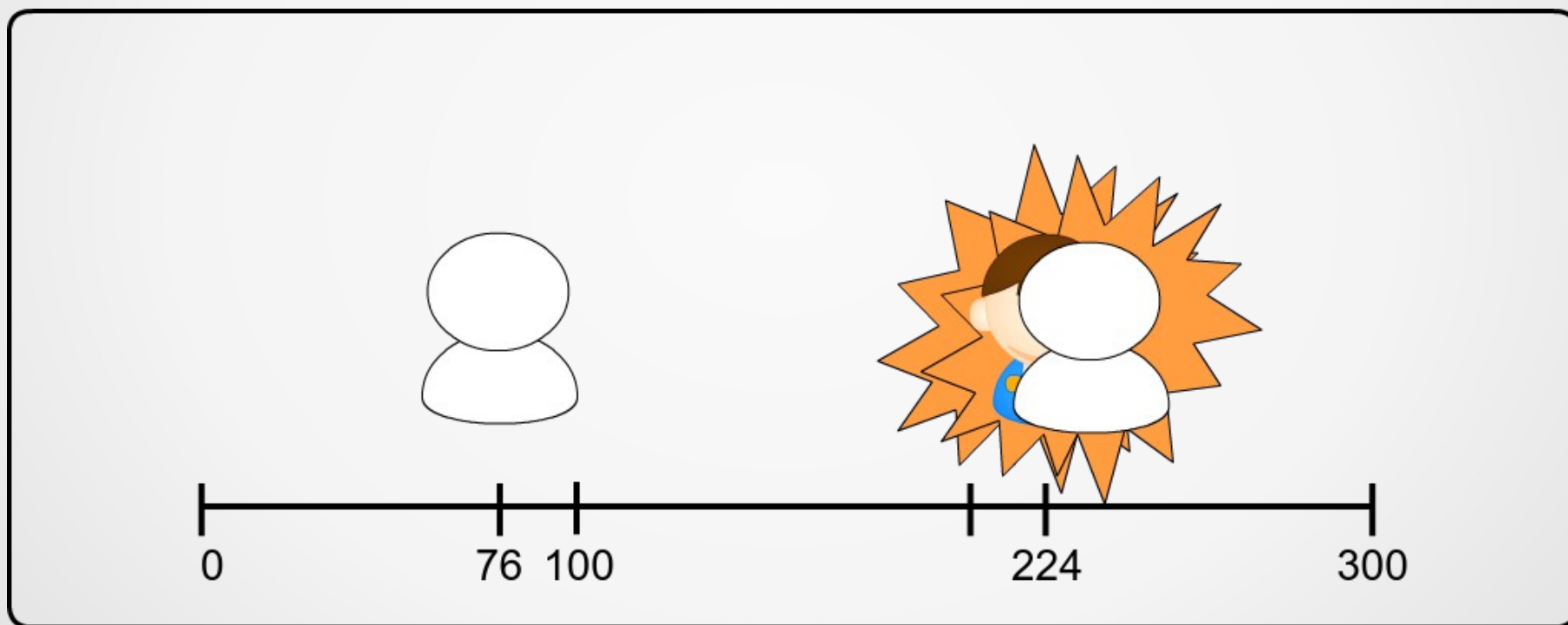
入力例2 ($s=8$)

- 時刻 9.5



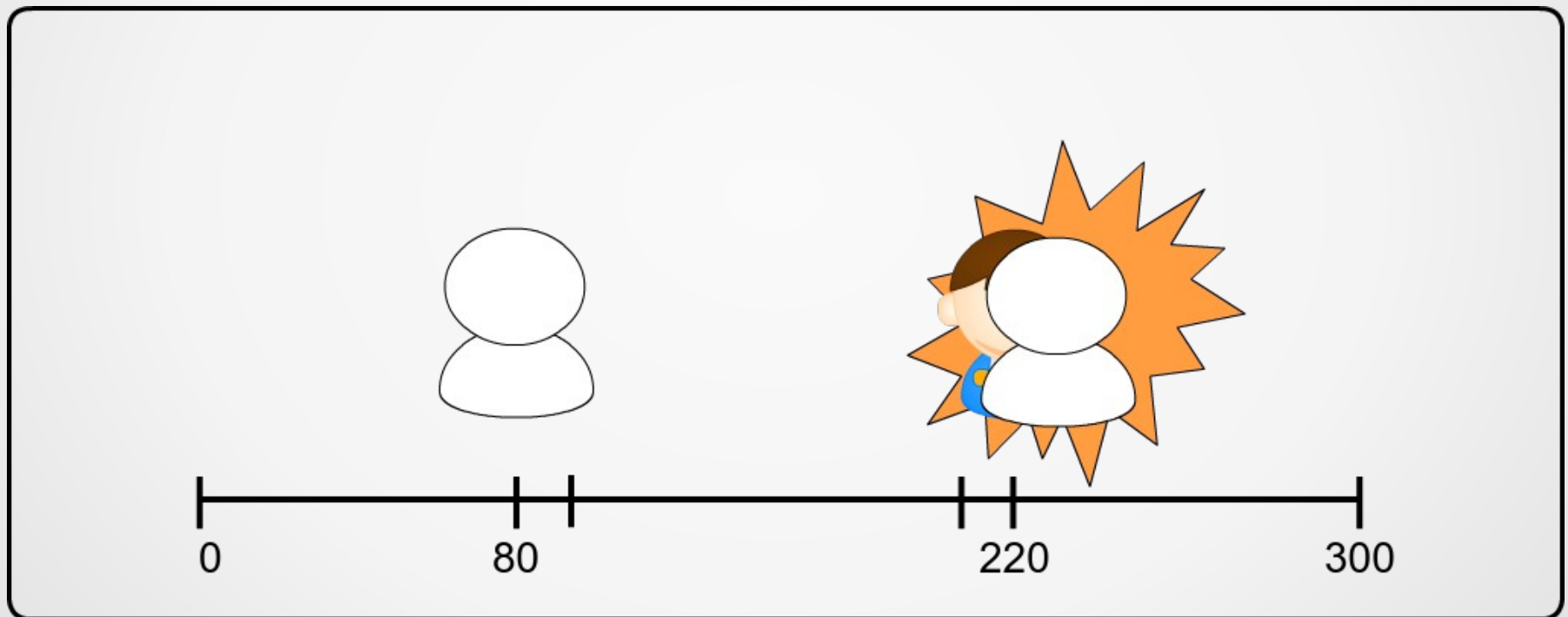
入力例2 ($s=8$)

- 時刻 9.5(火を移す、JOI君移動再開)



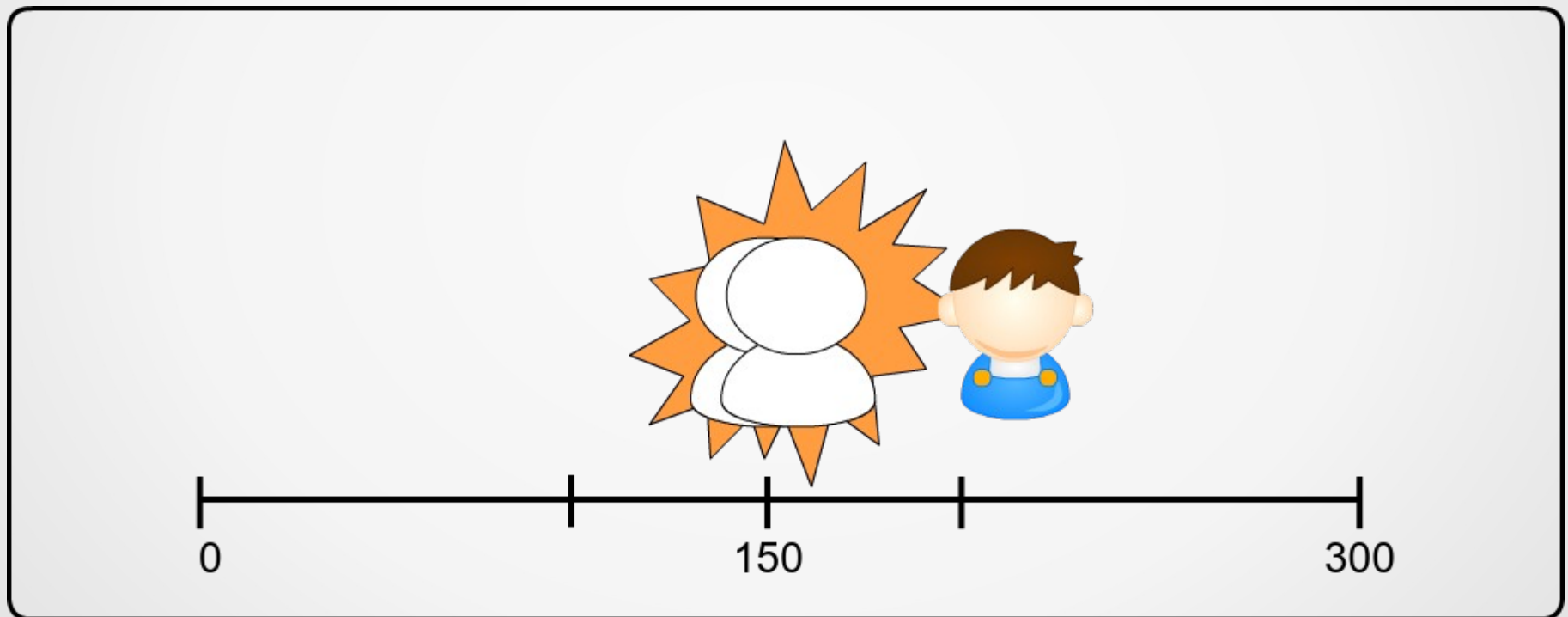
入力例2 ($s=8$)

- 時刻 10(JOI君燃え尽きる)



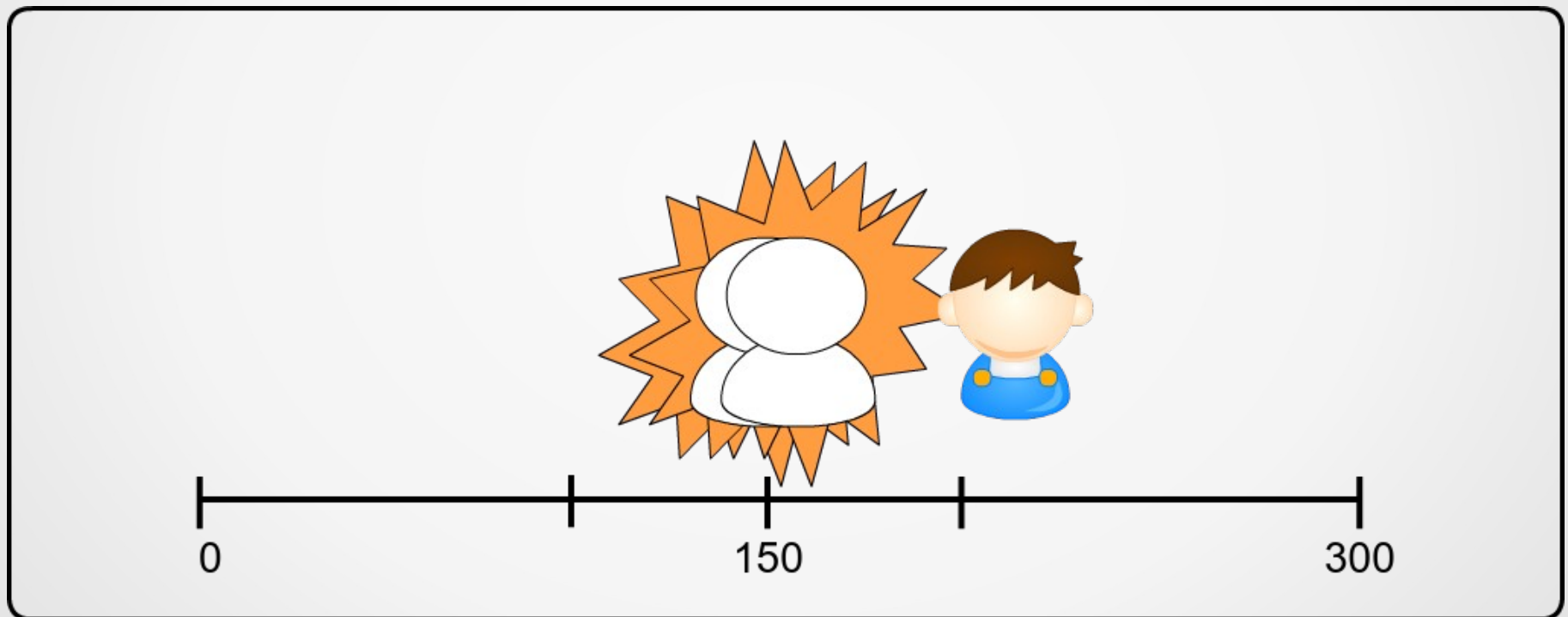
入力例2 ($s=8$)

- 時刻 18.75



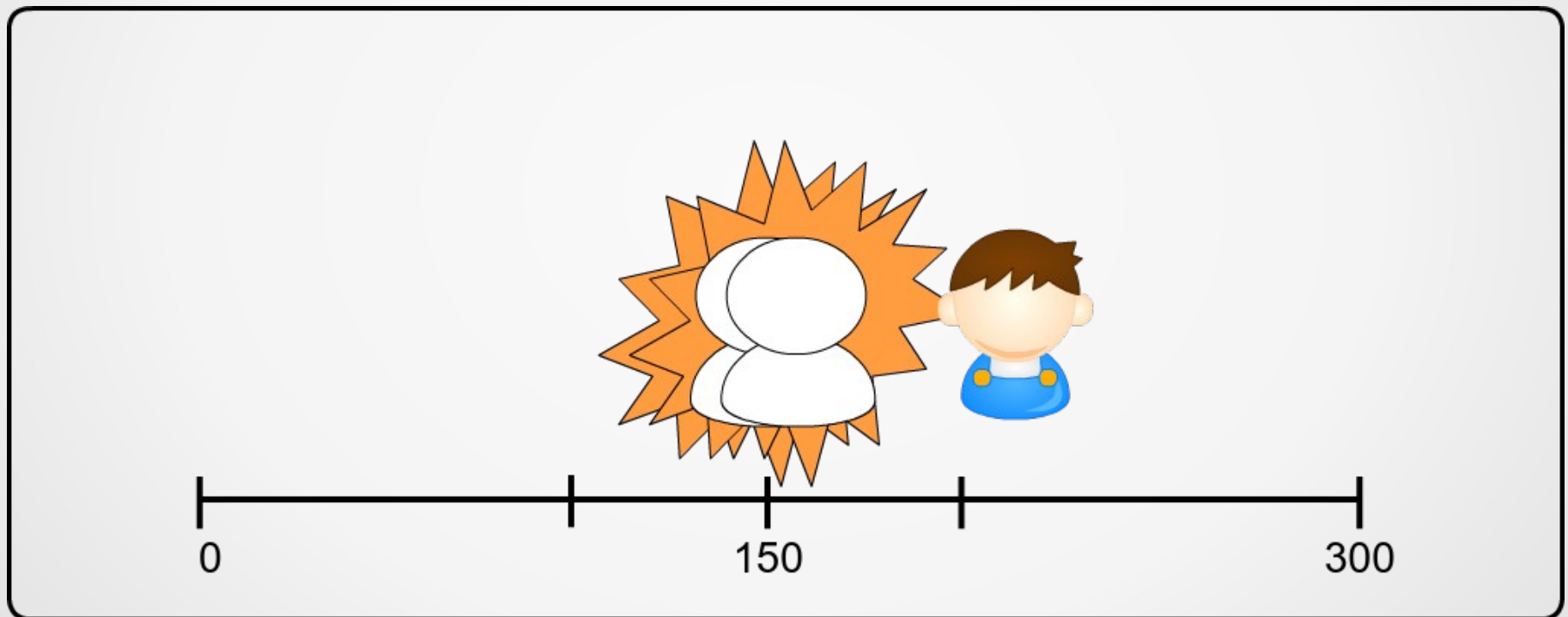
入力例2 ($s=8$)

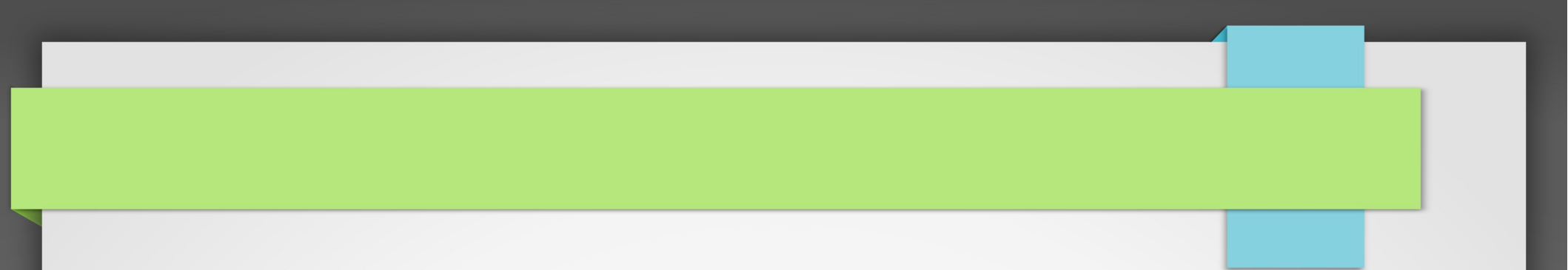
- 時刻 18.75(火を移す)



入力例2 ($s=8$)

- 時刻 18.75(全員に火が移った!)





感想

感想

- 全く同じ配置でも、違う動きをすることがあるようだ
- JOI君たちは立ち止まったり、引き返したりすることがあるようだ
- 同時に複数人の花火に火が付いていることもあるようだ

感想

- 全く同じ配置でも、違う動きをすることがあるようだ
- JOI君たちは立ち止まったり、引き返したりすることがあるようだ
- 同時に複数人の花火に火が付いていることもあるようだ

実は罫！

考察1

- 全く同じ配置でも、違う動きをすることがあるようだ
 - 入力例1は入力例2と同じ動きでも、全員に火を移すことができる

考察2

- JOI君たちは立ち止まったり、引き返したりすることがあるようだ
 - 引き返すことはあるが、立ち止まる必要はない
 - t 秒立ち止まりたければ、 $t/2$ 秒西に進んで、 $t/2$ 秒東に進めば良い
- JOI君たちは常に秒速 s メートルで進んでよい

考察 3

- 同時に複数人の花火に火が付いていることもあるようだ
 - その必要はない
 - 火を移した直後に、火を移しあった2人が同じ方向に進む必要はない
 - それならば別れる時か片方が燃え尽きるときに移せば良い
 - 火を移した直後に、2人が違う方向に進むならば、片方が燃え尽きるまで、火を移すのを延期して良い

教訓

入出力例はたまに意地悪

考察 4(重要)

- 考察3より、着火している花火は、常にひとつである
- まだ着火していない人は、着火している花火に近づかないメリットがない
 - みんな貪欲的に火の付いた花火に近づく
- 火の付いた花火を持ってる人は、次に火を移したい人にむかって近づくのが良い
 - 出会ったら、一緒に並走して、さらに次に火を移したい人に向かって近づくのが良い

考察 4(重要)

- つまり、火をつける順番を決めれば、それぞれの動きも確定できる
- ついに全探索ができるようになった!

小課題0

小課題0 (0点)
 $N \leq 10$

小課題0 ($N \leq 10$) 解法

- 火をつける順番を全通り試す
 - $O((N-1)!)$ 通りの順番がある
- 各順番については貪欲にシミュレート
 - $O(N^2)$ で容易にシミュレートできる
- あわせて $O(N * N!)$
- 速度制限に関する二分法をするのでトータルで
 - $O(N * N! * \log(X))$

0点獲得

小課題1 ($N \leq 20$)

小課題1 (30点)
 $N \leq 20$

小課題1 ($N \leq 20$) 考察

- X,Y,Zの順に人が並んでいるときにYを飛ばしてXがZに火を移すようなことは、意味がない
 - 一度Yに火を移してからZに火を移しても、デメリットがない
- よって、着火したことがある人の集合は、開始時の位置について連続した区間となる
- よって一連の操作は、区間を東もしくは西に1つ拡大することの繰り返しなので、火をつける順番は 2^N 通りしかない

小課題1 ($N \leq 20$) 解法

- 2^N 通りの火の移し方を全部試す
 - あとは小課題0と同様
 - トータルの計算量は
 - $O(N * 2^N * \log(X))$
- 30点獲得!

小課題2 ($N \leq 1000$)

小課題2 (20点)
 $N \leq 1000$

小課題2 ($N \leq 1000$) 考察

- 着火したことのあある人の集合が $[L, R]$ になった直後のことを考える。
- このとき L 番の人は開始時刻からずっと右向きに動いている
- R 番の人でも開始時刻からずっと左向きに動いている
- よって、この時刻 T を決定することができる

$$T = (X[R] - X[L]) / 2 * s$$

小課題2 ($N \leq 1000$) 考察

- このとき $[L, R]$ の人は全員同じ場所にいて並走している
- よって、燃えてる花火のそばには、着火したことがあるかないかを無視すると、全部で $R-L+1$ 個の花火がある
- いままで、同時にひとつしか花火は燃えてない条件で T 秒やってきたので、これ以上花火が増えないとしたときに、あと何秒くらい持つかも求まる

小課題2 ($N \leq 1000$) 考察

- よって以上の情報から
- $[L, R]$ の区間の人がJOI君と同じ場所にいるときに
(燃えている花火と並走しているときに)
- $[L, R+1]$ や $[L-1, R]$ に区間を拡大できるかどうかを
求めることができる

小課題2 ($N \leq 1000$) 考察

- 具体的には $[L, R]$ の間の人が一箇所に集まれる時、その集合を
- $[L, R+1]$ に拡大できる必要十分条件は
 - $(X[R+1] - X[L]) / 2s \leq (R-L+1)t$
 - (LとR+1が出会う時刻) $\leq$ $[L, R+1]$ の花火を燃やし尽くす時間
- $[L-1, R]$ に拡大できる必要十分条件は
 - $(X[R] - X[L-1]) / 2s \leq (R-L+1)t$

小課題2 ($N \leq 1000$) 解法

- この式を元に $[K, K]$ (JOI君のそばにJOI君しかいない状態) から $[1, N]$ (JOI君のそばに全員がいる状態)まで遷移できるかDPでチェックすれば良い
- $dp[L][R] = [K, K]$ から $[L, R]$ に遷移できるかどうか
- $dp[L][R]$ は $dp[L+1][R]$, $dp[L][R-1]$ から $O(1)$ で計算可能
- DPテーブルを埋めるのは $O(N^2)$
- 二分法をつかうとトータルで
 - $O(N^2 \log X)$

50点獲得

小課題3 ($N \leq 100000$)

小課題3 (50点)
 $N \leq 100000$

小課題3 ($N \leq 100000$) 考察

- 遷移が可能かどうかを調べる式を眺めてみる
$$(X[R+1] - X[L]) / 2s \leq (R-L+1)t$$
- $X[R+1] - 2st(R+1) \leq X[L] - 2stL$
- $x[i] = X[i] - 2sti$ とすると
- $[L, R+1]$ に遷移できる $\Leftrightarrow x[L] \geq x[R+1]$
- 同様に
- $[L-1, R]$ に遷移できる $\Leftrightarrow x[L-1] \geq x[R]$

小課題3 ($N \leq 100000$) 考察

- $x[i] = X[i] - 2sti$ とすると
- $[L, R]$ に遷移できる必要十分条件は
 - $x[L] \geq x[R]$
- つまり以下のように簡単な問題に置き換えることができる

小課題3 ($N \leq 100000$) 言い換え

- $x[i] = X[i] - 2sti$ とする
- $L=R=k$ から開始して
 $x[L] \geq x[R]$ を保ちながら
以下の操作で $L=1, R=N$ に遷移できるか？
- 操作
 - L をデクリメント、あるいは R をインクリメントする

入力例1

- $N=3, K=2, T=50$
- $X = [0, 200, 300]$
- $s = 2$ とすると
- $x = [0, 0, -100]$
- $[2,2] \rightarrow [1,2] \rightarrow [1,3]$ や $[2,2] \rightarrow [2,3] \rightarrow [1,3]$
という遷移ができる

入力例2

- $N=3, K=2, T=10$
- $X = [0, 200, 300]$
- $s = 8$ とすると
- $X = [0, 40, -20]$
- $[2,2] \rightarrow [2,3] \rightarrow [1,3]$ という遷移ができる

やったぜ！

わかりやすい問題に
言い換えできた！

新たな

考察

考察

- 保ちたいものは $x[L] \geq x[R]$
- よって $x[L'] \geq x[L]$ となる $L' > L$ があるとして、 $[L, R]$ から $[L', R]$ に直接じゃなくても移動できるなら、移動しても問題ない
- $x[R] \geq x[R']$ となる $R > R'$ があった時も同様
- この貪欲を続けていくことを考える

考察

- $L \geq K$ の中で $x[L]$ が最大と成る L を GL
 $K \geq R$ の中で $x[R]$ が最小と成る R を GR とする
- 先ほどの貪欲な操作をしつづけたときに、 $[GL, GR]$ に到達したら、もうこれ以上同じ操作はできない
- 逆に $[GL, GR]$ に到達できなかったら？
 - $\rightarrow$ どのような遷移をしても $[1, N]$ に到達できない
- $[K, K]$ から $[GL, GR]$ に遷移できなければ $[1, N]$ にも到達できない

考察

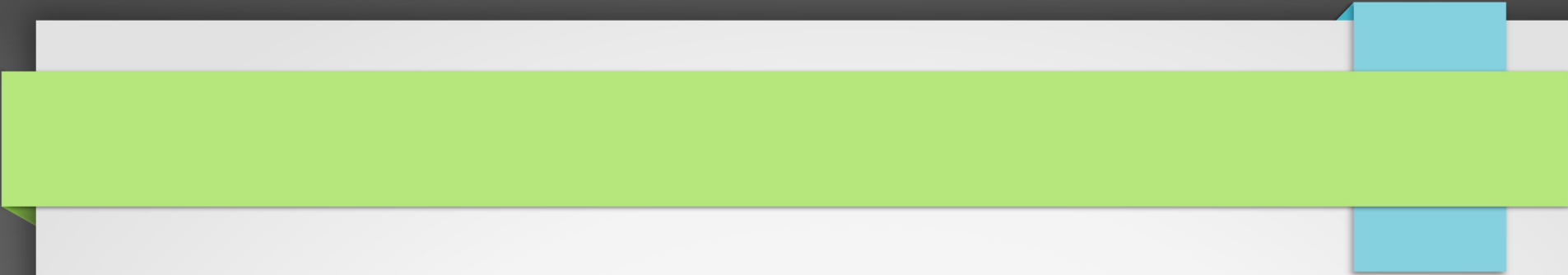
- 遷移は逆に辿ることができるので、この問題は以下のように言い換えることもできる。
- $L=1, R=N$ から開始して、以下の操作を繰り返して $[K, K]$ に遷移できるか？
 - L をインクリメント、あるいは R をデクリメント
- 先ほどと同様の貪欲な操作で同じ議論ができる
- $[1, N]$ から $[GL, GR]$ に遷移できなければ $[K, K]$ まで遷移できない

考察まとめ

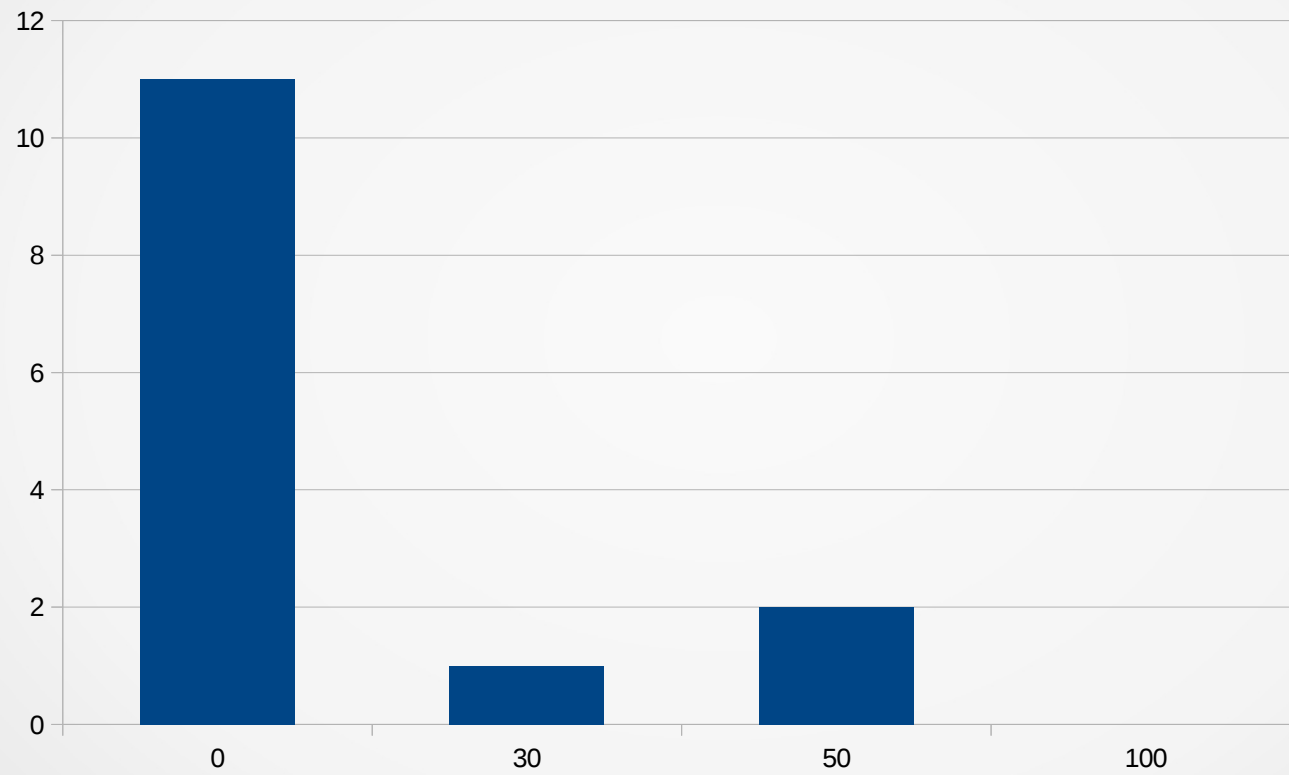
- Lに関しては増やせる限り増やす、Rに関しては減らせる限り減らすという貪欲な遷移を考える
- 以下の2つは同値
 - $[K, K]$ から $[L, R]$ に行けるかどうか
 - $[K, K]$ からも $[1, N]$ からも貪欲な遷移で $[GL, GR]$ に到達できる
- 後者は $O(N)$ でシミュレートすることができる

満点解法

- 言い換えた後の問題で
- $[K, K]$ と $[1, N]$ の両方から $[GL, GR]$ に遷移できるかどうかを調べる
- それが、元の問題で全員に火を着火できるかと同値
- シミュレーションは $O(N)$ でできる
- 二分法と合わせてトータルで
- $O(N \log X)$
 - 100点獲得!



得点分布





切符の手配 (Arranging Tickets) 解説

三谷庸

目次

- 問題概要
- 指数時間 (小課題 1)
- 多項式時間 (小課題 2, 3, 4)
- C が大きい場合 (小課題 5: 満点)

問題概要

- N 個の駅が円形に並んでいる
- 切符 i を使うと駅 i と駅 $i + 1$ の間を移動できる
- 駅 A_i から駅 B_i に行きたい人が C_i 人いる
- 各種類の切符の枚数の最大値を最小化したい

問題概要

- $N \leq 200,000$
- $M \leq 100,000$
- $C_i \leq 1,000,000,000$

問題概要

- $N \leq 200,000$
- $M \leq 100,000$
- $C_i \leq 1,000,000,000$

- 小課題 1, 2, 3, 4 では $C_i = 1$
- 小課題 1, 2, 3 では N, M が小さい

指数時間（小課題 1：10 点）

- 各人についてどちらの向きに回るかを試す
- 回り方を決めると、各切符の必要枚数がわかる
- 計算量 $O(2^M M^2)$ （または $O(2^M M)$ ）
 - $N = O(M)$ とした

多項式時間（小課題 2, 3, 4）

- 問題をもう少し考えやすい形に言い換えることから始めます

円を線分にする

- 円形に並んでいると考えにくいので線分上で考えたい
- 駅 1 と駅 N の間で切り開いて線分にする
-> それぞれの人の移動方法が一意に定まる

この状態から、何人かについて移動方向を逆にして目的の枚数を最小化する

言い換え後の問題

- 人が M 人いる
- 最初、人 i は切符 $[A_i, B_i)$ を使おうとしている (A, B の大小関係はもとの入力では両方あるので適当に入れ替える)
- 何人か選んで、区間を逆にする。つまり、番号が $[A_i, B_i)$ 以外の切符をすべて使うようにする
- 切符の枚数の最大値を最小化するにはどの区間を選べば良いだろうか

$[A_i, B_i)$ とは、「 A_i 以上 B_i 未満」

答えで二分探索できるか？

- (切符の枚数の)「最大値の最小値」と言われているので二分探索したい
- 本当に二分探索できる問題であるか確かめる
- 「条件を満たす最小の・・・」と言われているとき、
3: 満たす 4: 満たさない 5: 満たす
みたいな部分があると二分探索できない

答えで二分探索できるか？

- (切符の枚数の)「最大値の最小値」と言われているので二分探索したい
- 本当に二分探索できる問題であるか確かめる
- 「条件を満たす最小の・・・」と言われているとき、
3: 満たす 4: 満たさない 5: 満たす
みたいな部分があると二分探索できない
- この問題の場合、二分探索できる

二分探索できることの証明

- 最初の状態での切符の枚数の最大値を m とする
- 区間 $i_1, i_2, \dots, i_k$ を逆にすると最大値の最小値 m^* が得られるとする
- 適切に区間を逆にすることで最大値が $m^*, m^* + 1, \dots, m$ のすべてをとりうることをいえばよい

二分探索できることの証明

- 区間 $i_1, i_2, \dots, i_j$ を逆にしたときの最大値を m_j とする
- このとき、 $|m_j - m_{j+1}| \leq 1$ が成り立つ
 - 区間 1 つを逆にしたとき各切符の枚数の変化は $+1 / -1$ のいずれかであるから
- よって、 $m = m_0, m_1, m_2, \dots, m_k = m^*$ の中に $m^*, m^* + 1, \dots, m$ はすべて登場する

二分探索できることの証明

- 区間 $i_1, i_2, \dots, i_j$ を逆にしたときの最大値を m_j とする
- このとき、 $|m_j - m_{j+1}| \leq 1$ が成り立つ
 - 区間 1 つを逆にしたとき各切符の枚数の変化は $+1 / -1$ のいずれかであるから
- よって、 $m = m_0, m_1, m_2, \dots, m_k = m^*$ の中に $m^*, m^* + 1, \dots, m$ はすべて登場する
- 二分探索してよいことがわかった

逆にする区間が満たす条件

- たとえば、区間 $[2, 4)$ と区間 $[5, 6)$ の両方を逆にすることはあるだろうか
- 切符の枚数の変化を観察

逆にする区間が満たす条件

区間 $[2, 4)$ と区間 $[5, 6)$ の両方を逆にしてみる

切符	1	2	3	4	5	6	7
							
枚数変化	+2	0	0	+2	0	+2	+2

逆にする区間が満たす条件

- 全部の切符の枚数が増えている！
- 求めた区間の集合に $[2, 4)$ と $[5, 6)$ が含まれていれば、それら両方を逆にするのをやめることで枚数が改善（正確には、悪くなることはない）
- このようなものは含まれないとして良い

逆にする区間が満たす条件

- より一般に、 $a < b \leq c < d$ について、区間 $[a, b)$ と区間 $[c, d)$ の両方を逆にすることはない

逆にする区間が満たす条件

- より一般に、 $a < b \leq c < d$ について、区間 $[a, b)$ と区間 $[c, d)$ の両方を逆にすることはない
- したがって、ある t が存在して、逆にする区間はすべて t を含む
 - きちんと証明したければ、右端が一番左にある区間と左端が一番右にある区間に注目する

このことに気づくと、多項式時間解法はすぐそこです（小課題 2）
さらにいろいろ考察して計算量を落としていけば点数が増えます

逆にする区間の選び方

- t を決めただけでは逆にする区間を選ぶのは難しい
- 他に以下の2つのパラメータを決める
 - 最終的な切符の枚数の最大値 m
 - 逆にする区間の個数 n

逆にする区間の選び方

- 以下の 3 つの条件を満たす区間の選び方が存在するか判定できればよい
 - 逆にする区間が含むべき値 t
 - 最終的な切符の枚数の最大値 m
 - 逆にする区間の個数 n
- 貪欲に選んでゆけばよい

逆にする区間の選び方

- 各 $i \leq t$ について、「左端が i 以下である区間を $need_i$ 個以上逆にする必要がある」ような $need_i$ を求める
 - これを求めるために m, a が必要
 - $need_t = n$ とする
- 区間の右端の値を管理する集合 (multiset) S を用意する

そして、

以下のように処理すれば良い

```
for l = 0 to t
```

左端が l であるような各区間 $[l, r)$ について

S に r を追加

すでに逆にした区間の個数が $need_l$ より少ない場合

S のうち大きい方から必要なだけ取ってきてそれらに対応する区間を逆にする (S に含まれる要素が足りなければ「存在しない」と返す)

最後に、それでうまくいっているか確かめる

逆にする区間の選び方

- 左端が左にあるものから「選べる」ようにしていき、「選ばなければならない」ときは右端が右のものを優先

逆にする区間の選び方

正当性を証明する

- 選び方が存在しないときに「存在する」と言ってしまうことはない
 - 最後に確かめている
- 存在するときに「存在しない」と言ってしまうことはない
 - 求めた選び方では、どの $i > t$ についても、切符 i の枚数はすべての切符 $i \leq t$ の枚数が m 以下となるような選び方のうち最小値となっている

証明できた

小課題 2 : 35 点

ここまでの考察で多項式時間で解けるようになる

- 答えで二分探索 $O(\log M)$
- 二分探索の各ステップにおいて、
 - t, n を決めるのに $O(M^2)$
 - 決めたあと $O(M \log M)$

ただし $N = O(M)$ とした

全部で $O(M^3 (\log M)^2)$

$M \leq 300$ なら間に合う

小課題 2 のアルゴリズムのうち消せそうな部分は

- t をすべて試す
- n をすべて試す

の 2 つ

小課題 2 のアルゴリズムのうち消せそうな部分は

- t をすべて試す
- n をすべて試す

の 2 つ

実際、消せる

n をすべて試さなくてすむようにすることから考える

逆にする区間の個数

t のとり方に制限をつける

まず、逆にする区間を決めたとき、 a, b を以下のように定める

- $a_i =$ (最初の状態での切符 i の必要枚数)
- $b_i =$ (逆にしたあとの状態での切符 i の必要枚数)

逆にする区間の個数

t は以下を満たすようにとることにする

- 逆にする区間すべての共通部分を $[l, r)$ とすると、 b_t は $b_l, b_{l+1}, \dots, b_{r-1}$ のうちの最大値である

逆にする区間の個数

(命題) 逆にする区間の最適な選び方であって以下が成り立つものが存在する

- $b_t = \max b_i$ または $b_t = \max b_i - 1$

(証明)

最適な選び方のうち b_t が最大となるものが上を満たしていなかったとする
選んだ区間のうち左端が最も右にあるものと右端が最も左にあるものを取り、それらを選ぶのをやめる

$\max b_i$ の値は変わらないまま b_t の値は 1 または 2 増える (証明終)

逆にする区間の個数

- よって、 m, t を決めたあと、 n としては $a_t - m, a_t - (m - 1)$ だけを試せばよい

小課題 3 : 20 点

- 小課題 2 のアルゴリズムより $O(M)$ だけ計算量が良くなった
- 計算量 $O(M^2 (\log M)^2)$
- $M \leq 3000$ なら間に合う

次に、 t をすべて試さなくてもよいようにすることを考える

次に、 t をすべて試さなくてもよいようにすることを考える

b_t は b の中で大きい $\rightarrow a_t$ も a の中で大きそう

次に、 t をすべて試さなくてもよいようにすることを考える

b_t は b の中で大きい $\rightarrow a_t$ も a の中で大きそう

$b_i - a_i$ の値は i が t に近いほど小さくなり、特に選ばれたすべての区間に含まれる i で最小になることに注意

t の候補

(命題)

$a_t = \max a_i$ が成り立つ

t の候補

(証明)

- j を選んだ区間の共通部分に含まれない値とするとき

$$a_j > a_t$$

とならないことを示せばよい

- 上の式を書き換えて、

$$a_j \geq a_t + 1$$

t の候補

- j は選んだ区間の共通部分に含まれないことから、
$$b_t - a_t + 1 \leq b_j - a_j$$

- $a_t + 1 \leq a_j$ と両辺加えて

$$b_t + 2 \leq b_j$$

t の候補

- j は選んだ区間の共通部分に含まれないことから、

$$b_t - a_t + 1 \leq b_j - a_j$$

- $a_t + 1 \leq a_j$ と両辺加えて

$$b_t + 2 \leq b_j$$

- 一方 $b_t = \max b_i$ または $b_t = \max b_i - 1$ だったので矛盾

t の候補

- j は選んだ区間の共通部分に含まれないことから、

$$b_t - a_t + 1 \leq b_j - a_j$$

- $a_t + 1 \leq a_j$ と両辺加えて

$$b_t + 2 \leq b_j$$

- 一方 $b_t = \max b_i$ または $b_t = \max b_i - 1$ だったので矛盾

$a_t = \max a_i$ の証明終

t の候補

- まだ t の候補の個数を絞れていない
- a_i の値が最大となる i がたくさんあると困る

t の候補

(命題)

$m = \max a_i$ とし、 $a_i = m$ となる最小および最大の i をそれぞれ l, r とする

このとき、 $t = l$ または $t = r$ である

- つまり、 t は a が最大となる添字のうち左端 or 右端

t の候補

(証明)

背理法を使う

t が $l < t < r$ を満たすとする

($a_t = m$ はすでに示されているので、 $t < l$ や $t > r$ となることはない)

t の候補

(補題) 選んだ区間に含まれる $[x, y)$ であって $l < x < y \leq r$ を満たすものは存在しない

(証明)

$[x, y)$ を逆にするのをやめる。そのときの各切符の必要枚数を $b'_1, \dots, b'_N$ とする

このとき、以下の式が成り立つ

t の候補

(証明つづき)

- $b'_t = b_t + 1$
- $b'_l = b_l - 1$
- $b'_i \geq b'_t$

(最後の式は $b'_i - a_i$ も $b_i - a_i$ と同様 $i = t$ で最小となることから得られる)

よって、 $b_l - 1 \geq b_t + 1$ 、 $b_l \geq b_t + 2$

$b_t = \max b_i$ または $b_t = \max b_i - 1$ であつたので矛盾

(補題の証明終)

t の候補

- $t = l$ または $t = r$ の証明に戻る

逆にする区間 $[x_1, y_1), [x_2, y_2)$ が存在して、

- $l < x_1$
- $y_2 \leq r$

の両方が成り立つ

これらを逆にするのをやめるとどうなるか
やめたときの各切符の必要枚数を b' と書く

t の候補

(補題) $[x_1, y_1), [x_2, y_2)$ を選ぶのをやめても $\max b$ が悪くなることはない。
つまり、
 $\max b_i \geq \max b'_i$ が成り立つ

(証明)

$\max(b_l, b_{l+1}, \dots, b_r) = \max(b_l, b_r)$ が成り立つ。 b' も同様

(理由: $\max(a_l, \dots, a_r) = a_l, a_r$ であり、 $b_i - a_i$ は i が t から遠くなるほど大きくなる)

よって、 $\max b = \max(i \leq l, i \geq r) b$ となる。 b' も同様

この範囲の i について $b_i \geq b'_i$ が成り立つ

(補題の証明終)

t の候補

- したがって、選んだ区間すべての共通部分に l も r も含まれない場合、選ぶ区間を減らすことができる
- すべての共通部分に l か r が含まれるようになるまで続ければ良い

$t = l$ または $t = r$ の証明終

小課題 4 : 20 点

- 小課題 3 のアルゴリズムで t をすべて試していたのを 2 (or 1) 通りだけ試すようにする
- 計算量 $O(M (\log M)^2)$ となり、間に合う

小課題 5 : 15 点

- C の値が大きい
- 小課題 4 と同じことを高速に行えば良い

小課題 5 : 15 点

以下の操作ができれば良い

- 場所 i にものを x 個置く
- ものを右から x 個とる（とって削除する）

小課題 5 : 15 点

以下の操作ができれば良い

- 場所 i にものを x 個置く
- ものを右から x 個とる (とって削除する)
- Segment Tree を使えばできる (詳細略)

得点分布





壊れた機器 (Broken Device) 解説

伊佐 碩恭 (HIR180) – IOI 2014日本代表

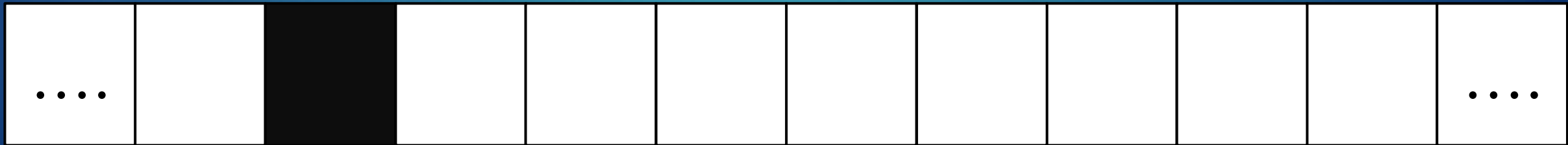
問題概要

- ▶ AnnaはBrunoに非負整数 X ($\leq 10^{18}$)を伝えるために長さ N ($=150$)の0/1列を送る。
- ▶ しかし、0/1列を送ることができるのは1回のみで、機器の故障のため特定の桁についてはAnnaの設定した値に関わらず0になってしまう。
- ▶ 機器の故障箇所はAnnaには分かるがBrunoには分からず、故障箇所の個数は K (≤ 40)である。
- ▶ 以上の操作を Q (≤ 1000)回繰り返す。

$L^* = 1$ (8点)

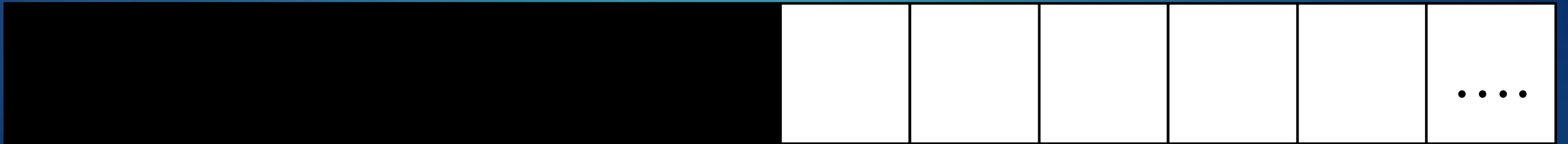
- ▶ 1個だけ壊れている場合を考える。
- ▶ X は高々 10^{18} なので、 $2^{59} < 10^{18} < 2^{60}$ に注意すると150個の桁のうち60個使えば良いことがわかる。
- ▶ 2つに分割して全部生きている方のみを使えばよい。

100



$L^* = 1$ (8点)

左半分は全く使わない。(すべて0に設定)



$L^* = 1$ (8点)

右半分はすべての桁が生きているので、普通に送れる。

					1	0	1	0	1	
--	--	--	--	--	---	---	---	---	---	------

$L^* = 15$ (41点)

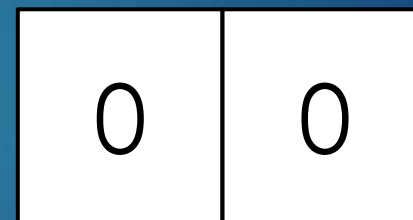
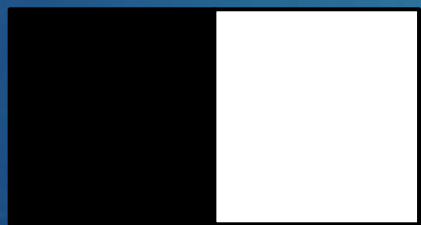
- ▶ さっきの解法はあまりに無駄が多すぎる。
- ▶ 150桁を2桁 $\times$ 75 に分けることを考える。
- ▶ 壊れている桁は高々15個だから $75 - 15 = 60$ 組は2桁とも生きているはず。
- ▶ 4枚前のスライドにも書いた通りXを伝えるには60個の0/1を送ればよいため、1組につき1個送ればよさそう。

$L^* = 15$ (41点)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は左に1、右に送りたい桁を設定する。

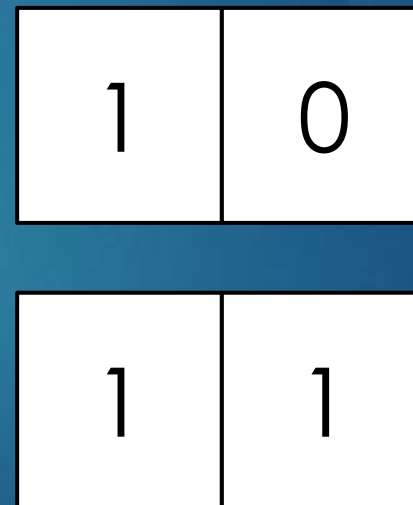
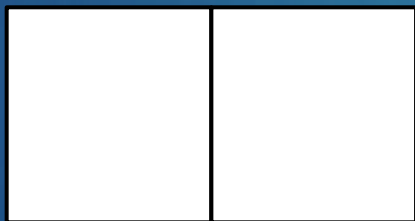
$L^* = 15$ (41点)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。



$L^* = 15$ (41点)

- ▶ 両方生きている場合は左に1、右に送りたい桁を設定する。



$$L^* = 15 \text{ (41点)}$$

- ▶ こうすることで、少なくとも $75 - 15 = 60$ 桁の情報を送れるため、 X を正しく伝えることができ、41点を得られる。

$L^* = 20$ (51点)

- ▶ さっきは2桁 × 75に区切ったが、今度は3桁 × 50に区切ってみる。
- ▶ 組の中で1つでも壊れている場合は無条件で「000」を送ることにする。
- ▶ 全て生きている場合は1番左に1、残り2桁に送りたい桁を設定する。

$$L^* = 20 \text{ (51点)}$$

- ▶ すると、Xに関する少なくとも $(150 \div 3 - 20) \times 2 = 60$ 桁の情報を送ることができ、51点が得られる。

$L^* = 24$ (59点)

- ▶ さっきの場合、3桁中1つしか壊れていない場合でも他の2つも無効化してしまっていて無駄が多い。
- ▶ 3桁中1つしか壊れていないような組も情報を持たせたい。
- ▶ 例えば以下のように3桁の0/1列と0,1,2を対応させる:
「001」 「110」 : 0
「010」 「101」 : 1
「100」 「011」 : 2

$L^* = 24$ (59点)

- ▶ このとき、3桁中高々1つしか壊れていないならば0,1,2のいずれが割り当てられても、対応された3桁の0/1列のどちらかは送ることができる。
- ▶ 最悪ケースでは50組中 $24 \div 2 = 12$ 組潰れる(3桁中2つずつ壊れる)ため、少なくとも $(50-12=)$ 38個の0,1,2を送ることができるため、 $3^{37} < 10^{18} < 3^{38}$ よりXを正しく送ることができ、59点が得られる。

$$L^* = 37 \text{ (85点)}$$

▶ $L^* = 15$ の解法をもう一度振り返ってみましょう

$L^* = 15$ (41点) (再掲)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は左に1、右に送りたい桁を設定する。

$L^* = 15$ (41点) (再掲)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は左に1、右に送りたい桁を設定する。
(「10」「11」)

$L^* = 15$ (41点) (再掲)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は左に1、右に送りたい桁を設定する。
(「10」「11」) ← ? ? ? ? ? ? ?

$L^* = 37$ (85点)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は「10」「11」「01」を送る

$$L^* = 37 \text{ (85点)}$$

- ▶ 「10」「11」「01」それぞれに0,1,2を割り振れば、少なくとも $150 \div 2 - 37 = 38$ 個の0,1,2を送れるため、正しくXを送ることができ、85点が得られる。

$L^* = 40$ (100点) (解法1)

▶ $L^* = 20$ の解法をもう一度振り返ってみましょう

$L^* = 20$ (51点) (再掲)

- ▶ さっきは2桁 × 75に区切ったが、今度は3桁 × 50に区切ってみる。
- ▶ 組の中で1つでも壊れている場合は無条件で「000」を送ることにする。
- ▶ 全て生きている場合は1番左に1、残り2桁に送りたい桁を設定する。

$L^* = 40$ (100点) (解法1)

- ▶ これまでと同様に3桁中2つ生きている組も情報を送ることを考える。
- ▶ 3桁の0/1列とXに関する情報をうまく対応させることで、3桁とも生きている組は2桁, 2つ生きている組は1桁の情報を必ず送ることができる例を示す。

$L^* = 40$ (100点) (解法1)

▶ 対応の例

「000」：何も情報を送らない

「001」：「1」を送る

「010」：「0」を送る

「011」：「10」を送る

「100」：「00」を送る

「101」：「01」を送る

「110」：「1」を送る

「111」：「11」を送る

$L^* = 40$ (100点) (解法1)

- ▶ 3桁とも使えるなら当然2桁の情報を送れる。
- ▶ 1つ壊れている時でも「1」を送ることはできる。(「001」と「110」がどちらも「1」を表し、どちらかは送ることができるから)
- ▶ 「0」を送りたいときは、「010」が対応するので、真ん中の桁が壊れていなければOKで、その時は「100」と「101」がそれぞれ「00」と「01」に対応しているから2桁の情報を送ることができる。

$L^* = 40$ (100点) (解法1)

- ▶ この時3桁とも生きている時は2桁、2つ生きている時は1桁、2つ以上壊れている時は0桁の情報を送ることができる。
- ▶ 壊れていなければ $150 \div 3 \times 2 = 100$ 桁の情報を送ることができる、1桁壊れるとき高々1桁減るだけだからXに関する情報が最低でも $100 - 40 = 60$ 桁送れるため、上手くいく。

$L^* = 40$ (100点) (解法2)

▶ $L^* = 37$ の解法をもう一度振り返ってみましょう

$L^* = 37$ (85点) (再掲)

- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。
- ▶ 両方生きている場合は「10」「11」「01」を送る

$L^* = 37$ (85点) (再掲)

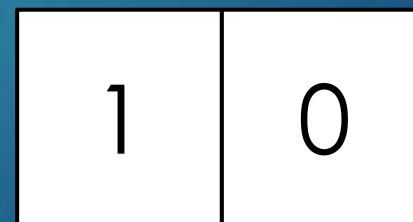
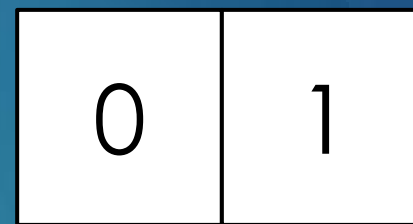
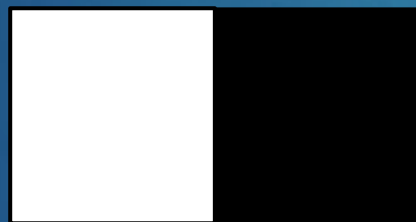
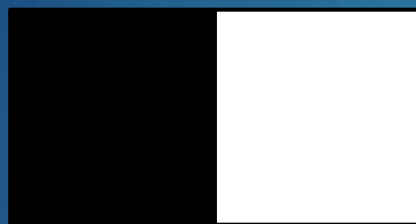
- ▶ 組の中で1つでも壊れている場合は無条件で「00」を送ることにする。

$L^* = 40$ (100点) (解法2)

- ▶ 2桁中片方生きているのにそれを潰すのは勿体ない

$L^* = 40$ (100点) (解法2)

- ▶ もし「01」か「10」を送りたいタイミングで、丁度良く2桁のうち片方が壊れた組が来てくれると想定より多く情報を送ることができる



$L^* = 40$ (100点) (解法2)

- ▶ 最悪ケースでは片側のみが潰れた組が40組存在し、もしランダムにXが選ばれるならば確率1/3で都合のよい値に割り当たる。(「01」「10」「11」のうちどれか1つ)
 - ▶ $L^* = 40$ のとき、 $150 \div 2 - 40 = 35$ 個の0,1,2を送れ、さらにランダムな場合には上の通り $40 \div 3 \div 13$ 個の0,1,2を送れる
- => 0,1,2は38個あればXが表現できたし、余裕そう？

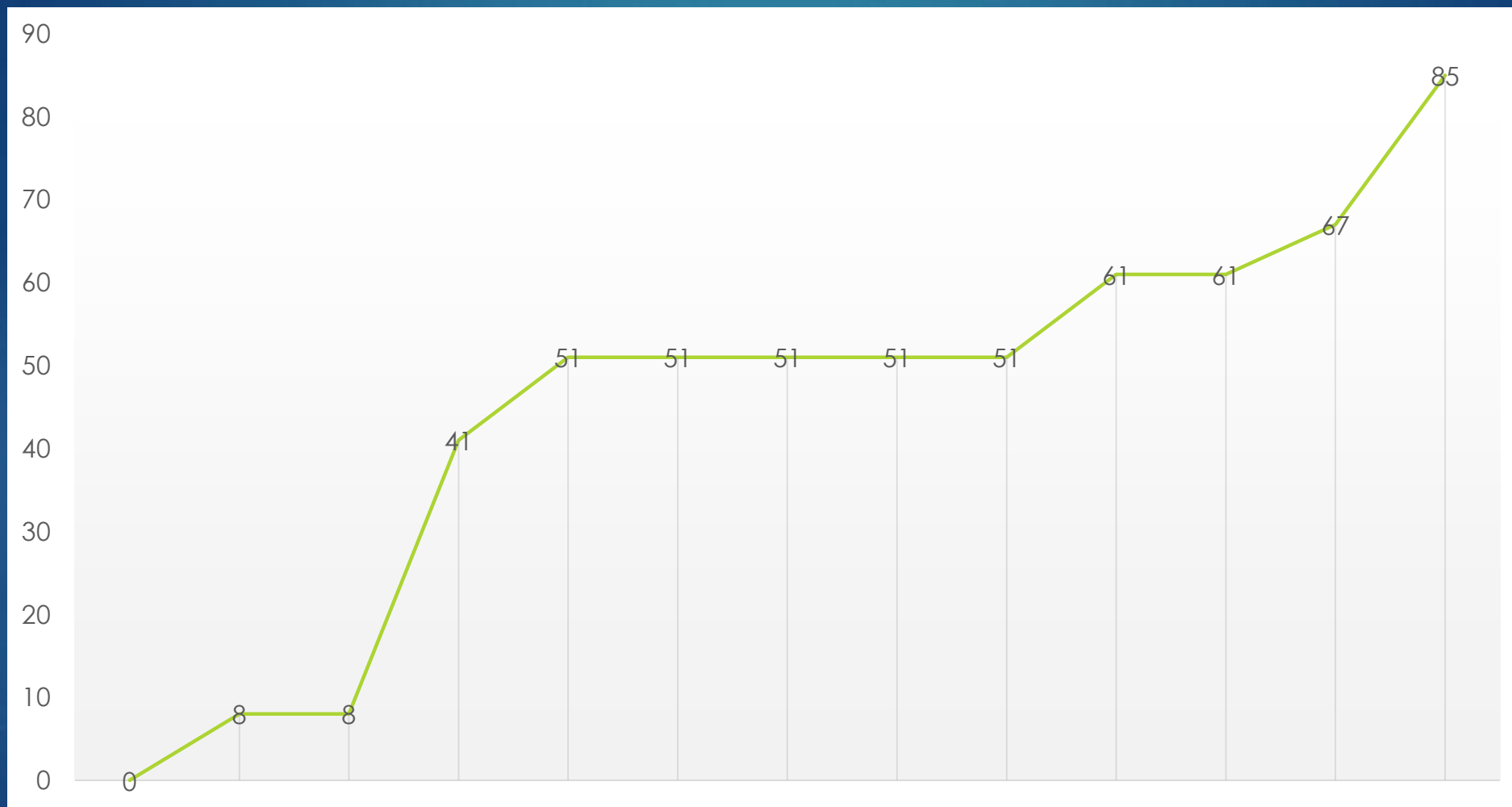
$L^* = 40$ (100点) (解法2)

- ▶ 実際はそのままでは悪意のあるケースで落ちる (例えば $X = 3^{38} - 1$ とすると、全ての組に「11」が割り振られてしまい、片方が潰れた組は全く情報を送れず、終了)
- ▶ 対処法として、適当な正整数 N をAnnaとBrunoの間で共有しておき、Annaは $X \wedge N$ ($\wedge$ はxor)を送り、Brunoは受け取った値と N でxorを取って答える、というものがあり、こうすることで X はランダムとみなせるようになり、38桁以上送ることができそう。

$L^* = 40$ (100点) (解法2)

- ▶ また、「2つとも桁が壊れている組」が3個以上存在すれば何も考えずにXを送ることができるので、悪意をもって作られたケースに対しても、桁の番号をシャッフルしておくことで(シャッフルの仕方はAnnaとBrunoで共有しておく)、壊れた箇所をランダムに配置できるとみなせる。
- ▶ 以上の乱択的な方針で、高確率で通すことができる。

得点分布



鉄道旅行 (Railway Trip) 解説

増田隆宏

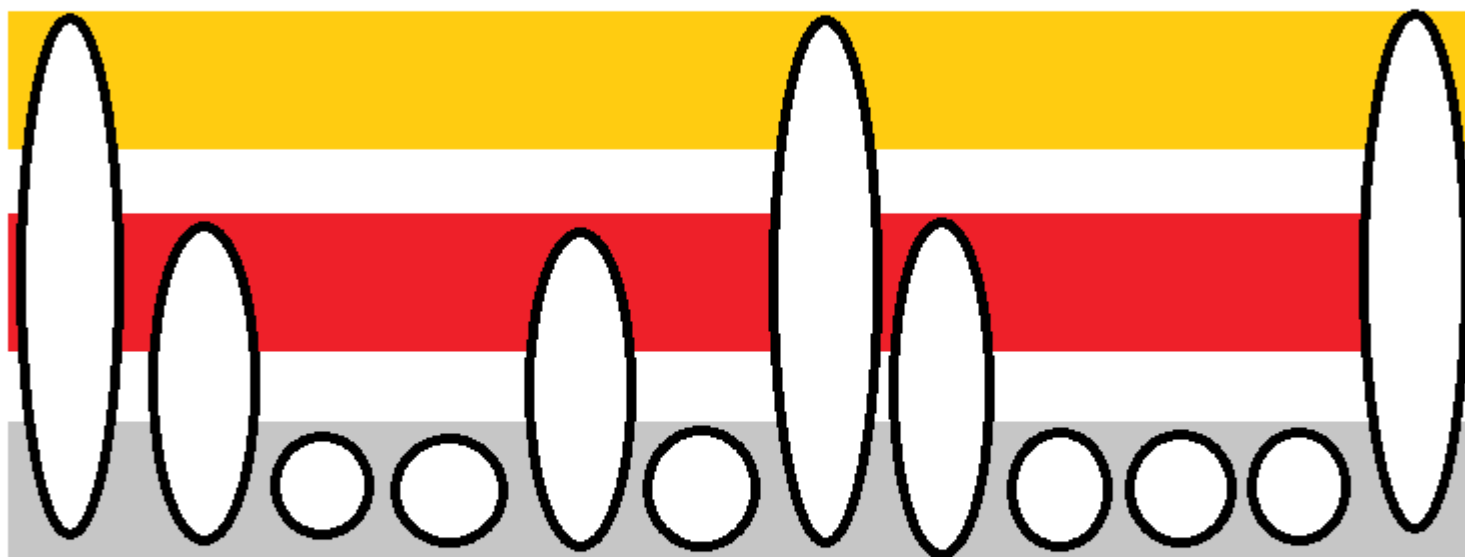
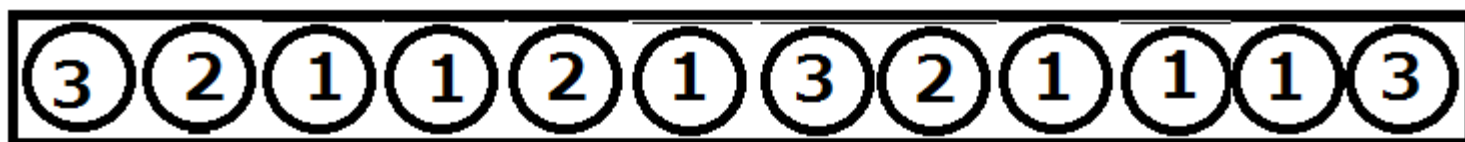
問題概要

- 一直線上に並んだ N 駅を結んだ路線がある
- それぞれの駅には $1 \sim K$ のランクがある
- 種別 j の列車はランク j 以上の駅のみに停車
- 各クエリごとに、出発駅と到着駅が与えられるので、移動するための途中の停車の回数の最小値を求めよ

こんな路線があったら



こんな路線があったら



途中の停車の回数の最小値とは

- 途中停車なしで行けるすべての二駅の組み合わせをコスト1の辺で結ぶ
- 「途中の停車の回数の最小値」 = 「出発駅から到着駅までの最小距離」 - 1
- なので
- これからは「出発駅から到着駅までのコストの最小値」を考えます

小課題 1 (5 点)

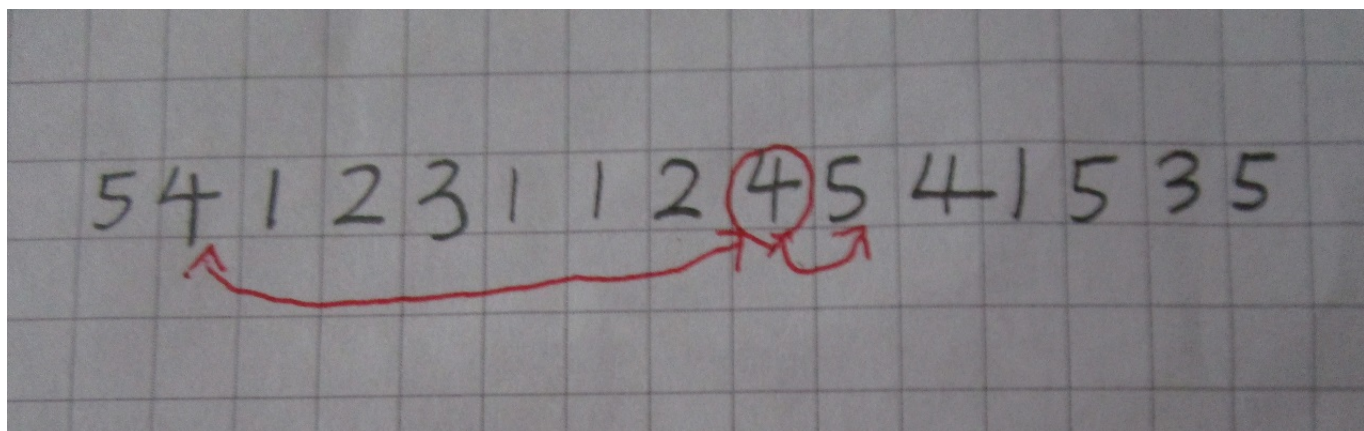
- $N \leq 100$
- $K \leq 50$
- $Q \leq 50$
- 1 ~ K の全ての種別において停車駅を全て求め、辺を張ると、ワーシャルフロイド法で解けます

小課題 2(5+15=20 点)

- $Q \leq 50$
- 辺を $O(NK)$ 本も張れない
- 使う辺を減らしてみる

小課題 2(5+15=20 点)

- 急行ー>各駅ー>急行 みたいなのは意味がない
- こう考えると、すべての駅において、自分より左にありレベルが自分以上となる最も右の駅、自分より右にありレベルが自分以上となる最も左の駅との間に双方向の辺を張るとよいことがわかる



- 辺を張る方法はスタックで $O(N)$ がおすすめ

小課題 2(5+15=20 点)

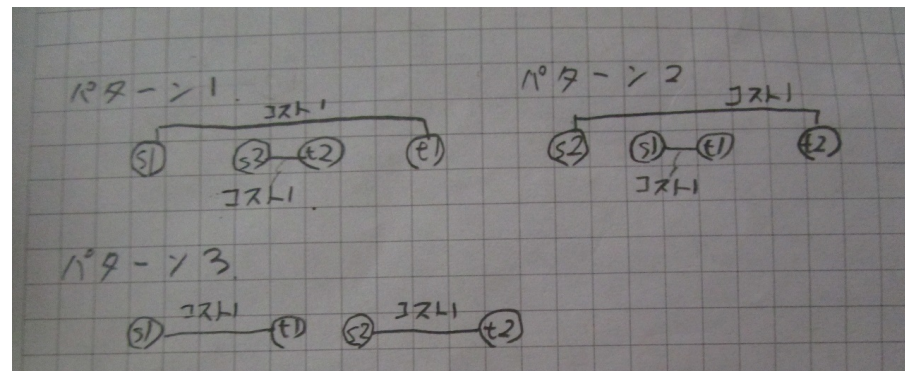
- このようにグラフを張れたら、各クエリごとに、頂点 N 、辺は高々 $2N$ 、辺のコストがすべて 1 の最短経路問題として解くことができる
- 幅優先探索を用いて、各クエリごとに $O(N)$
- よって、 $O(NQ)$ で解くことができ、20 点が得られる。

小課題 3(25 点)

- $K \leq 20$
- さらなる考察が必要そう
- というか必要でしょうね

ところで

- レベル i の駅から、次に現れるレベル i 以上の駅までの間の移動はコスト 1 で行える
- これらのタイプの移動により、コスト 1 で移動できる駅の組み合わせ (s_1, t_1) 、 (s_2, t_2) (ただし $s_1 < t_1, s_2 < t_2$) を考えると、区間 $(s_1 < x < t_1)$ と区間 $(s_2 < x < t_2)$ は内包 / 外包関係にあるか、まったくかぶらないかのどちらかとなる
- 理由: 「レベル i の駅」と「次に現れるレベル i 以上の駅」までの間は必ずレベル $i-1$ 以下だから

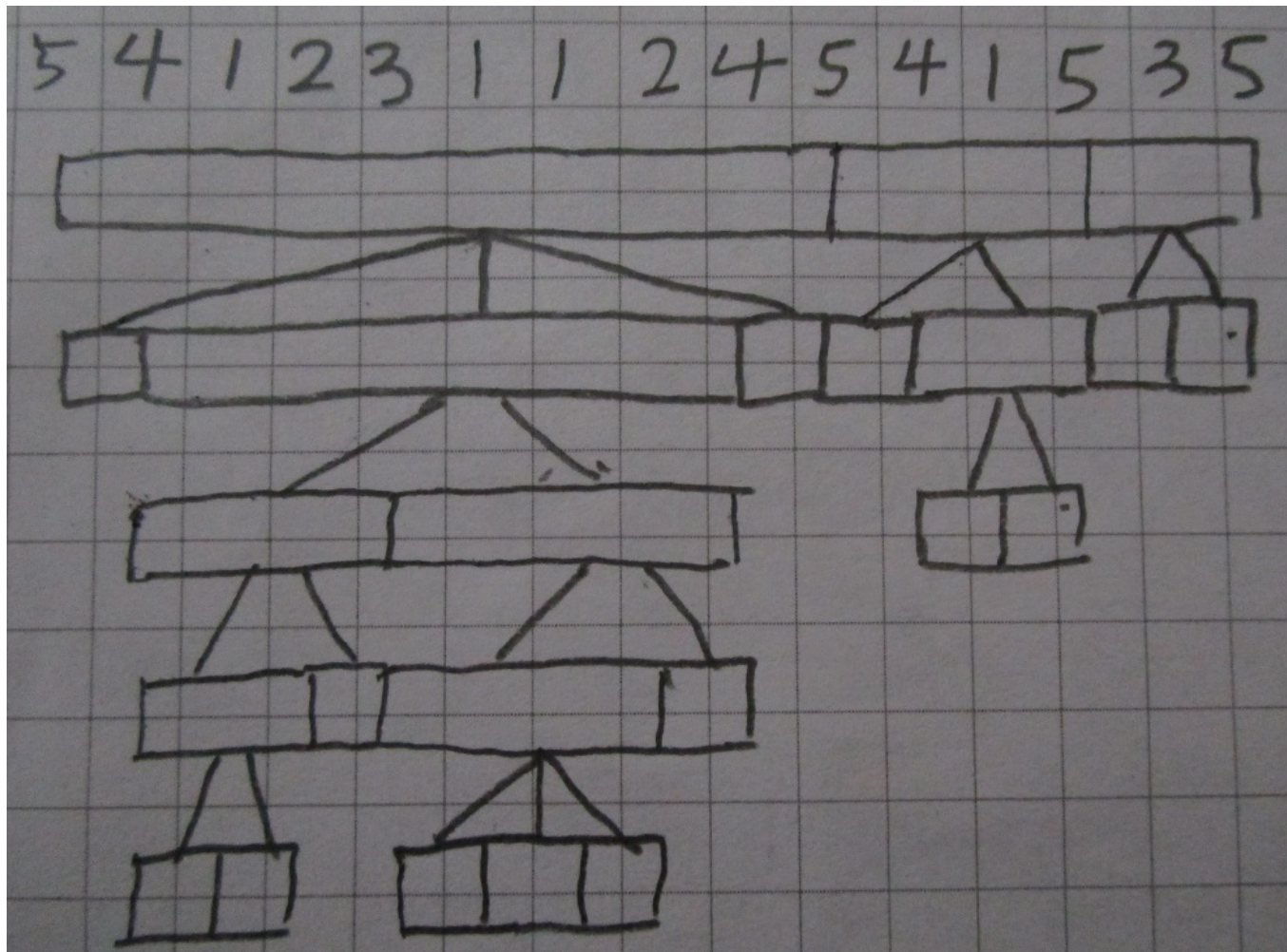


ところで

- ここで $K \leq 20$ を生かす
- 先述の組み合わせにより構成された区間を頂点としたグラフを考える
- 区間 $(s \leq x \leq t)$ に対応した頂点があったとき
 $s < x < t$ の中でレベルが最大となる駅たちを $v_1, v_2, \dots$ とする
子の区間として $(s \leq x \leq v_1), (v_1 \leq x \leq v_2), \dots$ を考える
- $K \leq 20$ なのでグラフの深さが現実的

入力例 3 のとき

- このようになる



で？

- 駅 s と駅 t の間がコスト 1 で移動できるとき
ある駅から駅 s へのコストが d_s 、駅 t へのコストが d_t なら
- $|d_s - d_t| \leq 1$ が明らかに成り立つ
- 自明

で？

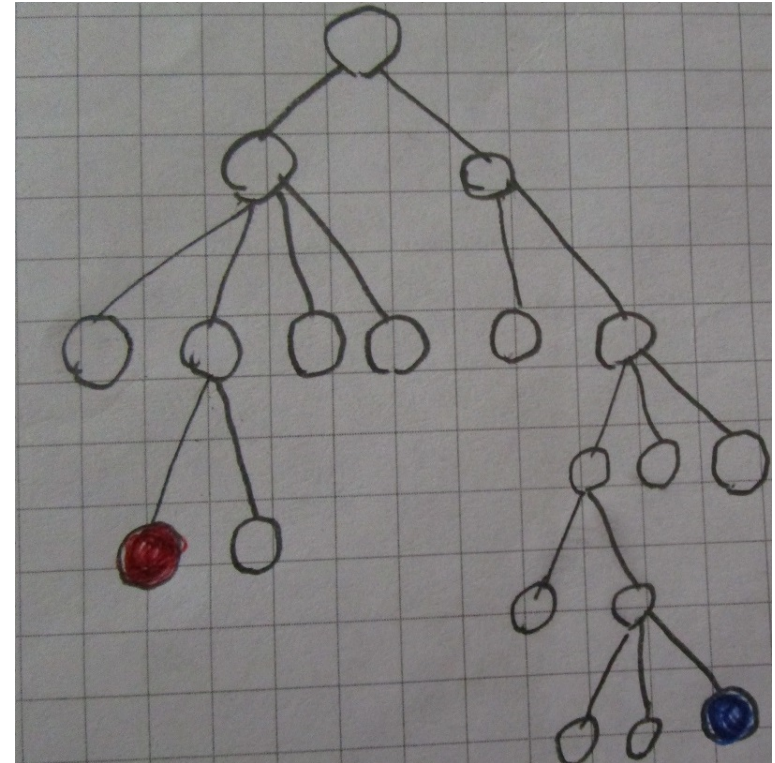
- 右端を経由したほうが良い場合 （タイプ A）
- どちらを経由しても同じ場合 （タイプ B）
- 左端を経由したほうが良い場合 （タイプ C）

これらの三つの場合に限られる

- すると、親方向への移動時にどのタイプに移動すべきか、そしてその移動にかかるコストがわかる

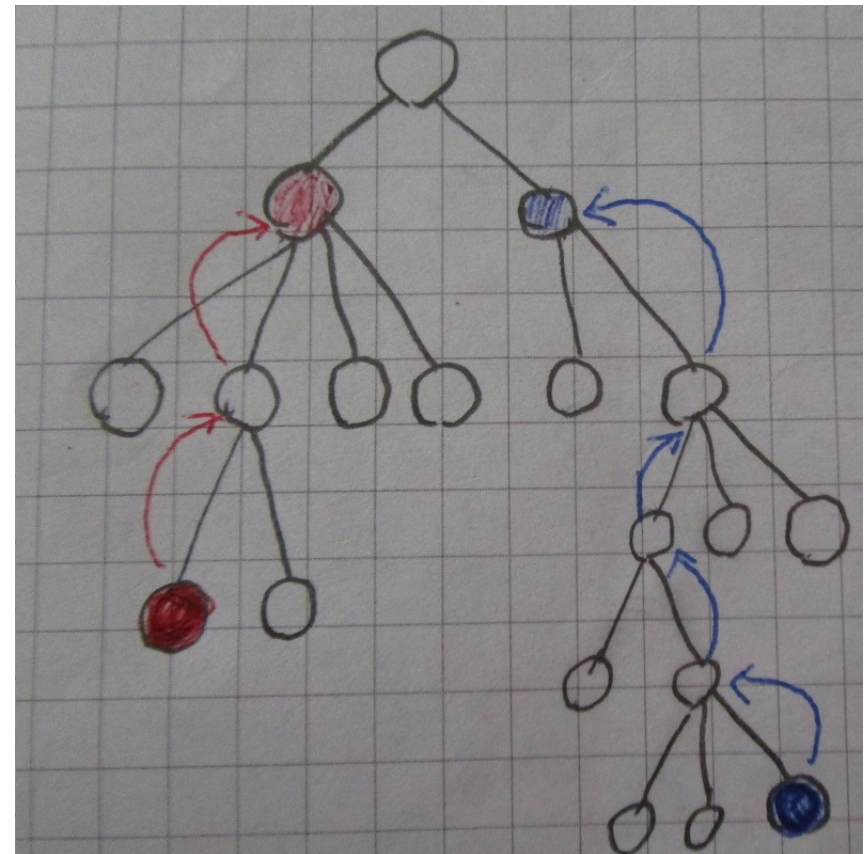
じゃあ駅 A から駅 B までどう移動するの？

- $A < B$ とします
- A から左に移動するとき、B から右に移動するときには必ず親方向への移動を伴うので、使う頂点は「A を左端に持つ区間に対応する頂点」と「B を右端に持つ区間に対応する頂点」でよい



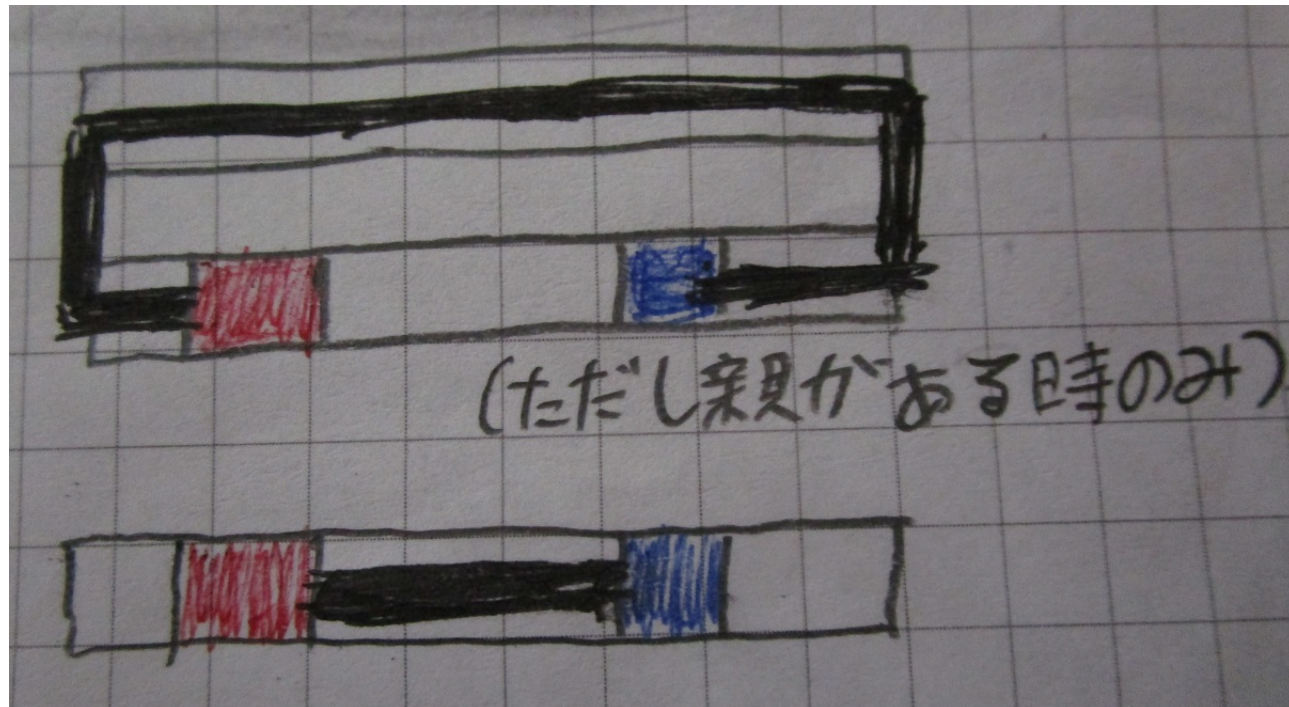
じゃあ駅 A から駅 B までどう移動するの？

- また、これらが共に「ある頂点の直接の子」になるまでは親方向に移動し続けてよい
- そのような頂点がなければ一番上の先祖まで移動して問題ない



じゃあ駅 A から駅 B までどう移動するの？

- この 2 点間の移動コストは簡単に求まる
- これら二つの最小値となる



はい。それで？

- 頂点の上り、下りを考えるとクエリ1つあたり $O(K)$ で処理できる
- クエリ処理全体で $O(KQ)$ となり 25 点が得られる
- 小課題 1,2 も通せばここまでで 45 点

小課題 4(5+15+25+55=100 点)

- はい。
- Q は仕方ないとして、クエリごとに $O(K)$ にかかるのはどうにかしたい
- (この頂点における子から親への移り方、なんか LCA のときと似てると感じませんか？)
- 1 回上った場合、2 回上った場合、4 回上った場合 ... を記録しておき、ダブリングを使えばクエリごとに $O(\log K)$
- 満点が得られる

ご清聴ありがとうございました

- 得点分布
- 2 人 (100 点)
- 1 人 (45 点)
- 4 人 (20 点)
- 5 人 (5 点)
- 1 人 (0 点)

Long Distance Coach

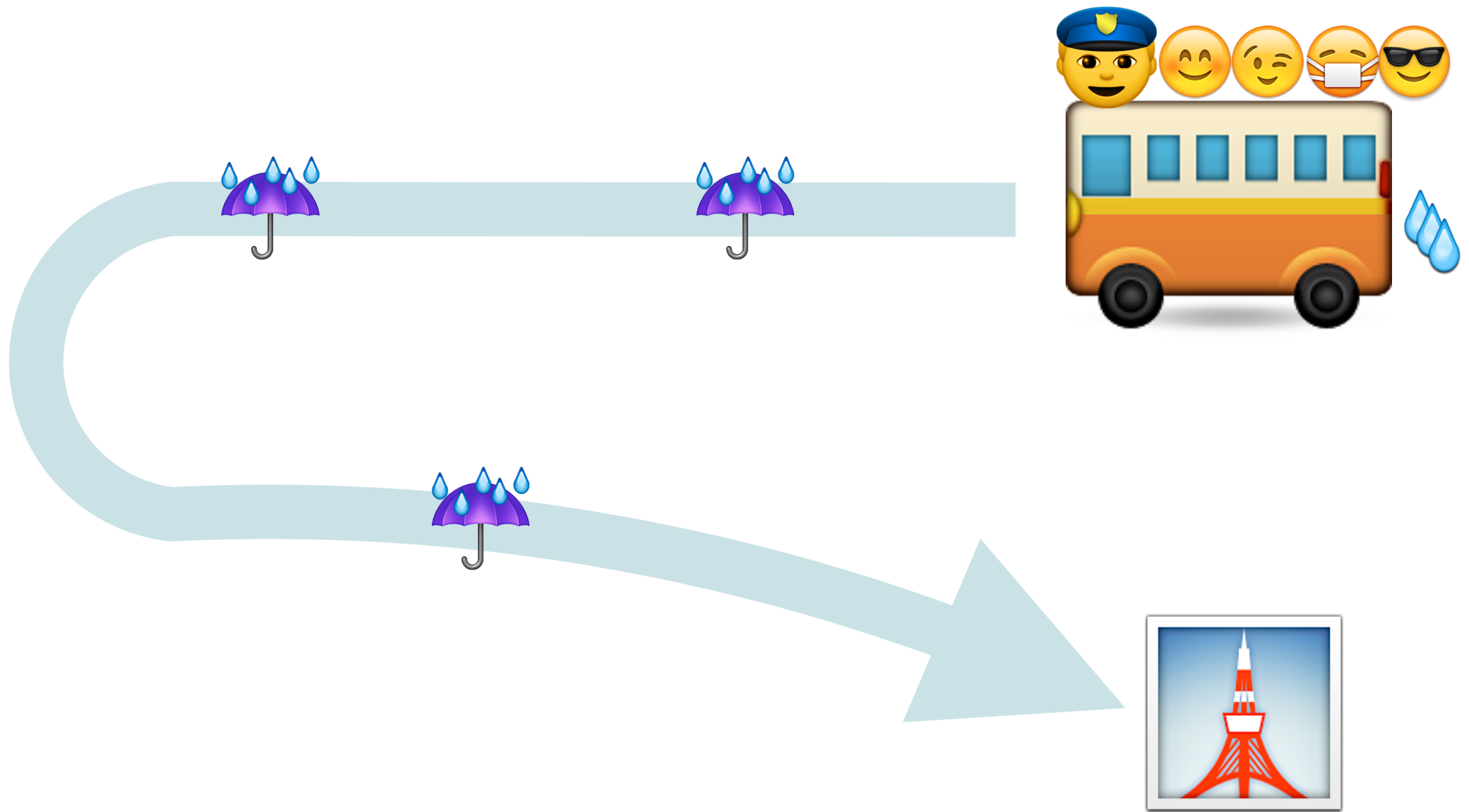
長距離バス 解説

今西 健介 (@japlj)

情報オリンピック 2016-2017 春季トレーニング合宿 競技日3

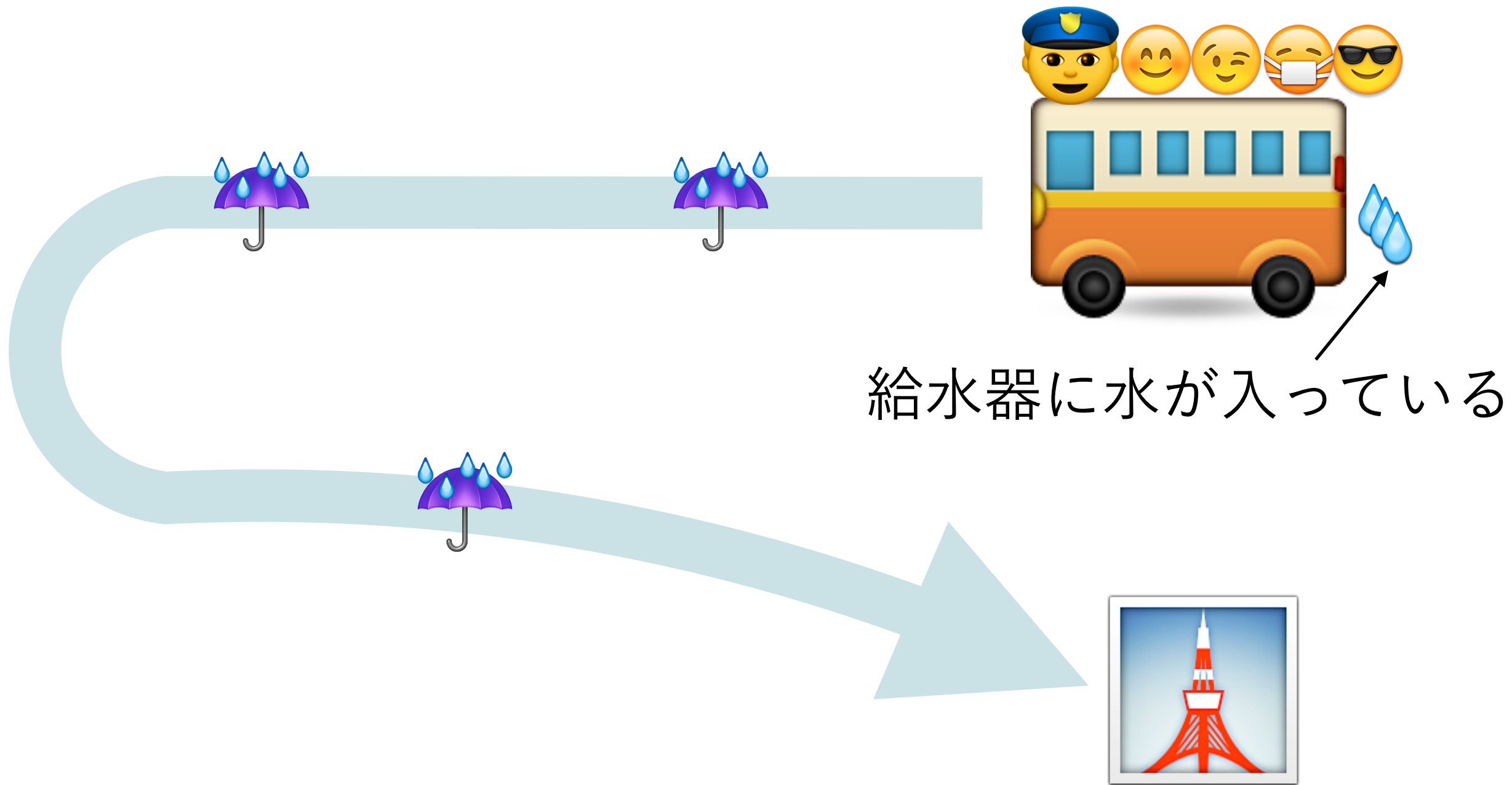
問題概要

長距離バスが目的地に行きたい



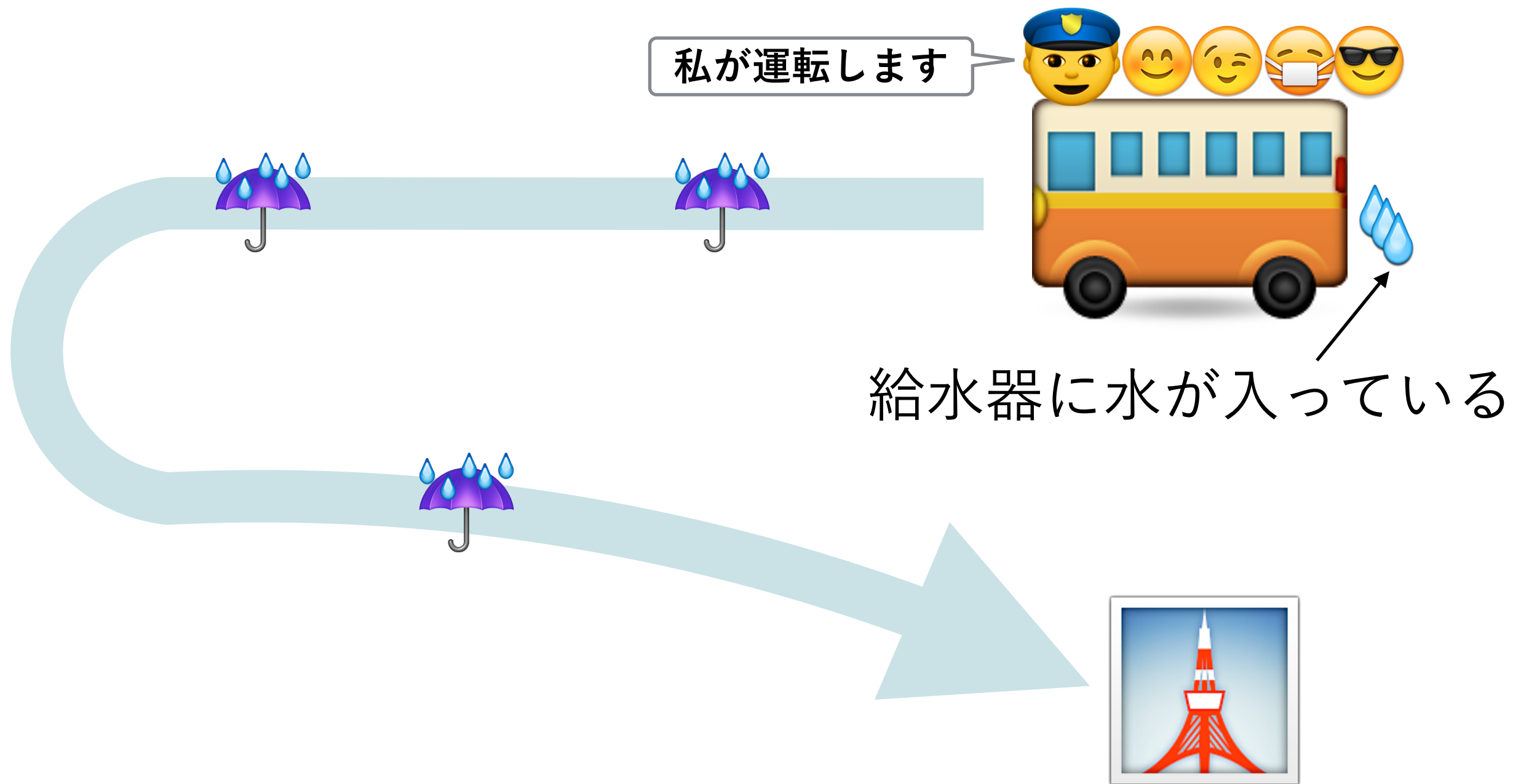
問題概要

長距離バスが目的地に行きたい



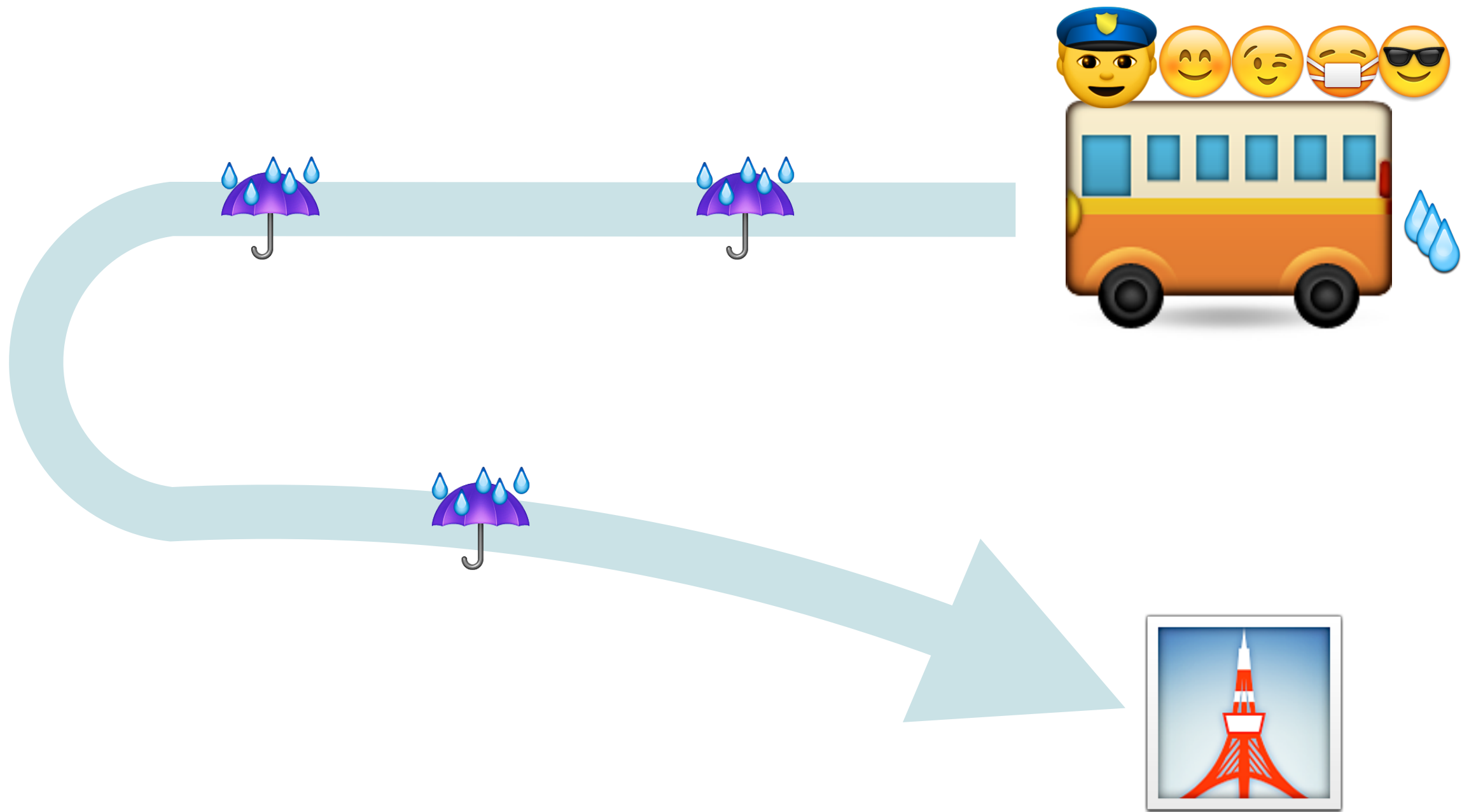
問題概要

長距離バスが目的地に行きたい



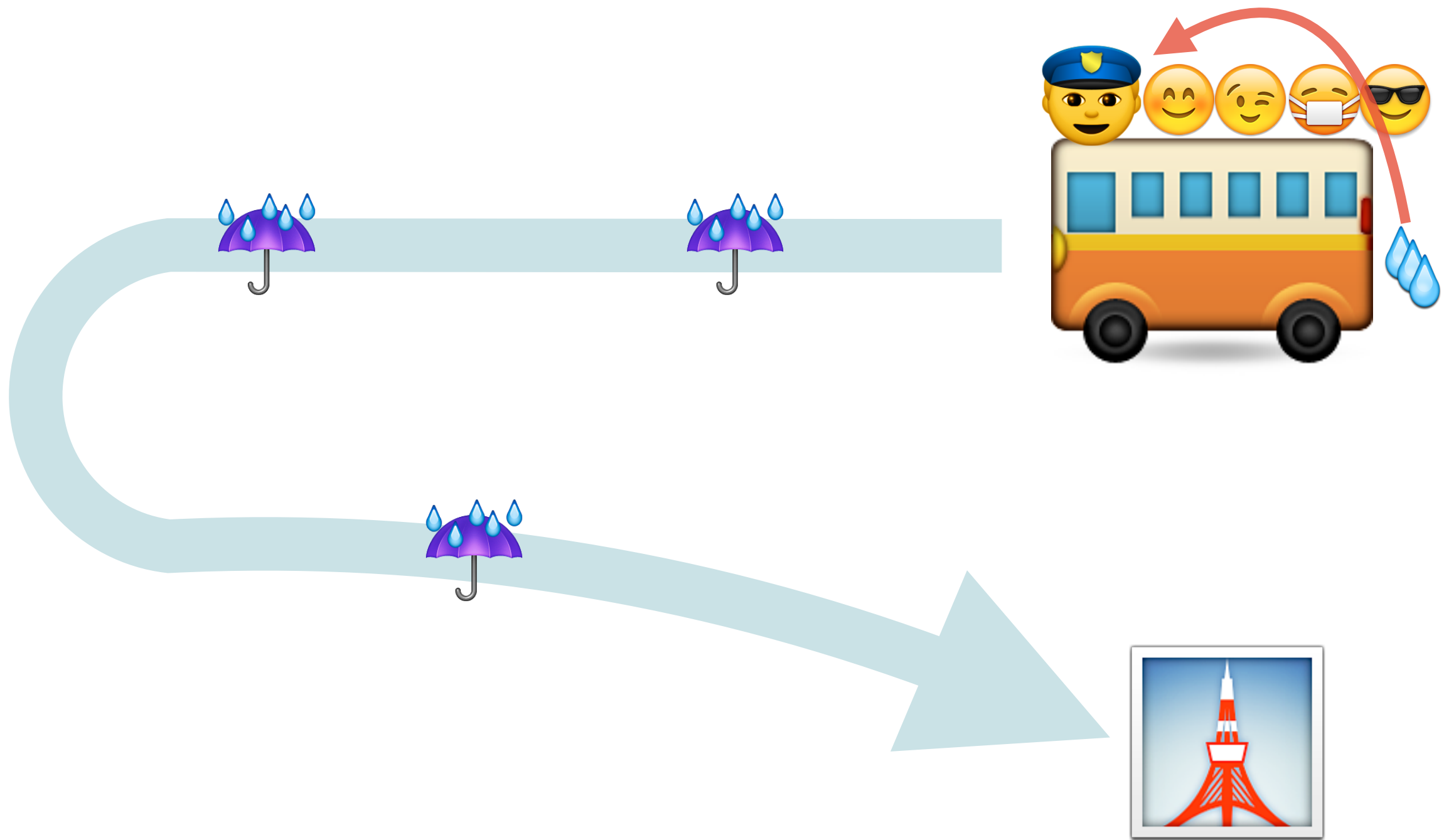
問題概要

長距離バスが目的地に行きたい



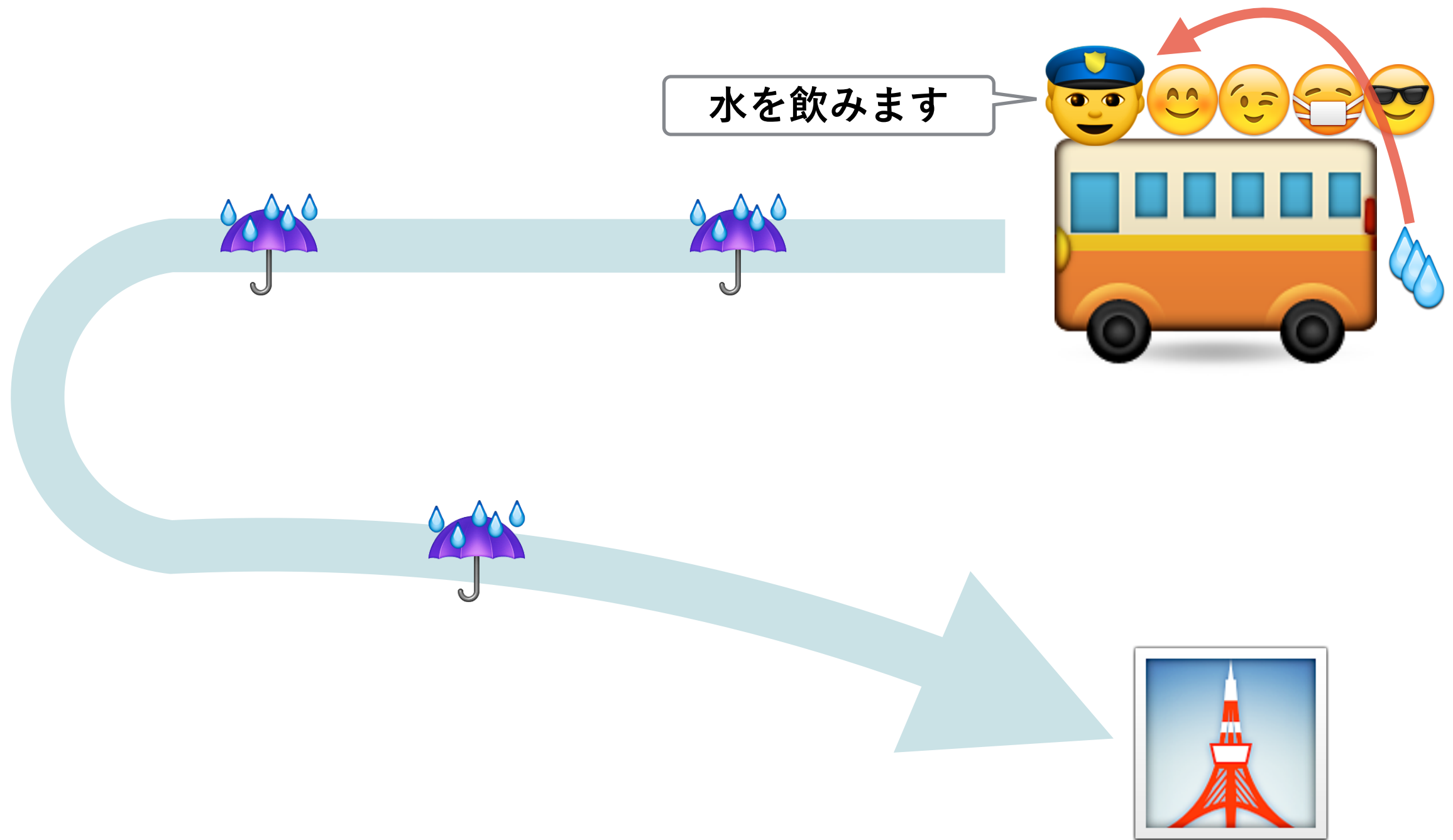
問題概要

長距離バスが目的地に行きたい



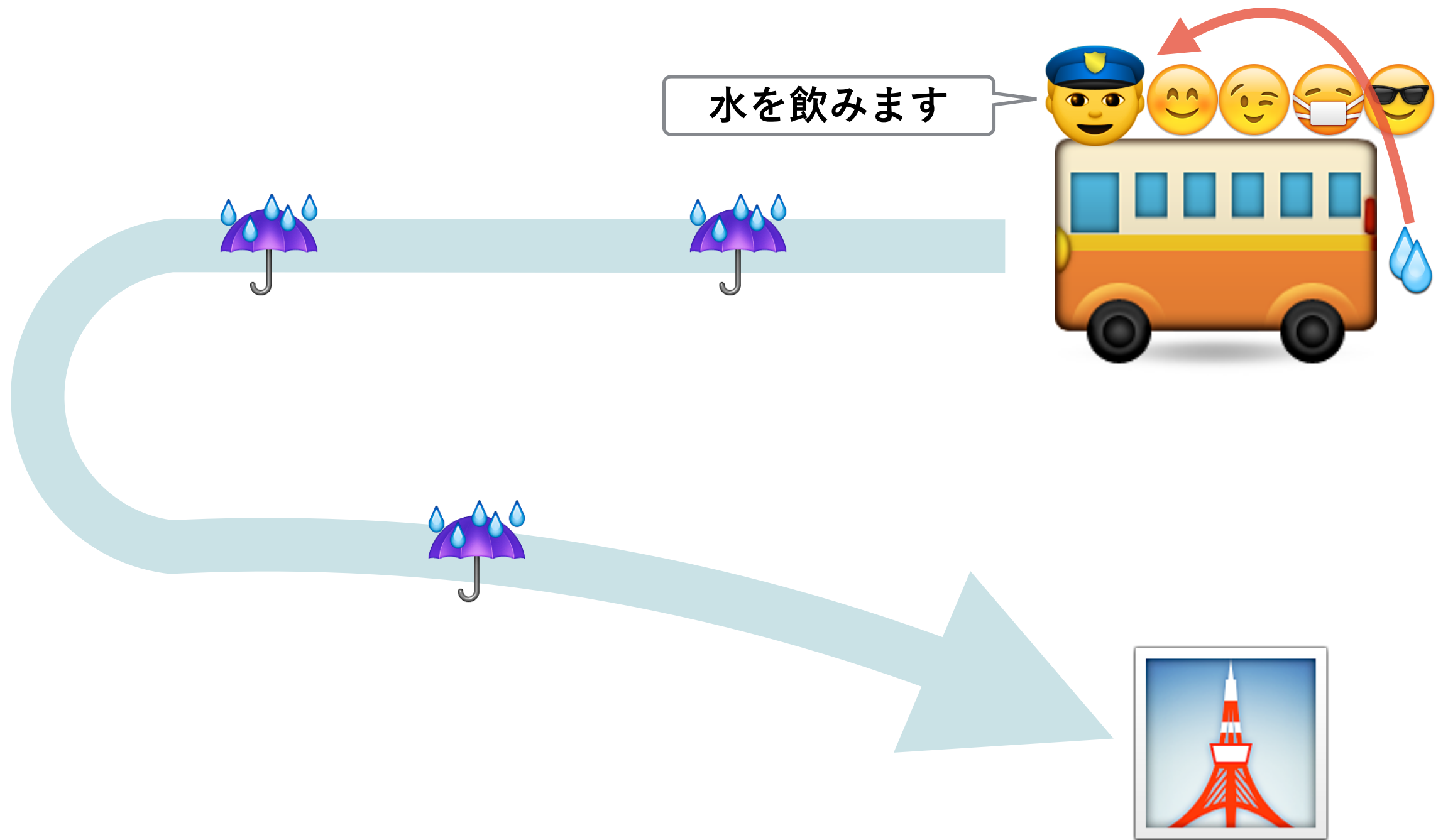
問題概要

長距離バスが目的地に行きたい



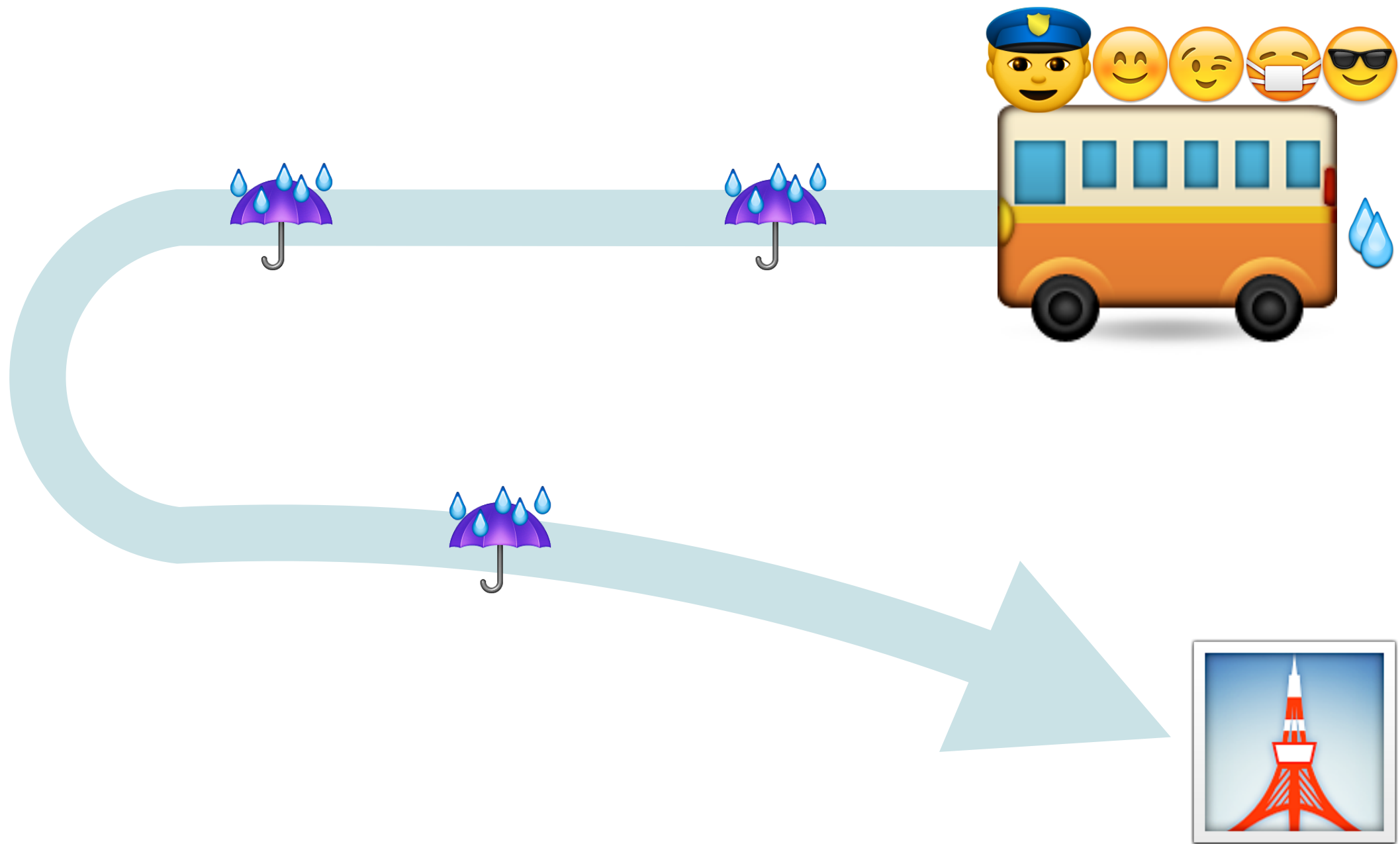
問題概要

長距離バスが目的地に行きたい



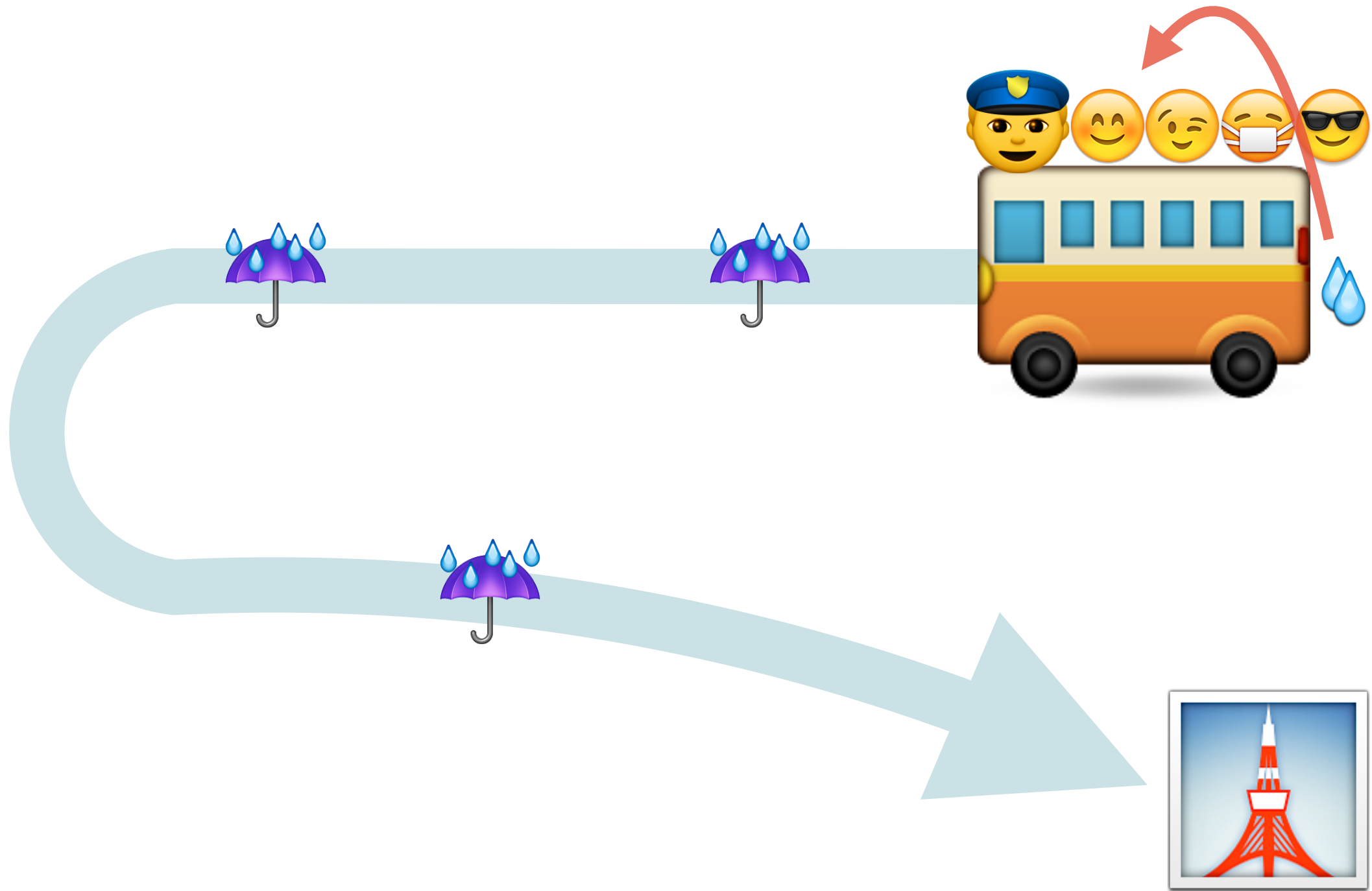
問題概要

長距離バスが目的地に行きたい



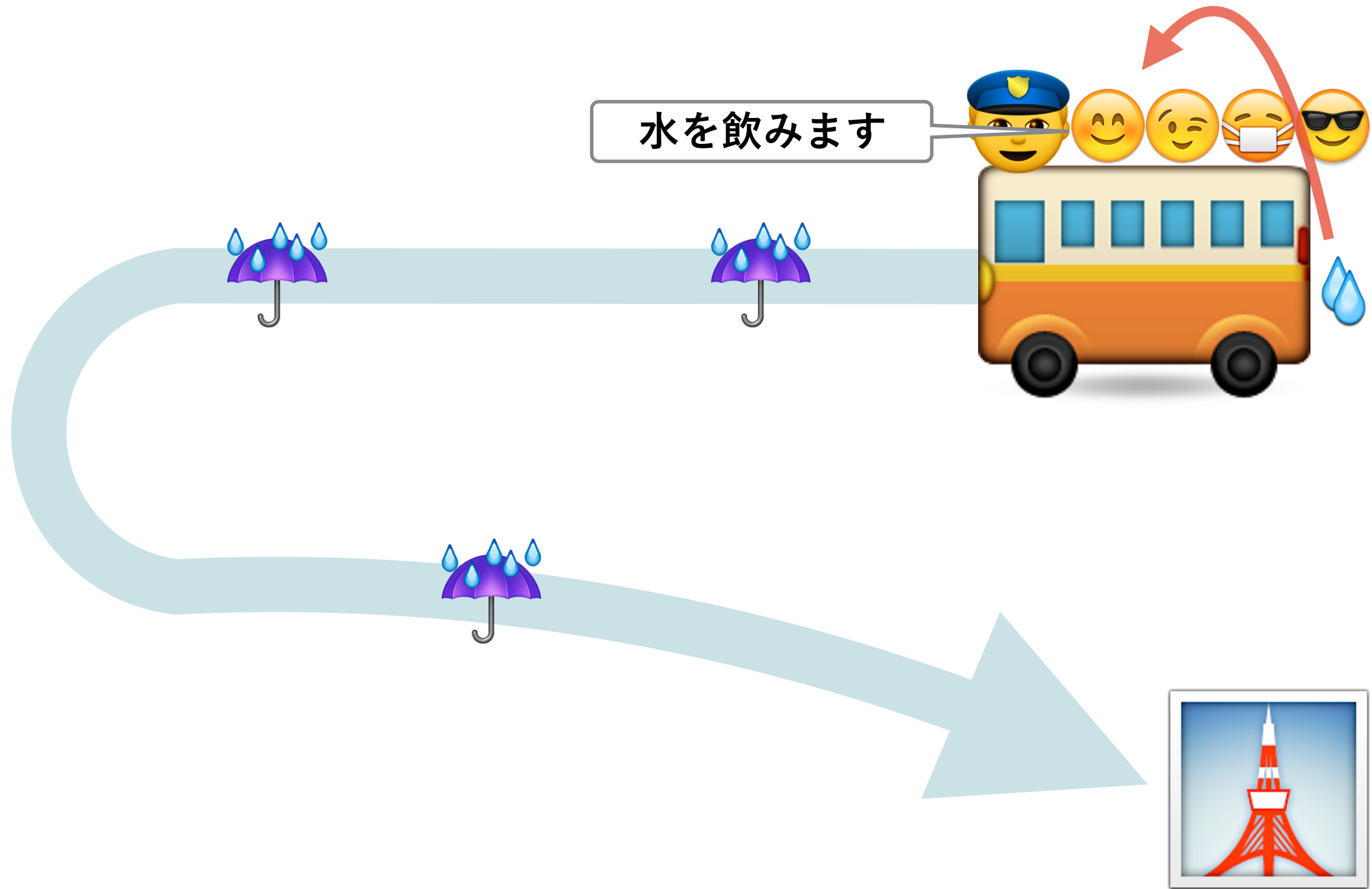
問題概要

長距離バスが目的地に行きたい



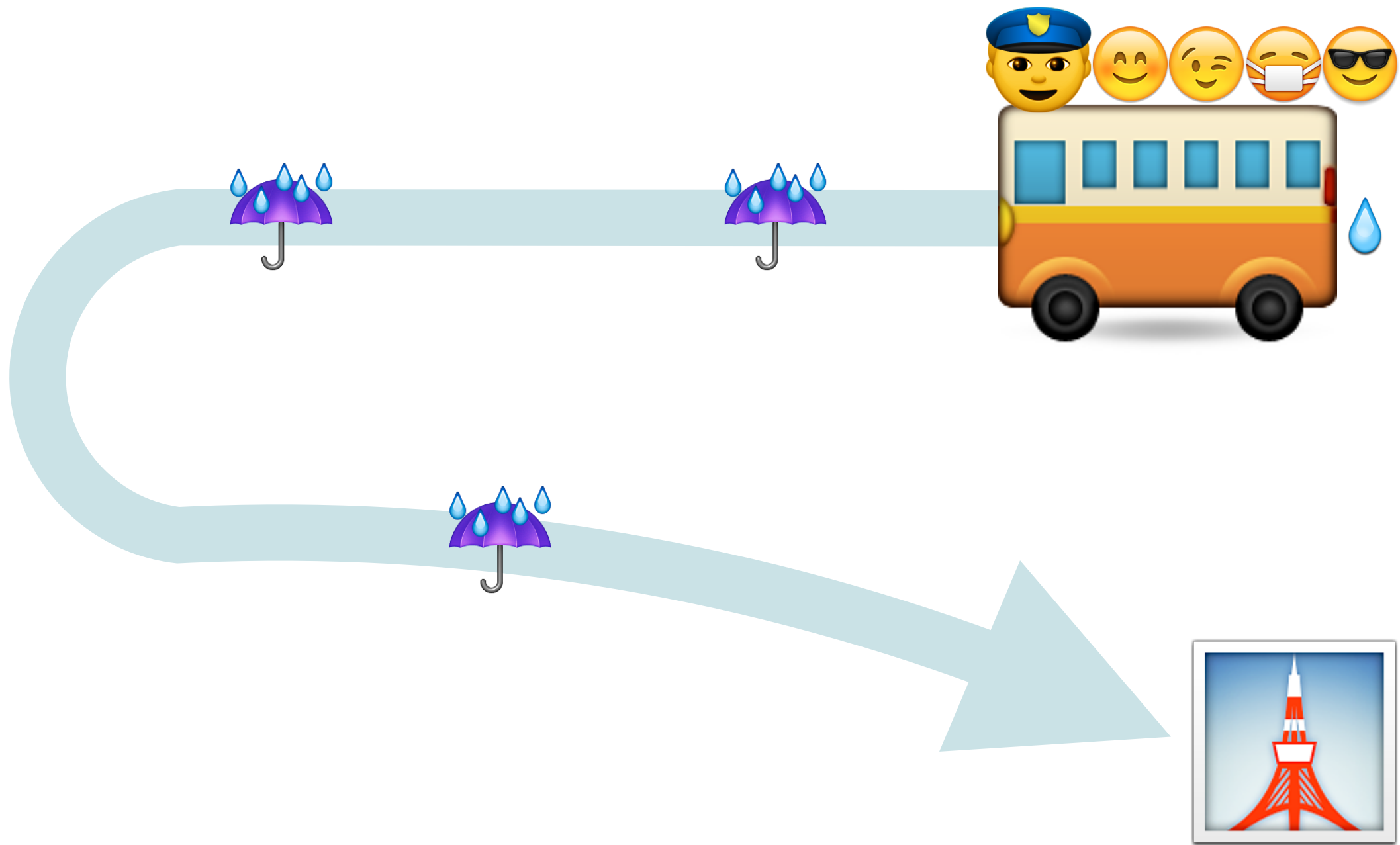
問題概要

長距離バスが目的地に行きたい



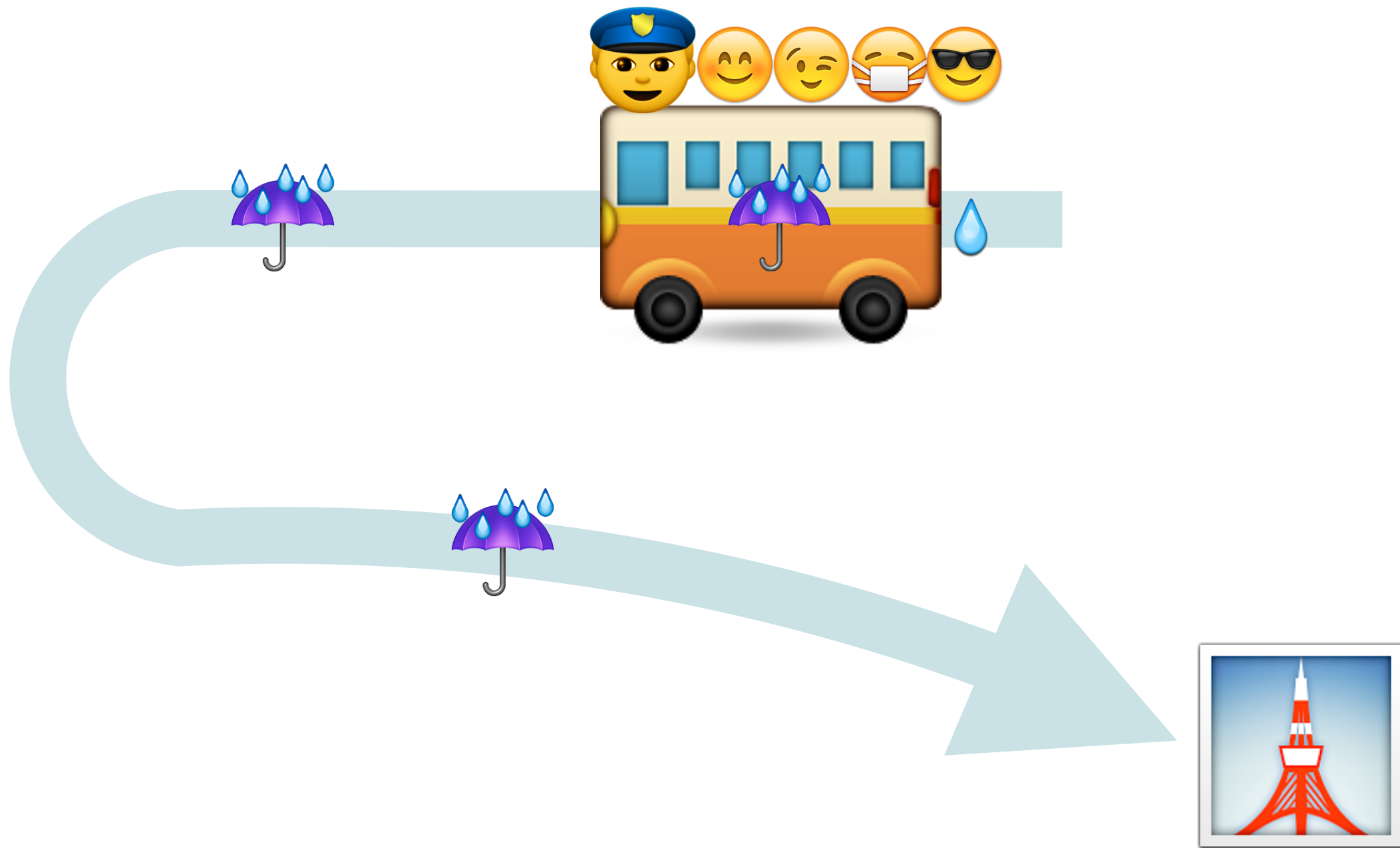
問題概要

長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい



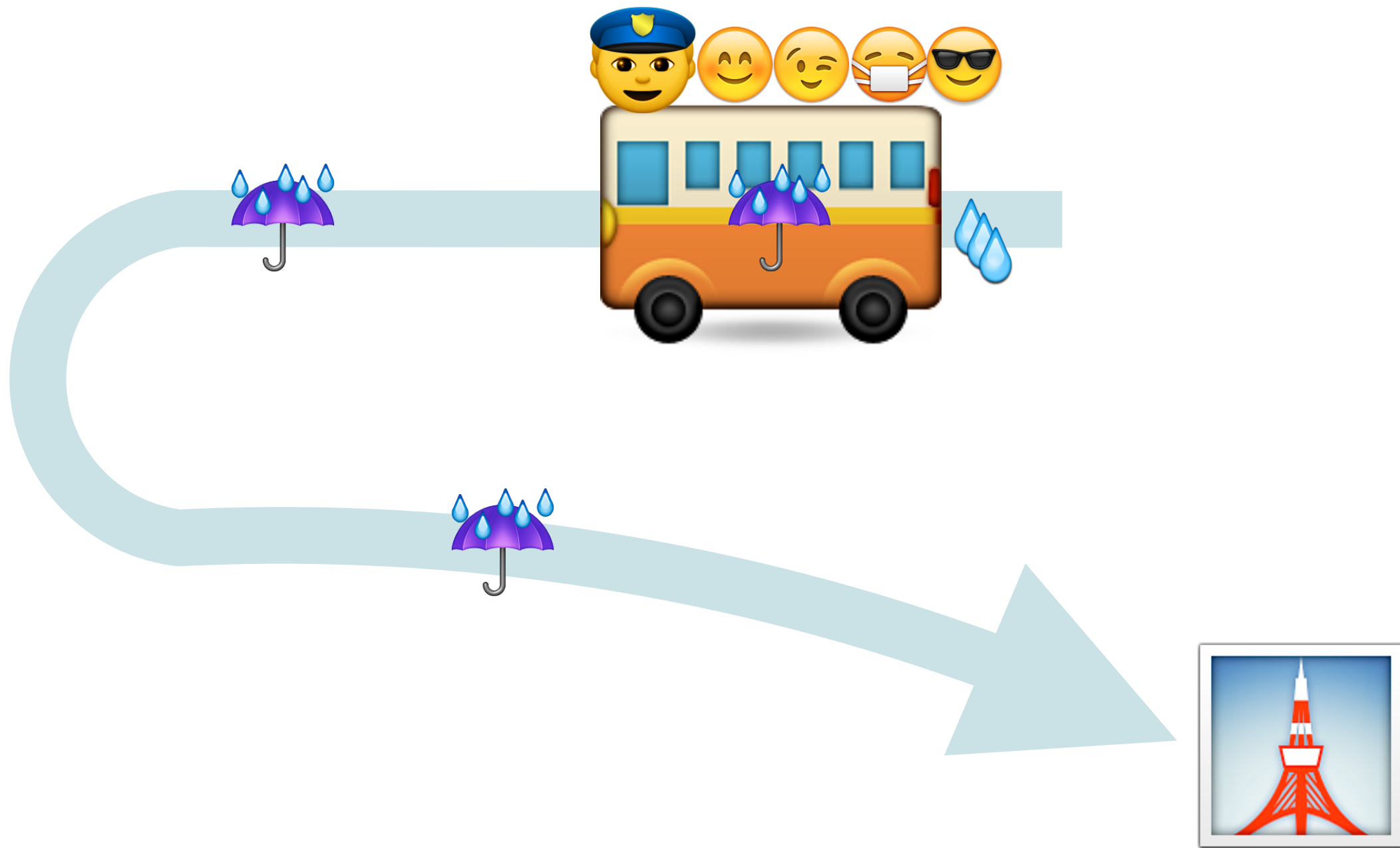
問題概要

長距離バスが目的地に行きたい



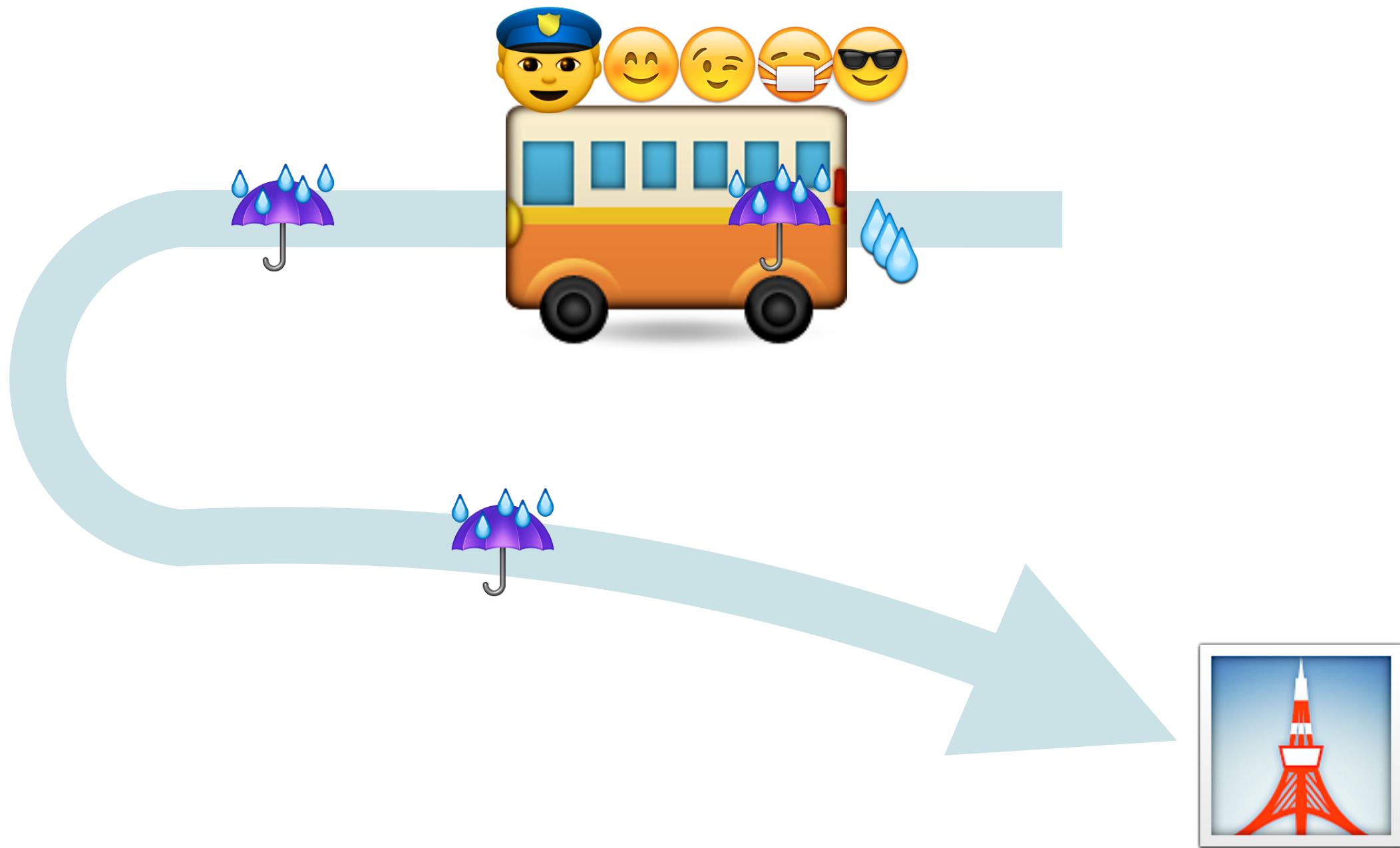
問題概要

長距離バスが目的地に行きたい



問題概要

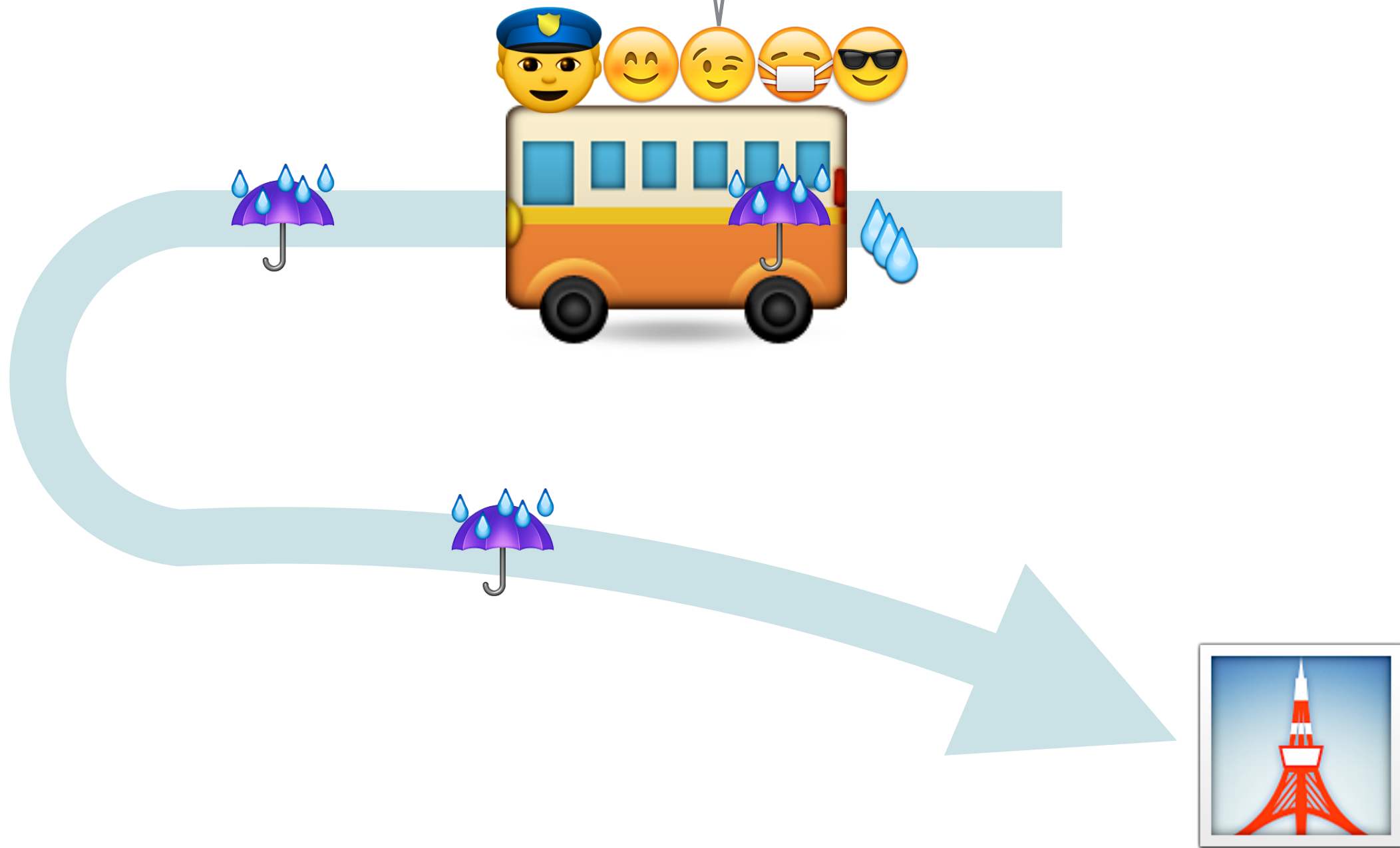
長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい

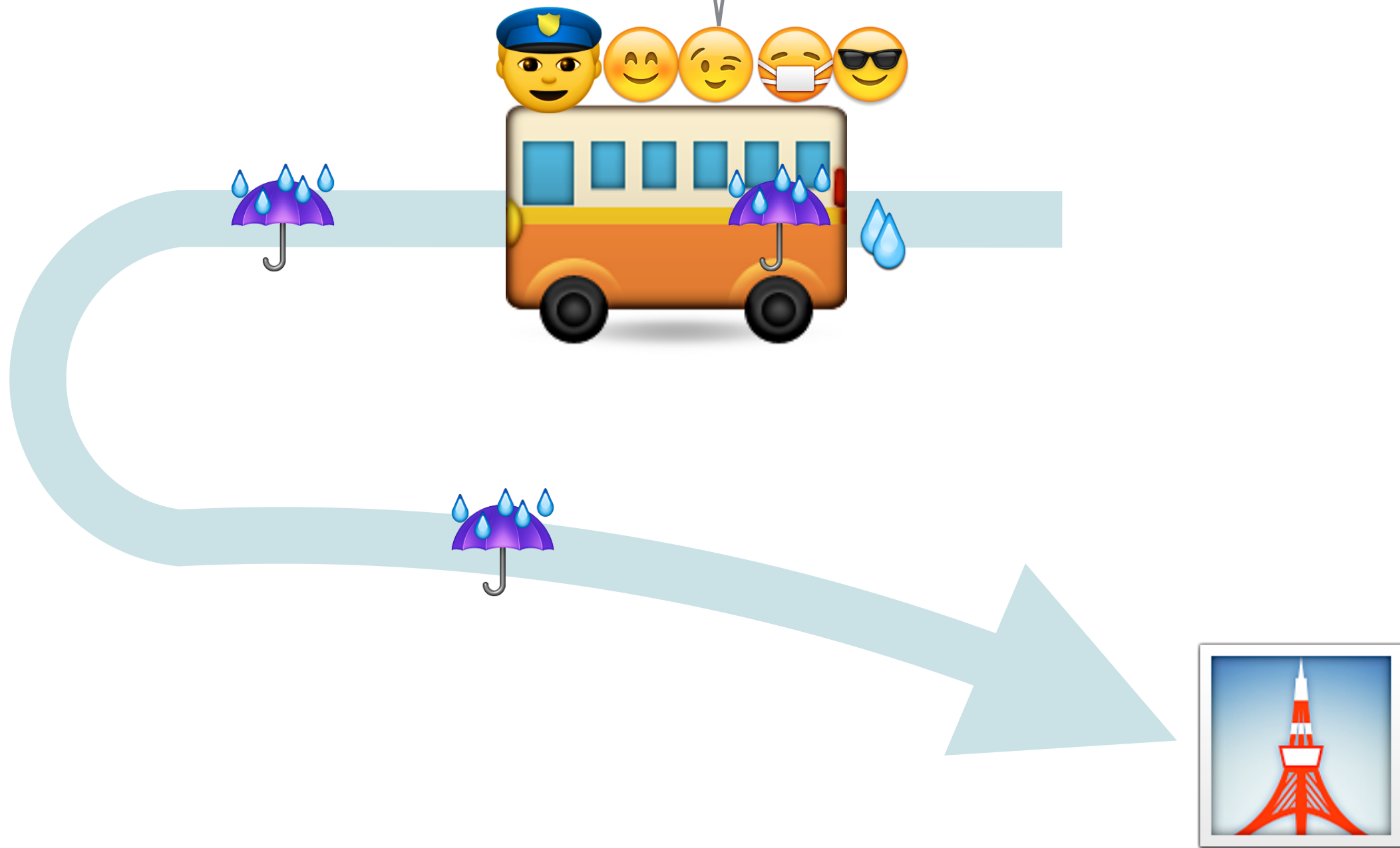
水を飲みます



問題概要

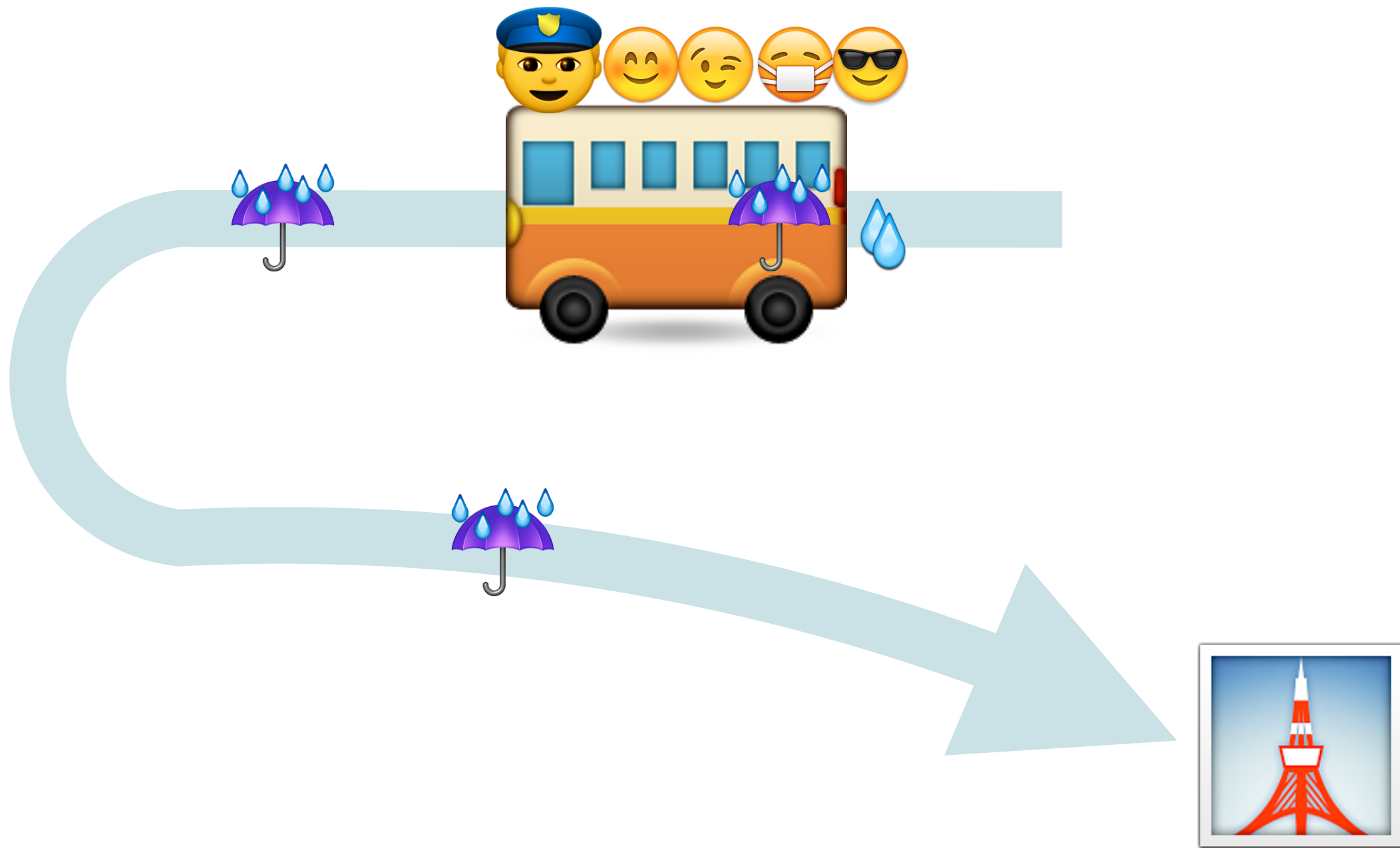
長距離バスが目的地に行きたい

水を飲みます



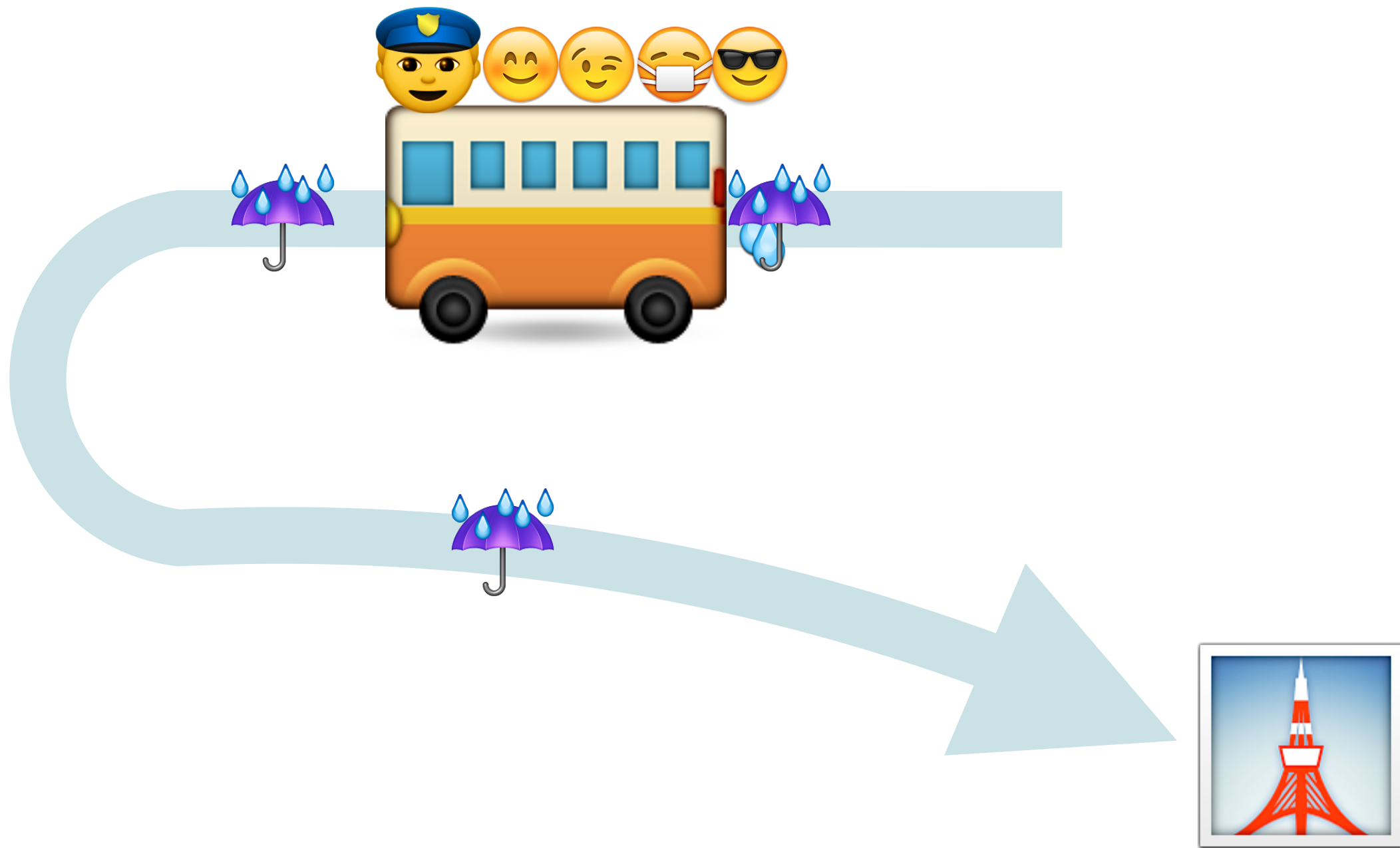
問題概要

長距離バスが目的地に行きたい



問題概要

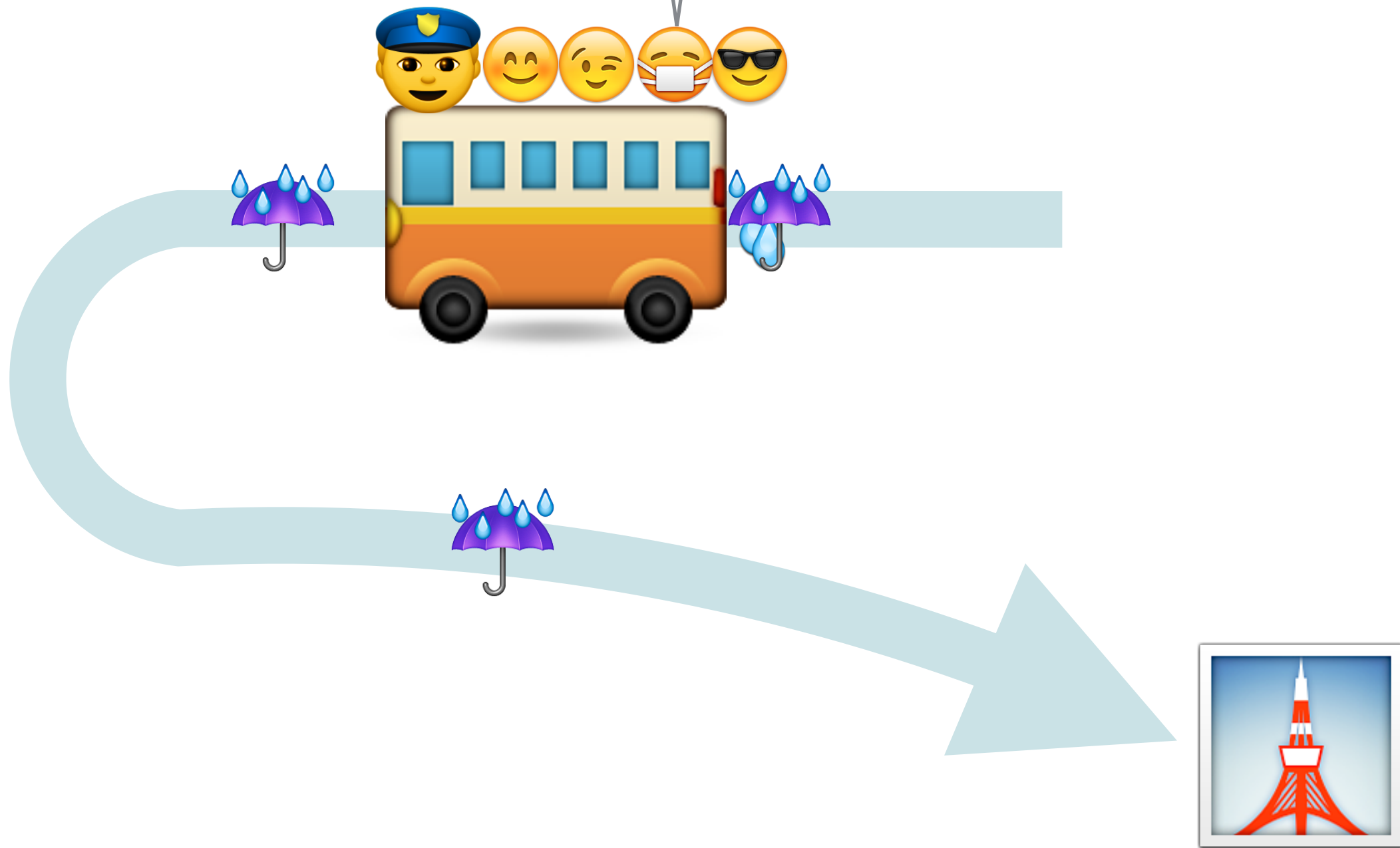
長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい

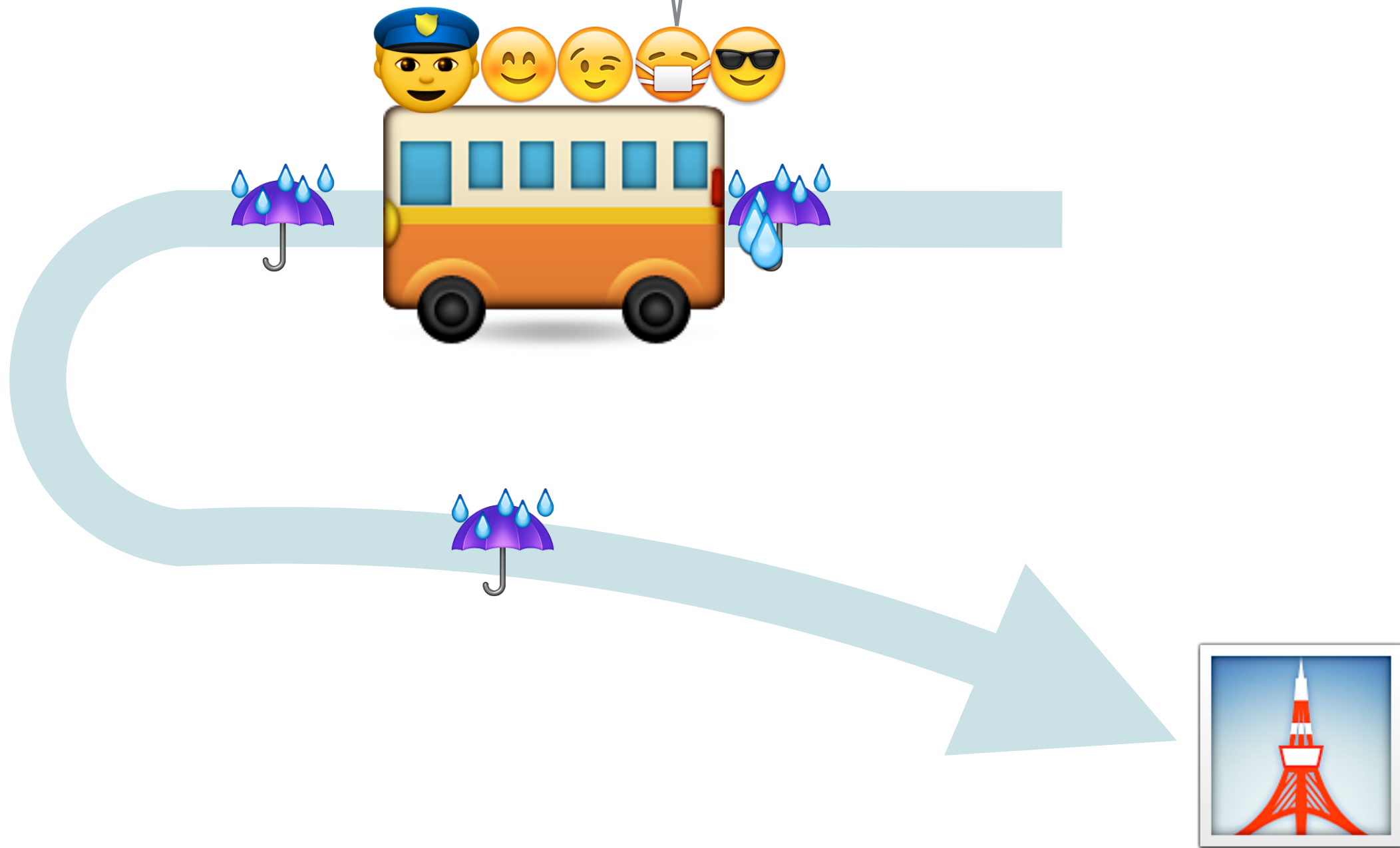
水を飲みます



問題概要

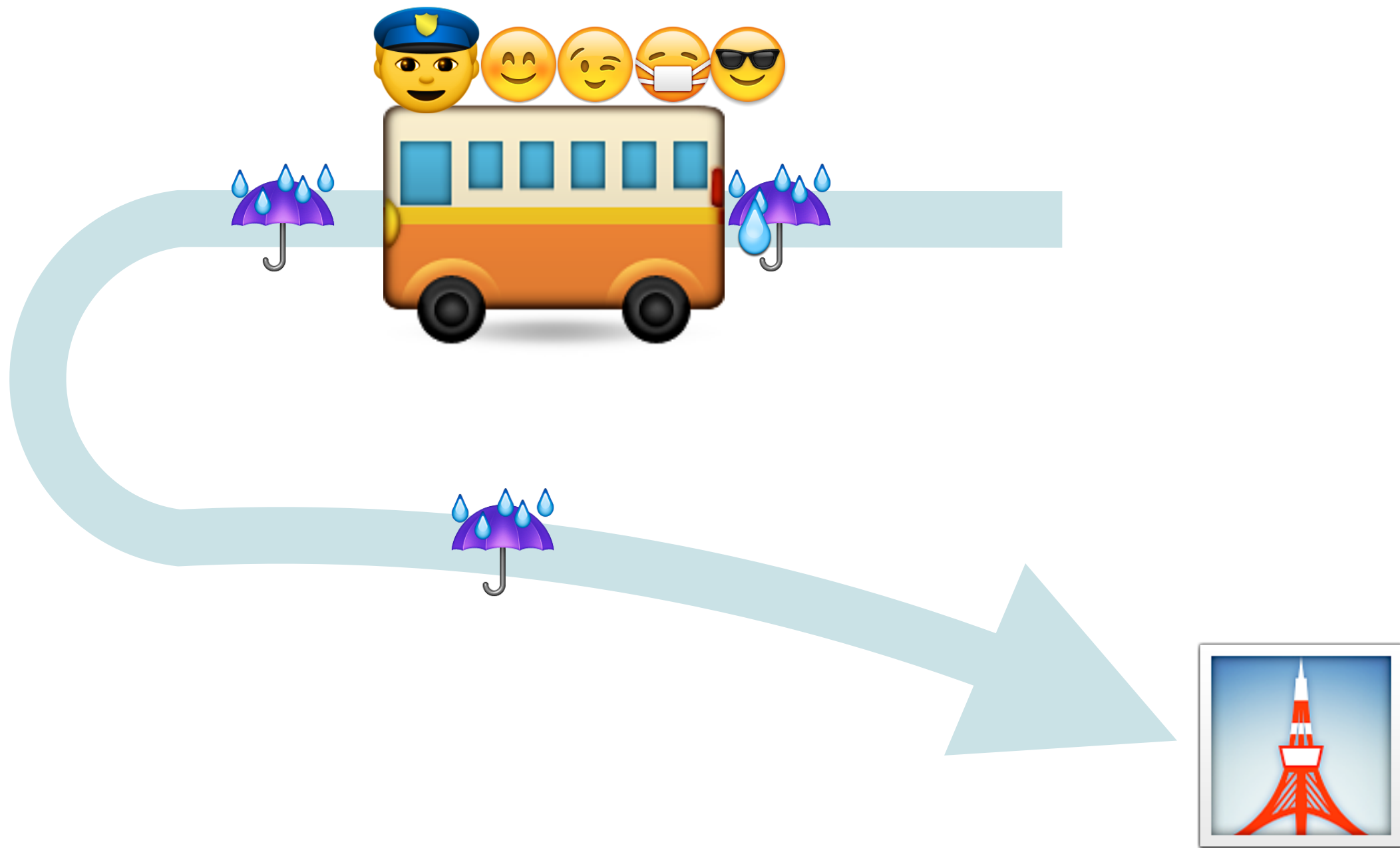
長距離バスが目的地に行きたい

水を飲みます



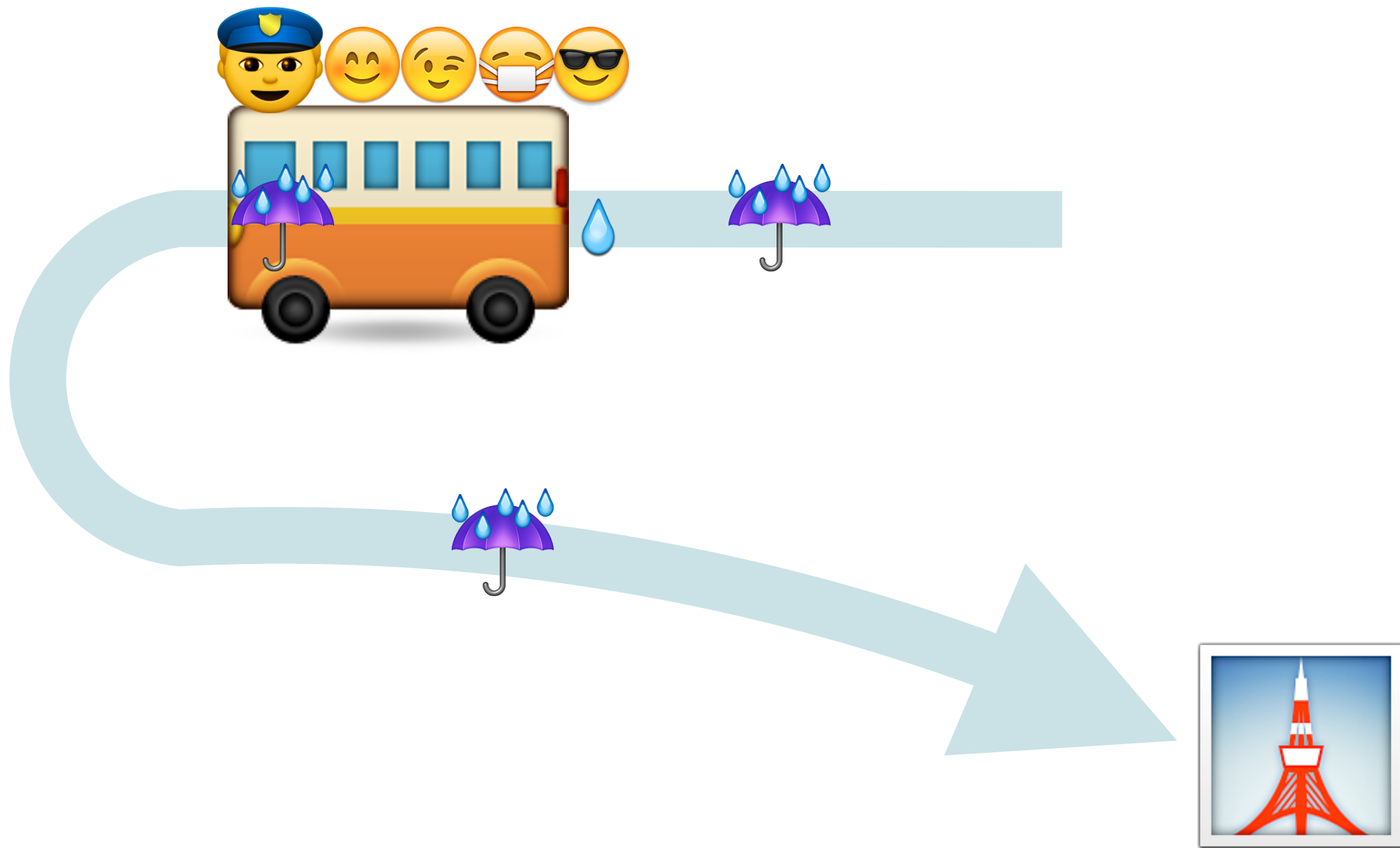
問題概要

長距離バスが目的地に行きたい



問題概要

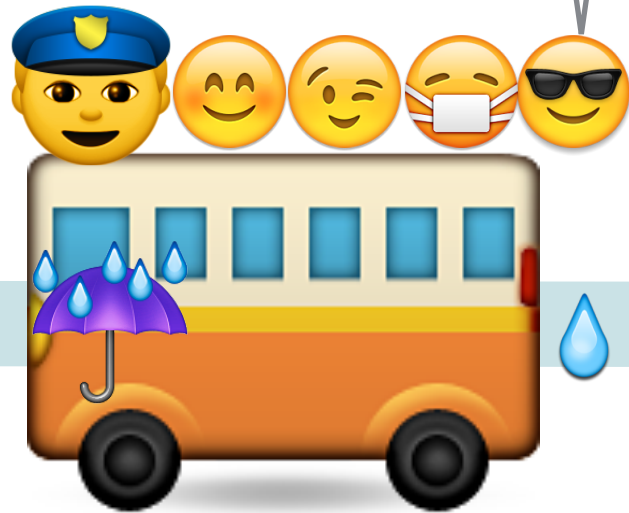
長距離バスが目的地に行きたい



問題概要

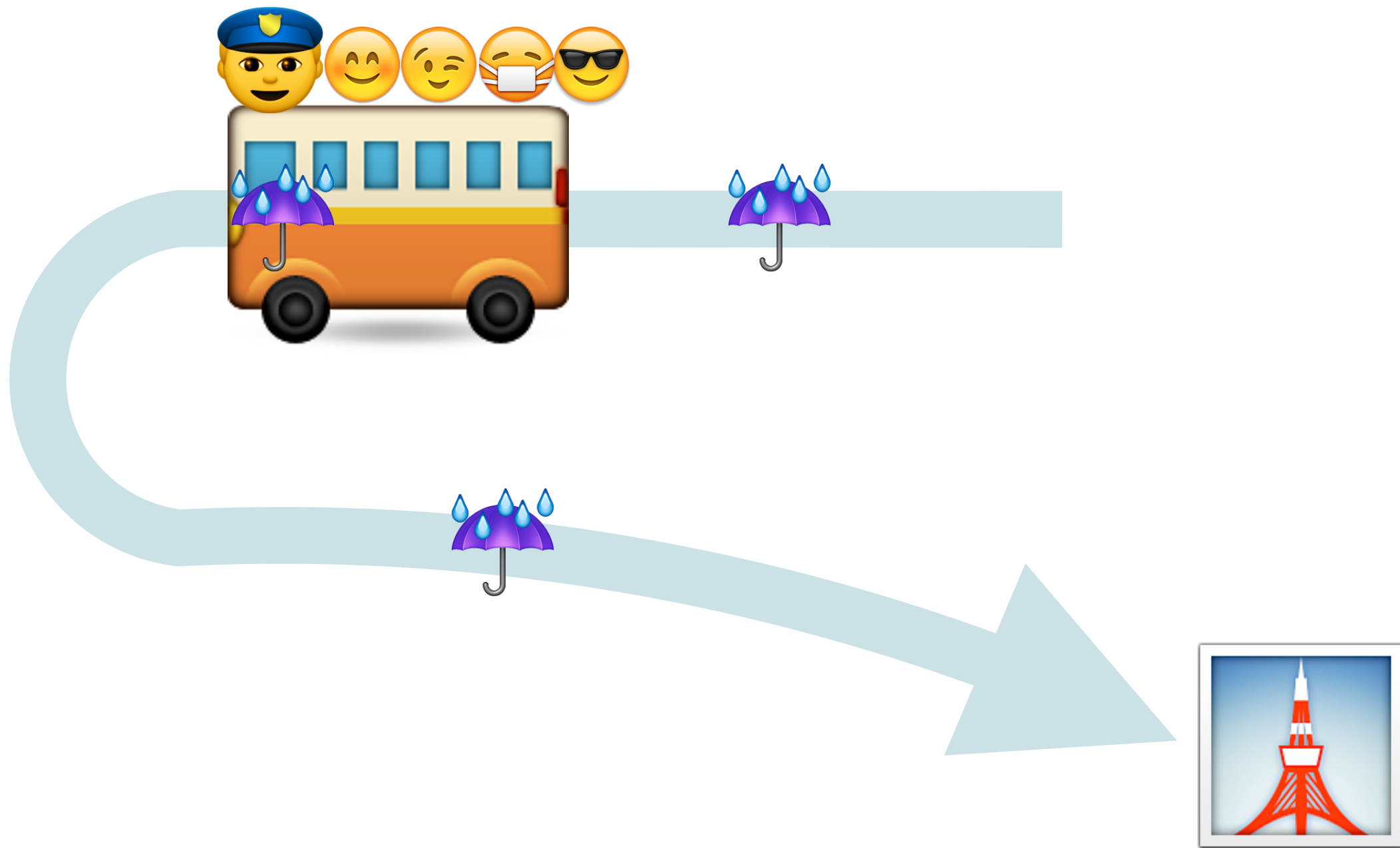
長距離バスが目的地に行きたい

水を飲みます



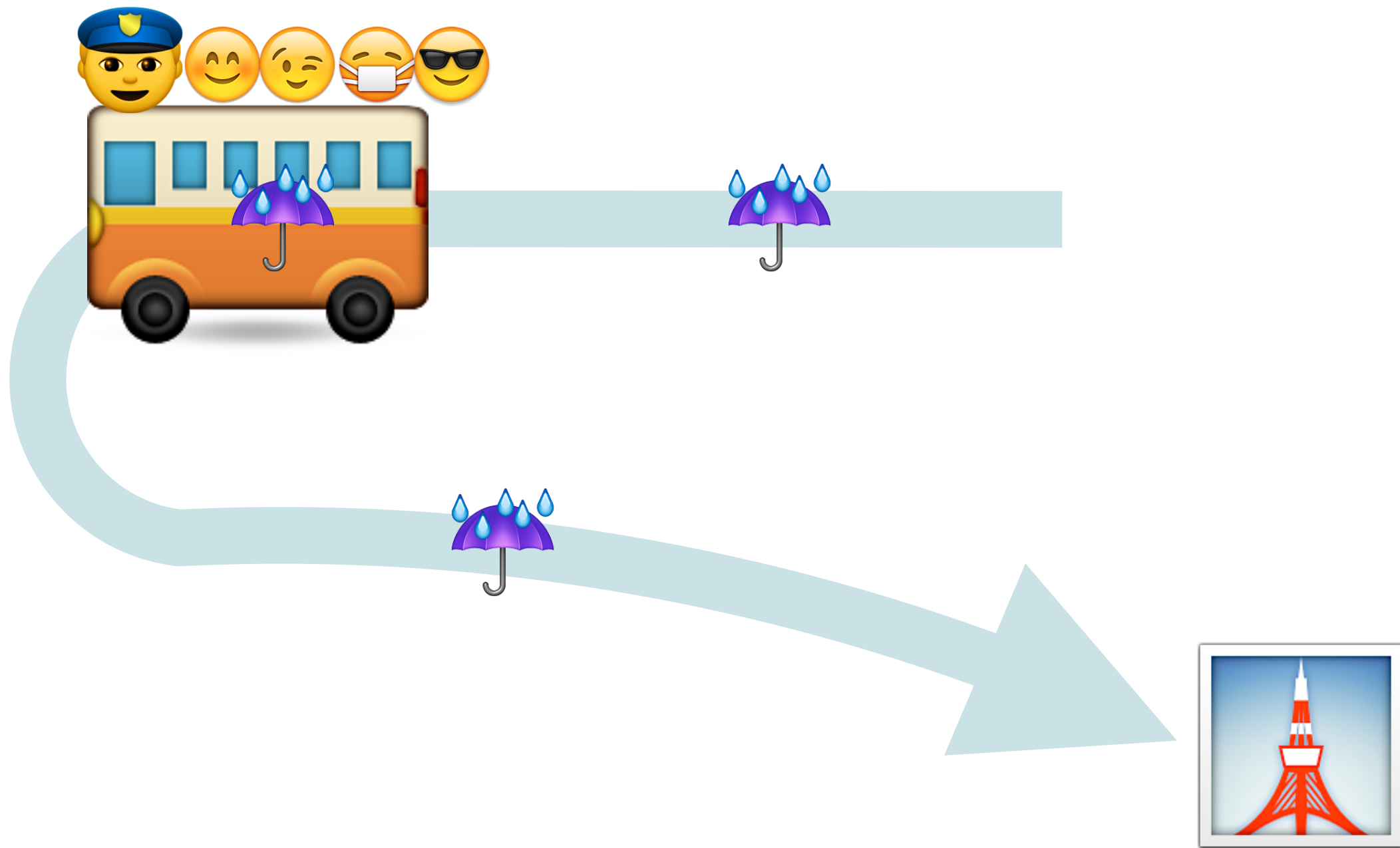
問題概要

長距離バスが目的地に行きたい



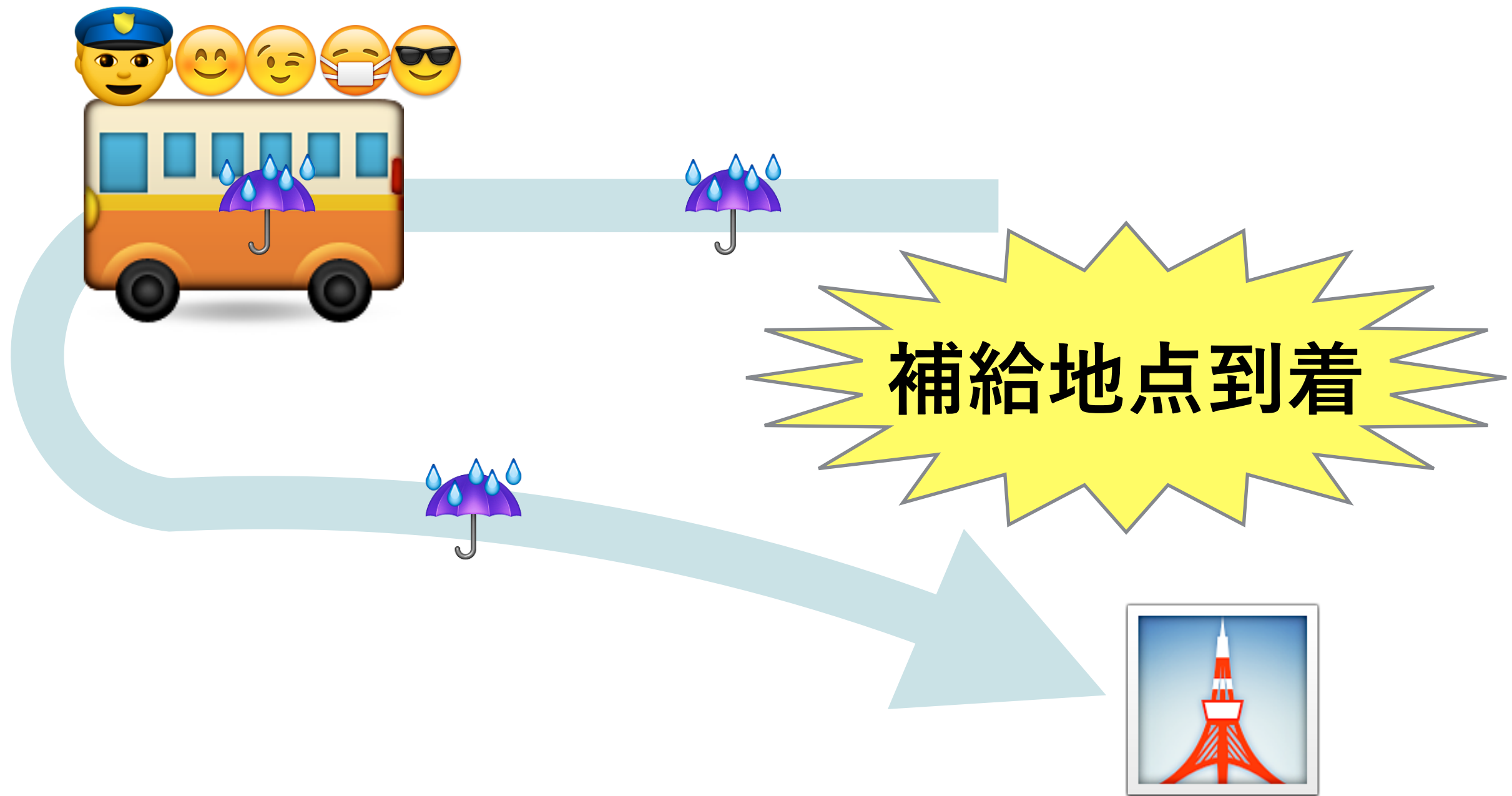
問題概要

長距離バスが目的地に行きたい



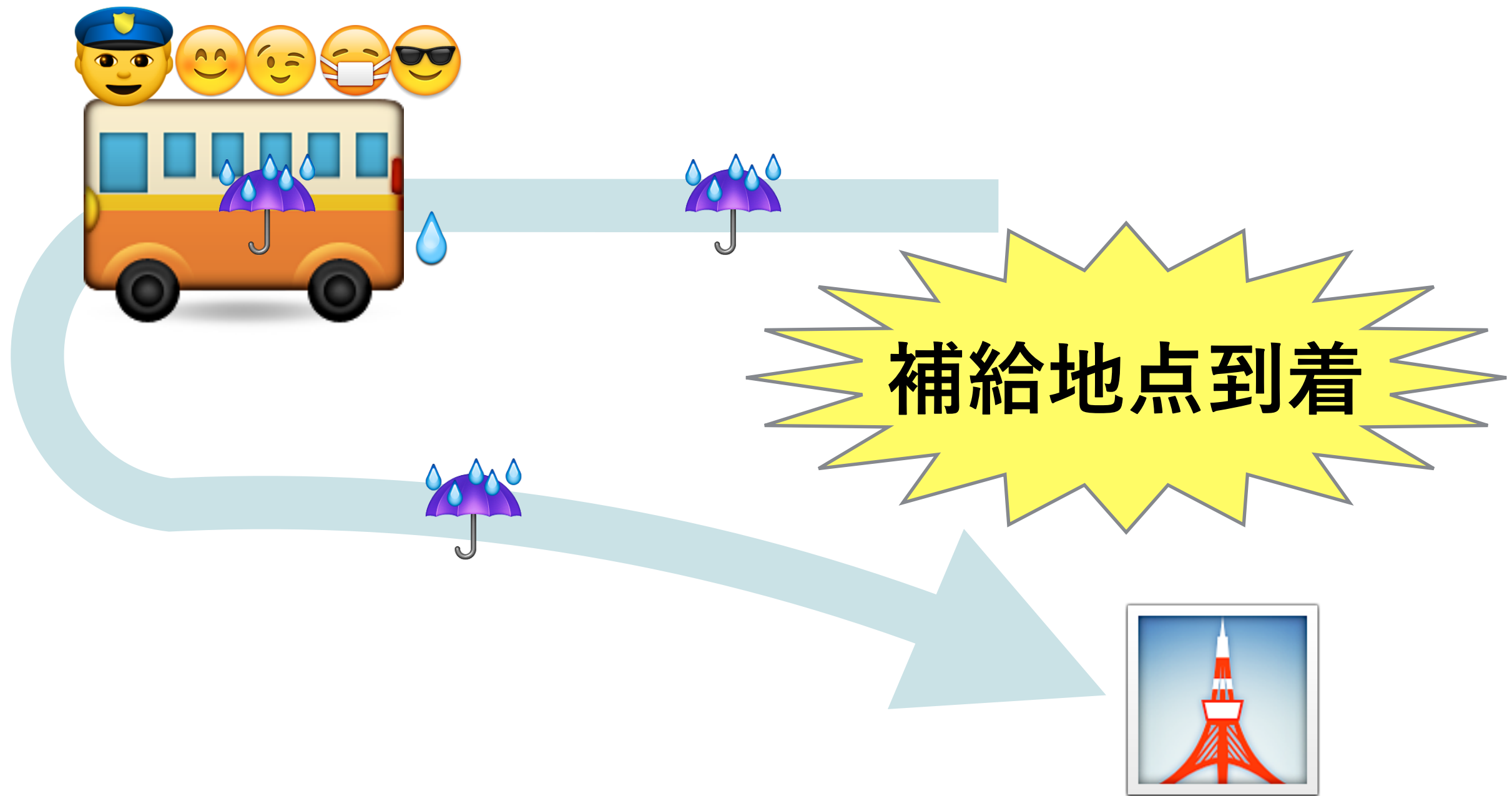
問題概要

長距離バスが目的地に行きたい



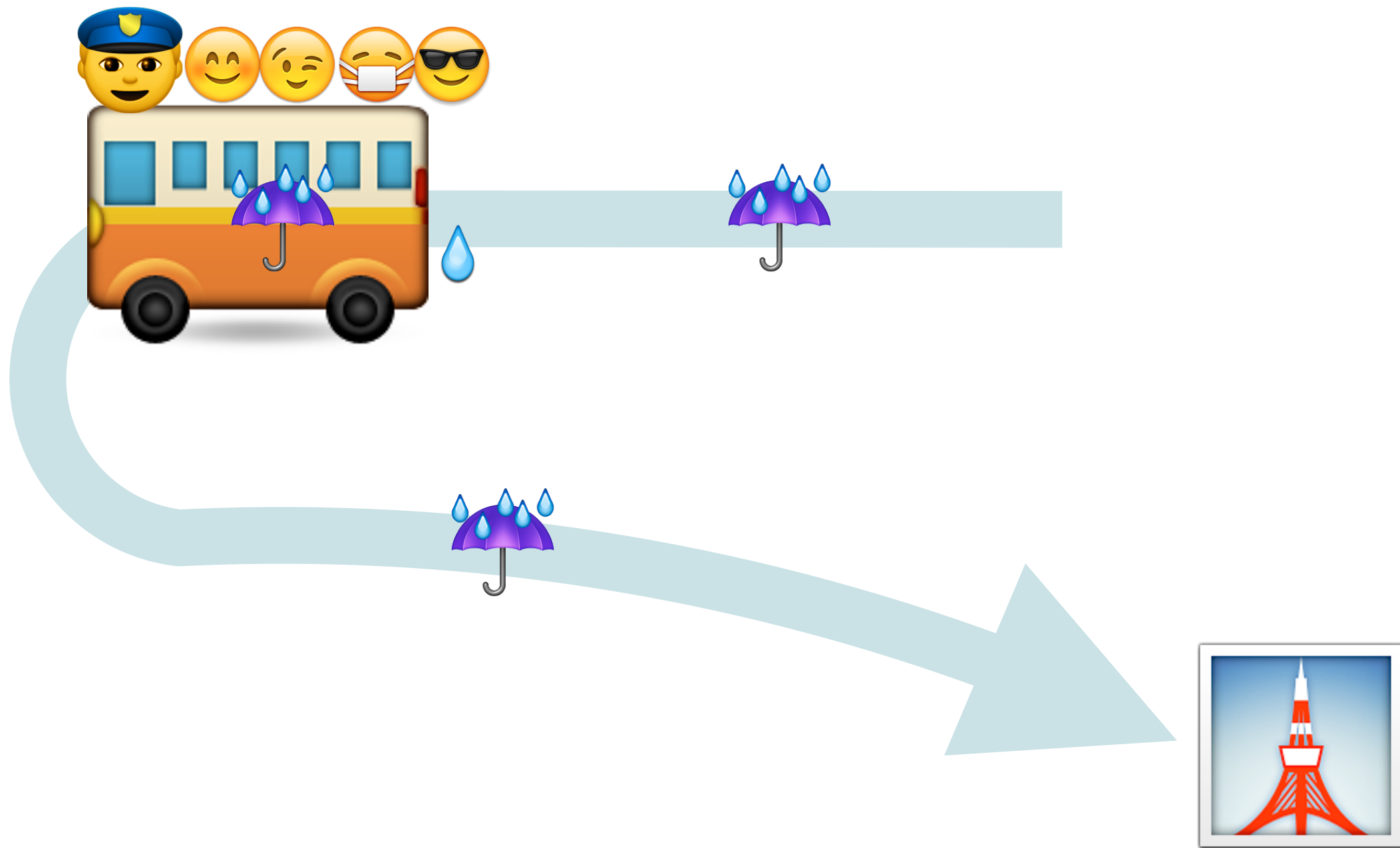
問題概要

長距離バスが目的地に行きたい



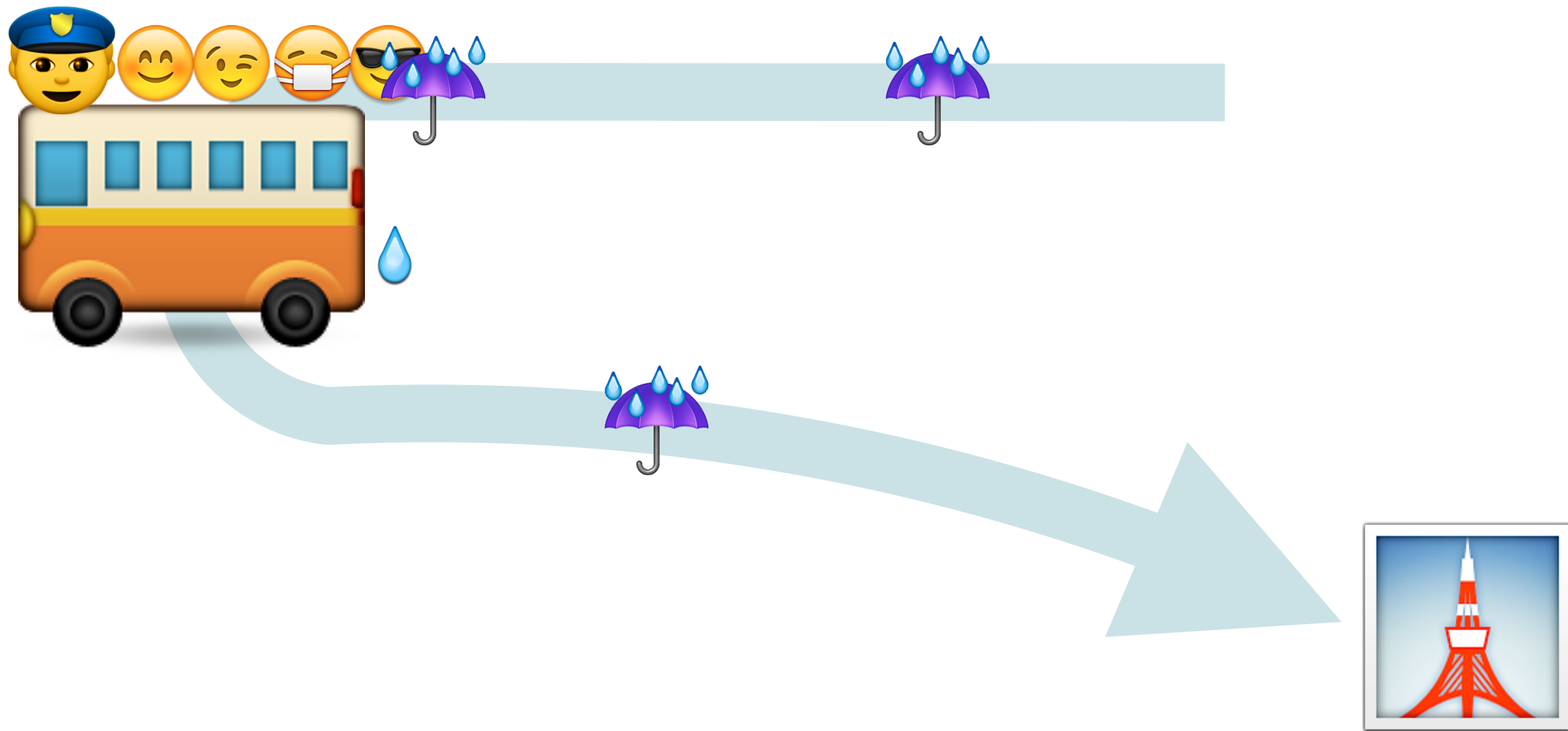
問題概要

長距離バスが目的地に行きたい



問題概要

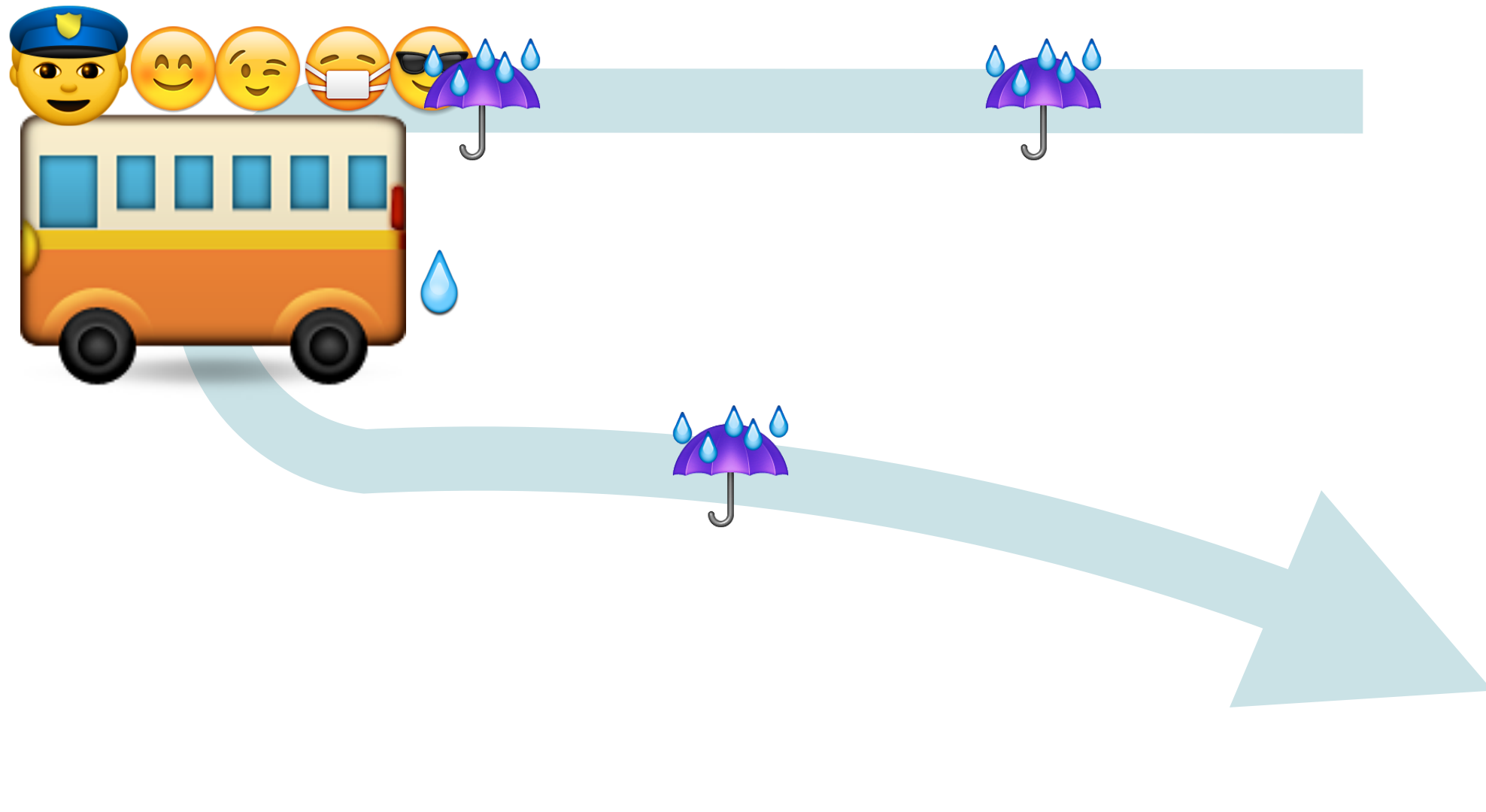
長距離バスが目的地に行きたい



問題概要

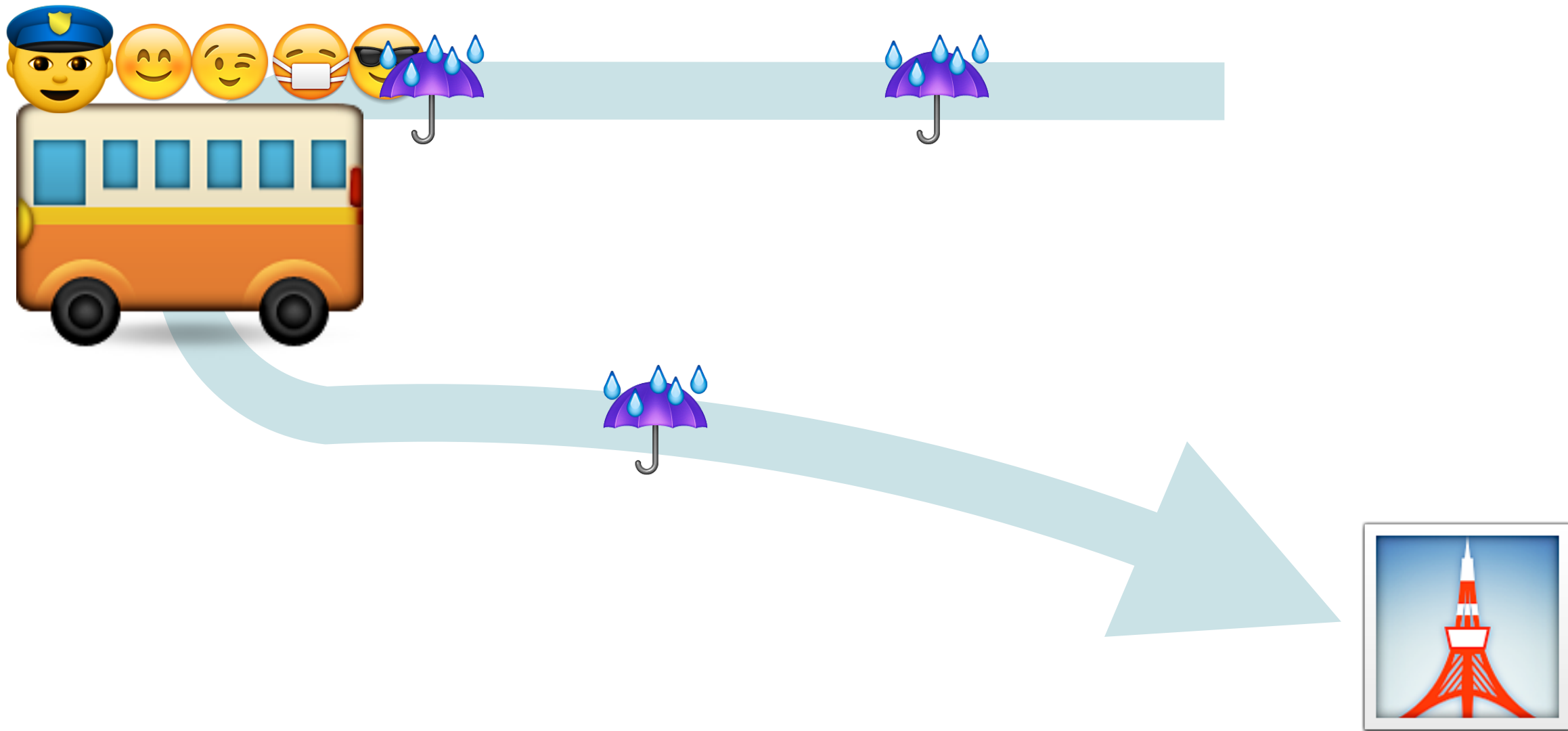
長距離バスが目的地に行きたい

水を飲みます



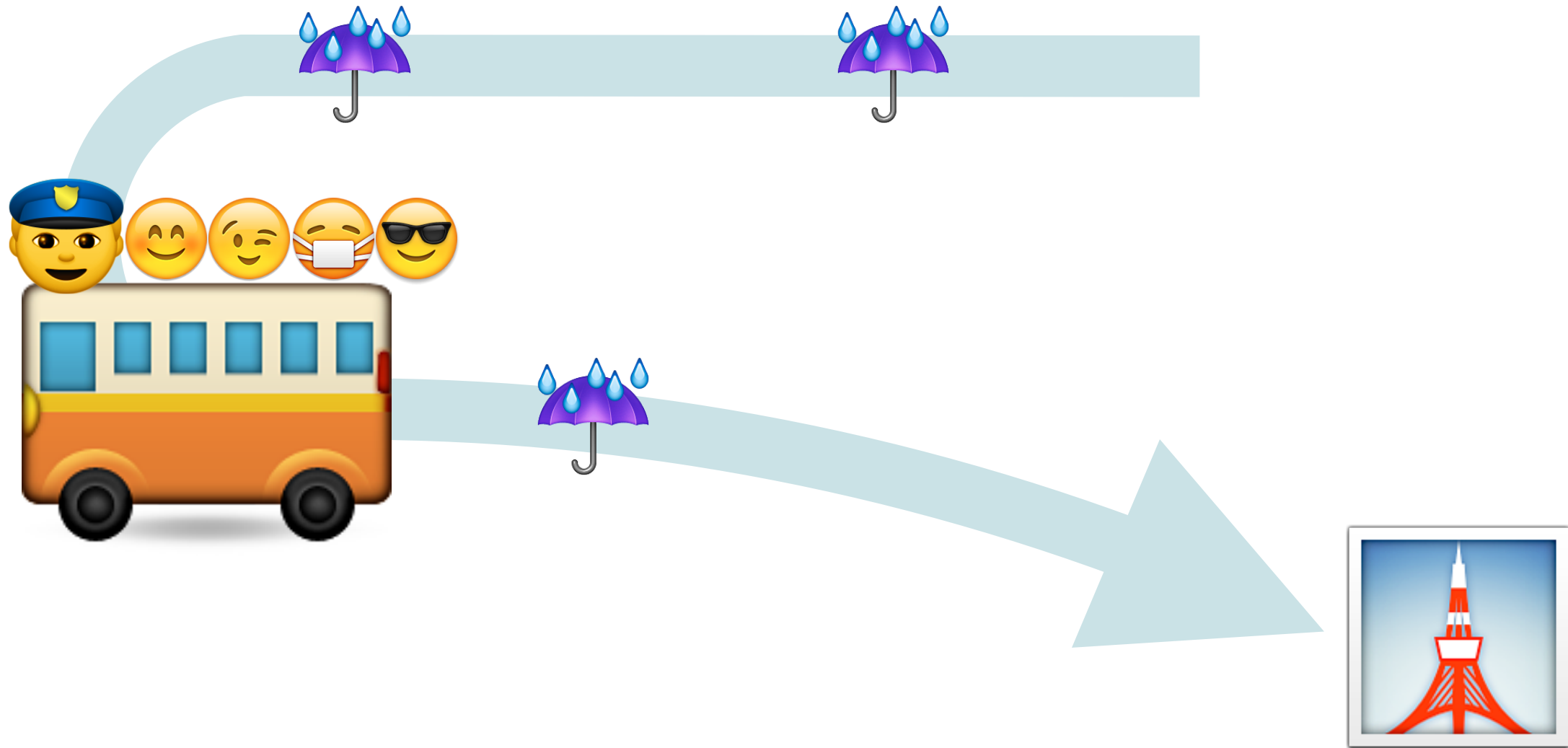
問題概要

長距離バスが目的地に行きたい



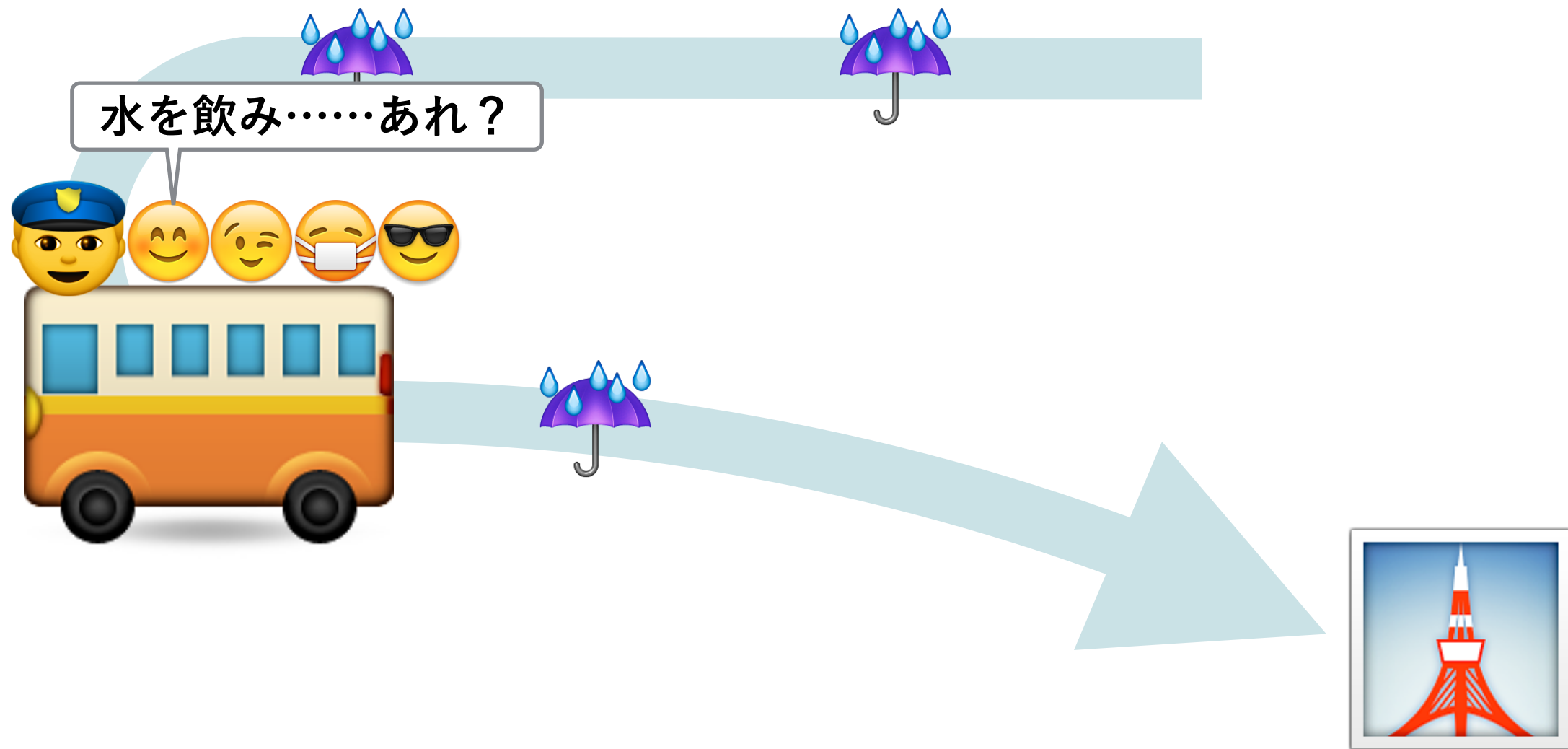
問題概要

長距離バスが目的地に行きたい



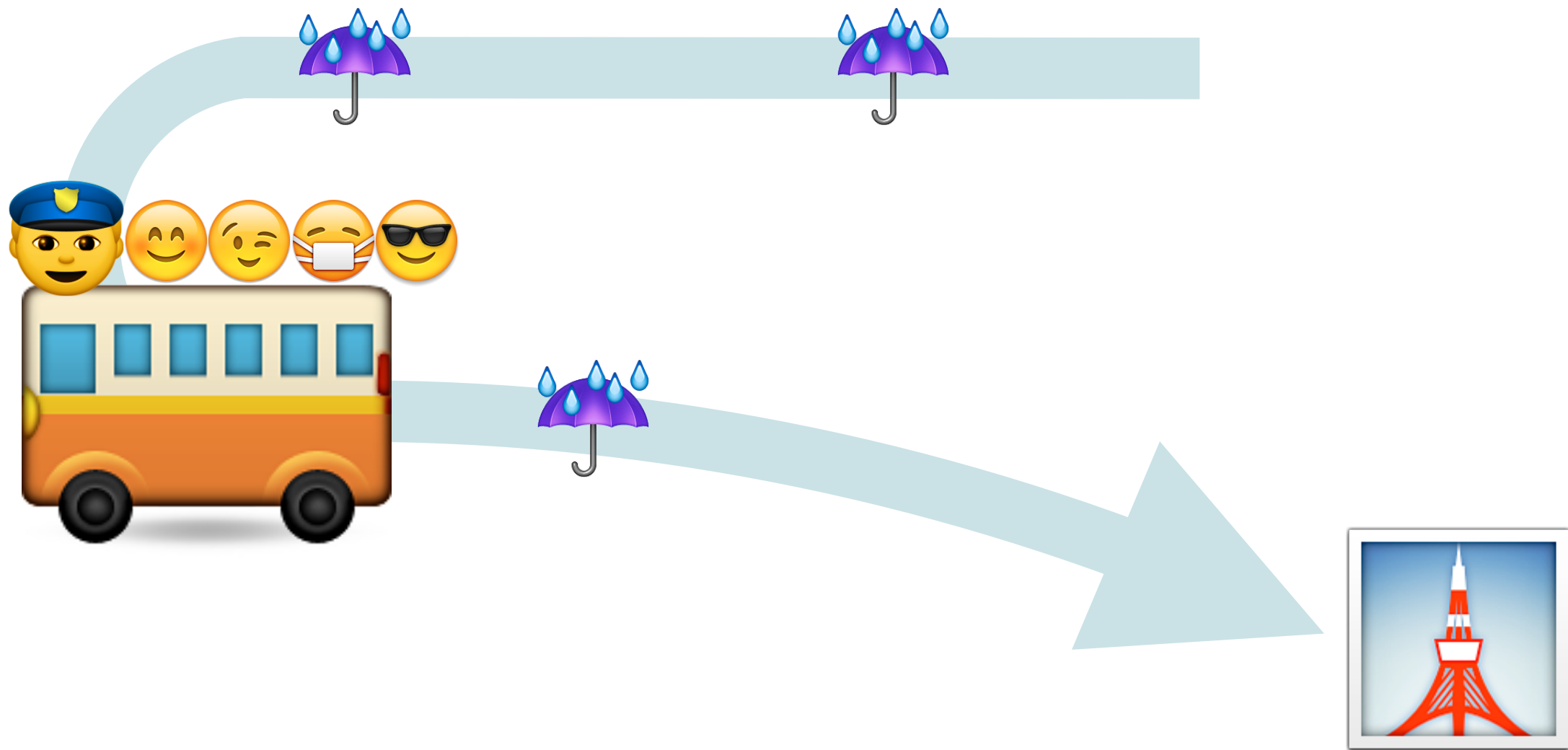
問題概要

長距離バスが目的地に行きたい



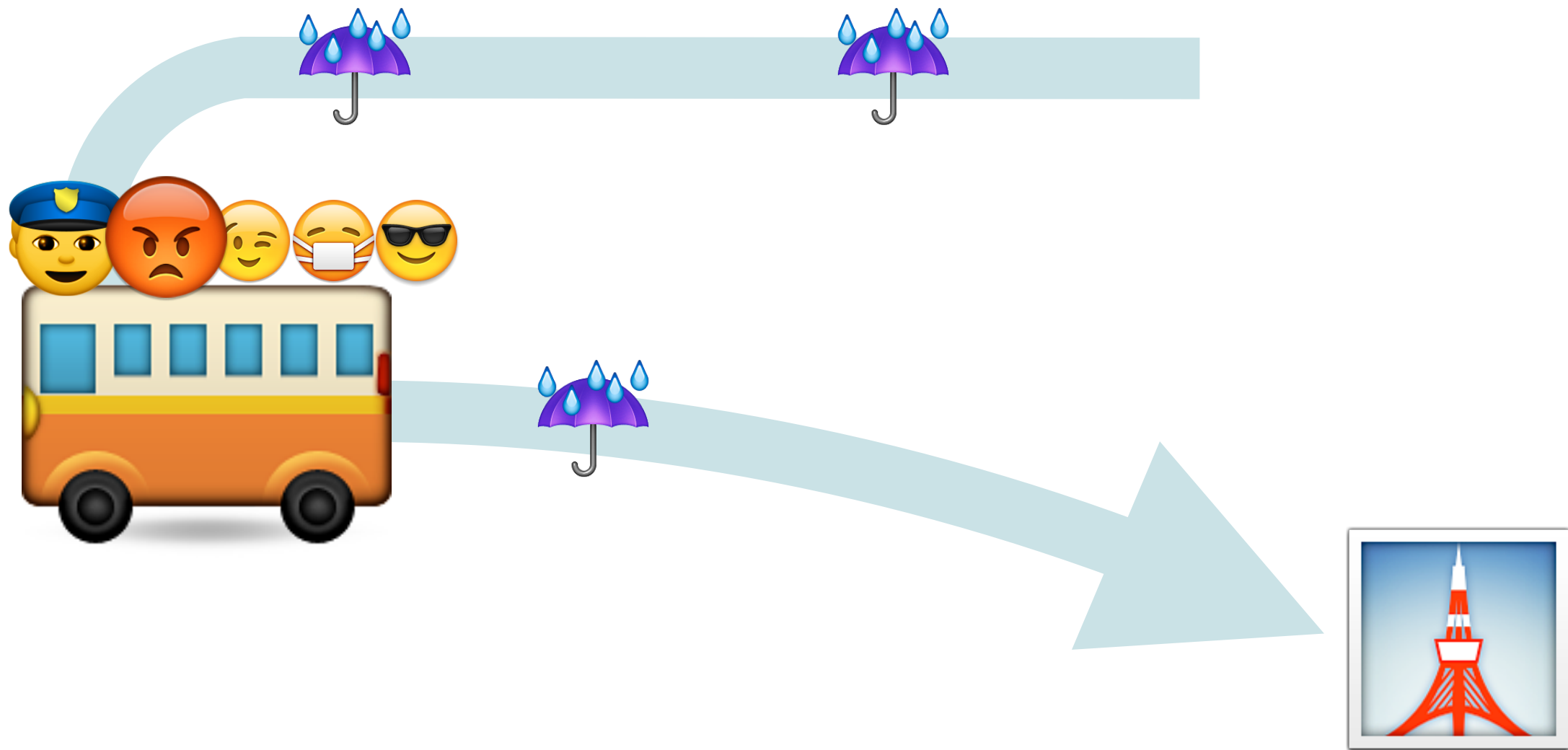
問題概要

長距離バスが目的地に行きたい



問題概要

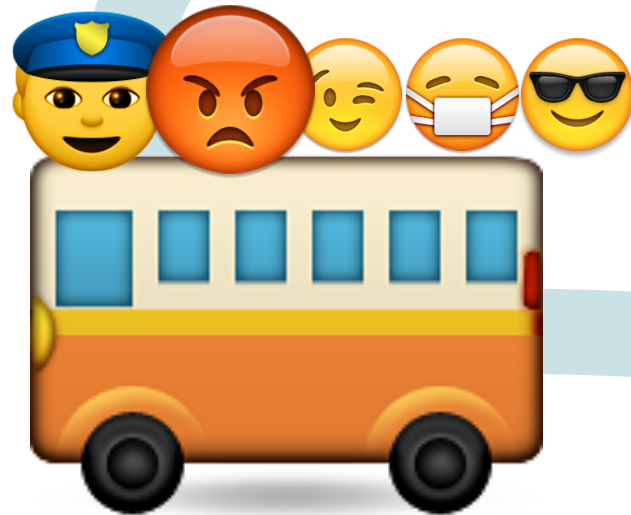
長距離バスが目的地に行きたい



問題概要

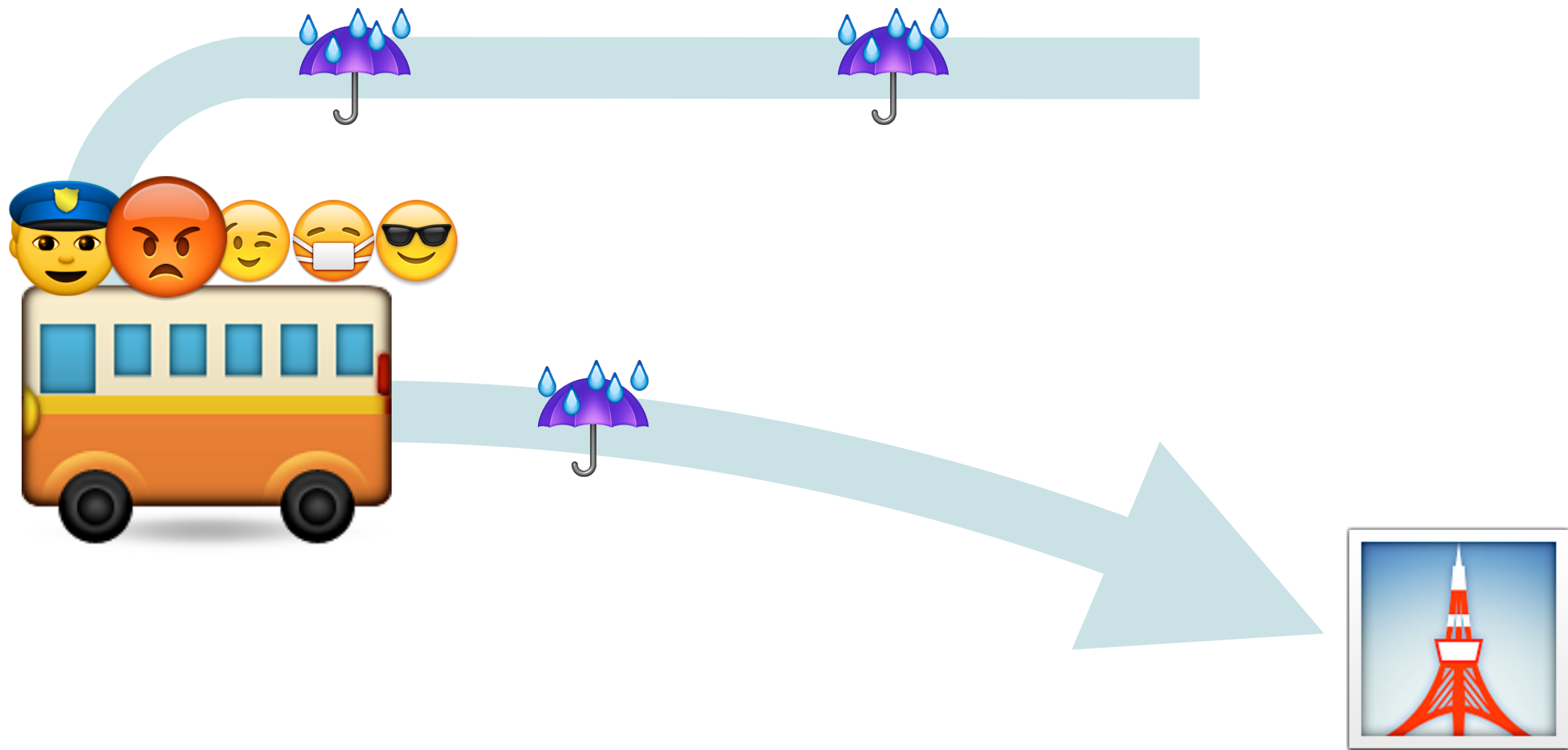
長距離バスが目的地に行きたい

水のないバスに乗っていただけるか！
帰らせてもらおう！！！！



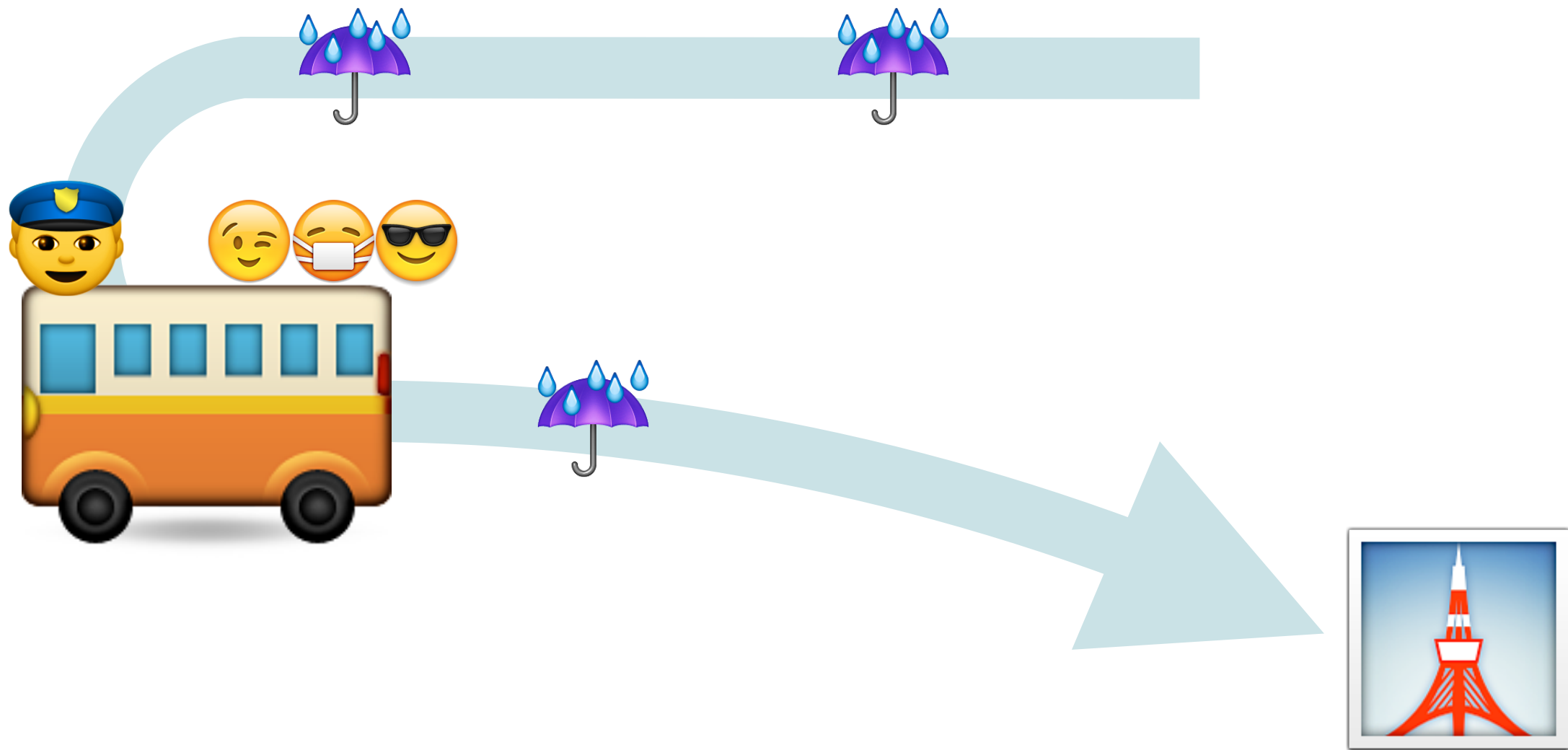
問題概要

長距離バスが目的地に行きたい



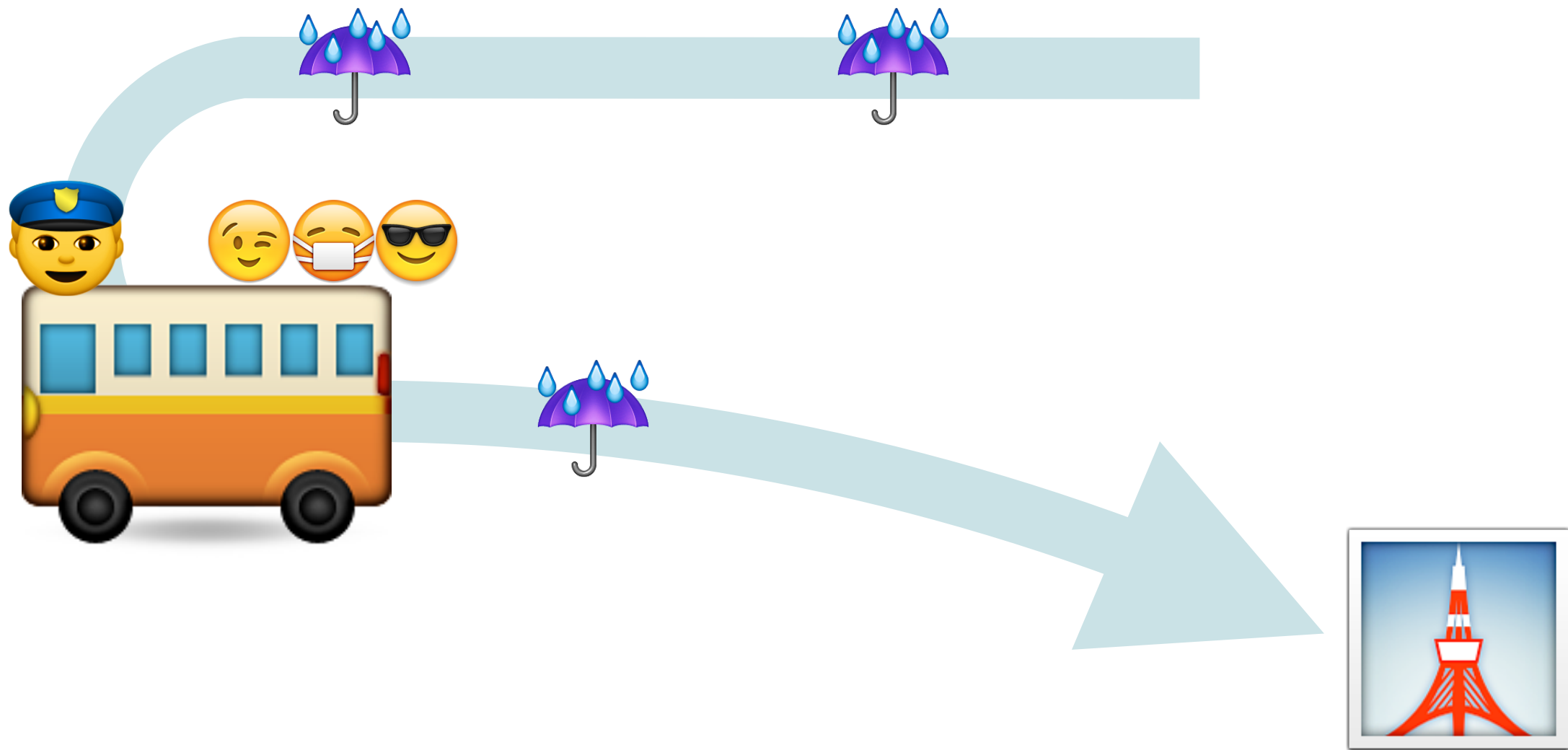
問題概要

長距離バスが目的地に行きたい



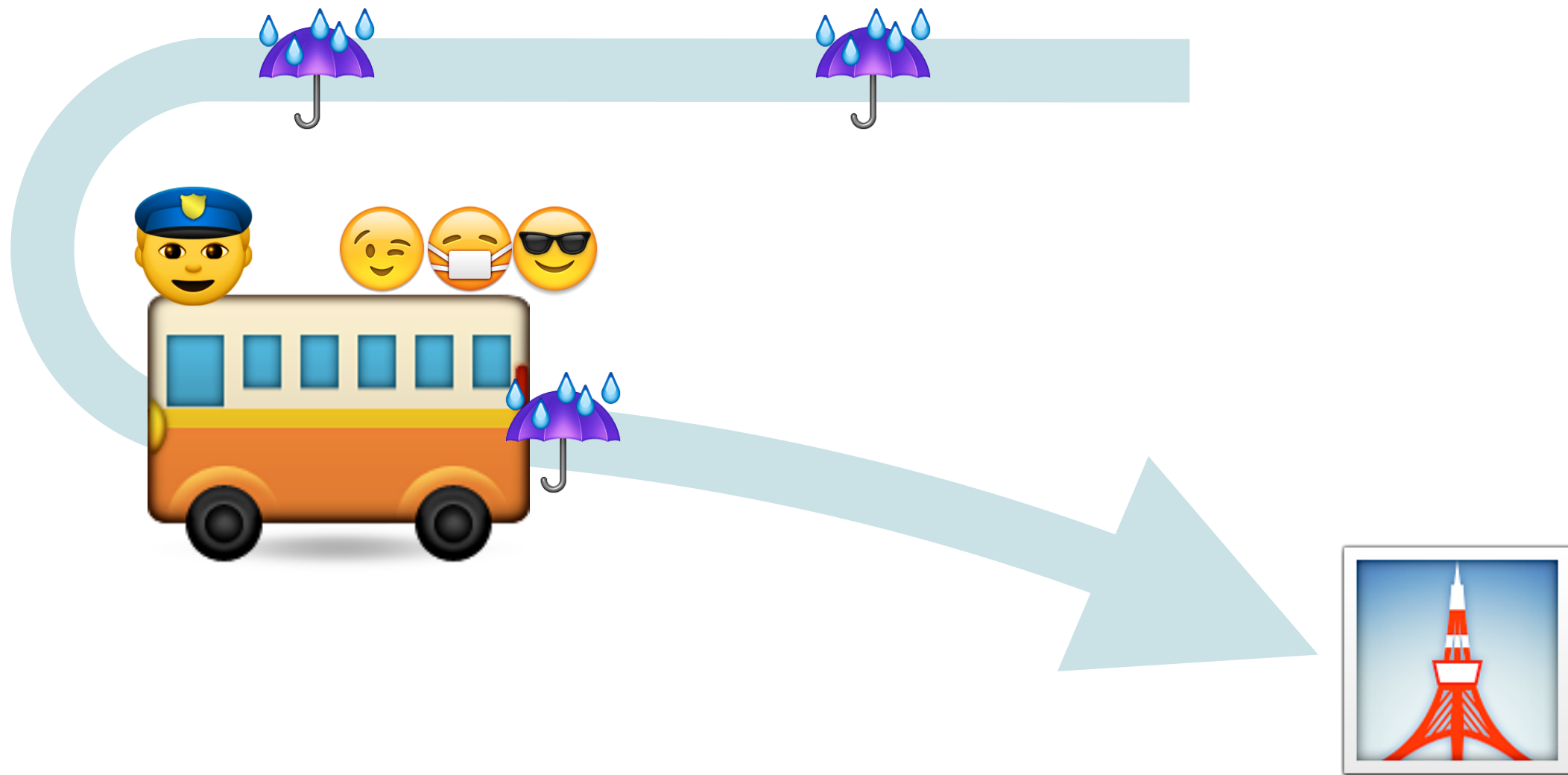
問題概要

長距離バスが目的地に行きたい



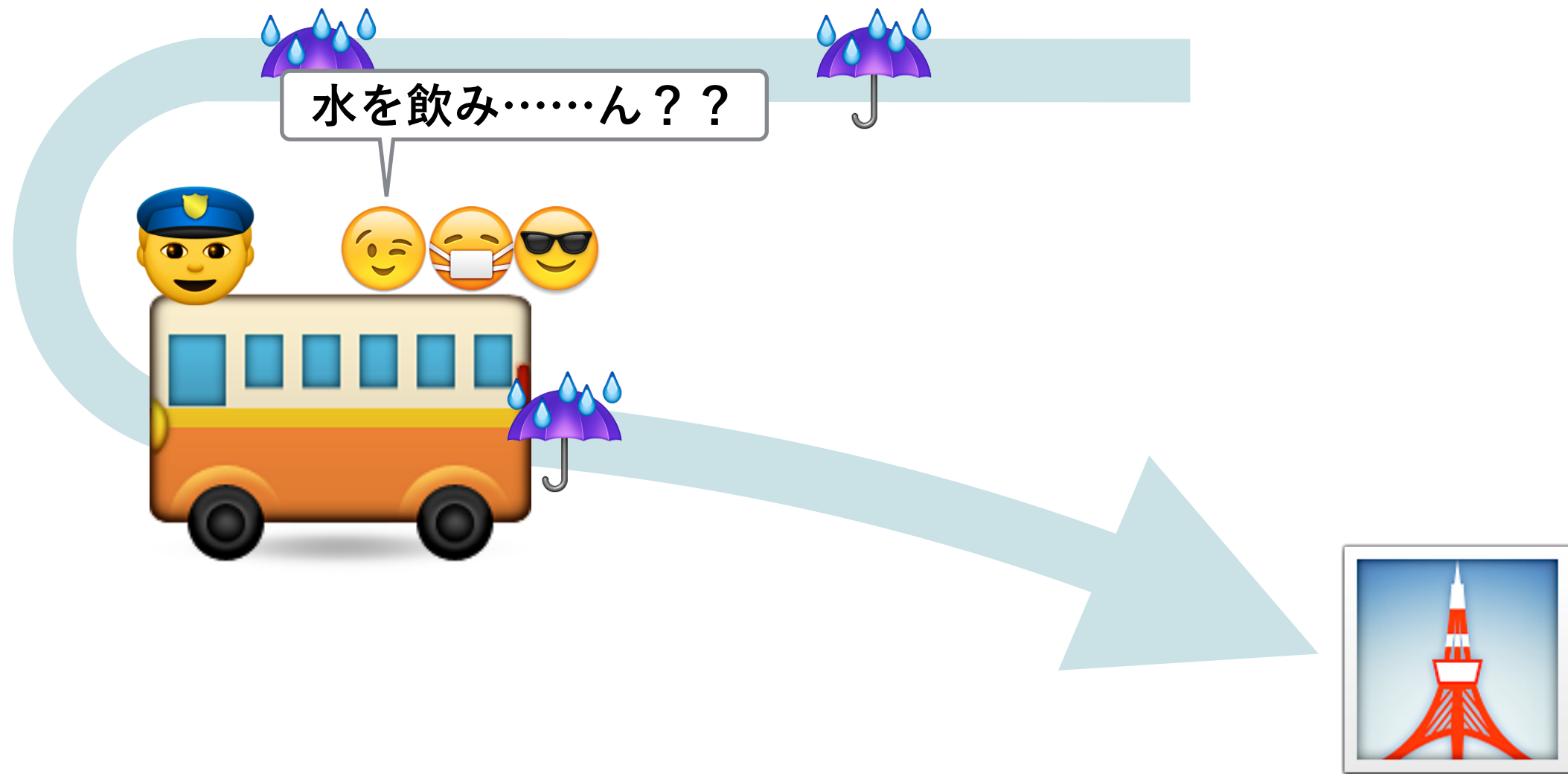
問題概要

長距離バスが目的地に行きたい



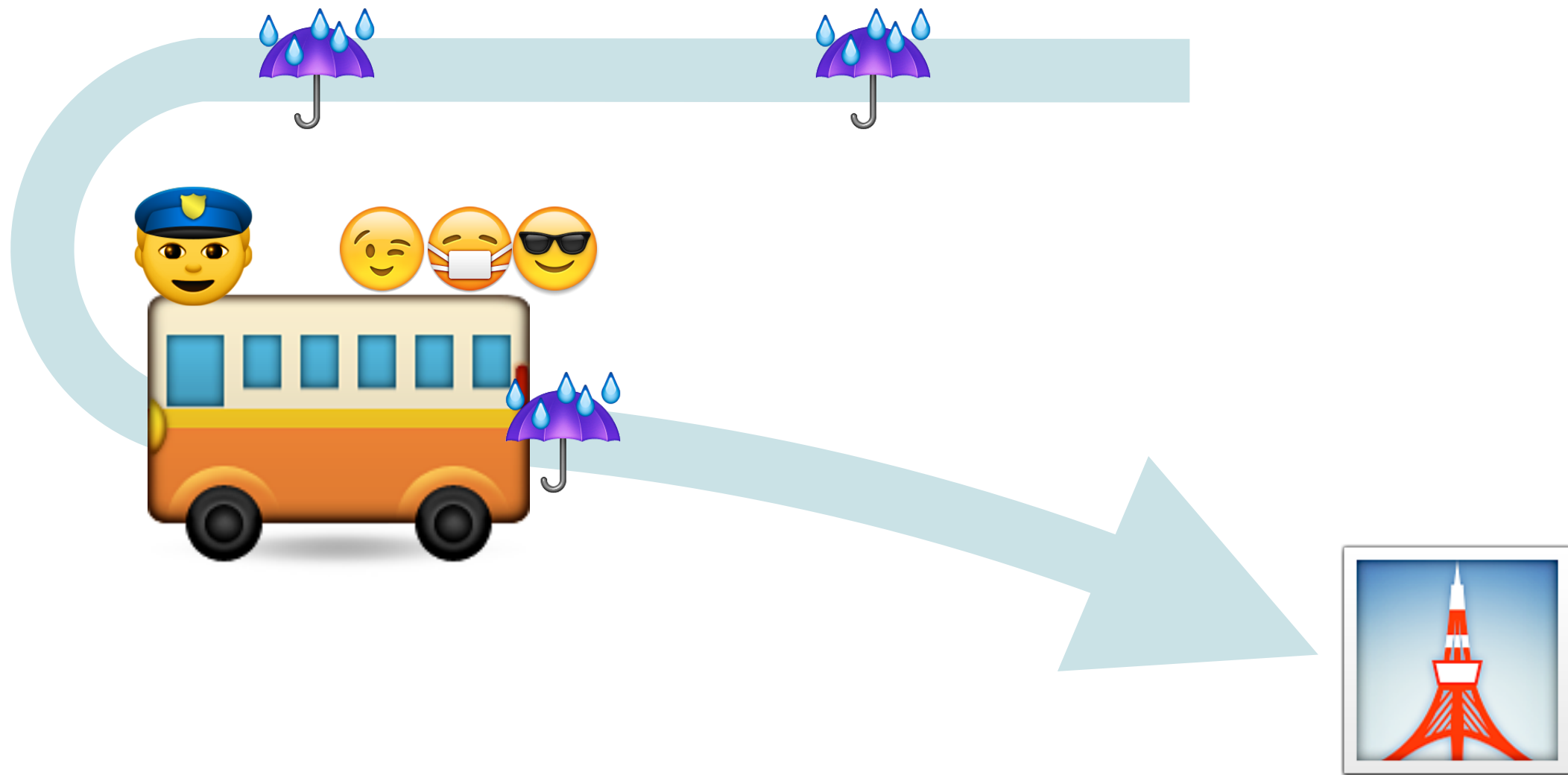
問題概要

長距離バスが目的地に行きたい



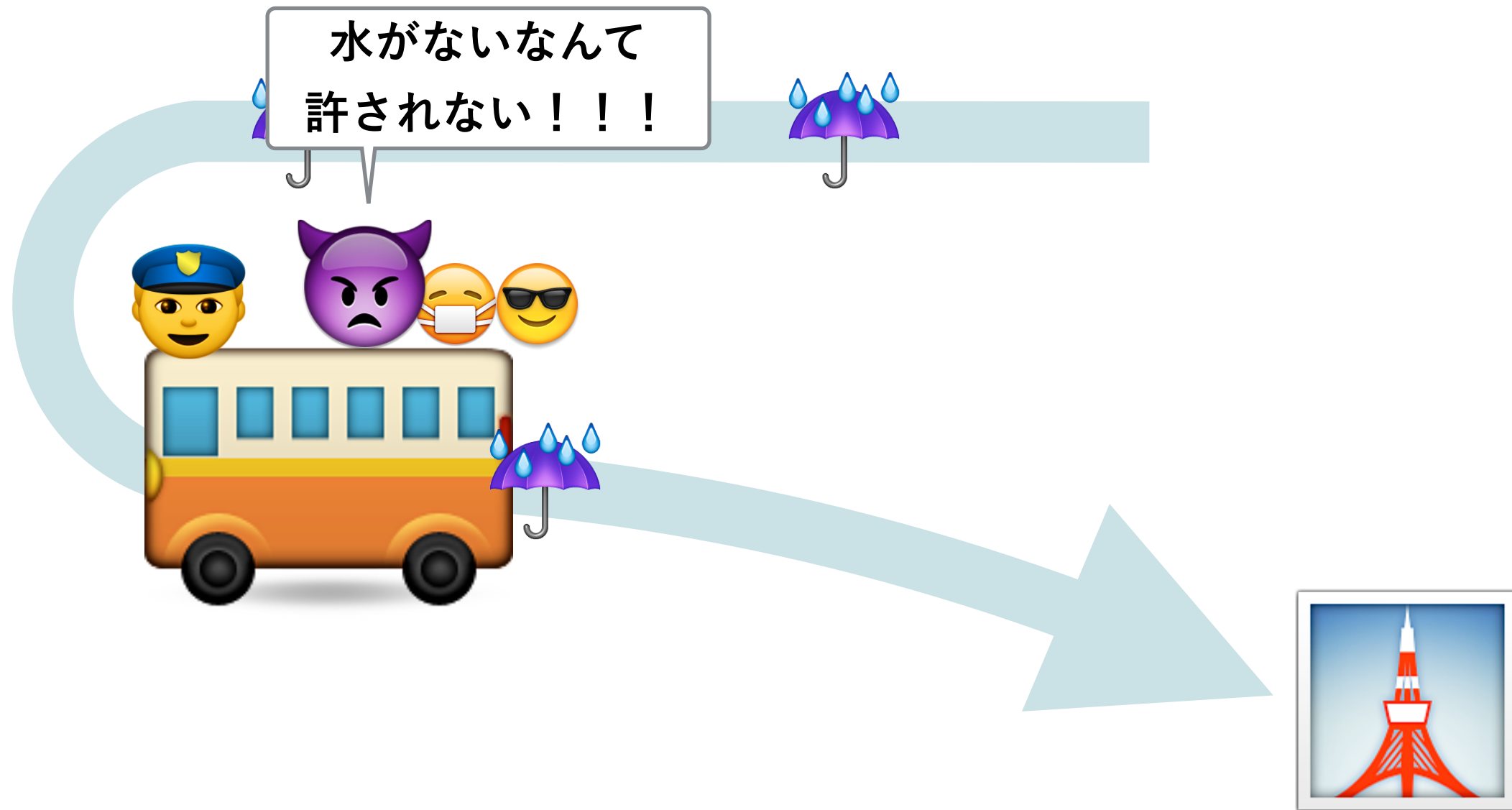
問題概要

長距離バスが目的地に行きたい



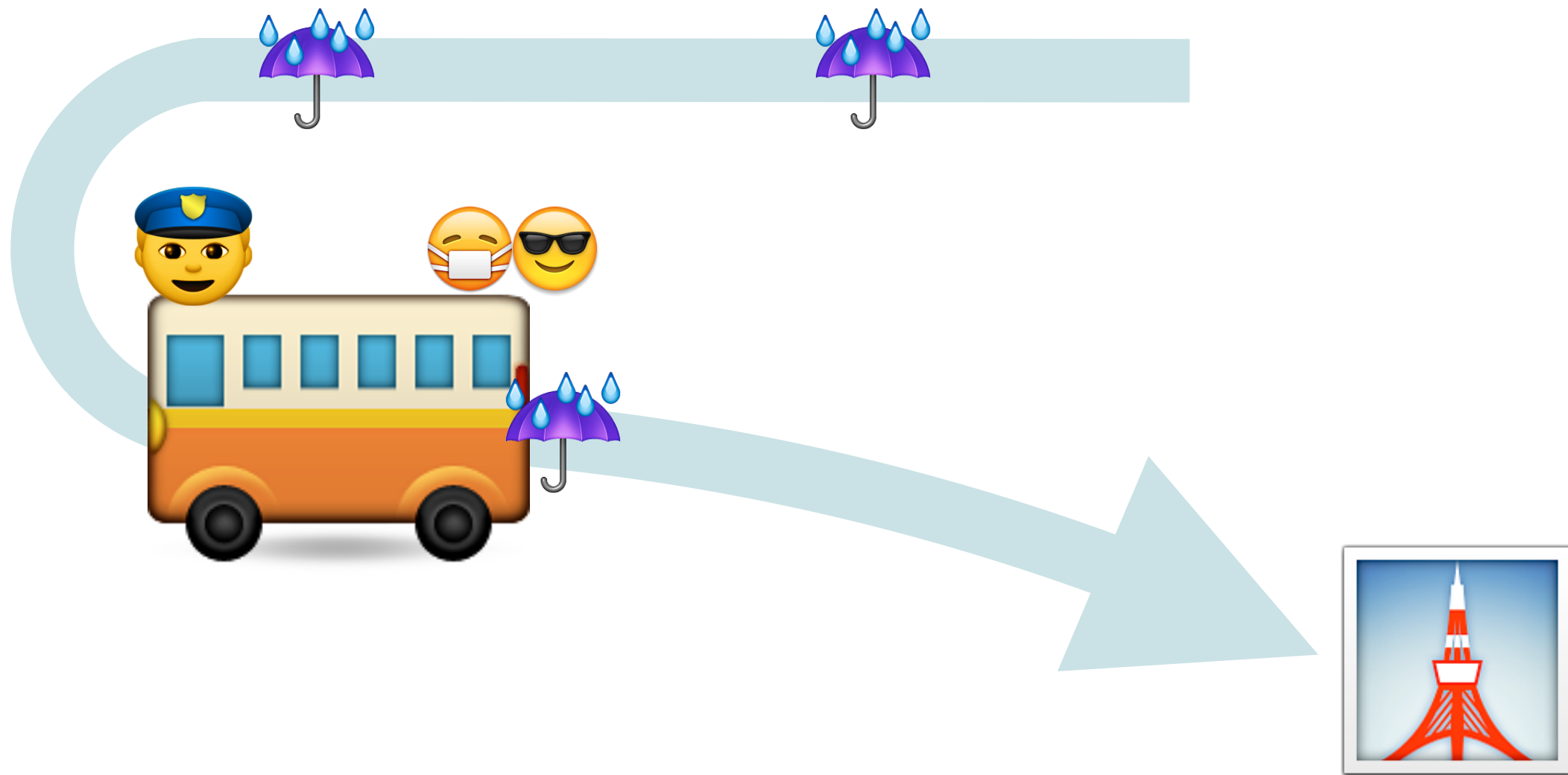
問題概要

長距離バスが目的地に行きたい



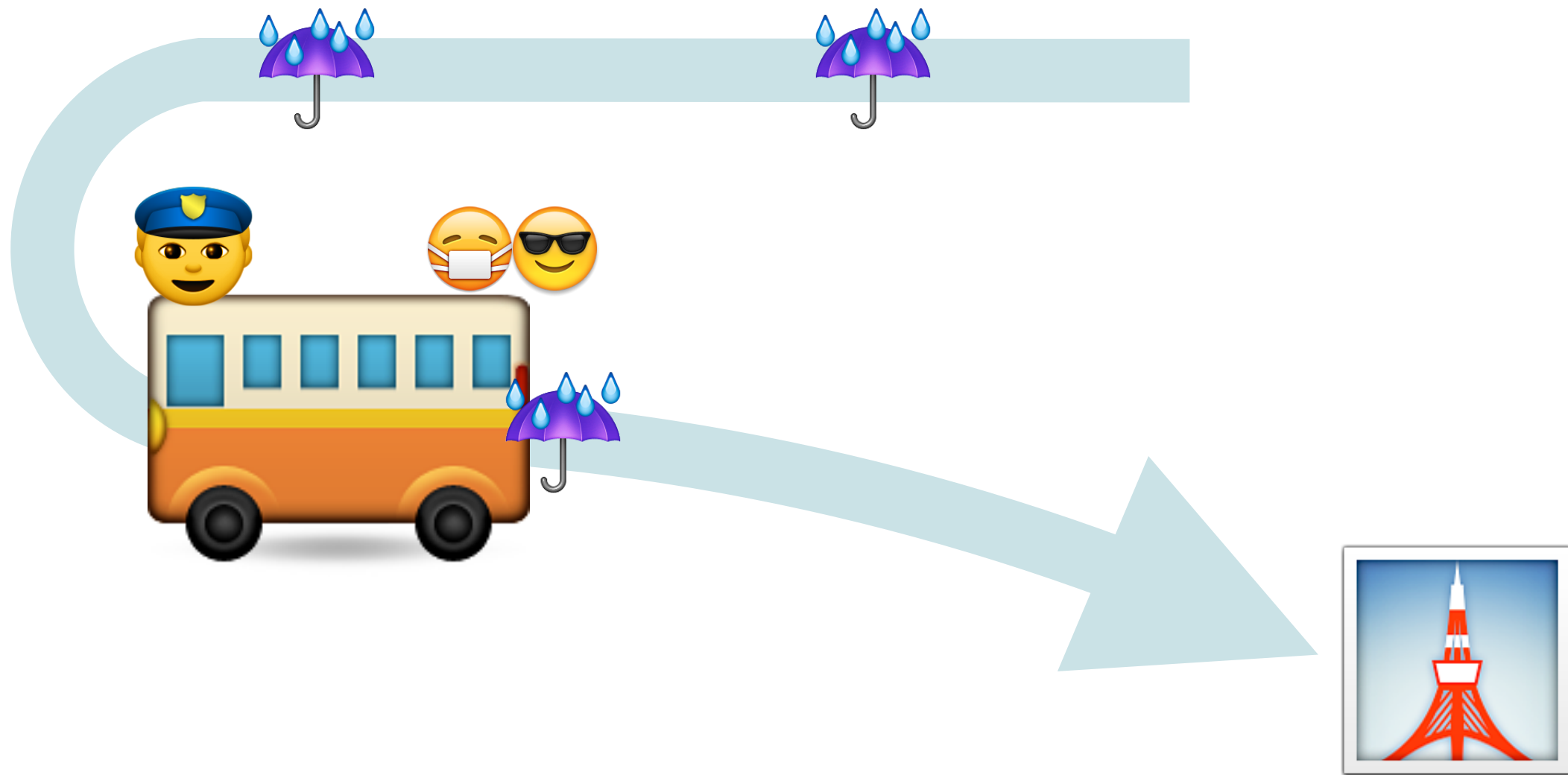
問題概要

長距離バスが目的地に行きたい



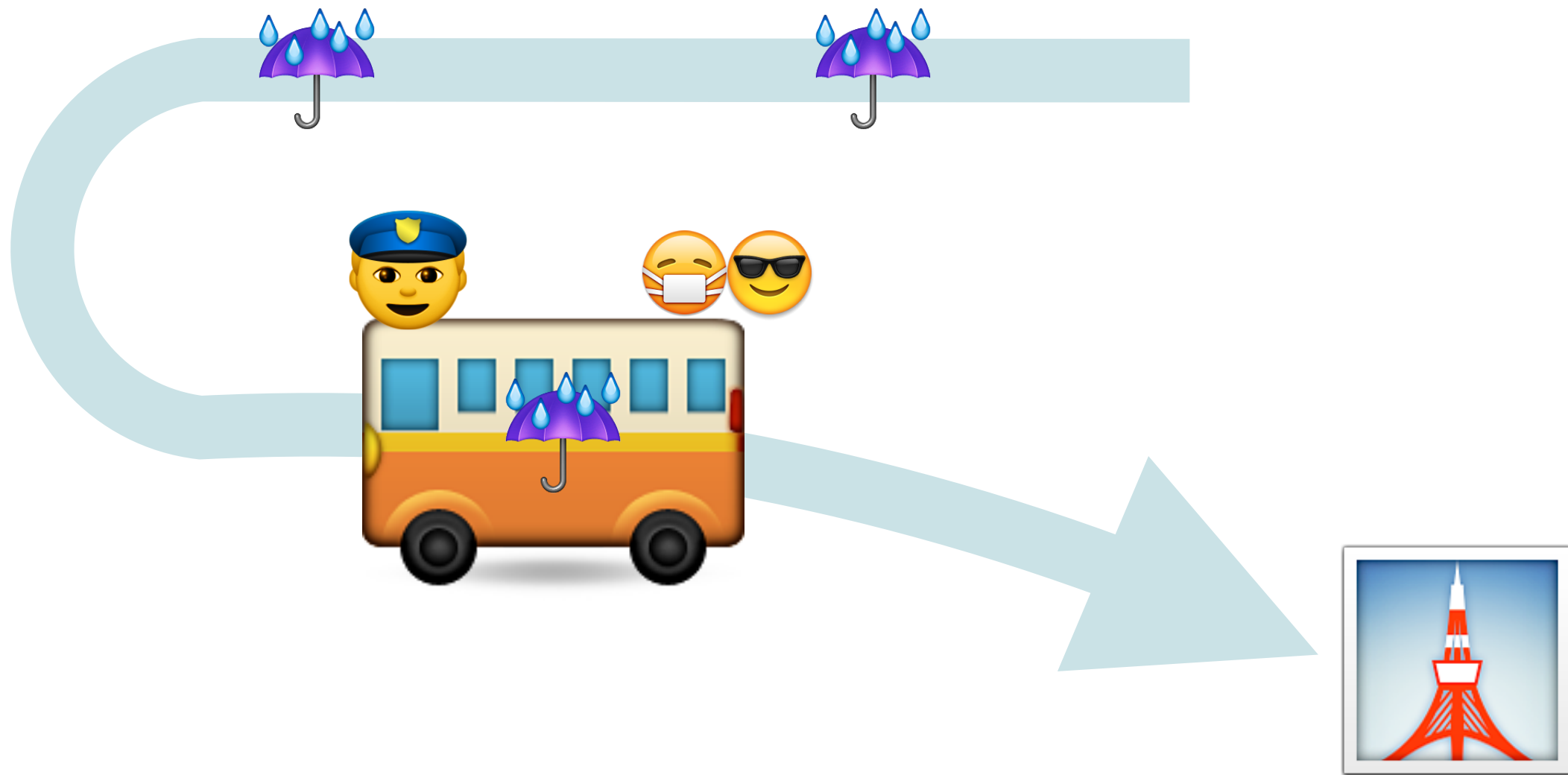
問題概要

長距離バスが目的地に行きたい



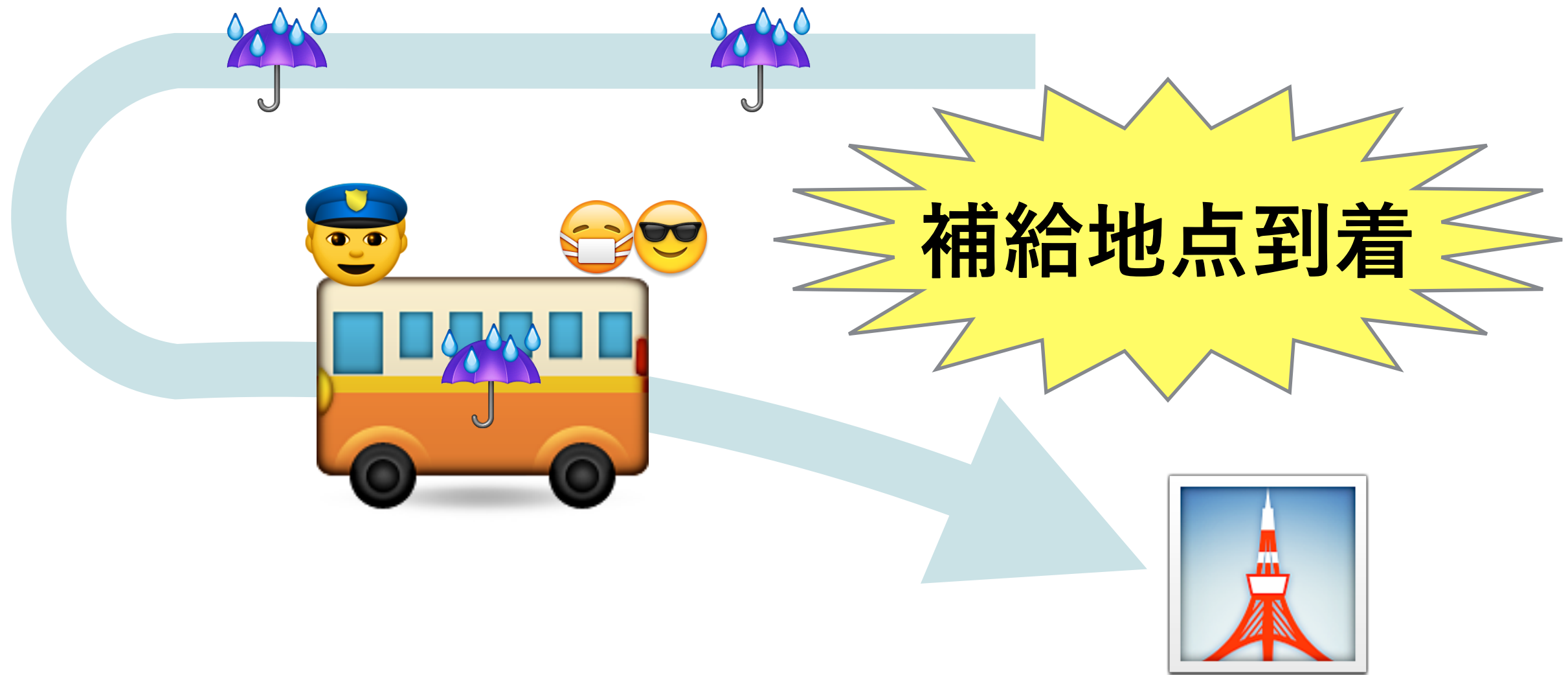
問題概要

長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい



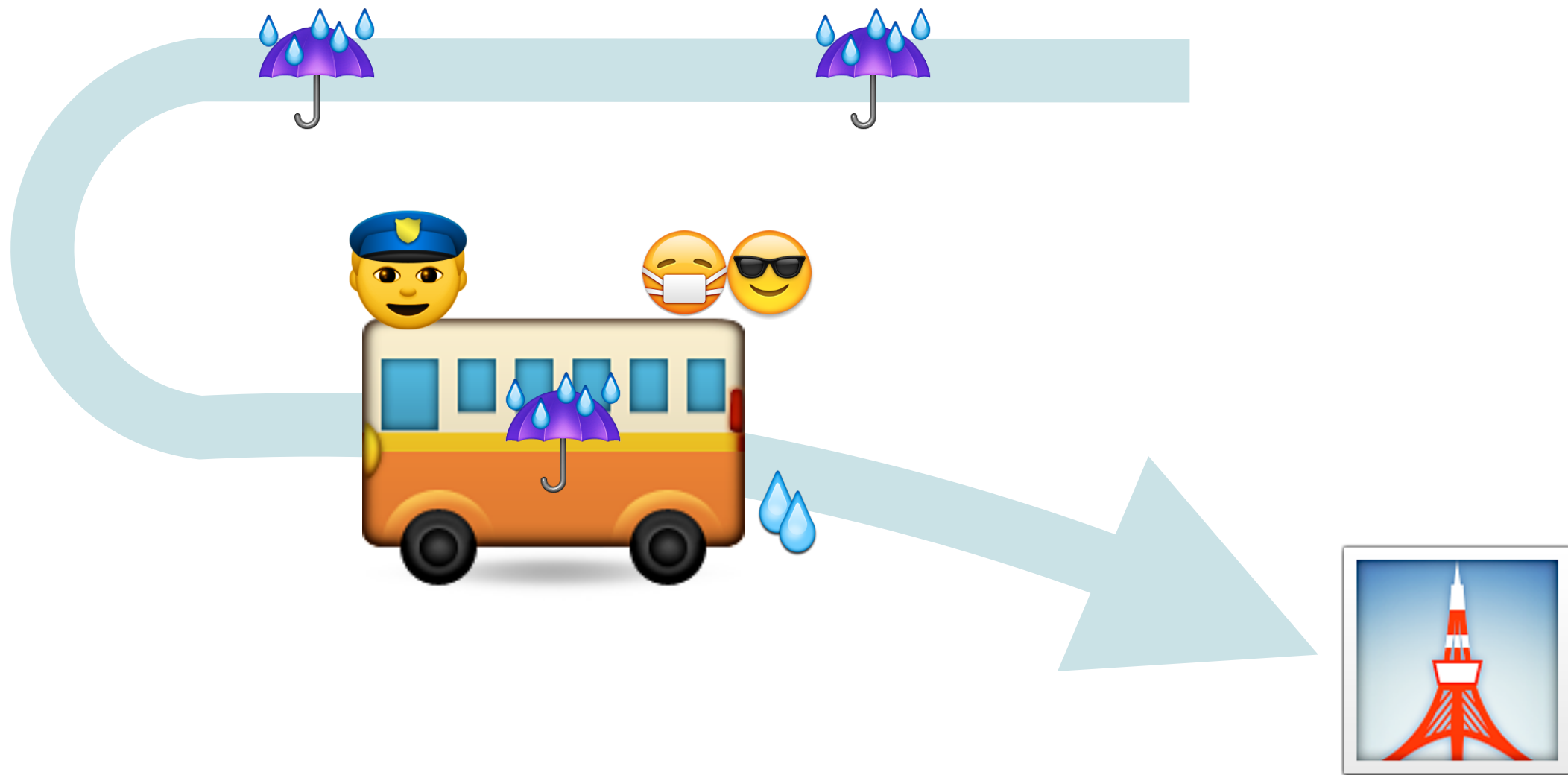
問題概要

長距離バスが目的地に行きたい



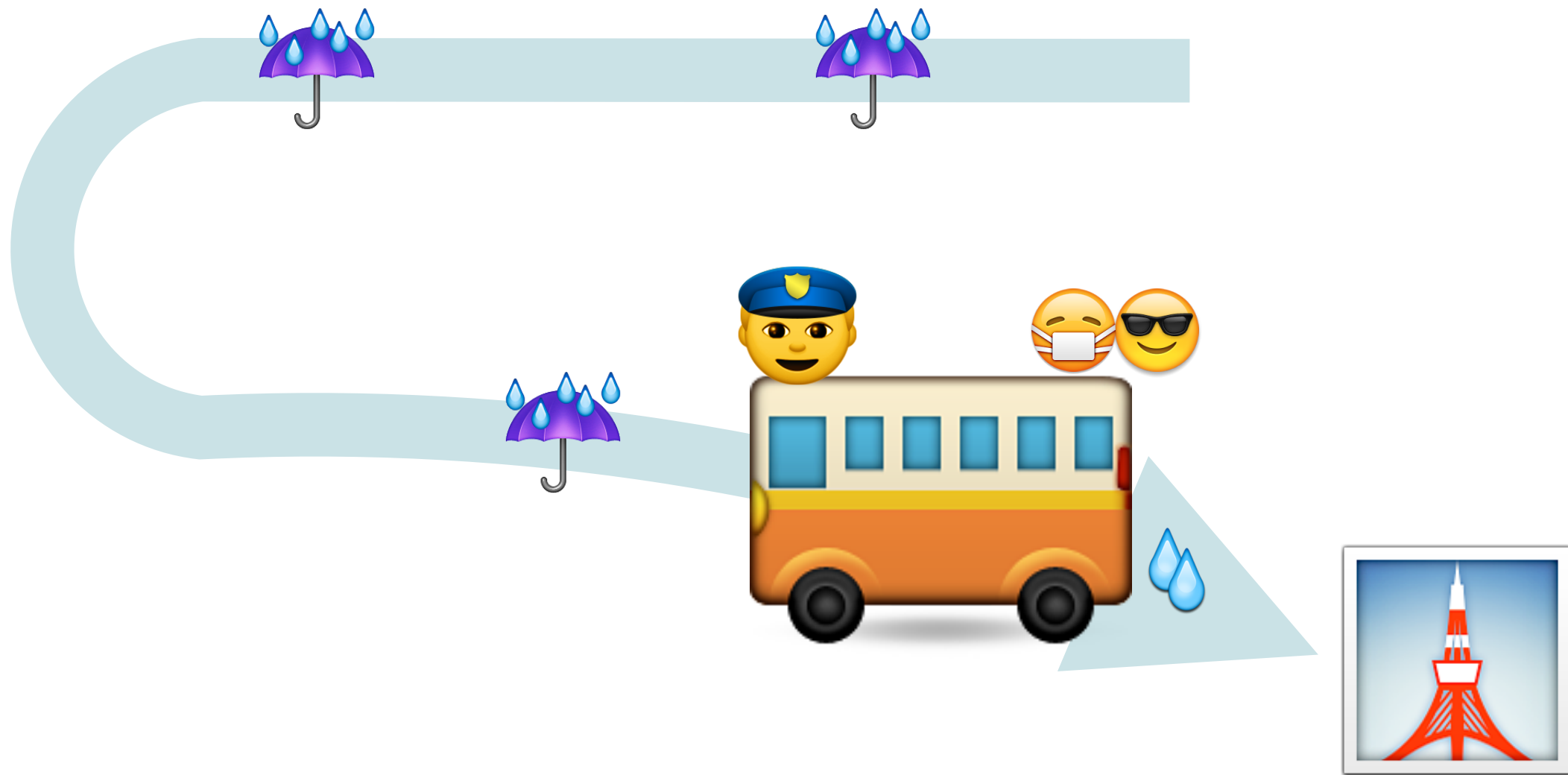
問題概要

長距離バスが目的地に行きたい



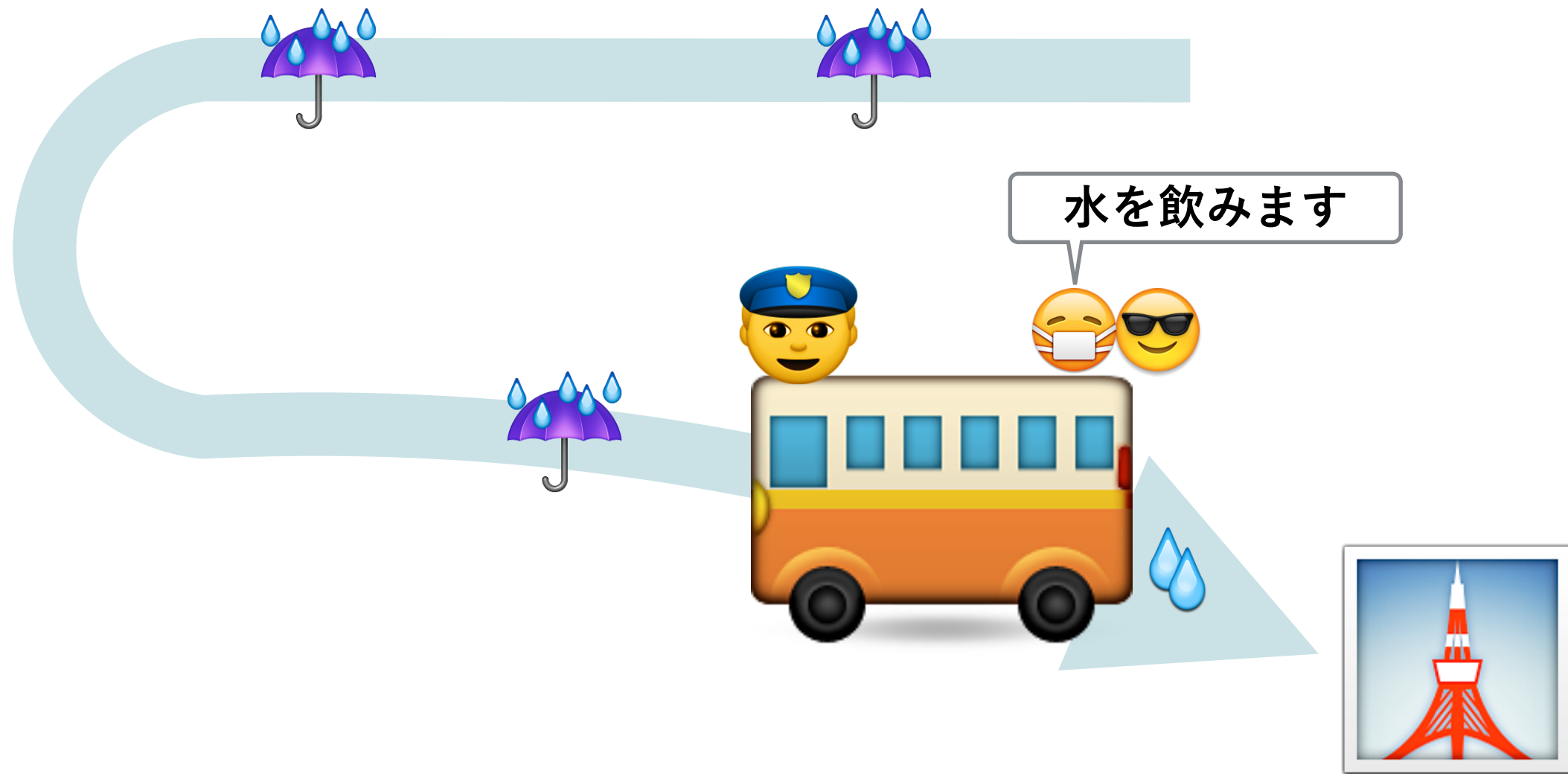
問題概要

長距離バスが目的地に行きたい



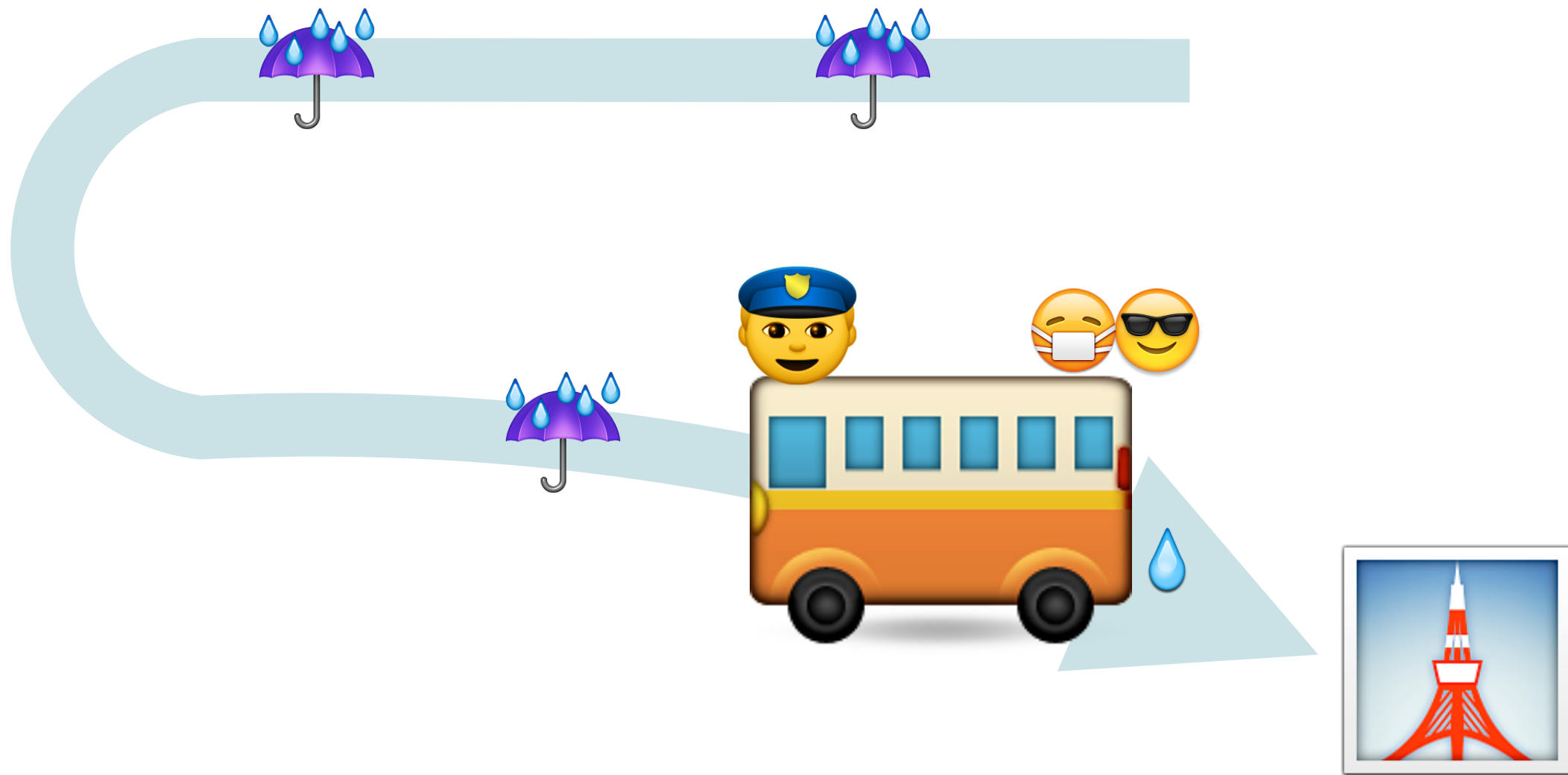
問題概要

長距離バスが目的地に行きたい



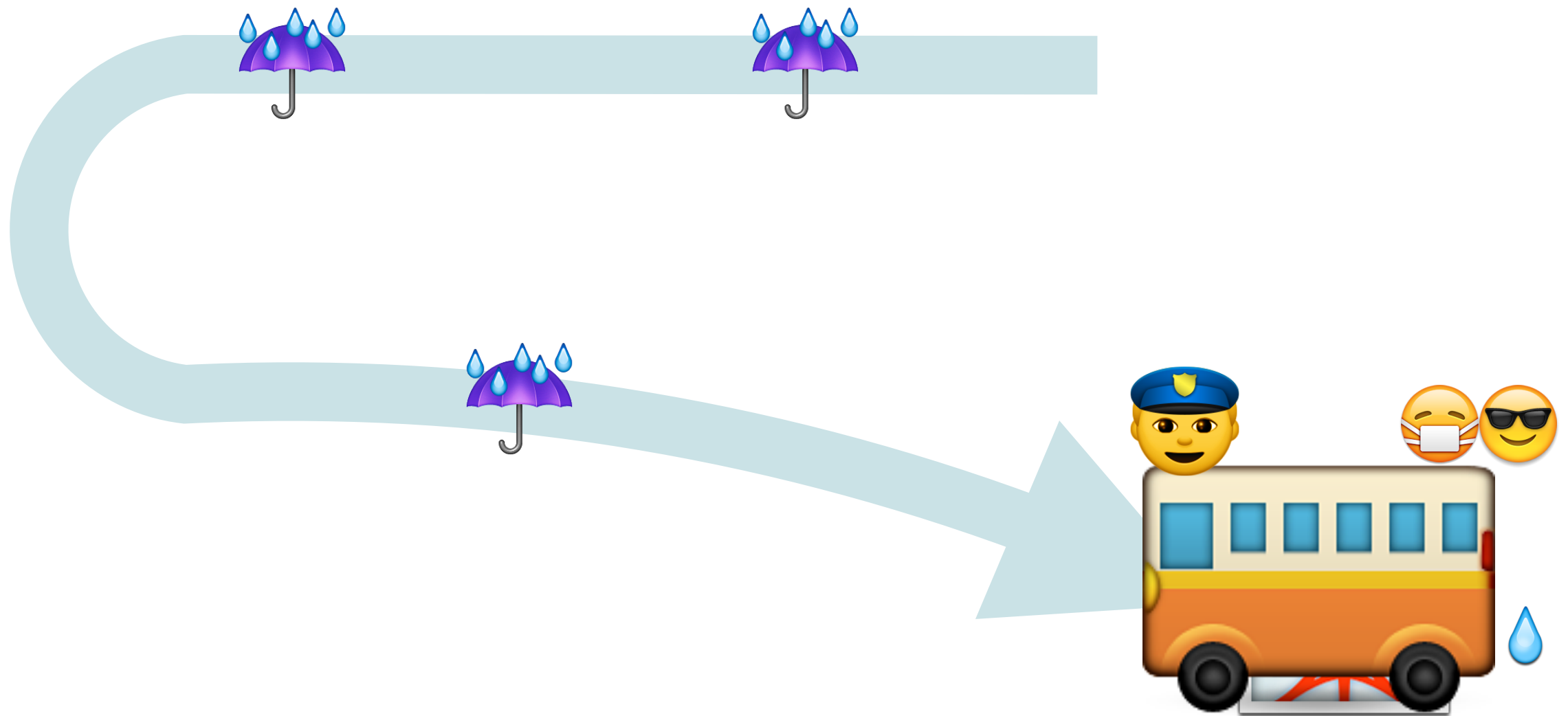
問題概要

長距離バスが目的地に行きたい



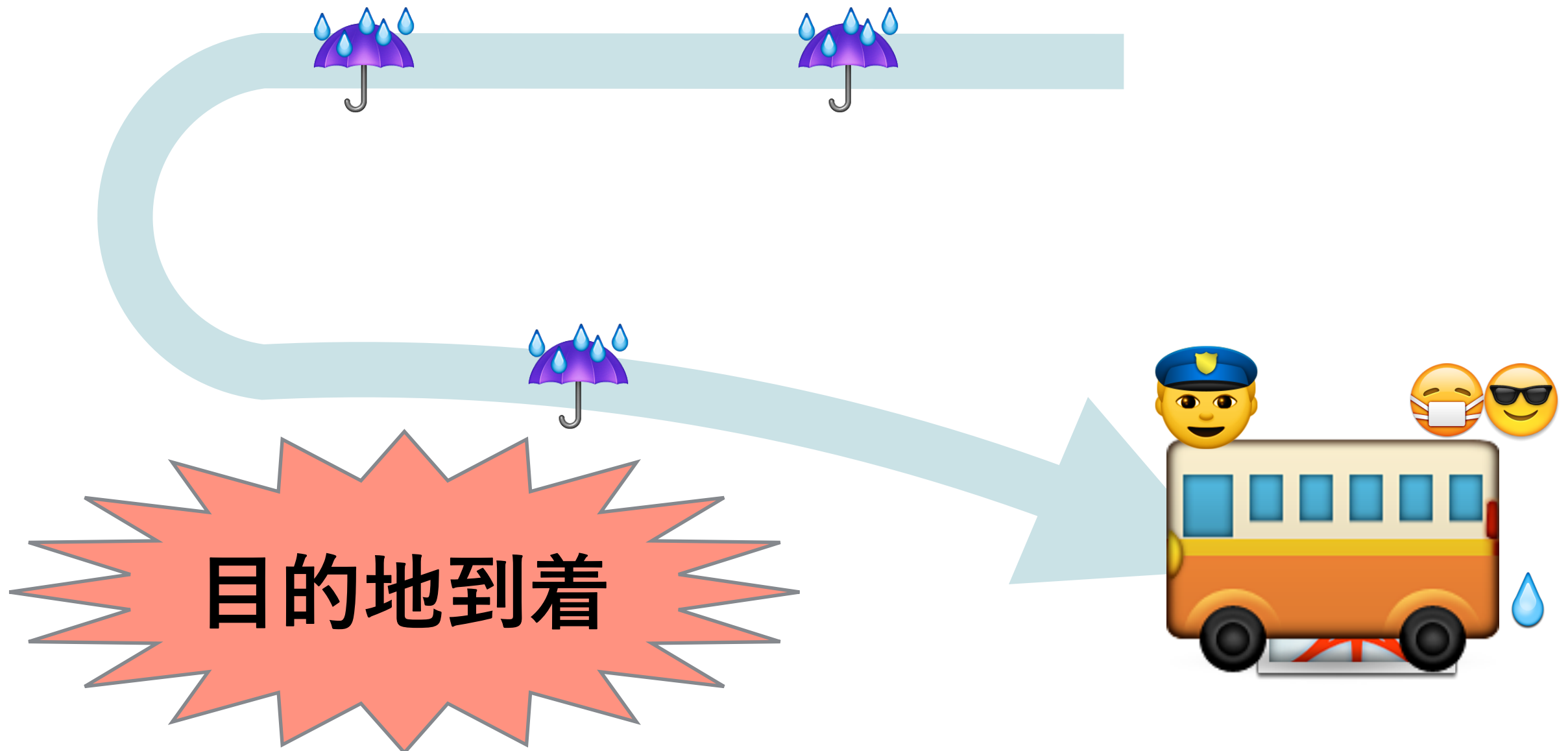
問題概要

長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい



問題概要

長距離バスが目的地に行きたい

- ▶ 途中の補給地点で給水器🌧️に💧を補給できる
 - 💧 1個で W 円かかる
- ▶ 運転手👮と乗客😊😌😷😎が**周期的に**💧を飲む
 - 乗客 j は時刻 $D_j + kT$ に💧を 1 個飲む
- ▶ 💧が無いと客は怒って帰る
 - 乗客 j が帰る 😊→😡と C_j 円かかる
 - **運転手が帰ると** 💣なので回避せねばならない

制約

💧 を飲む周期 目的地🗼までの時間

▶ $1 \leq T \leq X \leq 1,000,000,000,000$

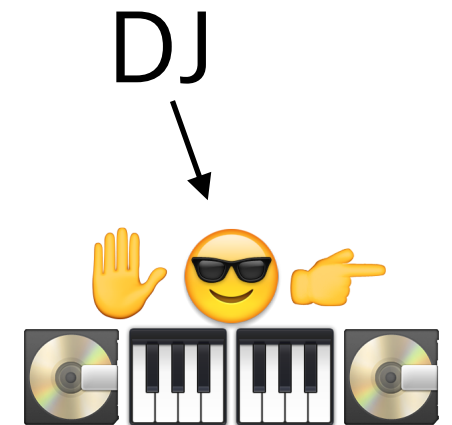
▶ $N \leq 200,000$ ←  (N 個)

▶ $M \leq 200,000$ ←  (M 人)

▶ $1 \leq D_j < T$ ← 💧 を最初に飲む時刻

▶ $1 \leq C_j \leq 1,000,000,000$

 →  のコスト



小課題 1 (16点)

▶ $N \leq 8$



< はい全探索で常勝で～す

▶ $M \leq 8$

? ? ? ? ? ? ? ? ? ?

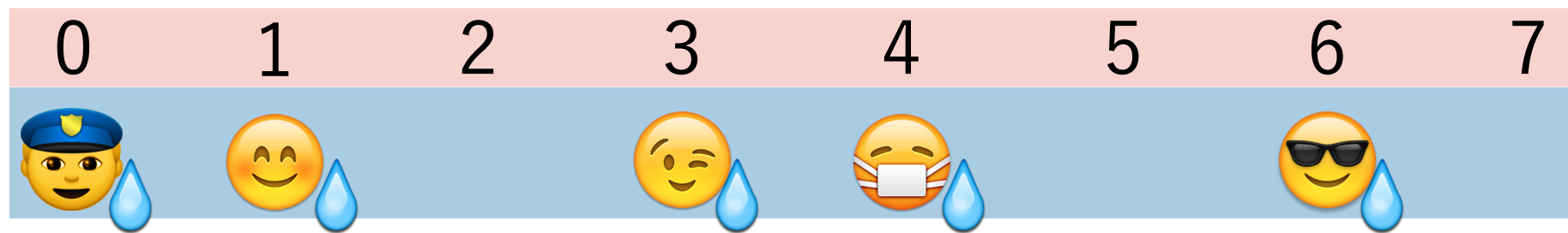
▶ $1 \leq T \leq X \leq 1,000,000,000,000$



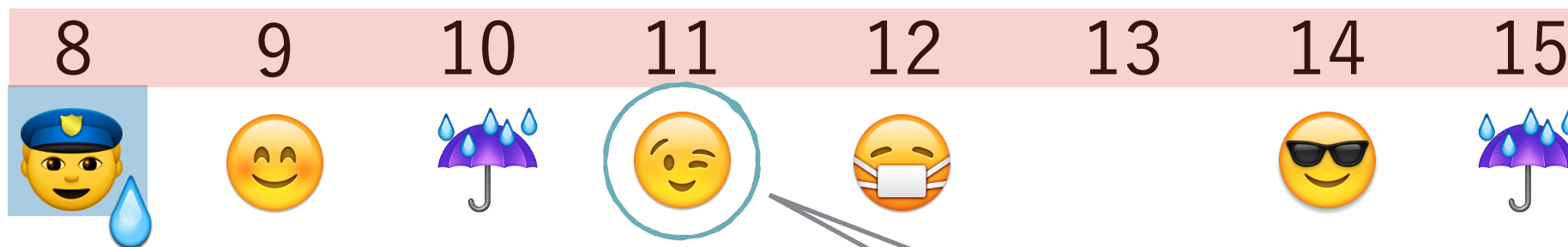
💧 を飲む回数が**数千億回**とか! ??! ?

図にする

ここで何個💧を補給するか？ → **6個は必要**



T

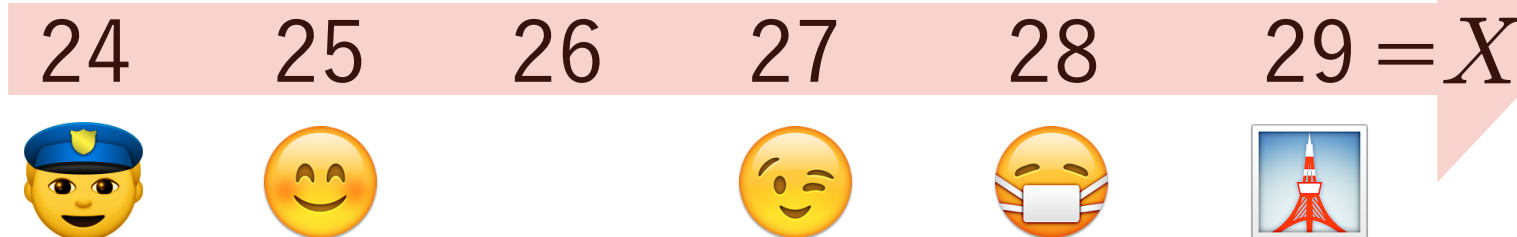


$2T$



この💧を最初に
補給する必要はない

$3T$



8個は不要

探索

ここで6個補給することにする



	0	1	2	3	4	5	6	7
T	8	9	10	11	12	13	14	15
$2T$	16	17	18	19	20	21	22	23
$3T$	24	25	26	27	28	29 = X		

探索

ここで6個補給することにする



	0	1	2	3	4	5	6	7
								
T	8	9	10	11	12	13	14	15
								
$2T$	16	17	18	19	20	21	22	23
								
$3T$	24	25	26	27	28	29 = X		
								

探索

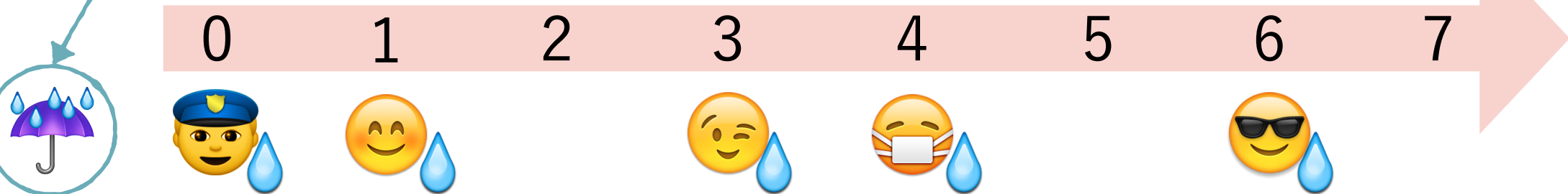
ここで6個補給することにする



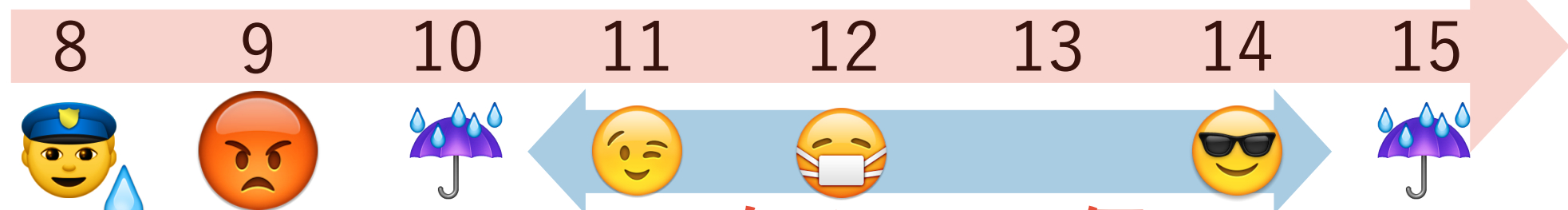
	0	1	2	3	4	5	6	7
T	8	9	10	11	12	13	14	15
$2T$	16	17	18	19	20	21	22	23
$3T$	24	25	26	27	28	29 = X		

探索

ここで6個補給することにする

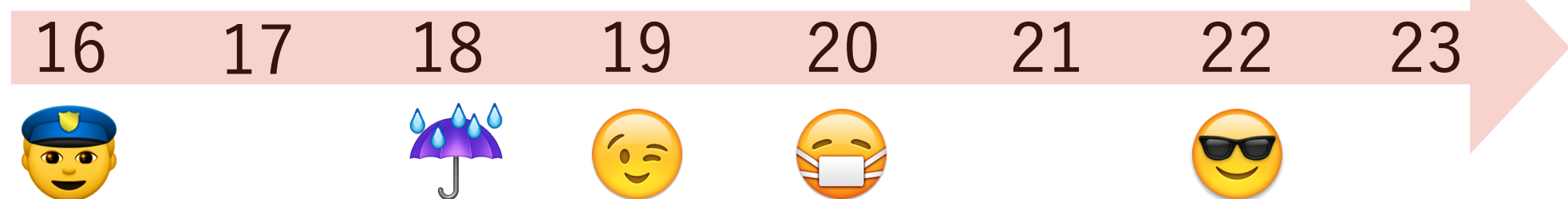


T



次は 0~3 個

$2T$



$3T$



探索

ここで6個補給することにする



探索

- ▶ 各🌧️につき補給する💧の個数の候補は $O(M)$ 通り
- ▶ 探索候補は全部で $O((M+1)^{N+1})$ 通り
- ▶ 実際はもっと少ない

😓 2秒で間に合うかなあ……？

実装によっては間に合わないよ🤪



もうすこしよく見る

この補給地点は **この人** を帰らせる権利を持っている



もうすこしよく見る

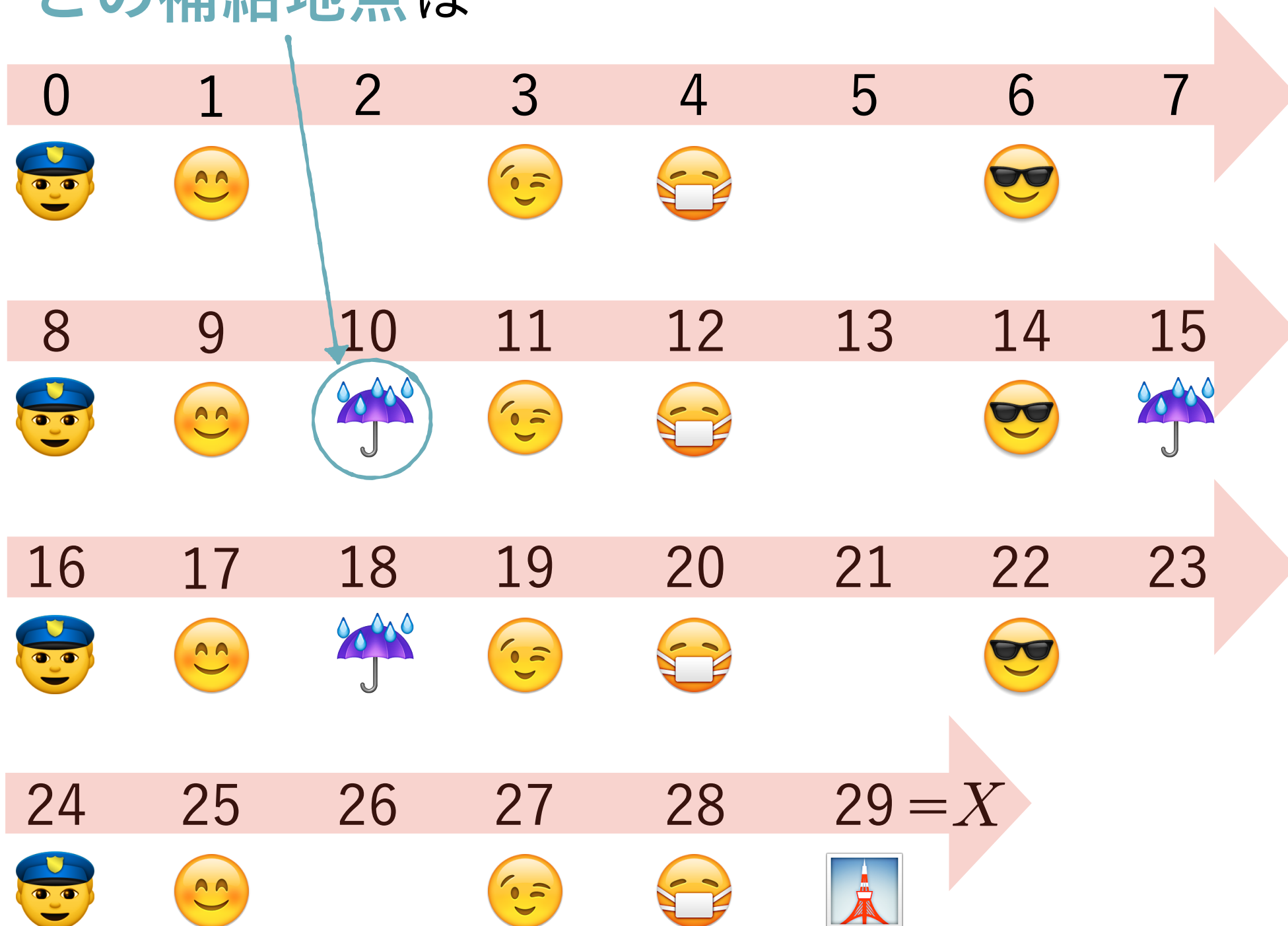
この補給地点は



T

$2T$

$3T$



もうすこしよく見る

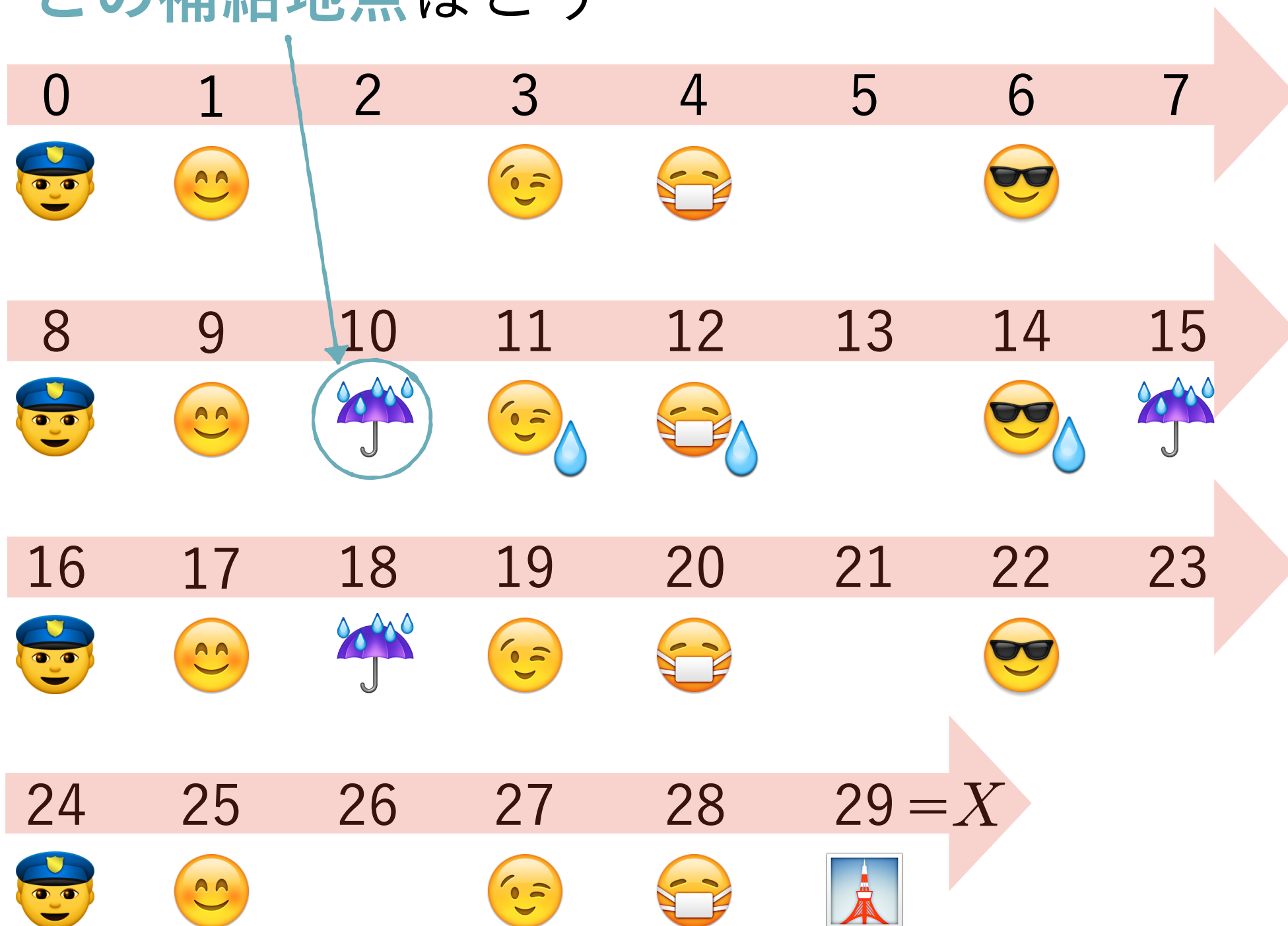
この補給地点はこう



T

$2T$

$3T$



もうすこしよく見る

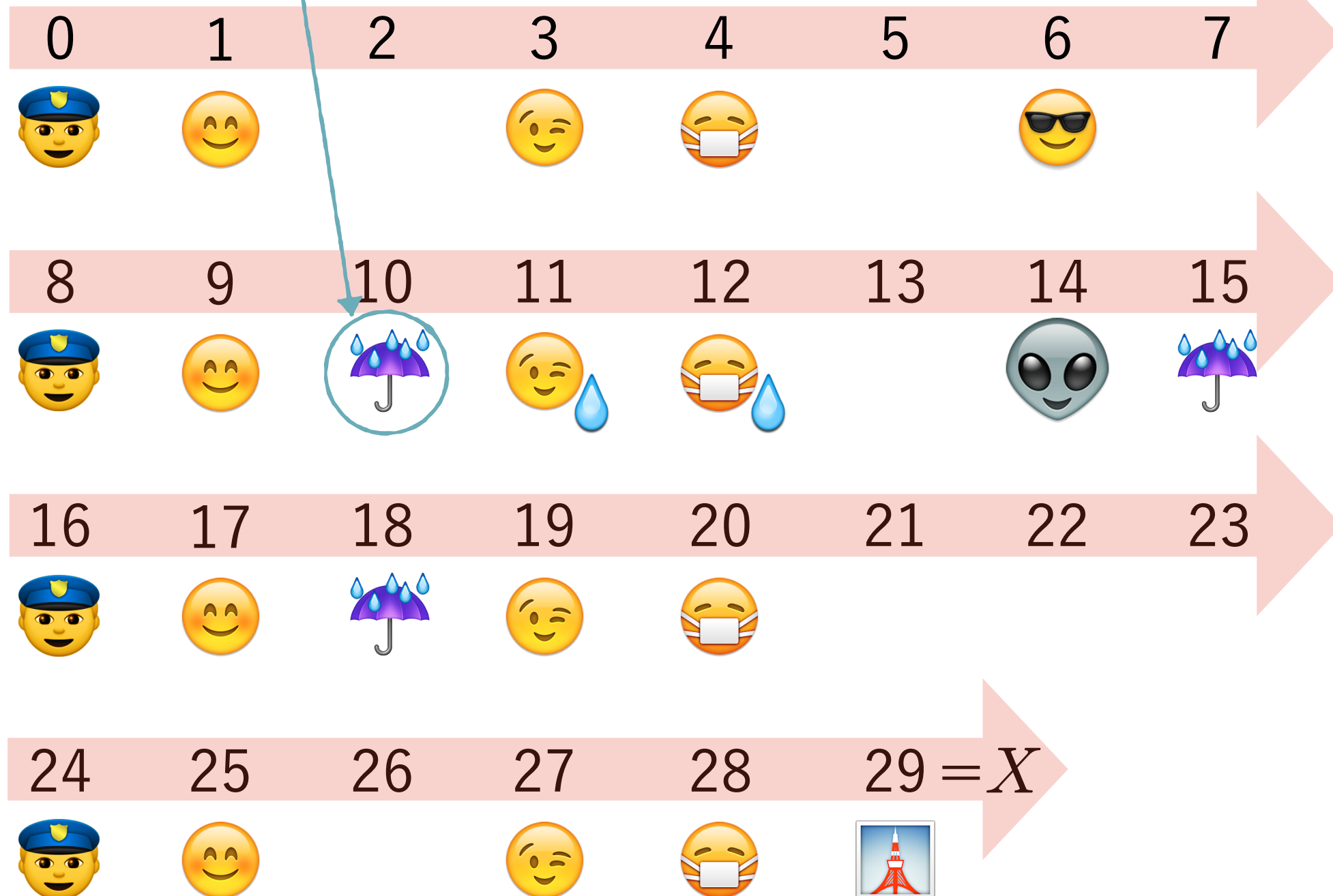
この補給地点はどうかこう



T

$2T$

$3T$



もうすこしよく見る

この補給地点はどうかこうかこう



T

$2T$

$3T$



もうすこしよく見る

この補給地点はこうかこうかこうかこう



T

$2T$

$3T$

0 1 2 3 4 5 6 7



8 9 10 11 12 13 14 15



16 17 18 19 20 21 22 23



24 25 26 27 28 29 = X



安心できる小課題 1 の解法

- ▶ 先にそれぞれの客を帰すか送り届けるか決める
 - ▶ これは $O(2^M)$ 通り
- ▶ すると各🌧️で何個💧を補給するか決まる

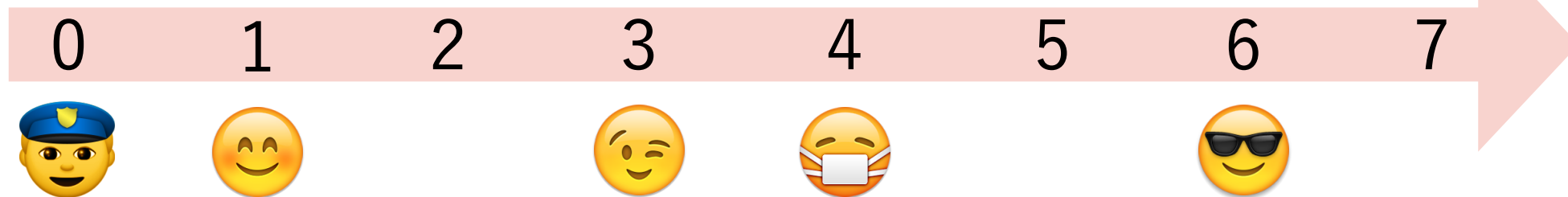
😓 2秒で間に合うかなあ……？

余裕があるよ😊

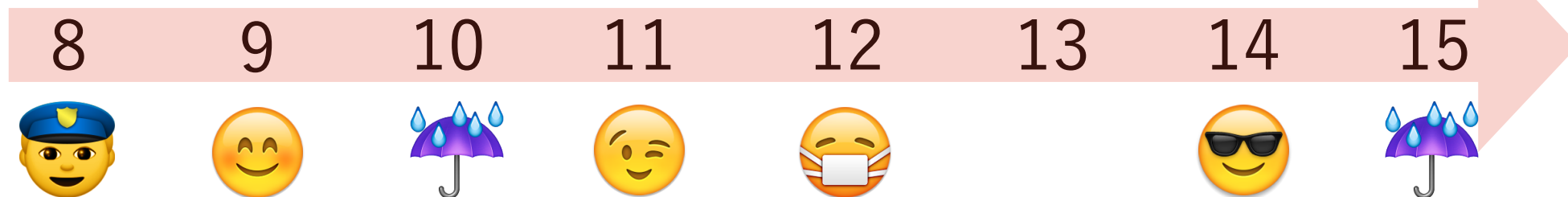


問題の言い換え

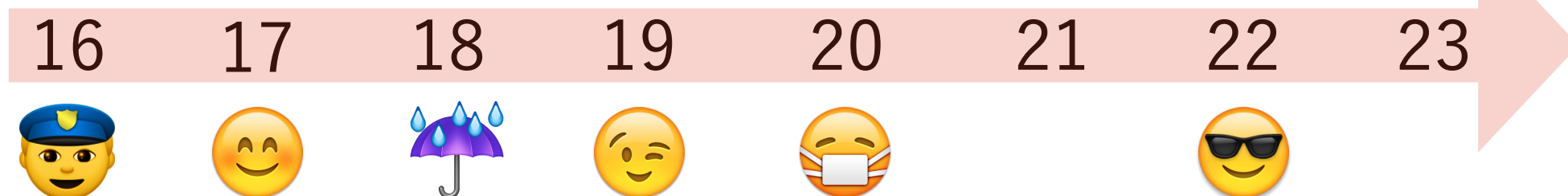
補給地点 → 帰せる人の区間



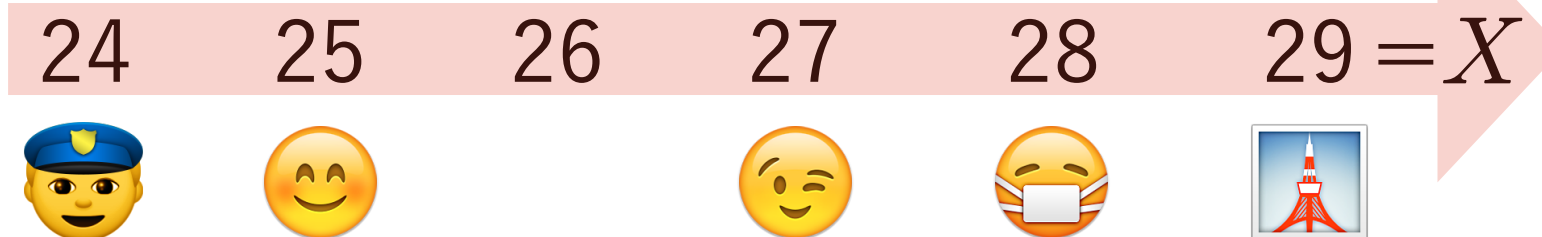
T



$2T$



$3T$



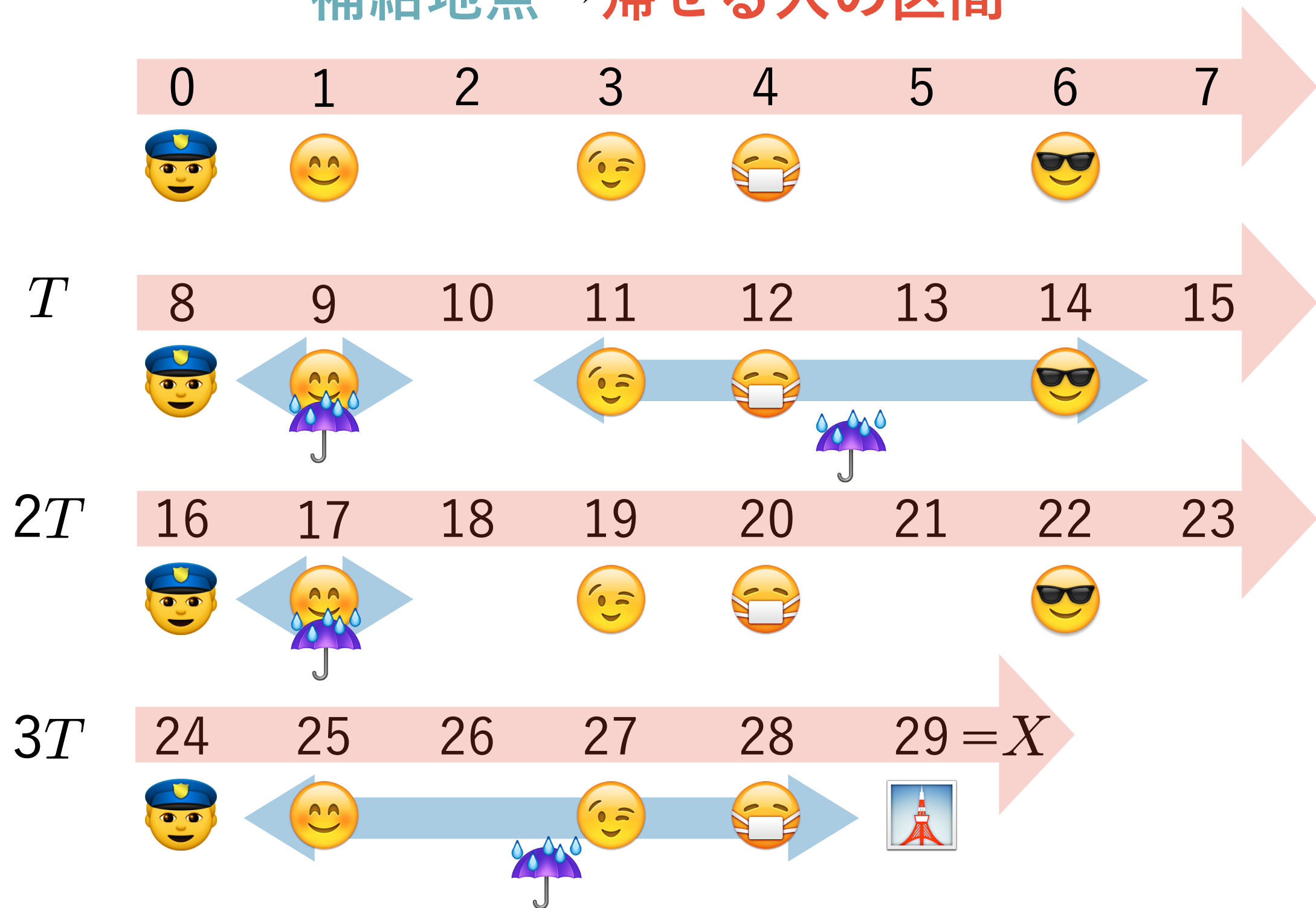
問題の言い換え

補給地点 → 帰せる人の区間



問題の言い換え

補給地点 → 帰せる人の区間



多項式になったで賞 (46点)

各🌂を「後ろから連続する何人かを帰せる区間」と

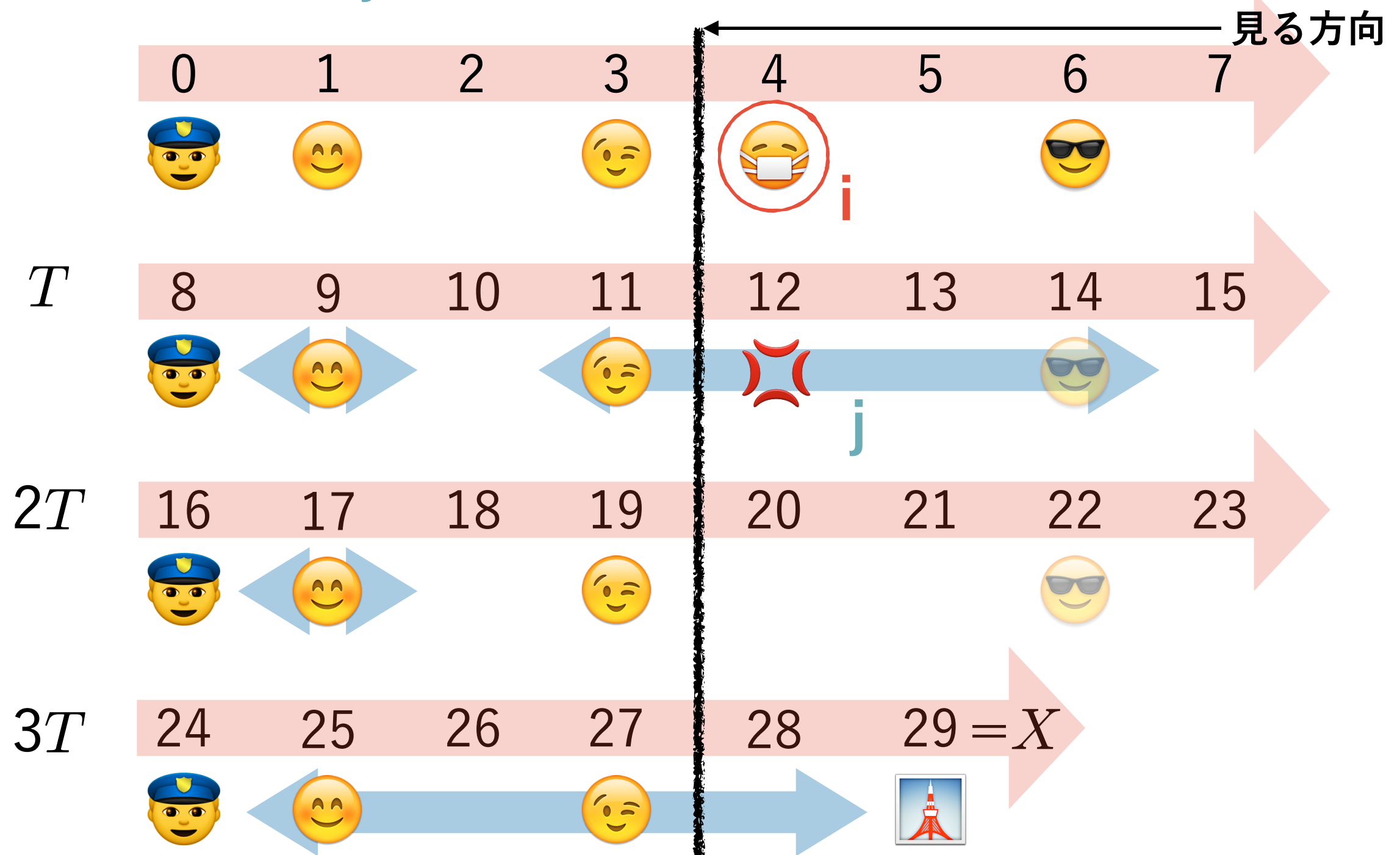
見てなんか DP とかしてください

3 乗以内になれば大丈夫です多分

ちゃんと DP できたで賞 (71点)

$dp[i][j] :=$ 人 i まで見ていて

人 i を🌧️ j で追い返したときの最小コスト



すこし詳しく

$dp[i][j] :=$ 人 i まで見ていて

人 i を🌂 j で追いついたときの最小コスト

▶ $dp[i][\text{FREE}] :=$ 人 i まで見ていて人 i を送り届ける場合

更新式

▶ $dp[i][j] += dp[i + 1][j] + (\text{人 } i \text{ を🌂 } j \text{ で帰すコスト})$
- 🌂 j がすでにある場合

▶ $dp[i][j] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を🌂 } j \text{ で帰す})$
▶ 🌂 j が人 i と $i+1$ の間で出てくる場合

▶ $dp[i][\text{FREE}] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を送り届ける})$

DP 早くなっただ賞 (100点)

DP の更新式をよく見てみよう

人\🌧	FREE	1	2	3	4	5	6	7	8
9									
8									
7									
6									
5									
4									
3									
2									
1									

すこし詳しく

$dp[i][j] :=$ 人 i まで見ていて

人 i を🌂 j で追いついたときの最小コスト

▶ $dp[i][\text{FREE}] :=$ 人 i まで見ていて人 i を送り届ける場合

更新式

▶ $dp[i][j] += dp[i + 1][j] + (\text{人 } i \text{ を🌂 } j \text{ で帰すコスト})$
- 🌂 j がすでにある場合

▶ $dp[i][j] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を🌂 } j \text{ で帰す})$
▶ 🌂 j が人 i と $i+1$ の間で出てくる場合

▶ $dp[i][\text{FREE}] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を送り届ける})$

DP 早くなっただけ賞 (100点)

DP の更新式をよく見てみよう

人\🌂	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	*	*	*	*	*
8									
7									
6									
5									
4									
3									
2									
1									

DP 早くなっただけ賞 (100点)

DP の更新式をよく見てみよう

人\🌂	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	*	*	*	*	*
8	*	*	*	*	*	*	*	*	*
7									
6									
5									
4									
3									
2									
1									

すこし詳しく

$dp[i][j] :=$ 人 i まで見ていて

人 i を🌂 j で追いついたときの最小コスト

▶ $dp[i][\text{FREE}] :=$ 人 i まで見ていて人 i を送り届ける場合

更新式

▶ $dp[i][j] += dp[i + 1][j] + (\text{人 } i \text{ を🌂 } j \text{ で帰すコスト})$
- 🌂 j がすでにある場合

▶ $dp[i][j] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を🌂 } j \text{ で帰す})$

▶ 🌂 j が人 i と $i+1$ の間で出てくる場合

▶ $dp[i][\text{FREE}] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を送り届ける})$

DP 早くなっただ賞 (100点)

DP の更新式をよく見てみよう

人 \ 	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6									
5									
4									
3									
2									
1									

人 6 と 7 の間で
 4 が始まる

DP 早くなっただけ賞 (100点)

DP の更新式をよく見てみよう

人 \ 	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6									
5									
4									
3									
2									
1									

min をとる

人 6 と 7 の間で
 4 が始まる

すこし詳しく

$dp[i][j] :=$ 人 i まで見ていて

人 i を🌂 j で追いついたときの最小コスト

▶ $dp[i][\text{FREE}] :=$ 人 i まで見ていて人 i を送り届ける場合

更新式

▶ $dp[i][j] += dp[i + 1][j] + (\text{人 } i \text{ を🌂 } j \text{ で帰すコスト})$
- 🌂 j がすでにある場合

▶ $dp[i][j] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を🌂 } j \text{ で帰す})$

▶ 🌂 j が人 i と $i+1$ の間で出てくる場合

▶ $dp[i][\text{FREE}] = \min\{dp[i + 1][\star]\} + (\text{人 } i \text{ を送り届ける})$

DP 早くなつたで賞 (100点)

DP の更新式をよく見てみよう

人 \ 傘	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6	*	*	*	*	*	*	*	*	*
5									
4									
3									
2									
1									

人5を 傘 4で帰すコスト

DP 早くなっただけ賞 (100点)

DP の更新式をよく見てみよう

人 \ 傘	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6	*	*	*	*	*	*	*	*	*
5					*				
4									
3									
2									
1									

人5を傘4で帰すコスト

人4を傘4で帰すコスト

DP 早くなっただけ賞 (100点)

DP の更新式をよく見てみよう

人 \ 傘	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6	*	*	*	*	*	*	*	*	*
5					*				
4									
3									
2									
1									

人5を傘4で帰すコスト

人4を傘4で帰すコスト

他の列によらず決まる

DP 早くなっただ賞 (100点)

人 i を  j で帰すコスト = $W * (S[j]/T)$ + $C[i]$
人 に 依存 しない 人 に 依存

人 \ 	FREE	1	2	3	4	5	6	7	8
9	*	*	*	*	∞	*	*	*	*
8	*	*	*	*	∞	*	*	*	*
7	*	*	*	*	∞	*	*	*	*
6	*	*	*	*	*	*	*	*	*
5					*				
4									
3									
2									
1									

人5を  4で帰すコスト

人4を  4で帰すコスト

他の列によらず決まる

DP 早くなっただ賞 (100点)

人 i を  j で帰すコスト = $W * (S[j]/T)$ + $C[i]$
人 に 依存 しない 人 に 依存

人 \ 	FREE	1	2	3	4	5	6	7	8
9									
8									
7	あらかじめ足されていると思うしておく (min に影響はないことに注意)								
6									
5									
4				⋮					
3				+C[3]+⋯+C[9]					
2				+C[2]+⋯+C[9]					
1				+C[1]+⋯+C[9]					

DP 早くなつたで賞 (100点)

人 i を🌧️ j で帰すコスト = $W * (S[j]/T) + C[i]$
 人に依存しない 人に依存

人\🌧️	FREE	1	2	3	4	5	6	7	8
9									
8									
7									
6					登場				
5									
4									
3									
2									
1									

↓

直線的に増加

DP 早くなっただ賞 (100点)

人 i を  j で帰すコスト = $W * (S[j]/T)$ + $C[i]$
人に依存しない 人に依存

人 \ 	FREE	1	2	3	4	5	6	7	8
9									
8			登場						
7		登場					登場		
6					登場			登場	
5						登場			
4				登場					
3									登場
2									
1									

DP 早くなったで賞 (100点)

人 i を🌧️ j で帰すコスト = $W * (S[j]/T) + C[i]$
 人に依存しない 人に依存

人 \ 🌂

FREE

1 2 3 4 5 6 7 8

9

8 登場

7 登場 登場

6 登場 登場

5 登場

4 登場

3 登場

2

1

直線がたくさん！

min がとりたい

Convex Hull Trick

Convex Hull Trick

▶ 汎用テク & バリエーションも多いので詳細は各自

- 蟻本にもある？ ネット上にも結構解説がある

▶ 要らない直線を管理しつつ頑張っていけば……

😅 いやあ大変だったなあ……

? ? ? ? ? ? ? ? ? ?

▶ $1 \leq T \leq X \leq 1,000,000,000,000$



要らない直線の判定法

▶ 3 直線 $y_i = a_i x + b_i$ ($i=1,2,3$) が $a_1 > a_2 > a_3$ のとき

$$(a_2 - a_1)(b_3 - b_2) \geq (b_2 - b_1)(a_3 - a_2)$$

? ? ? ? ? ? ? ? ? ?

▶ $1 \leq T \leq X \leq 1,000,000,000,000$



世はまさに大オーバーフロー時代

- ▶ 回避方法1: 頑張る

- ▶ $a/d \geq b/c$ を判定しようとする

- ▶ 整数部分が違うなら簡単で

- 同じ時は mod をとって小数部分を再帰比較する

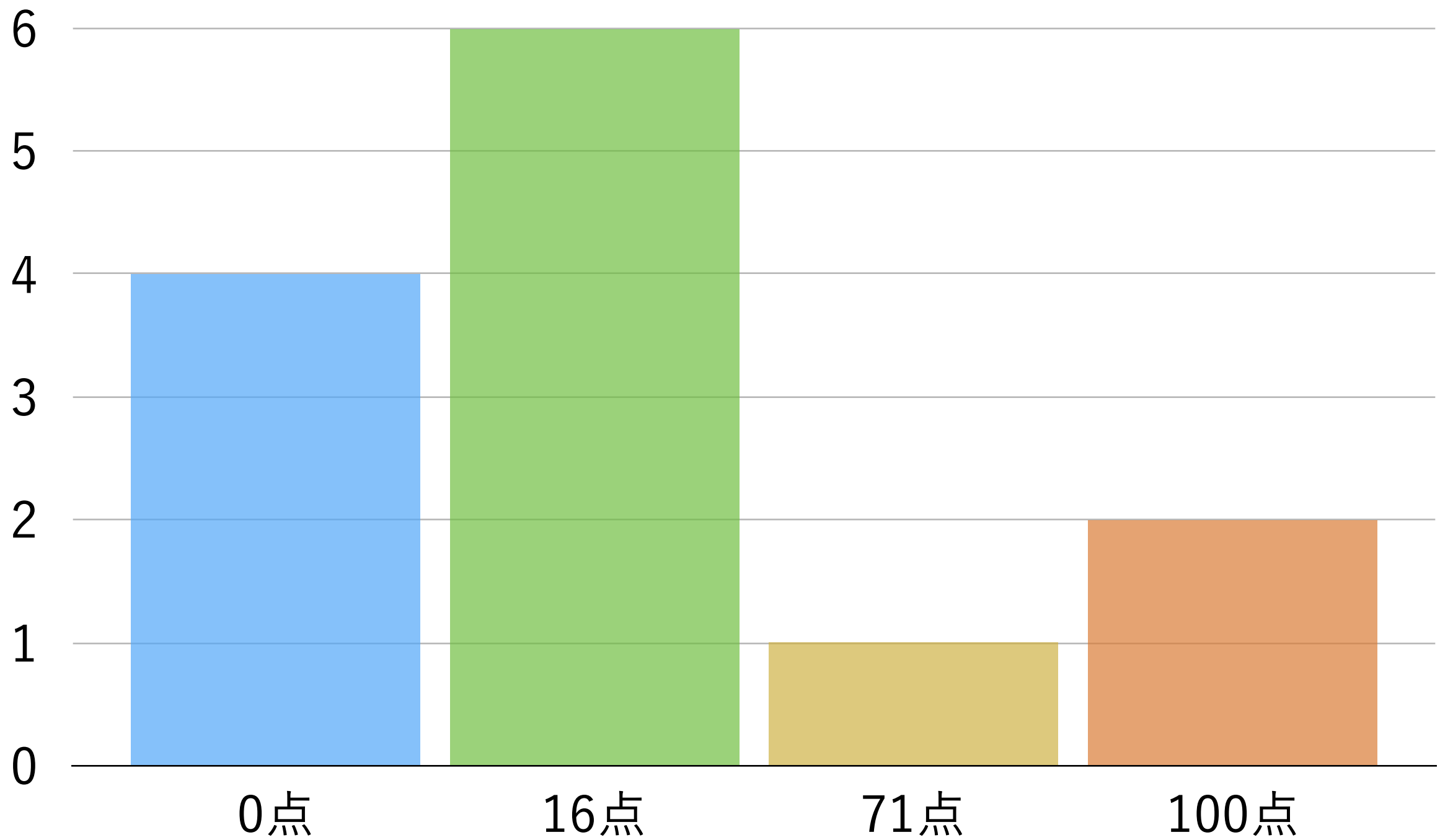
- ▶ 回避方法2: doubleる

- ▶ どうせそんなきわどいケースないでしょ……

- ▶ 回避方法3: `__int128`

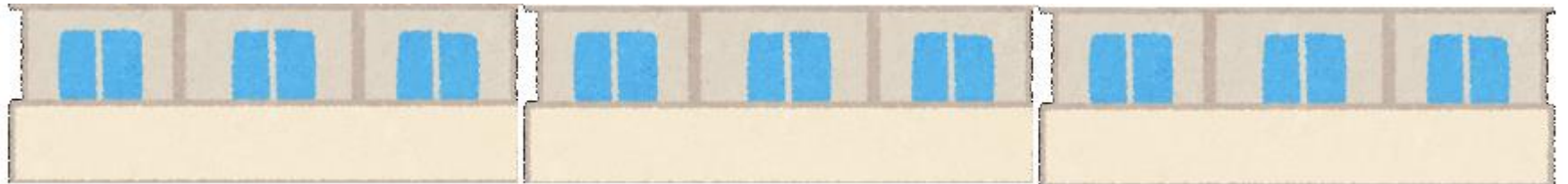
- ▶ パソコンに詳しい人向け

得点分布



細長い屋敷(Long Mansion)

hogloid



問題概要

- 部屋がN個直線形につながっており、隣接する部屋への移動には鍵が1つ必要
- 部屋には鍵が落ちてる。一度拾った鍵は何度でも使える
- クエリをQ個処理:ある部屋からある部屋へ行けるか？
- $N, Q, \text{鍵の総和}(=M) \leq 10^5$
- 小課題3:鍵の種類($=K$) ≤ 20

小課題1

- クエリごとに独立に処理
- ある点から進み到達できる範囲は必ず区間になる、区間を求めるには開始地点のみを含む区間を左右に拡張し鍵を回収し続けることで求まる
 - 鍵を取ることによって損することはないため
- クエリの目的地が始点から行ける区間に含まれるか求める。 $O(Q(N+M))$

小課題2

- N個の始点について到達可能区間を予め求めておく。クエリーには $O(1)$ で答える。
- $O(Q+NM)$

小課題3

- N個の始点について到達可能区間を予め求めておく。クエリーには $O(1)$ で答える。
- 始点から右に向かって初めてある鍵が必要になる廊下のみが区間の右端になりうる
- 始点から右に向かって初めてある鍵が手に入る部屋でのみ回収することに価値がある
 - 左についても同様。見る必要のある部屋は高々 $O(K)$
- これで座標圧縮をすると始点ごとに $O(K \log K)$

満点(その1)

- i から到達できる区間を $[l_i, r_i]$ とする
 - i から j に到達できるなら、 $[l_i, r_i]$ は $[l_j, r_j]$ を含む・・・
- ①
- これを使って区間を左右に拡張するメモ化再帰ができるのでは？
 - 単純にやると、できない
 - 互いに到達可能な始点があると遷移にサイクルがあり終了
 - しかし、①より互いに到達可能なら区間は等しい

満点(その1)

- よって、無限ループに陥りそうになるとき、その遷移のサイクルに属する始点全ての区間は等しい
- メモ化再帰を行うと遷移はdfs木になるので、無限ループが起こりそうになる遷移は後退辺で、サイクルに属する始点はdfs木の現在の始点の祖先
- サイクルで最も祖先の始点に戻るまで、区間を併合しながらdfsをやめてしまえばよい。dfsをやめてしまった始点はUnionFindで「どの始点と答えが同じか」を記憶しておく

満点(その1)

- 計算量は？

- 全ての始点について、併合によらない区間の拡張回数が全体の計算量
- 右端を k から $k+1$ に拡張する始点 $i, j (i < j \leq k)$ があったとする
 - このとき、 i は右端を k から $k+1$ に拡張する前に j について再帰的に問題を解いており、始点 j からの区間を含むため、矛盾
- よって、各廊下について左から拡張される回数は1回、右からについても同様
- 区間の拡張可能性判定(=鍵を持っているかどうか)に $O(\log N)$ かかる。UnionFindも使うため、合計 $O(N \log N)$

満点(その2)

- 全ての始点 i について、はじめ $[i,i]$ から始め、右・左に拡張していくが、1ステップでそれ以上右に拡張できなくなるまで拡張する
 - 右への拡張を考える。すべての廊下について、それより左で最も右の廊下を通る鍵が落ちている部屋の場所を覚えておく
 - 右へ拡張できなくなる場所は、初めてその廊下を通るのに必要な鍵が、現在の区間より左にある場所
 - ・ これは二分探索segment treeを使えばいけます

満点(その2)

- 毎ステップに左右へ拡張し、答えを求める
 - TLEします(真ん中に $N/2$ 個互いに通行可能な始点があり、その周りをジグザグ動くようなケース)
- 区間の経過全てについて、答えをメモしておき、既に求めた区間を訪れたらメモした答えを返すと、状態数は $O(N)$ になる！

満点(その2)

- それ以上拡張できない区間を終区間と呼ぶ
- 終区間・長さ1の区間は $O(N)$ 個しかないので、それ以外を数える
- 左端が l で、これから右に拡張していく区間について考える
- $[l, r_1], [l, r_2], \dots, [l, r_K]$ とおく。 $(r_1 > r_2 > \dots)$
- $[l, r_i]$ を右に拡張すると、 $[l, s_i]$ になるとする。
- $s_i < r_1$ なら $[l, s_i]$ は終区間 ($[l, r_1]$ は左に拡張できなかった
ので、 $[l, s_i]$ も左に拡張できない)
- $s_i > r_1$ なら $s_i = s_1$ ($[l, s_i]$ は $[l, r_1]$ を含んでいるので、右端が
 r_1 に達した時点でどこまで拡張されるかは同じ)
- なので、ある左端 l から遷移する非終区間は1つ。

満点(その他)

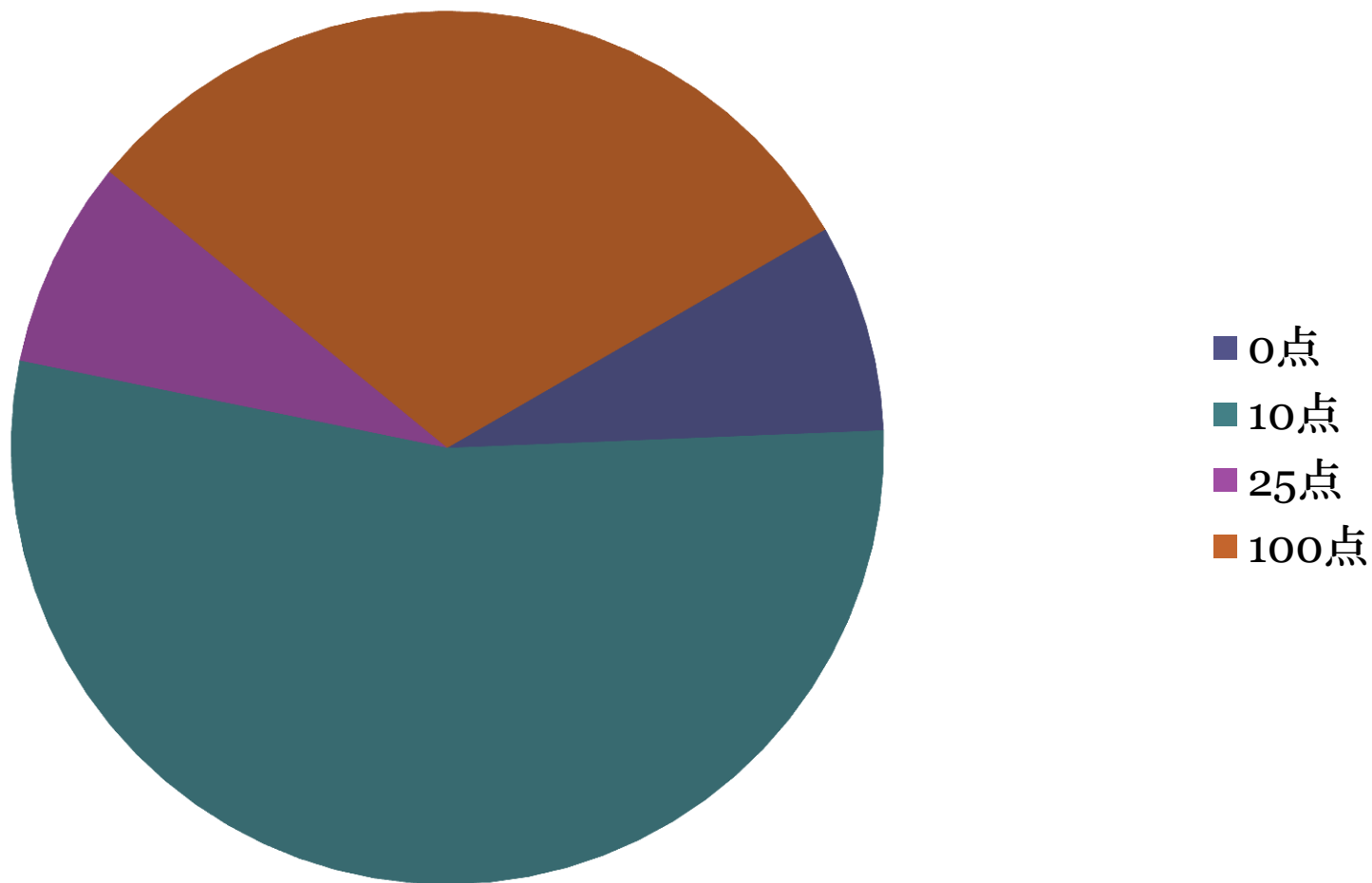
- ダブリング、segment tree、etc...
 - 時間が余ったら口頭で説明します

得点分布



- 0点
- 10点
- 25点
- 100点

得点分布



春合宿 Day 3

自然公園(Natural Park) 解説

チューター: 城下 慎也 – IOI 2011 タイ大会 日本代表

2017/3/22

問題概要

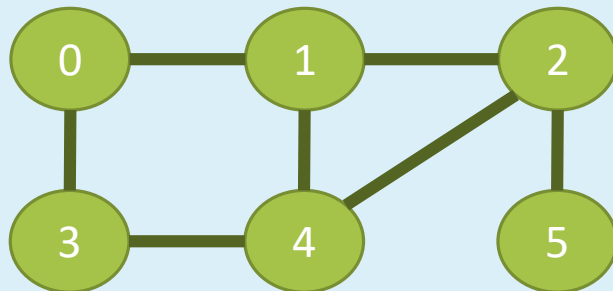
- N 頂点 M 辺のグラフが与えられる。
- 2 つの頂点 A, B と、他に使用可能な頂点の集合を指定した際に、 A, B 間を移動可能かを質問することができる。
- M 本の辺すべてを特定せよ。
- $N \leq 1,400, M \leq 1,500$, 質問回数 $\leq 45,000$
- **どの頂点も次数が 7 以下。**

問題概要

- N 頂点 M 辺のグラフが与えられる。
 - 2 つの頂点 A, B と、他に使用可能な頂点の集合を指定した際に、 A, B 間を移動可能かを質問することができる。
 - M 本の辺すべてを特定せよ。
-
- $N \leq 1,400, M \leq 1,500$, 質問回数 $\leq 45,000$
 - *どの頂点も次数が 7 以下。*
- 一見重要そうではあるが実は罠で、**小課題 5 まで使用しない**。

例

- サンプルの図で考える。



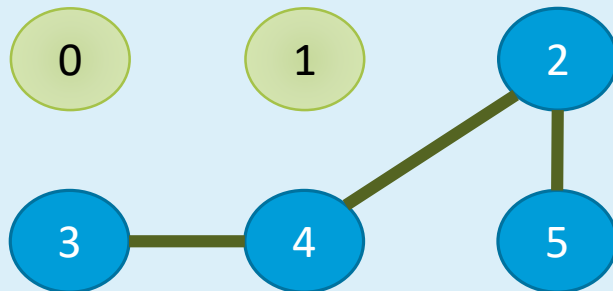
質問[1]
頂点 2,3,4,5 のみ使って
頂点 3 から 5 に行ける？



IOIちゃん(想像図)

例

- サンプルの図で考える。



行ける

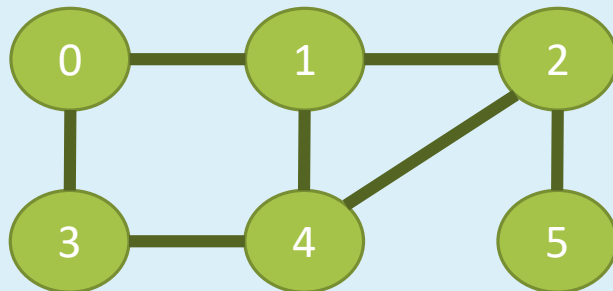
質問[1]
頂点 2,3,4,5 のみ使って
頂点 3 から 5 に行ける？



IOIちゃん(想像図)

例

- サンプルの図で考える。



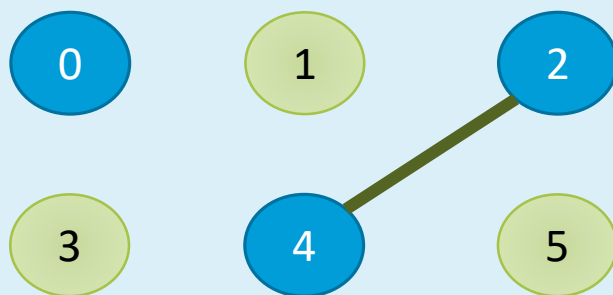
質問[2]
頂点 0,2,4 のみ使って
頂点 0 から 4 に行ける？



IOIちゃん(想像図)

例

- サンプルの図で考える。



行けない



IOIちゃん(想像図)

質問[2]
頂点 0,2,4 のみ使って
頂点 0 から 4 に行ける？

小課題 1 (10 点) 解法

- N が小さい。
- N が小さいので、すべての A, B の組について質問する余裕がある。
→ A, B 以外の頂点を使用不可能にすれば、結果は A, B 間の道の有無にしか依存しない！
- よって、すべての A, B の組について、その組となる辺があるかを判定すればよい。

小課題 2 (10 点)

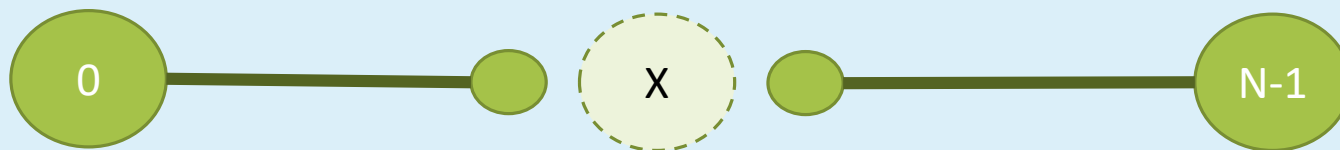
- 島の構造が直線である (頂点 0 および $N - 1$ が両端)。
- 頂点 0 から隣を求めていこうかな...

小課題 2 (10 点)

- 島の構造が直線である (頂点 0 および $N - 1$ が両端)。
- 頂点 0 から隣を求めていこうかな... → 残念ながらうまくいかない。
- いろいろ試してみて手がかりを探っていこう！

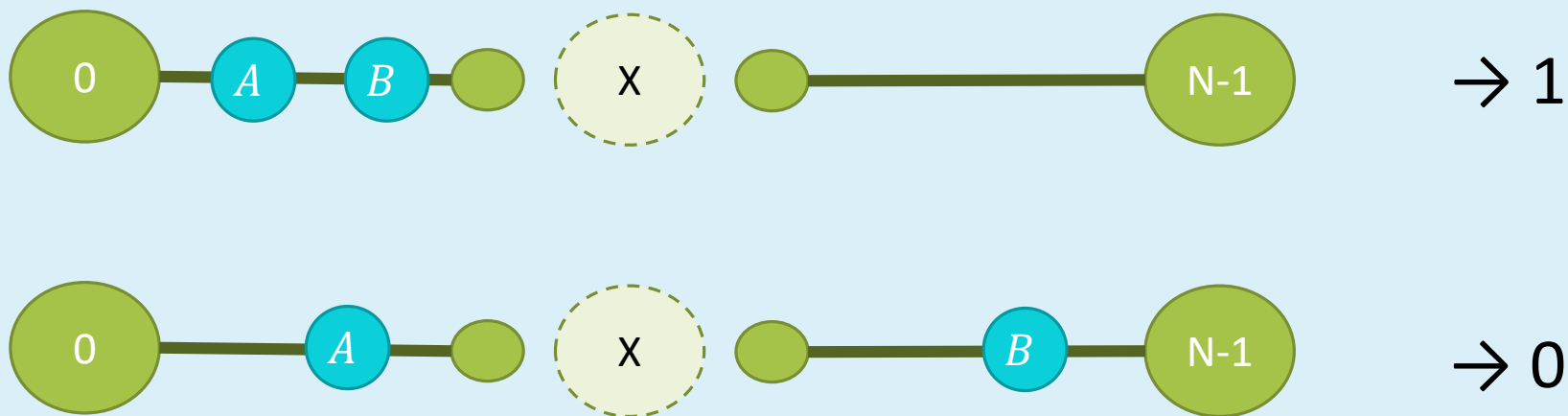
小課題 2 (10 点)

- そもそも小課題 1 では質問の性質を(ほぼ)使っていない。
- ある頂点 X のみを使用しないことにした場合にどうなるか考えてみよう。



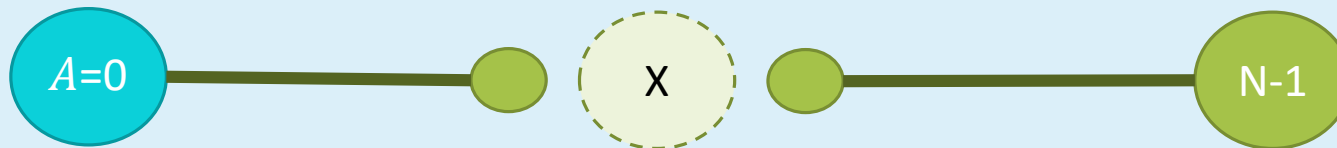
小課題 2 (10 点)

- そもそも小課題 1 では質問の性質を(ほぼ)使っていない。
 - ある頂点 X のみを使用しないことにした場合にどうなるか考えてみよう。
- A, B が同じ側なら 1、そうでないなら 0 が返ってくる。



小課題 2 (10 点)

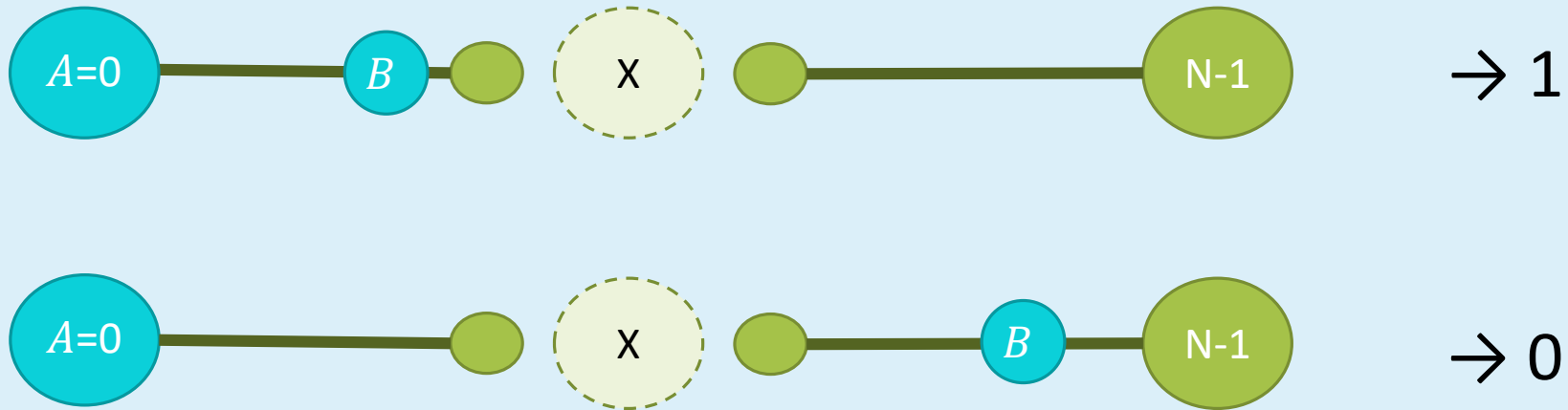
- じゃあ、さらに $A = 0$ と固定した場合は？



小課題 2 (10 点)

- じゃあ、さらに $A = 0$ と固定した場合は？

→ B が X より A に**より近い**なら 1、そうでないなら 0 が返ってくる。

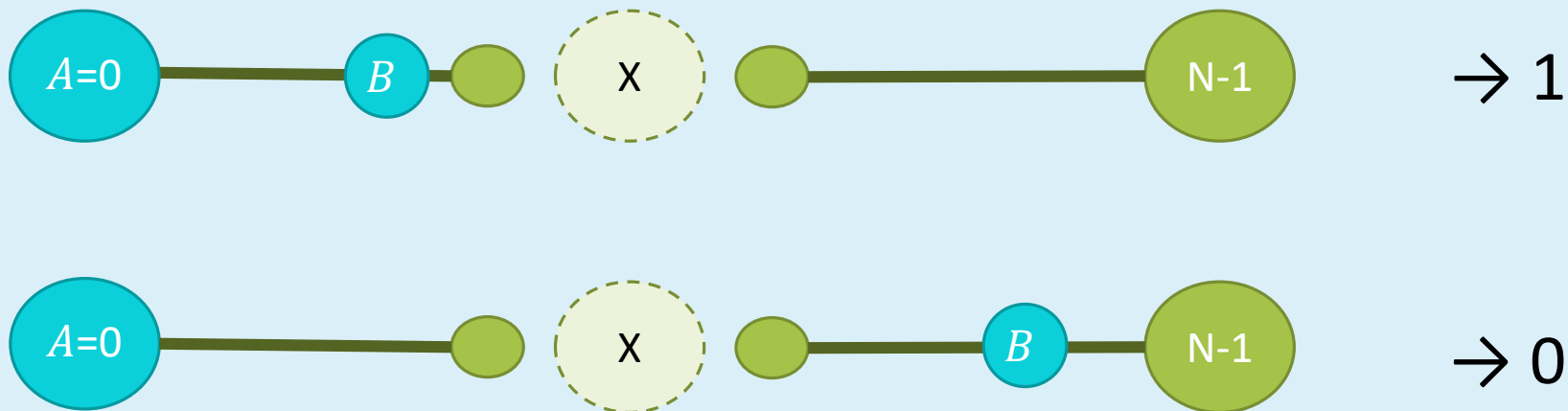


小課題 2 (10 点)

- じゃあ、さらに $A = 0$ と固定した場合は？

→ B が X より A により近いなら 1、そうでないなら 0 が返ってくる。

↪ 比較関数では！？



小課題 2 (10 点) 解法

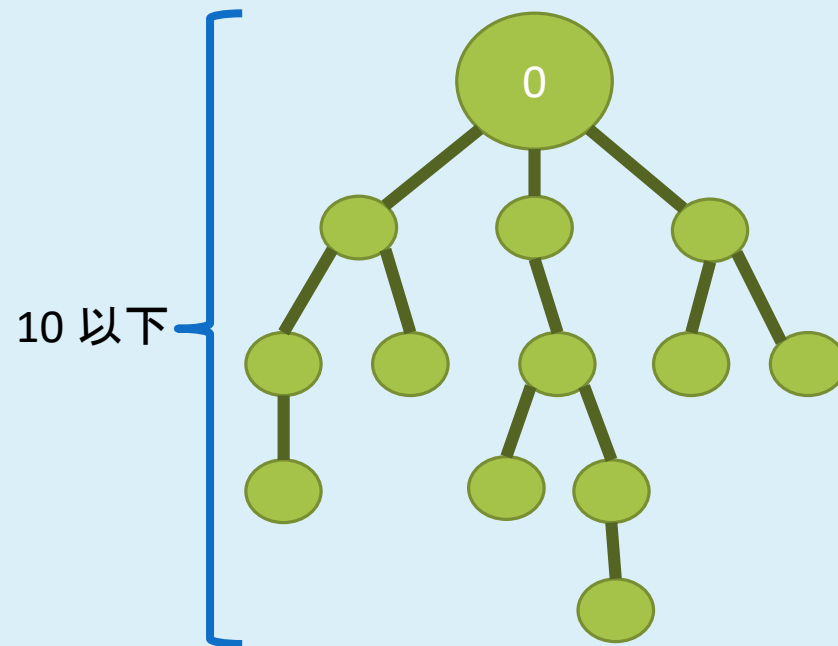
- $1, 2, \dots, N - 2$ をソートします。
- 頂点 A, B を比べたいとき、比較関数は次のようにすればよい:
 1. 頂点 A のみ使用不可、それ以外使用可能にして頂点 0 から頂点 B に行けるか尋ねる。
 2. もし行けるなら、 B の方が小さい(0 に近い)
 2. 行けないなら、 A の方が小さい(0 に近い)
- ソート後、 $0, [\text{ソート列}], N - 1$ の順に繋いで出力します。
- $O(N \log N)$ のソートを用いれば良い。

小課題 3 (27 点)

- 小課題 2 とは全く異なる形状となっている。
- 頂点 0 から 8 個以下の中継地しか必要とせずにすべての頂点に移動可能な木構造である。→どういうこと？

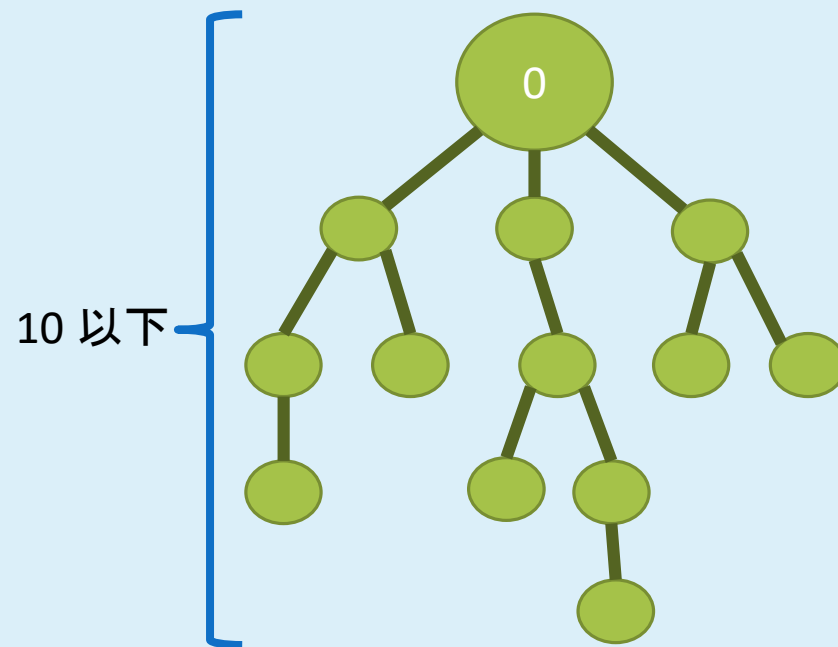
小課題 3 (27 点)

- 小課題 2 とは全く異なる形状となっている。
- 頂点 0 から 8 個以下の中継地しか必要とせずにすべての頂点に移動可能な木構造である。→どういうこと？
- 右図のように、浅い深さの木しか登場しない。



小課題 3 (27 点)

- 小課題 2 とは全く異なる形状となっている。
- 頂点 0 から 8 個以下の中継地しか必要とせずにすべての頂点に移動可能な木構造である。→どういうこと？
- 右図のように、浅い深さの木しか登場しない。
- 浅い木って何か嬉しいの？



小課題 3 (27 点)

- 結論から言うと、**それぞれのノードの深さを決定**することができる。

小課題 3 (27 点)

- 結論から言うと、**それぞれのノードの深さを決定**することができる。

- やり方: 深さ 0 の頂点は頂点 0 のみ。

深さ k ($k \geq 0$) 以下のノード全体からなる集合に対し、それ以外の頂点それぞれについて**直接繋がっているか判定**する。直接繋がっているなら深さ $k + 1$ である。

小課題 3 (27 点)

- 結論から言うと、**それぞれのノードの深さを決定**することができる。

- やり方: 深さ 0 の頂点は頂点 0 のみ。

深さ k ($k \geq 0$) 以下のノード全体からなる集合に対し、それ以外の頂点それぞれについて**直接繋がっているか判定**する。直接繋がっているなら深さ $k + 1$ である。

これは各頂点に対し、深さ k 以下全体のみを使用
可能とした際に 0 とつながっているかで判定

小課題 3 (27 点)

- 結論から言うと、**それぞれのノードの深さを決定**することができる。

- やり方: 深さ 0 の頂点は頂点 0 のみ。

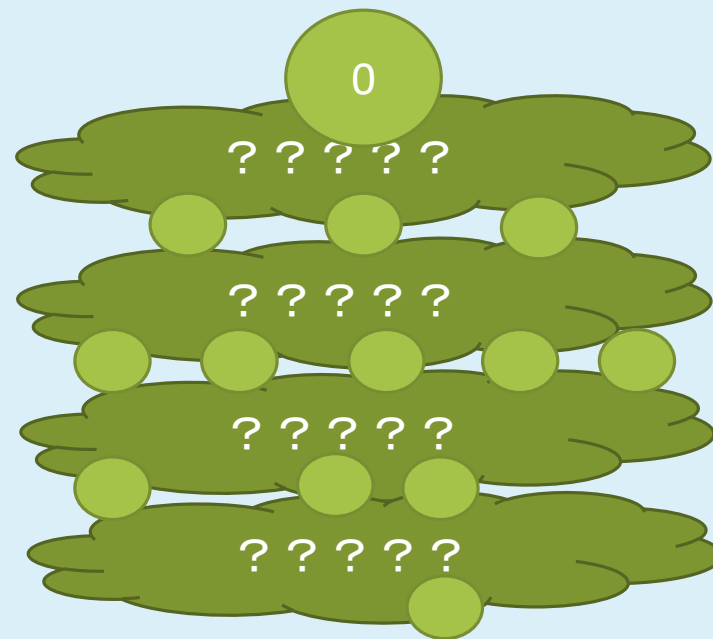
深さ k ($k \geq 0$) 以下のノード全体からなる集合に対し、それ以外の頂点それぞれについて**直接繋がっているか判定**する。直接繋がっているなら深さ $k + 1$ である。

これは各頂点に対し、深さ k 以下全体のみを使用
可能とした際に 0 とつながっているかで判定

- 各深さごとに最悪 $O(N)$ 回かかってしまうが、小課題 3 の性質より $9N$ 回以内にすべて判定できる。

小課題 3 (27 点)

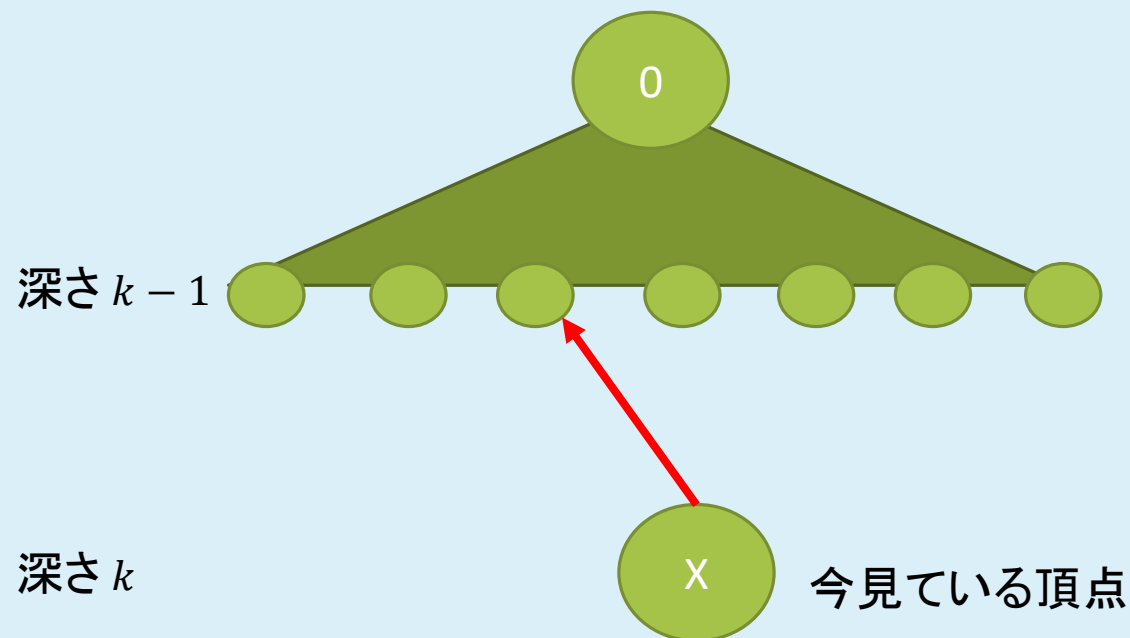
- しかしながら深さが分かっただけでは正解は得られない。辺が分かっていないためである。
- 辺を特定するためにどうすればいいのか考えてみる。



小課題 3 (27 点)

- 分かっていること:

1. 頂点 0 以外のどの頂点からも、それより深さが 1 低い頂点に辺が出ている。
2. そのような辺は各頂点ごとに 1 本だけである(そうでないと木にならない)。



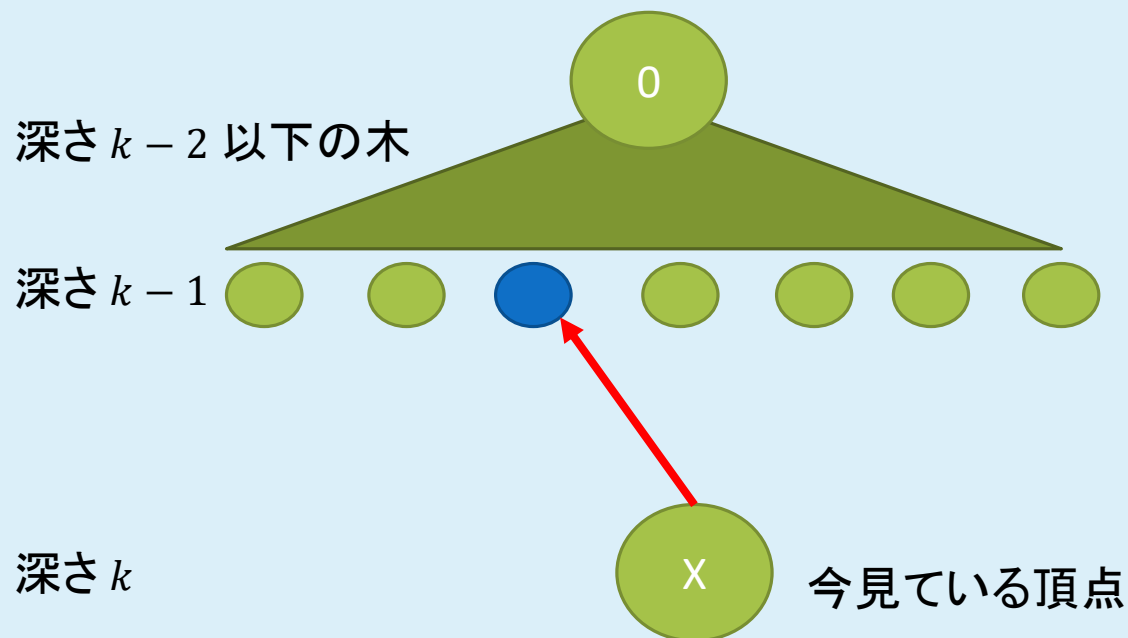
小課題 3 (27 点)

- 分かっていること:

1. 頂点 0 以外のどの頂点からも、それより深さが 1 低い頂点に辺が出ている。
2. そのような辺は各頂点ごとに 1 本だけである(そうでないと木にならない)。

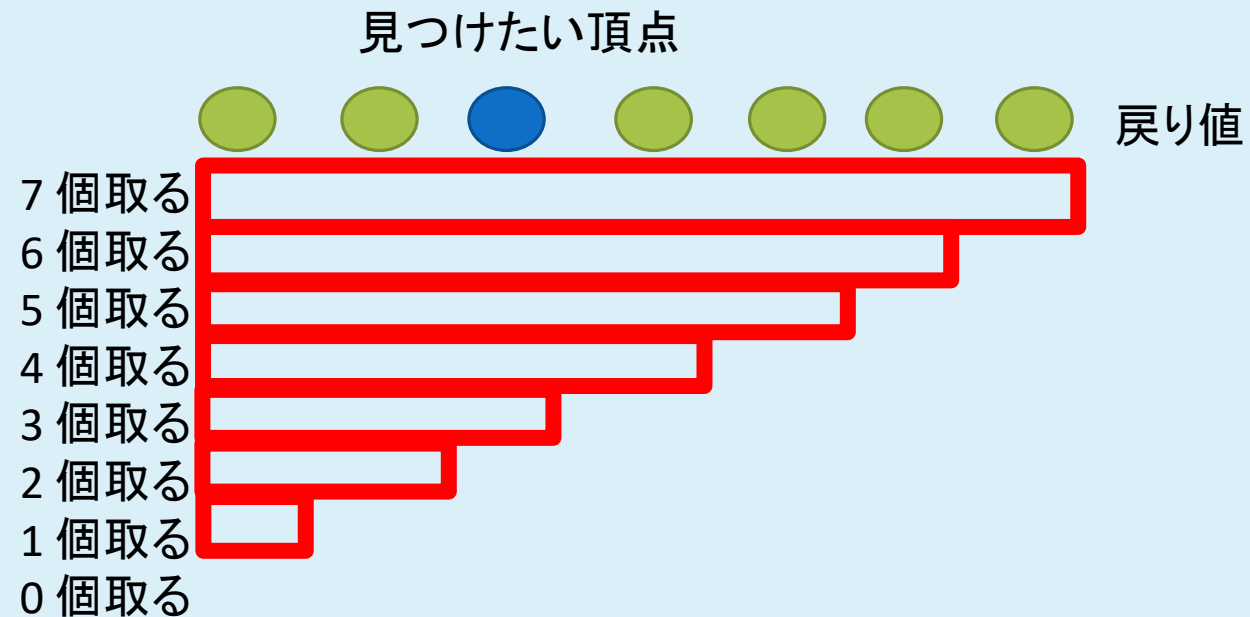
- 深さ $k - 2$ 以下の頂点からなる木は必ず採用することにして、追加で深さ $k - 1$ の頂点集合をどのように選択すると頂点 0 と x が連結になる/ならないか考えてみる。

- 他の深さ $k-1$ の頂点集合の選び方によらず、 x の親となる頂点を頂点を選べば 1、そうでなければ 0 が返ってくる。



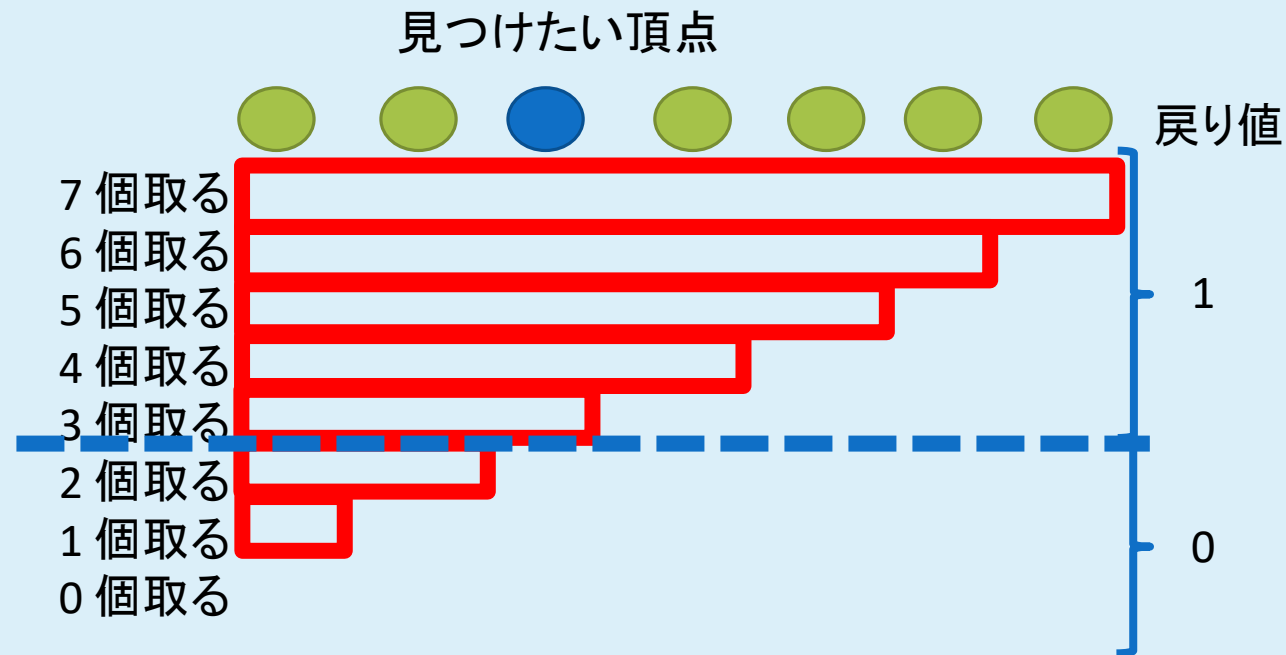
小課題 3 (27 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。



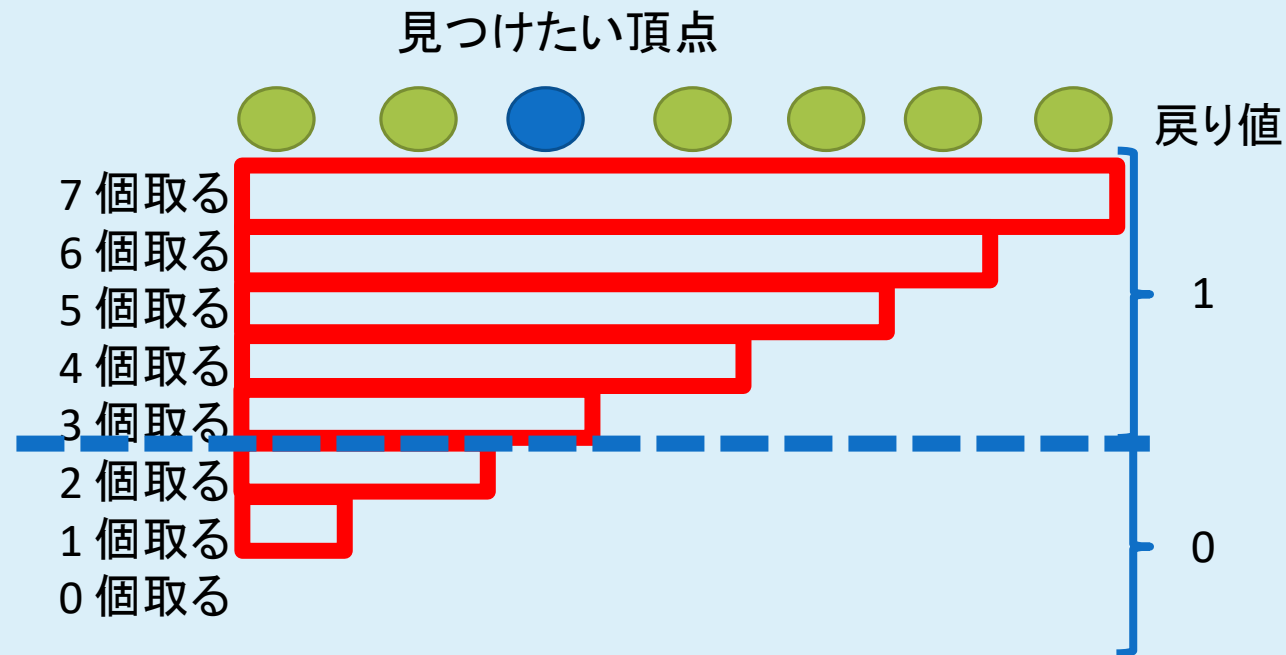
小課題 3 (27 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。
- 選んだ頂点を**境目**にして **0/1 が二分** される。



小課題 3 (27 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。
- 選んだ頂点を**境目**にして **0/1 が二分** される。
- **二分探索** を用いればどの辺も $\log N$ ステップで発見することができる。



小課題 3 (27 点) 解法

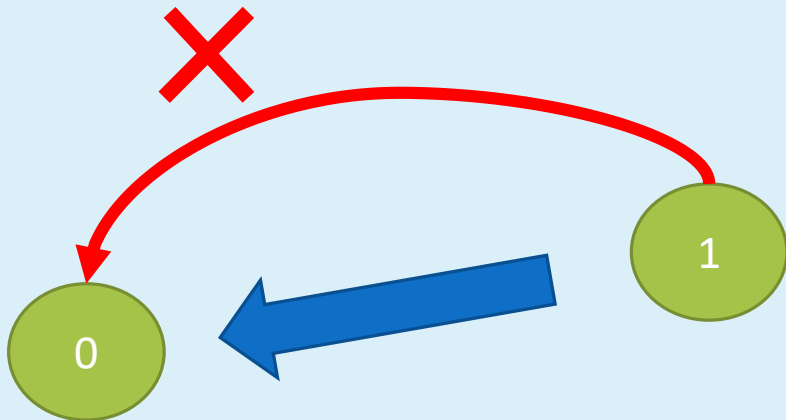
- $9N$ 回調べて各頂点の深さを調べる。
- 各頂点について、**二分探索**を用いれば親ノードを特定でき、結果すべての辺を発見できる。
- 全体で $N(\log N + 9)$ 回の質問回数、 $N = 1,400$ とすれば 28,000 回くらいの質問で解くことができる。

小課題 4 (30 点)

- ここから難易度が跳ね上がる。
- 木構造のすべてが登場する。
 - 当然、小課題 2 や 3 の方法では対処しきれないケースも出てくる。
- しかしながら、うまく閃けば小課題 2,3 をすっ飛ばして一気に 3 種類解くことができる！

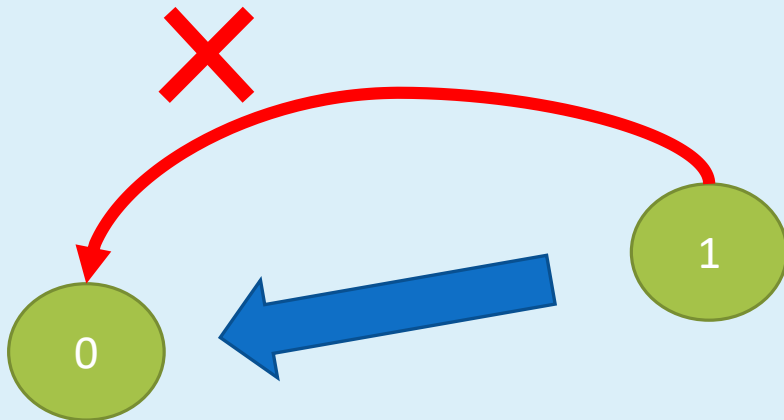
小課題 4 (30 点)

- 頂点 0 から拡張していく方針を考える。
- 例えば頂点 1 を含めよう！
 - うまく直接つながっていれば最高だが、現実はそう甘くはない。



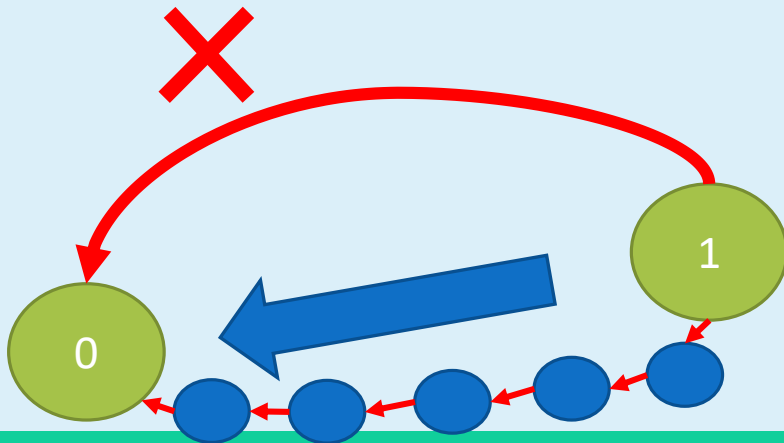
小課題 4 (30 点)

- 頂点 0 から拡張していく方針を考える。
- 例えば頂点 1 を含めよう！
 - うまく直接つながっていれば最高だが、現実はその甘くはない。
 - こういった場合、残念ながら頂点 1 を**直ちに繋ぐことは厳しい**。



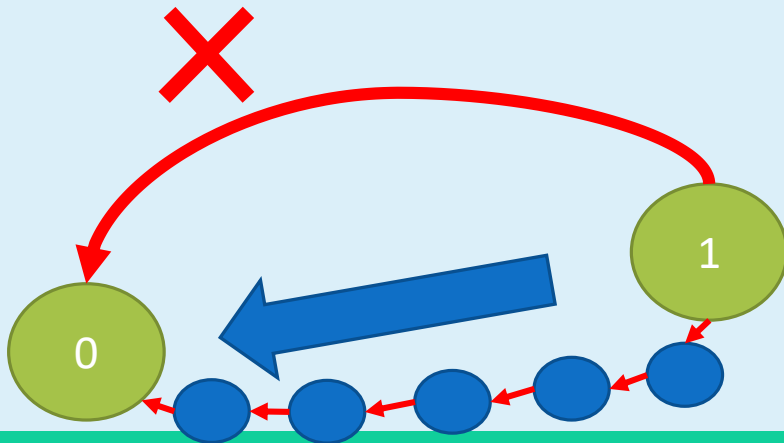
小課題 4 (30 点)

- 頂点 0 から拡張していく方針を考える。
- 例えば頂点 1 を含めよう！
 - うまく直接つながっていれば最高だが、現実はその甘くはない。
 - こういった場合、残念ながら頂点 1 を**直ちに繋ぐことは厳しい**。
 - 頂点 1 がよくなかった理由は、頂点 1 より近い別の頂点が間に挟まっているからである。



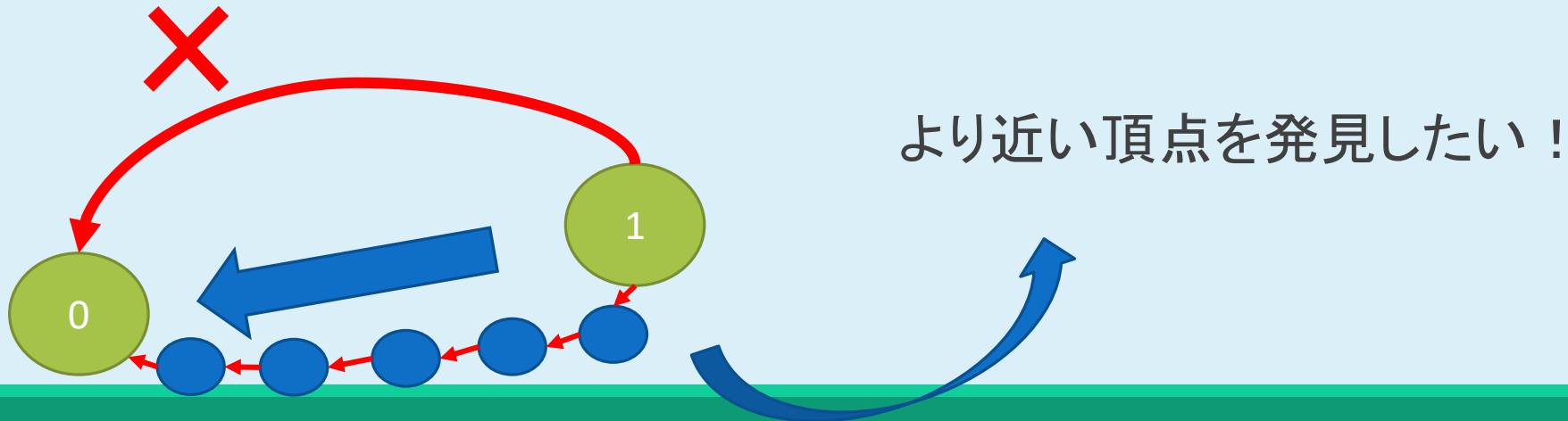
小課題 4 (30 点)

- 頂点 0 から拡張していく方針を考える。
- 例えば頂点 1 を含めよう！
 - うまく直接つながっていれば最高だが、現実はその甘くはない。
 - こういった場合、残念ながら頂点 1 を**直ちに繋ぐことは厳しい**。
 - 頂点 1 がよくなかった理由は、頂点 1 より近い別の頂点が間に挟まっているからである。
→ その頂点は直接頂点 0 とは繋がっているかもしれない！



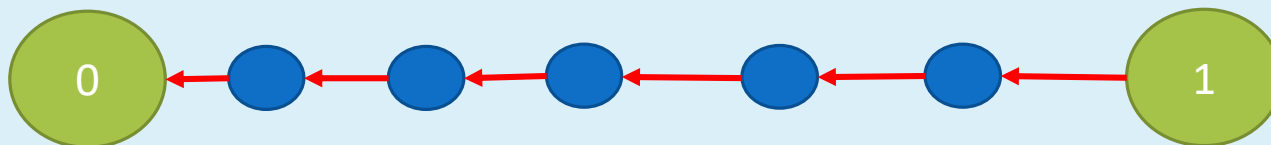
小課題 4 (30 点)

- 頂点 0 から拡張していく方針を考える。
- 例えば頂点 1 を含めよう！
 - うまく直接つながっていれば最高だが、現実はその甘くはない。
 - こういった場合、残念ながら頂点 1 を**直ちに繋ぐことは厳しい**。
 - 頂点 1 がよくなかった理由は、頂点 1 より近い別の頂点が間に挟まっているからである。
→ その頂点は直接頂点 0 とは繋がっているかもしれない！



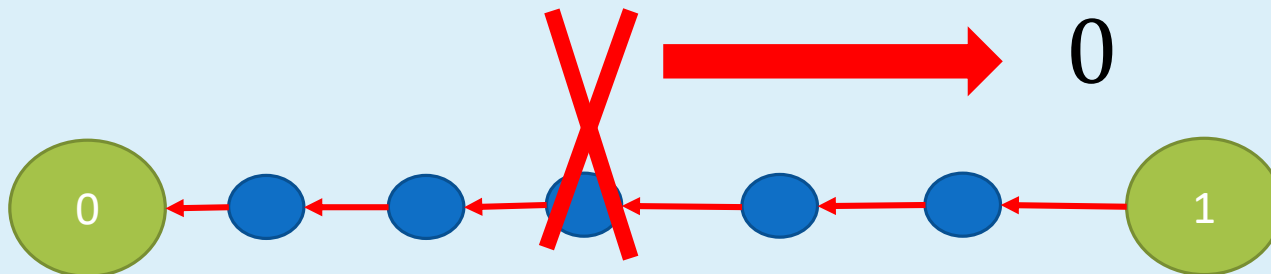
小課題 4 (30 点)

- 頂点 0 と 1 の移動可能性は何に影響されるか考えてみる。



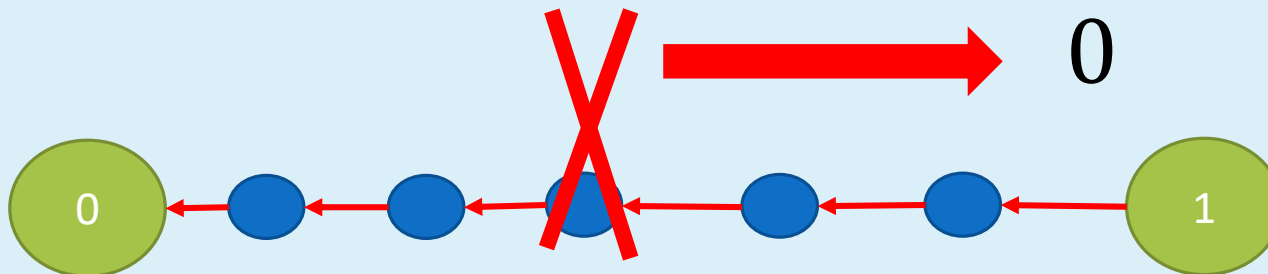
小課題 4 (30 点)

- 頂点 0 と 1 の移動可能性は何に影響されるか考えてみる。
- 中間にあるノードが消滅した瞬間、移動不可能になる(木なので(点素)パスは 1 通りしかないため)



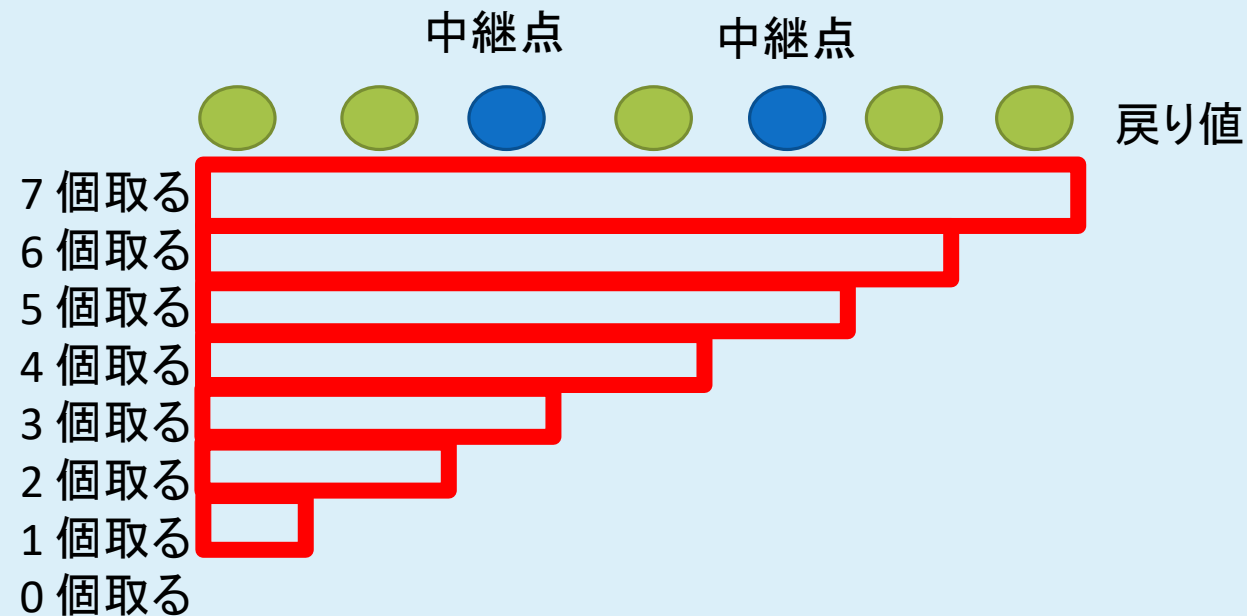
小課題 4 (30 点)

- 頂点 0 と 1 の移動可能性は何に影響されるか考えてみる。
- 中間にあるノードが消滅した瞬間、移動不可能になる(木なので(点素)パスは 1 通りしかないため)
- 実は、小課題 3 のような**二分探索**が役に立ちます。



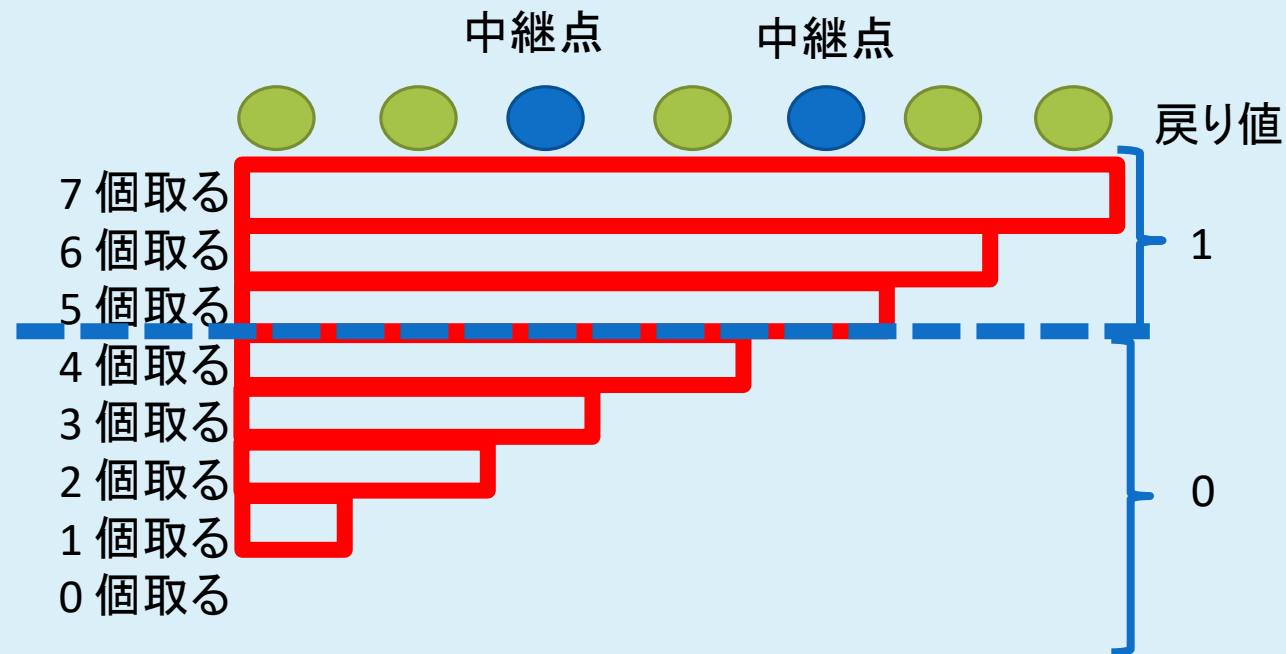
小課題 4 (30 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。



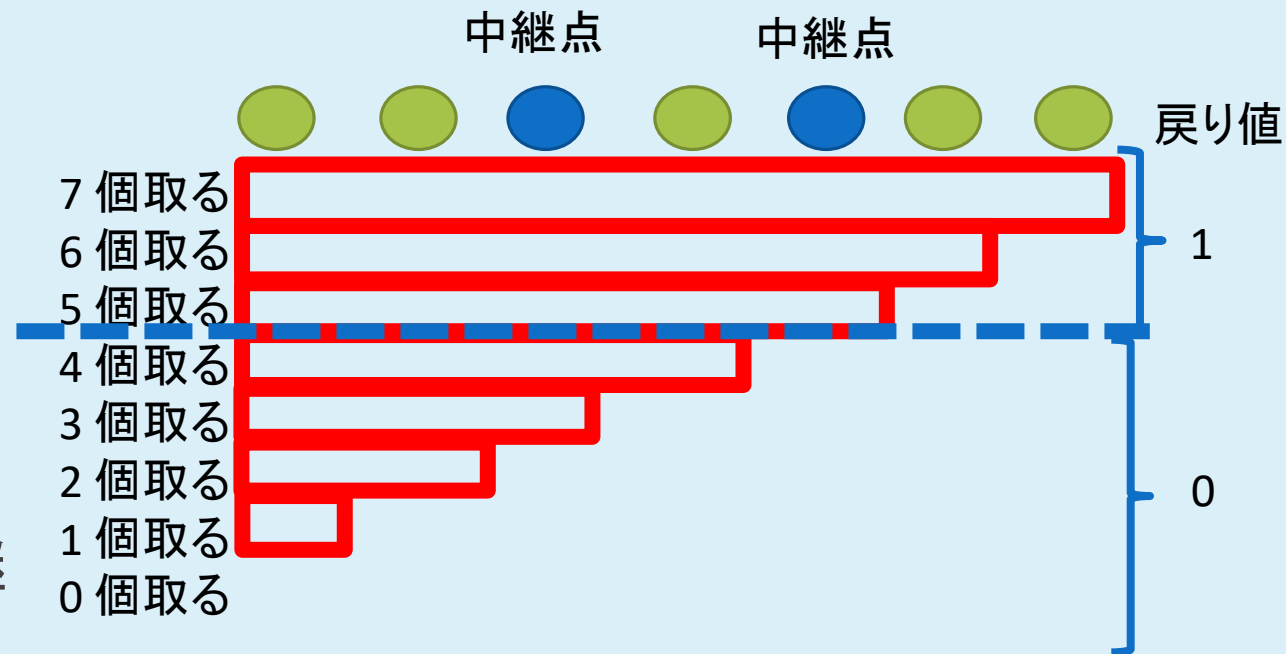
小課題 4 (30 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。
- 中継点のうち**最も右にあるものを境目**にして **0/1 が二分**される。



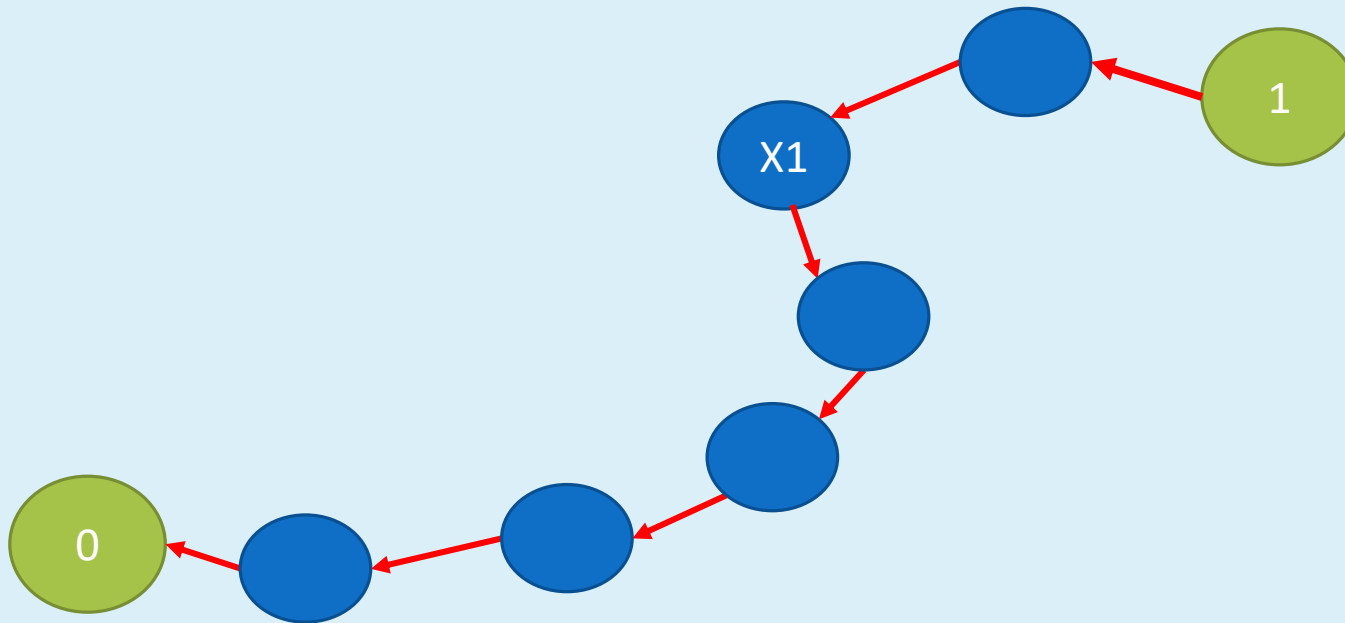
小課題 4 (30 点)

- 適当に左から右に並べて、左から連続していくつかとる方針で考えてみる。
- 中継点のうち**最も右にあるものを境目**にして **0/1** が二分される。
- **二分探索**を用いればいずれかの中継点を $\log N$ ステップで発見することができる！



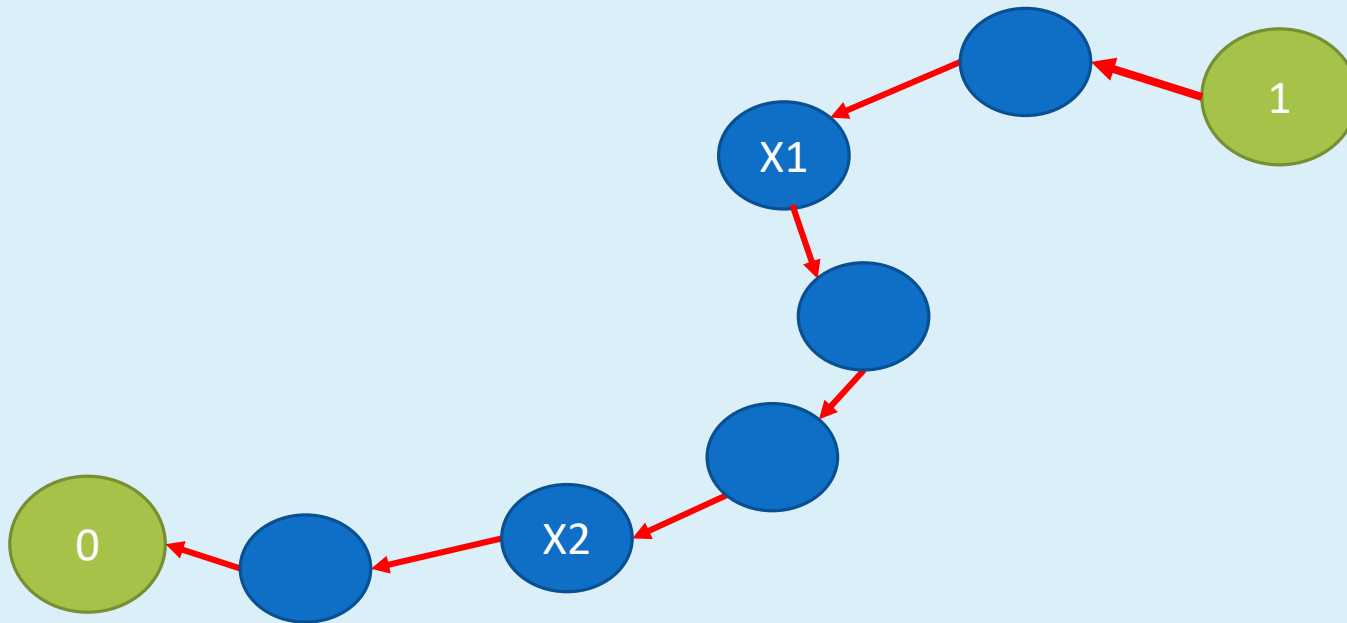
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。



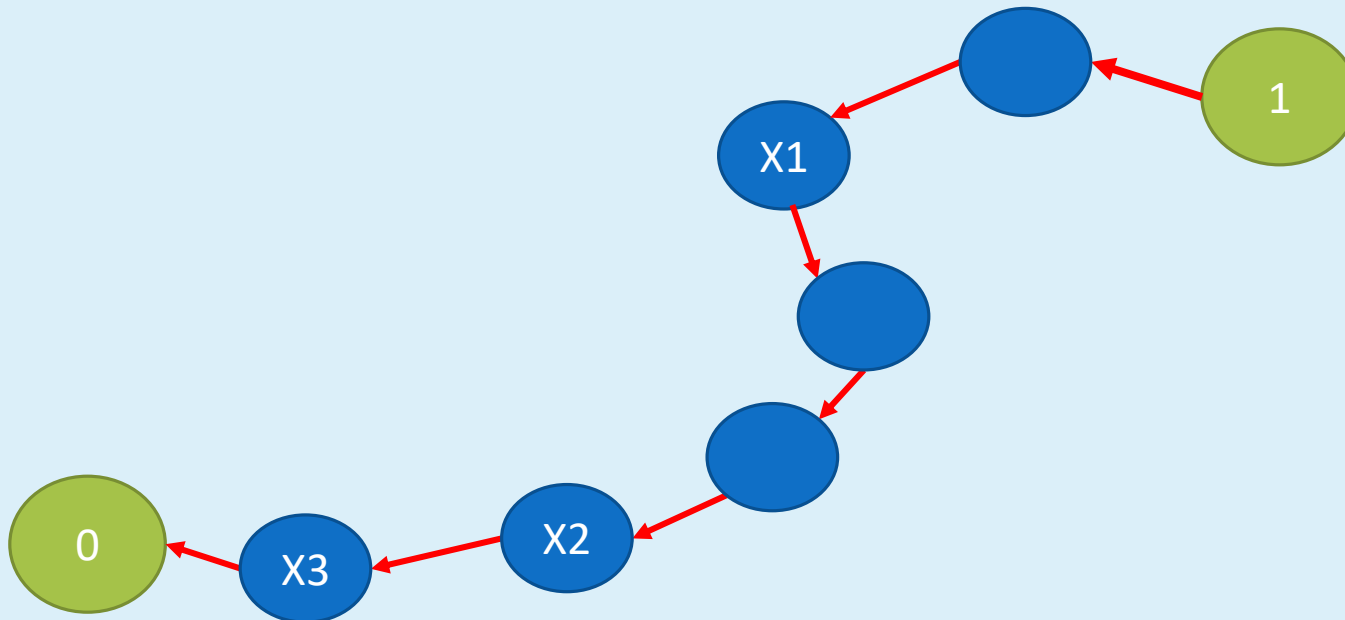
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。



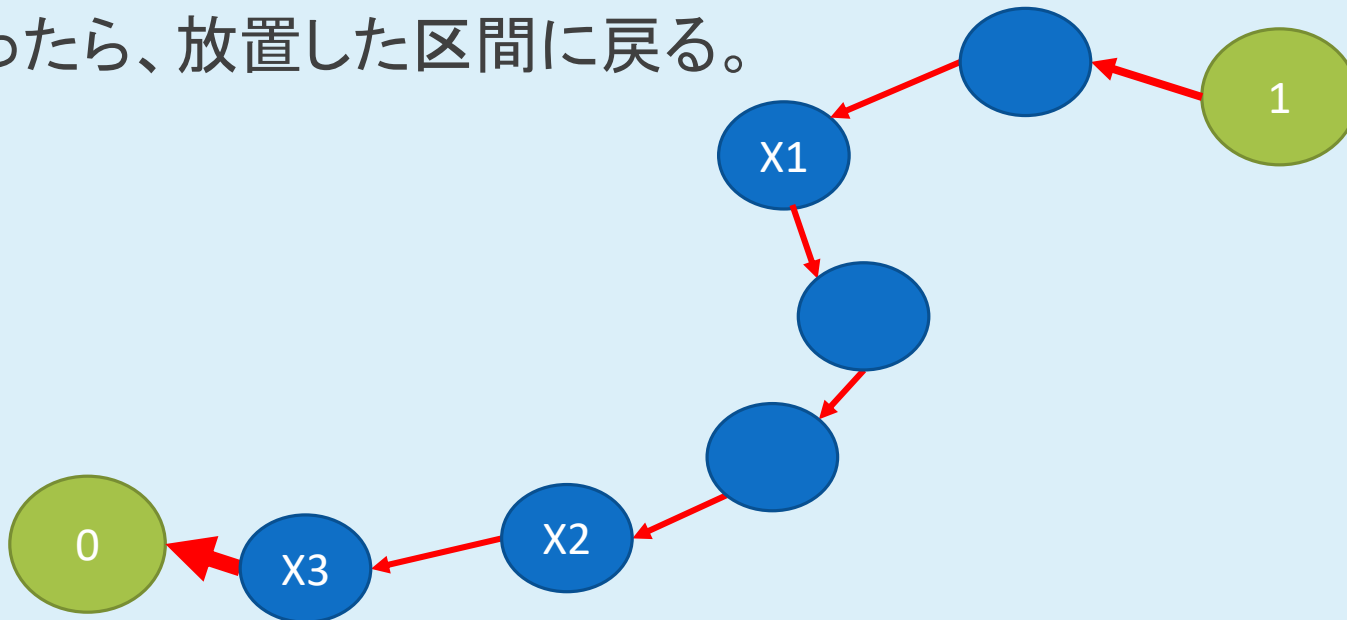
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)



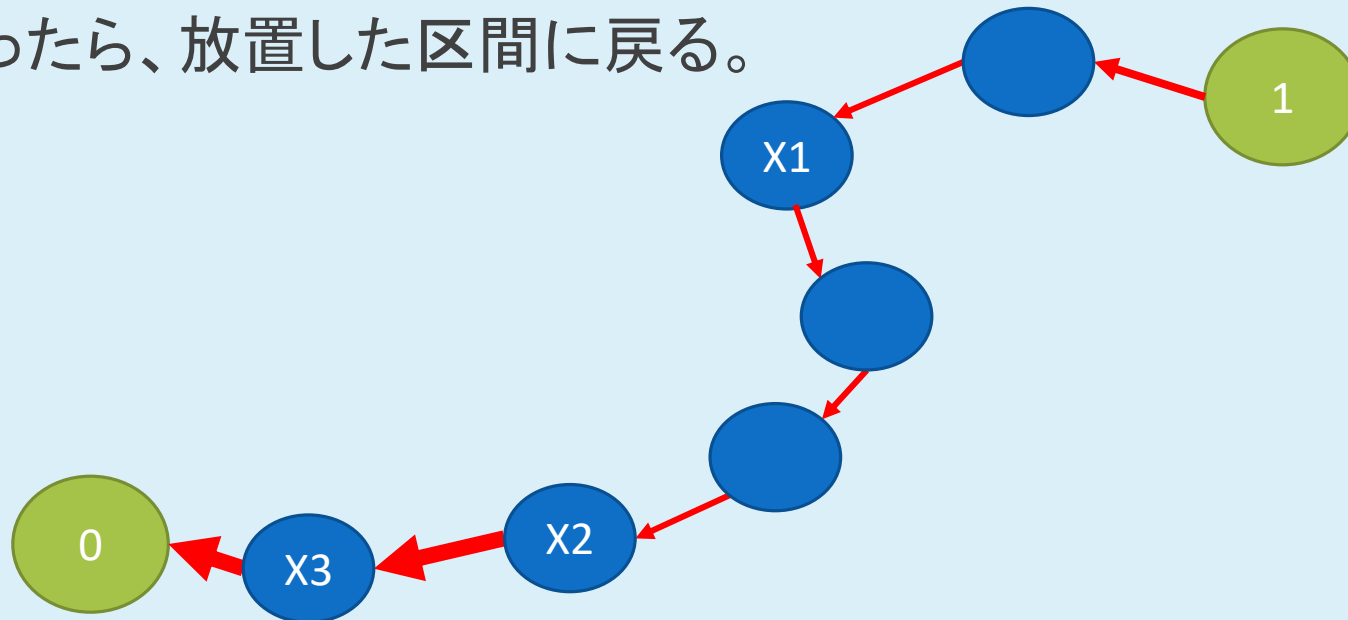
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



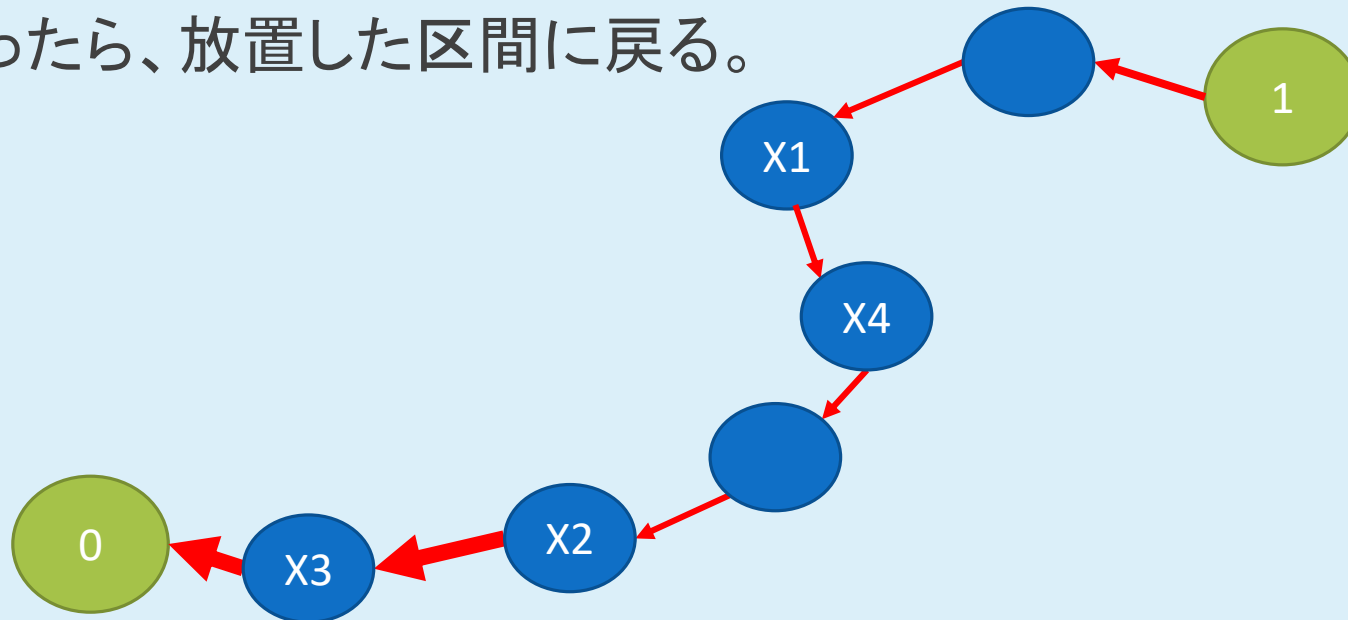
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



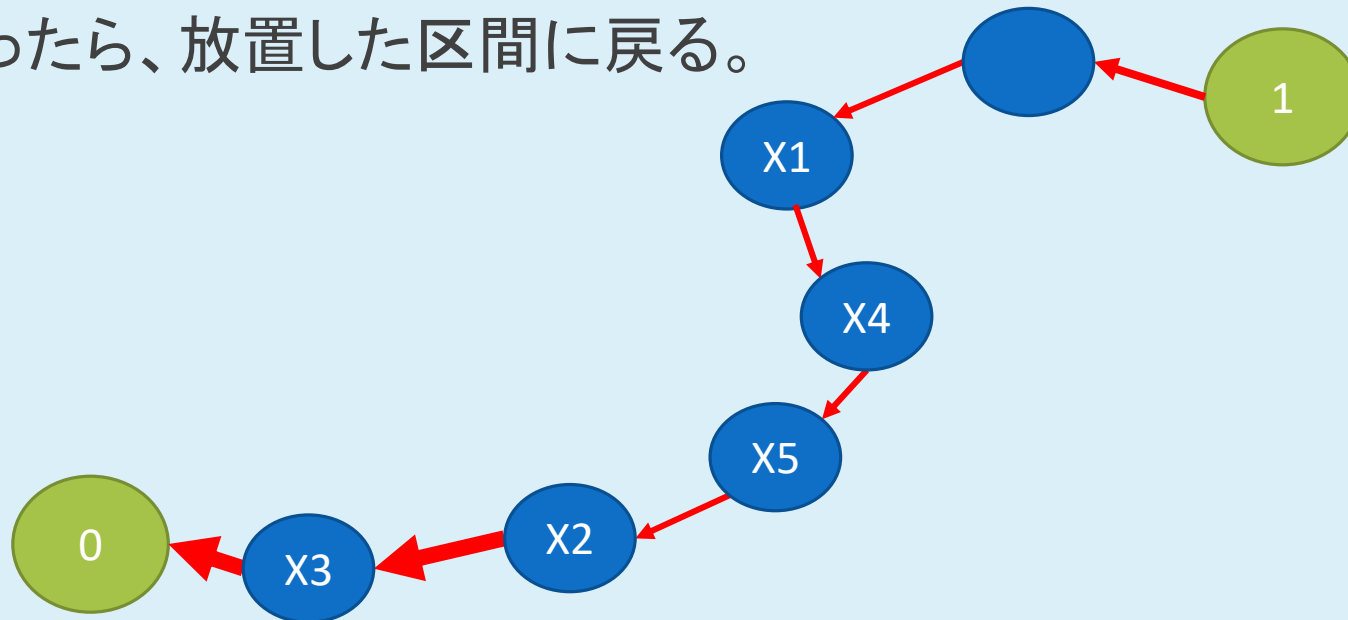
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



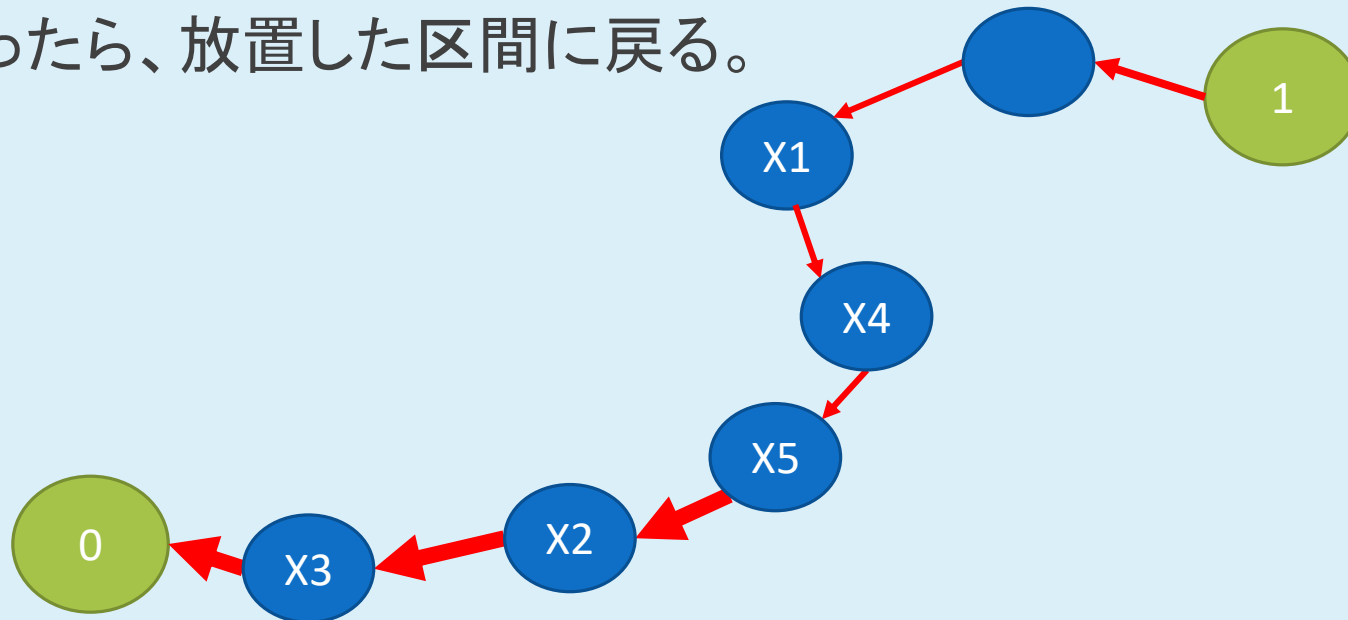
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



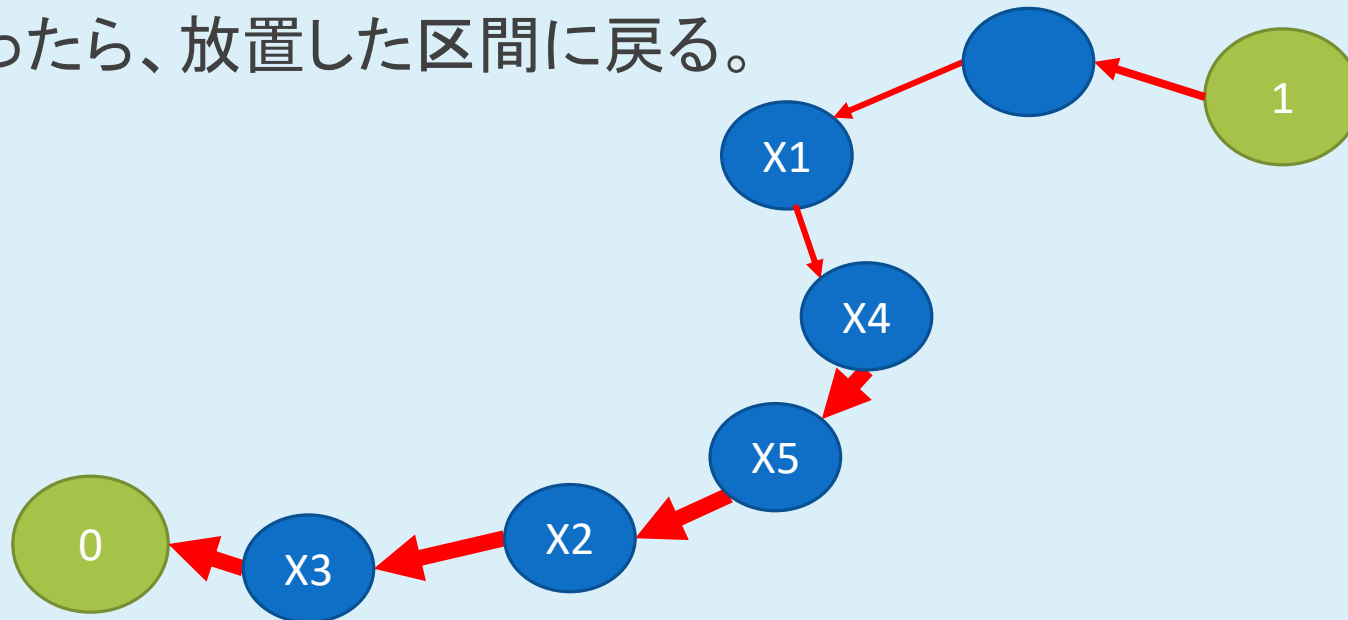
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



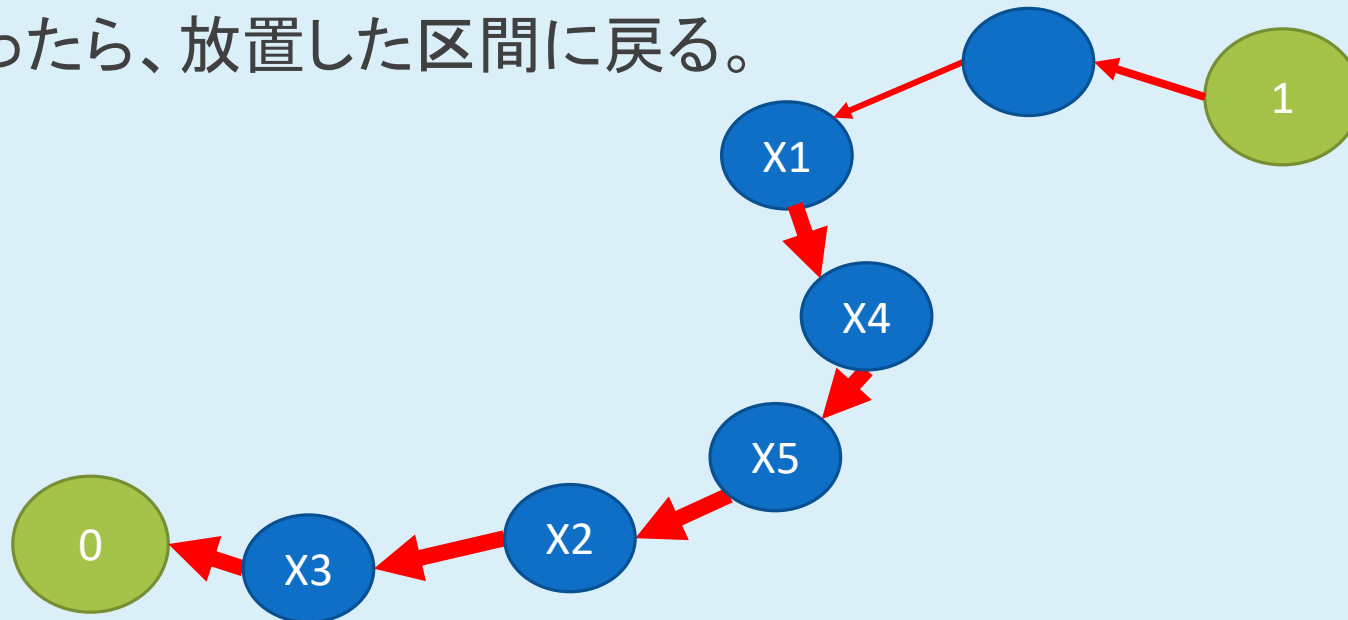
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



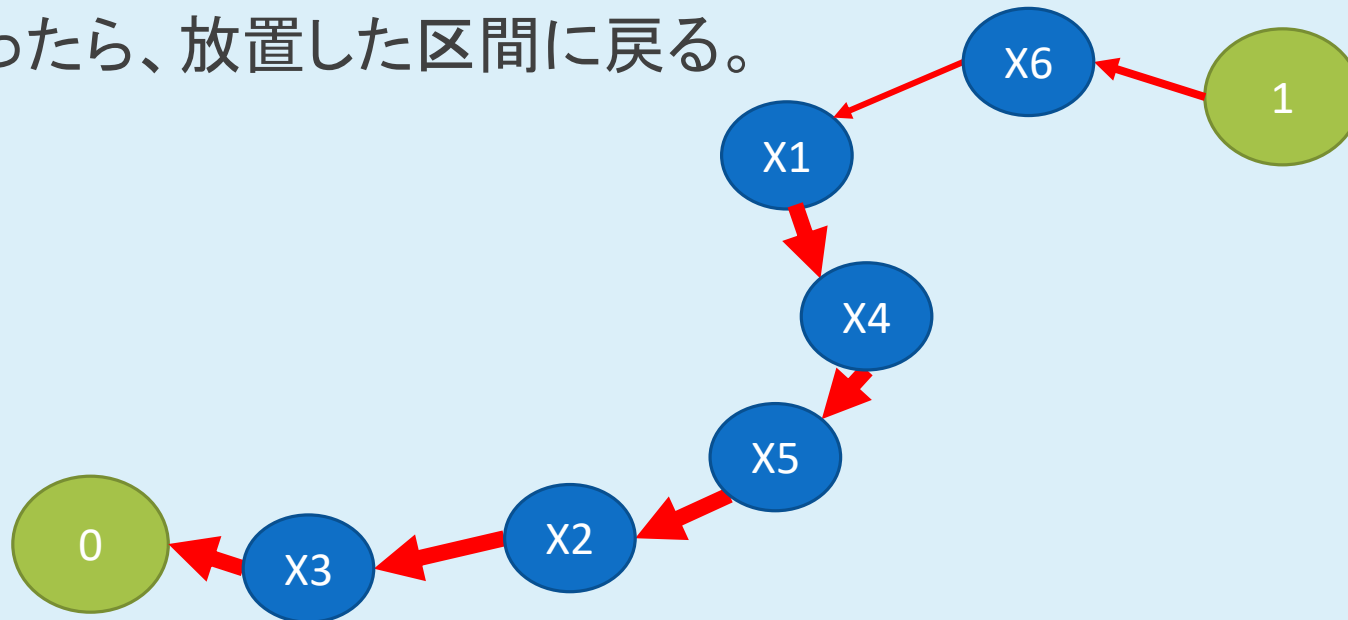
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



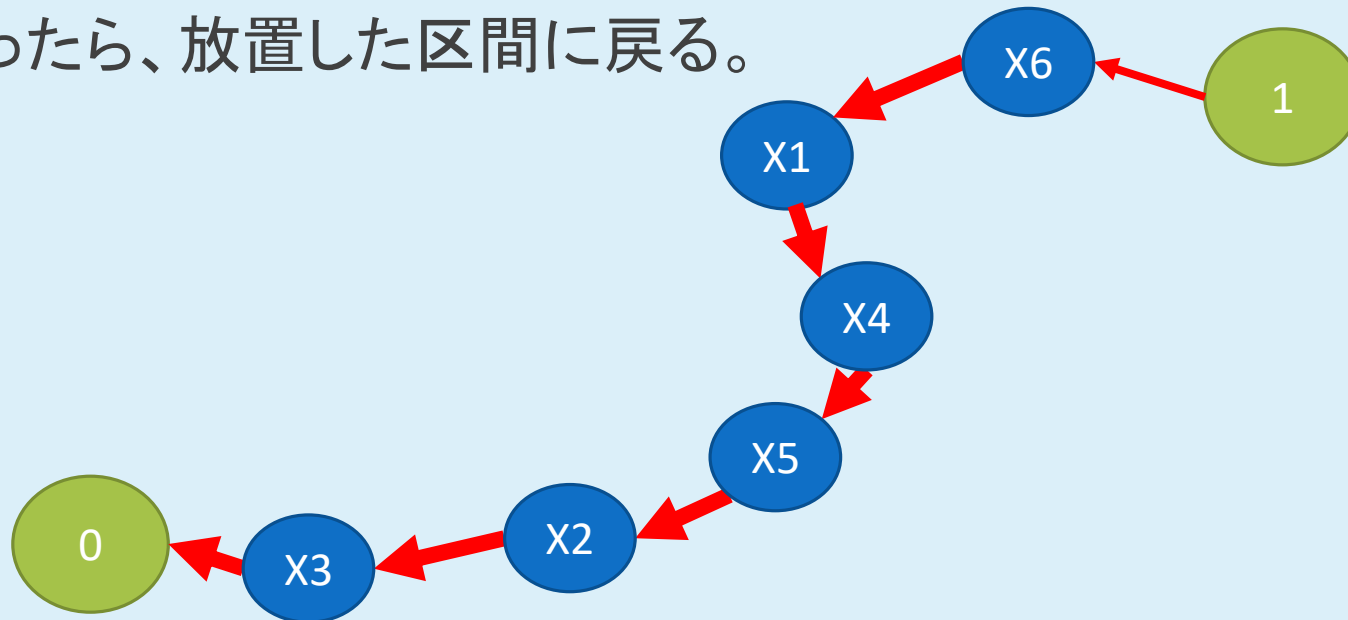
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



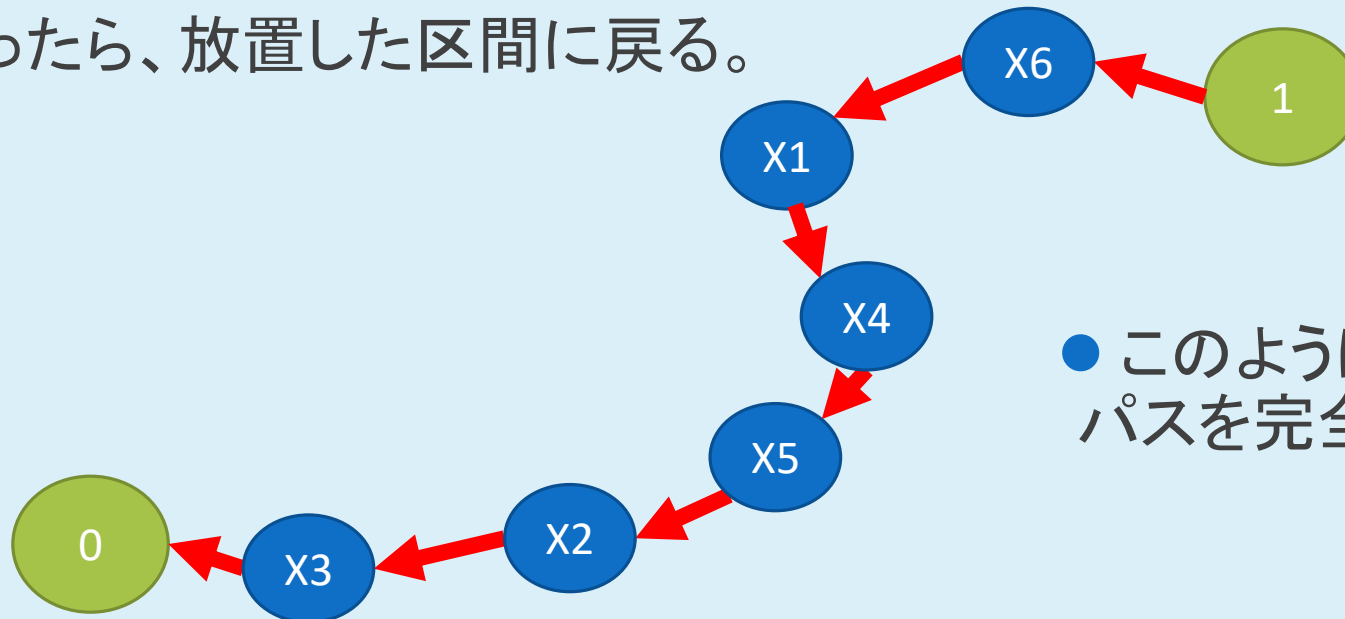
小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



小課題 4 (30 点)

- とりあえず中継点 (X1 とおく) が決まった。この後どうしよう？
- 残念ながら直接つながるのはやはり稀。
- じゃあ X1 を頂点 1 と同様に扱おう？ → より近い頂点 X2 が出る。
- それでもだめなら繰り返す (X3 → ...)
- 繋がったら、放置した区間に戻る。



- このようにすれば、頂点 0 と 1 をつなぐパスを完全に特定できる！

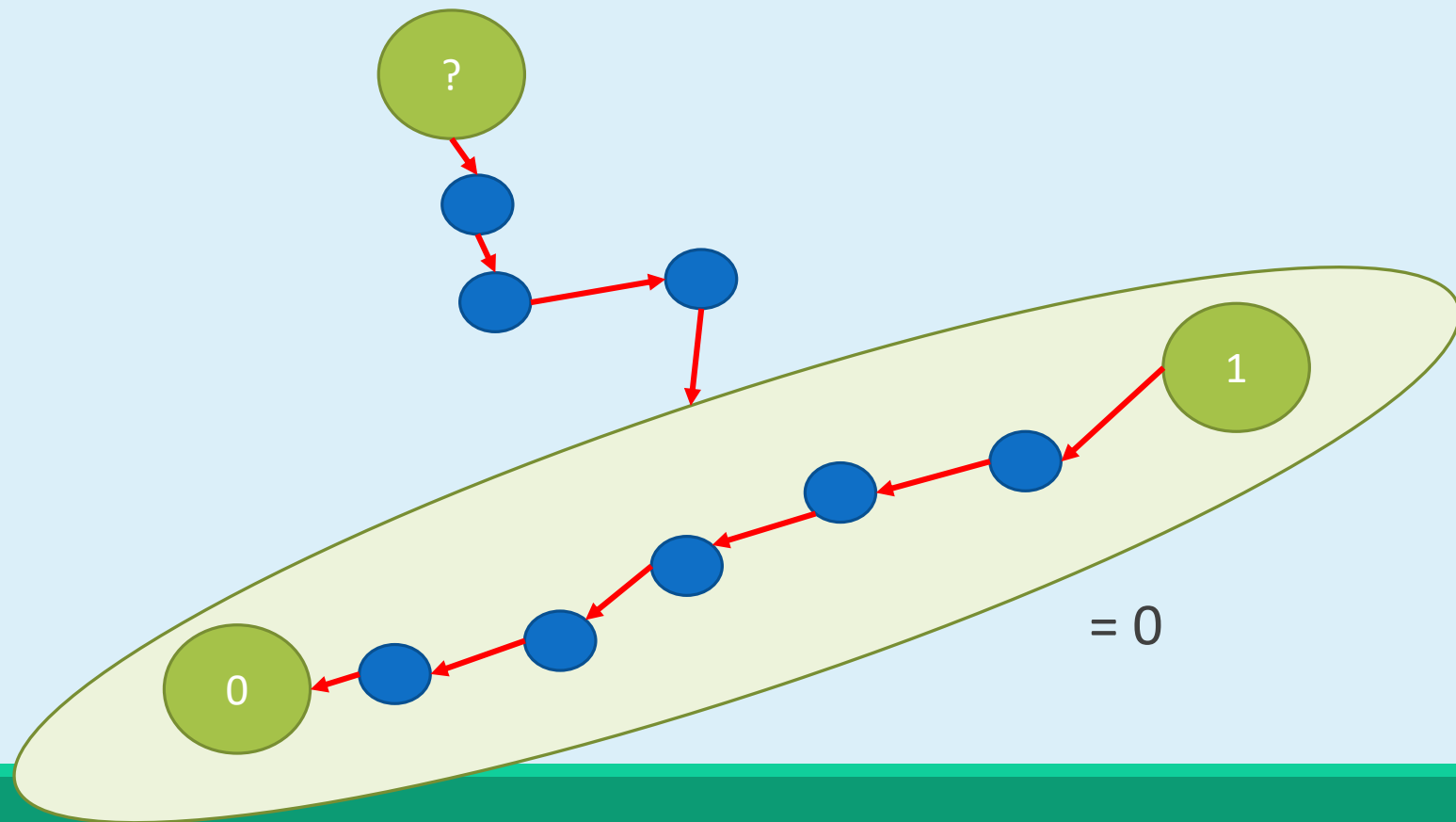
小課題 4 (30 点)

- それが終わった後は？



小課題 4 (30 点)

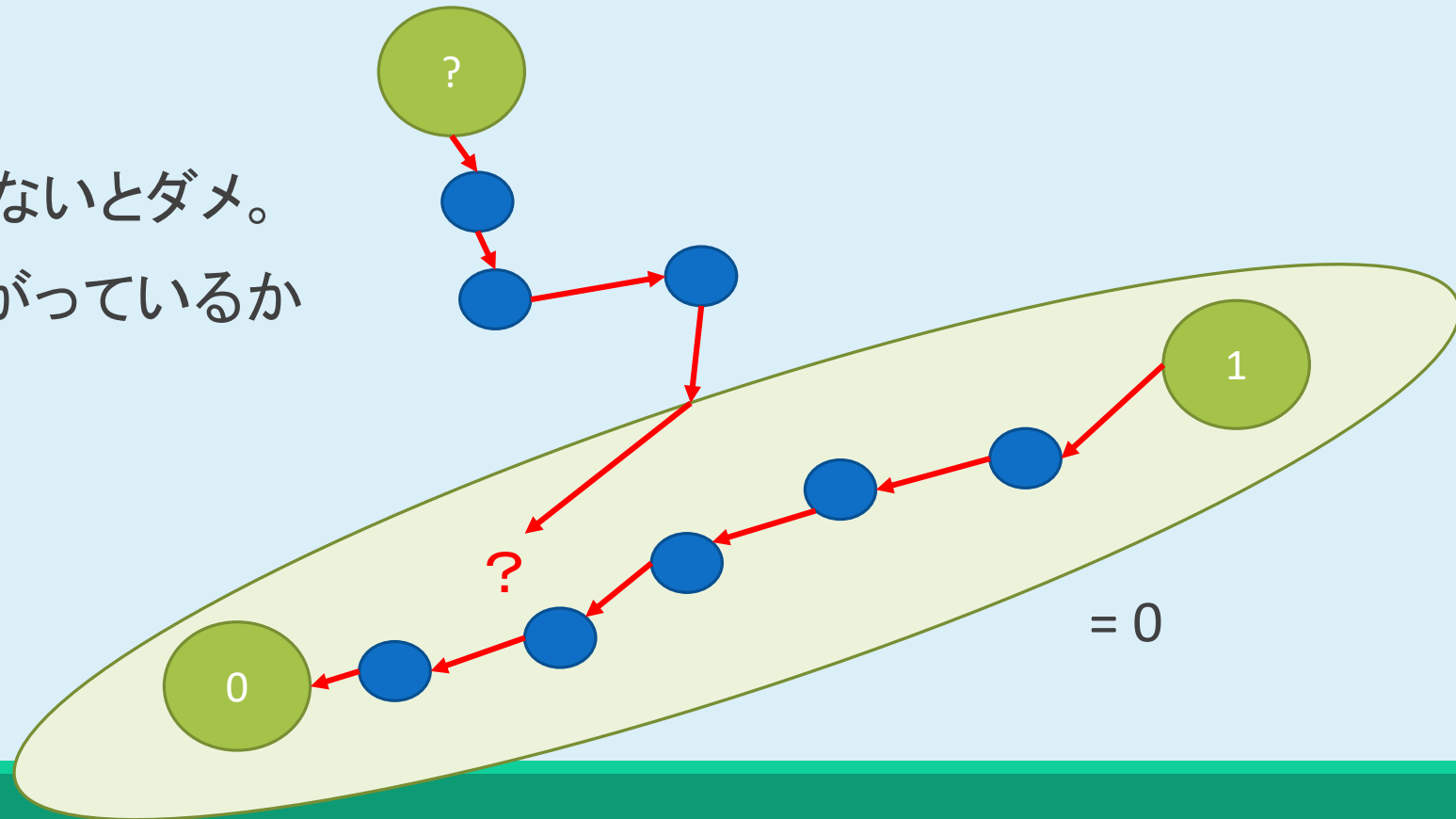
- それが終わった後は？
- わかっている部分全体を頂点 0 と同じに扱えば、同様にできる。



小課題 4 (30 点)

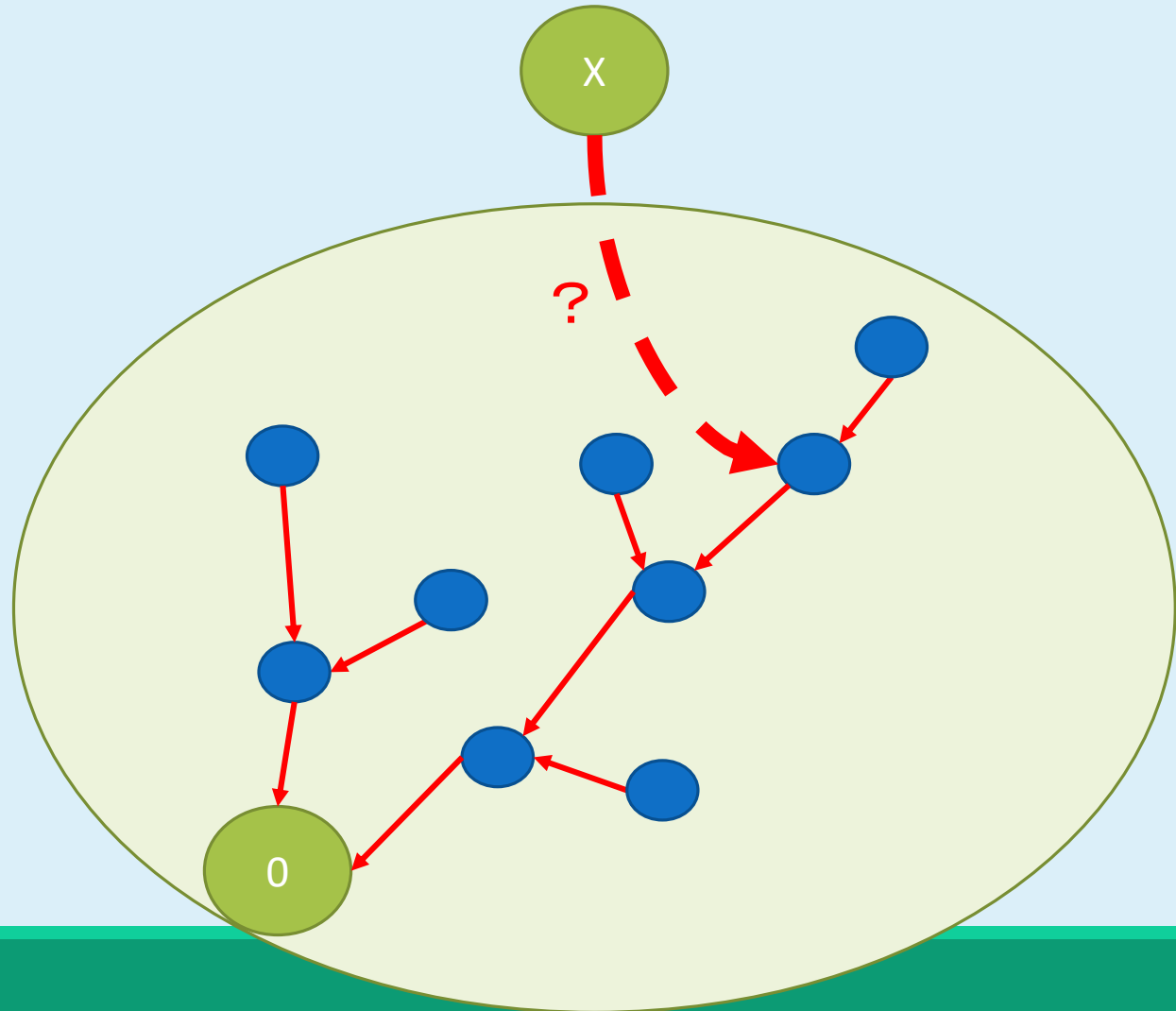
- それが終わった後は？
- わかっている部分全体を頂点 0 と同じに扱えば、同様にできる。

- ただし最後の辺の扱いは変えないとダメ。
(既知の頂点集合のどれにつながっているか
分からないため。)



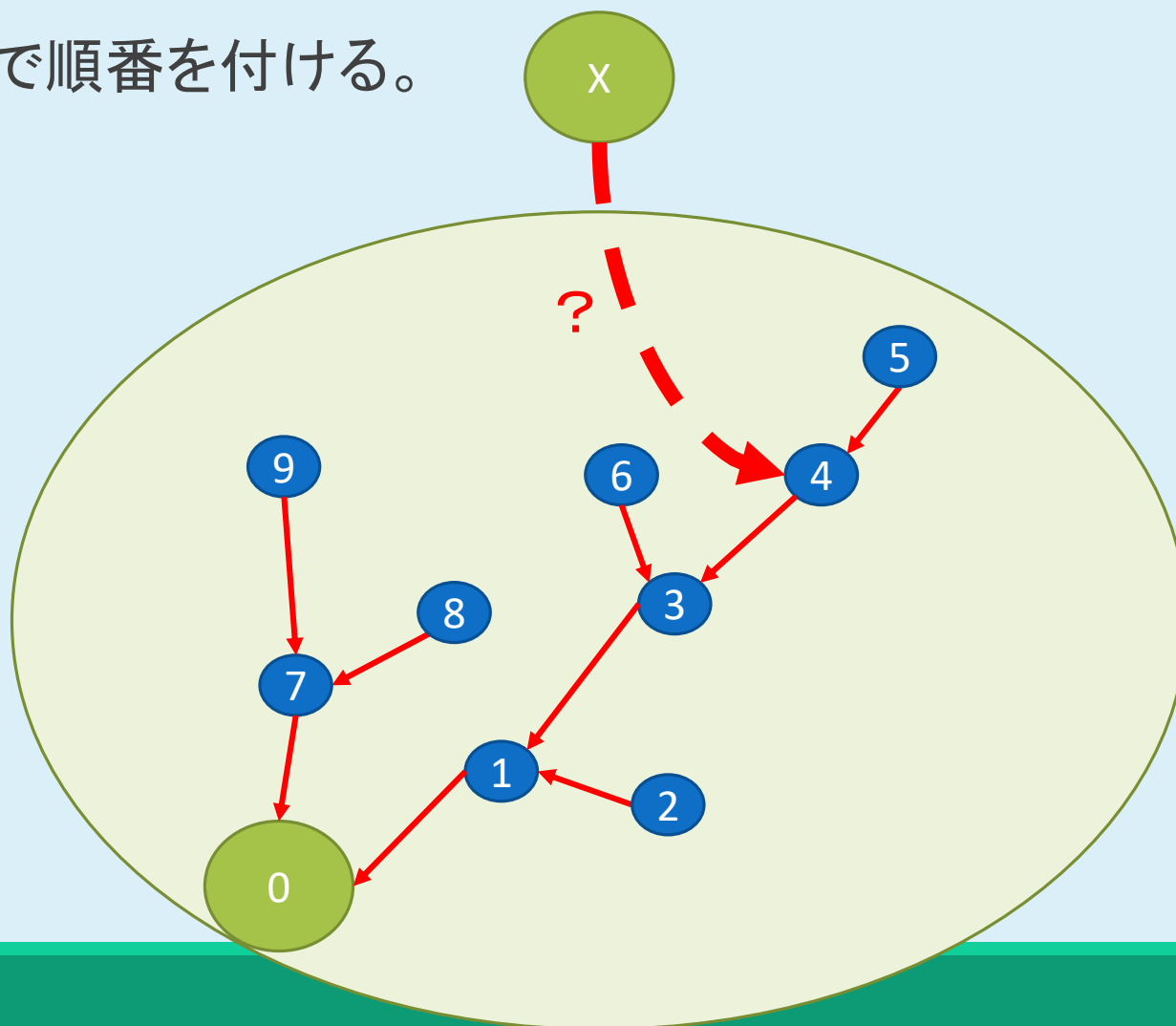
小課題 4 (30 点)

- これも**二分探索**で特定できる。



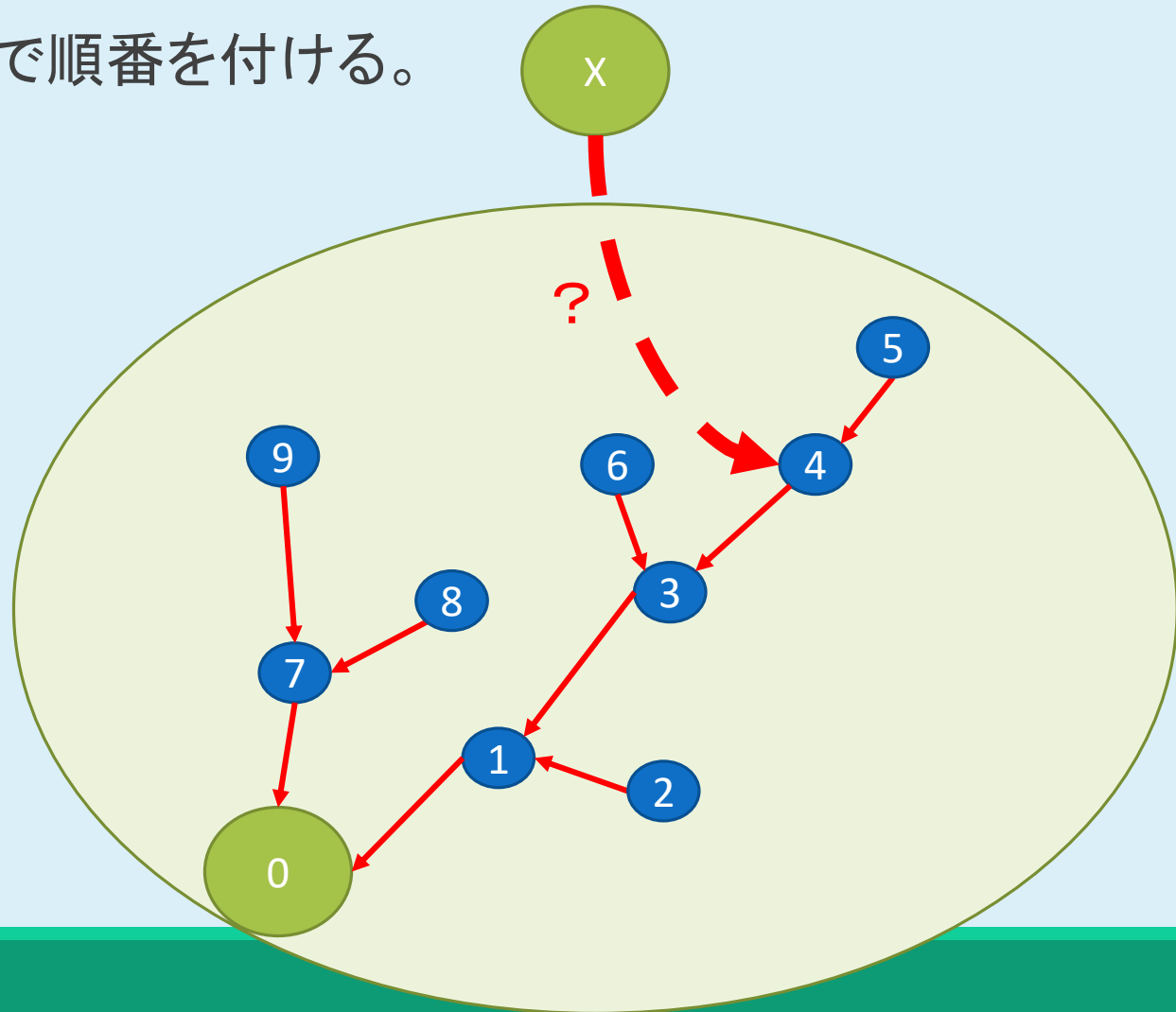
小課題 4 (30 点)

- これも**二分探索**で特定できる。
- 木に DFS 順 (BFS 順でも良い) で順番を付ける。



小課題 4 (30 点)

- これも**二分探索**で特定できる。
- 木に DFS 順 (BFS 順でも良い) で順番を付ける。
- 先頭から何番目まで含めると頂点 0 に到達できるかで**二分探索**する。



小課題 4 (30 点) 解法

- 以降の説明のため、スタックを導入します(既知ノード集合: 0 との接続関係が確定した木構造とする。初期は頂点 0 のみ)。

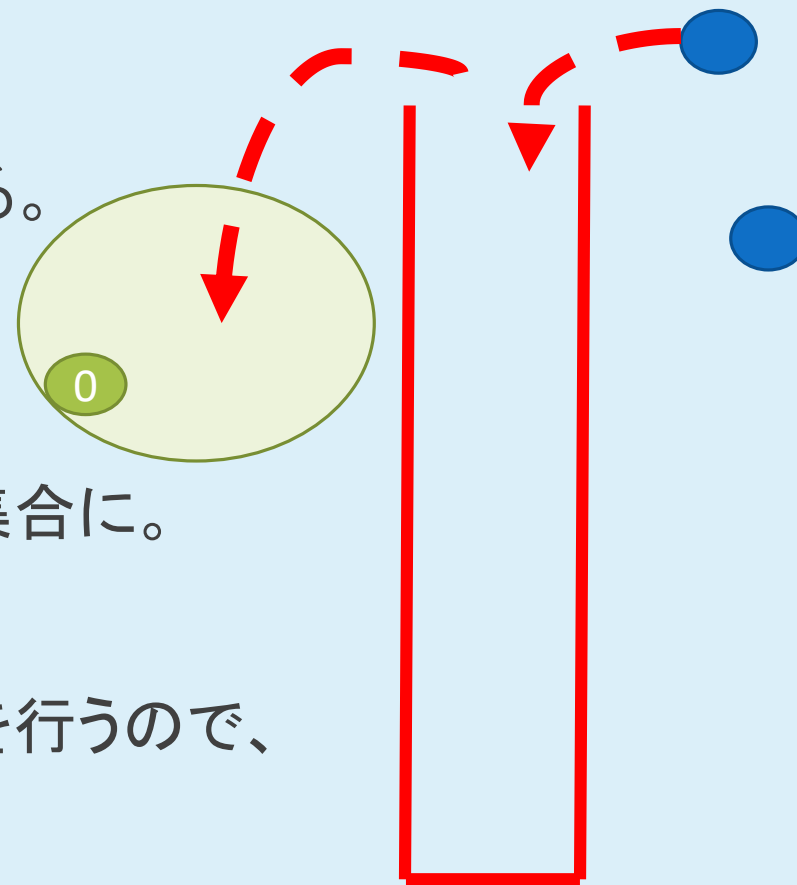
1. スタックが空なら未探索ノードを入れる。
2. 今のスタックトップが既知ノードとつながっているか見る。

→ つながっていないなら二分探索して間の頂点を見つけてスタックに追加する

→ つながっているなら 3. をしてスタックから既知ノード集合に。

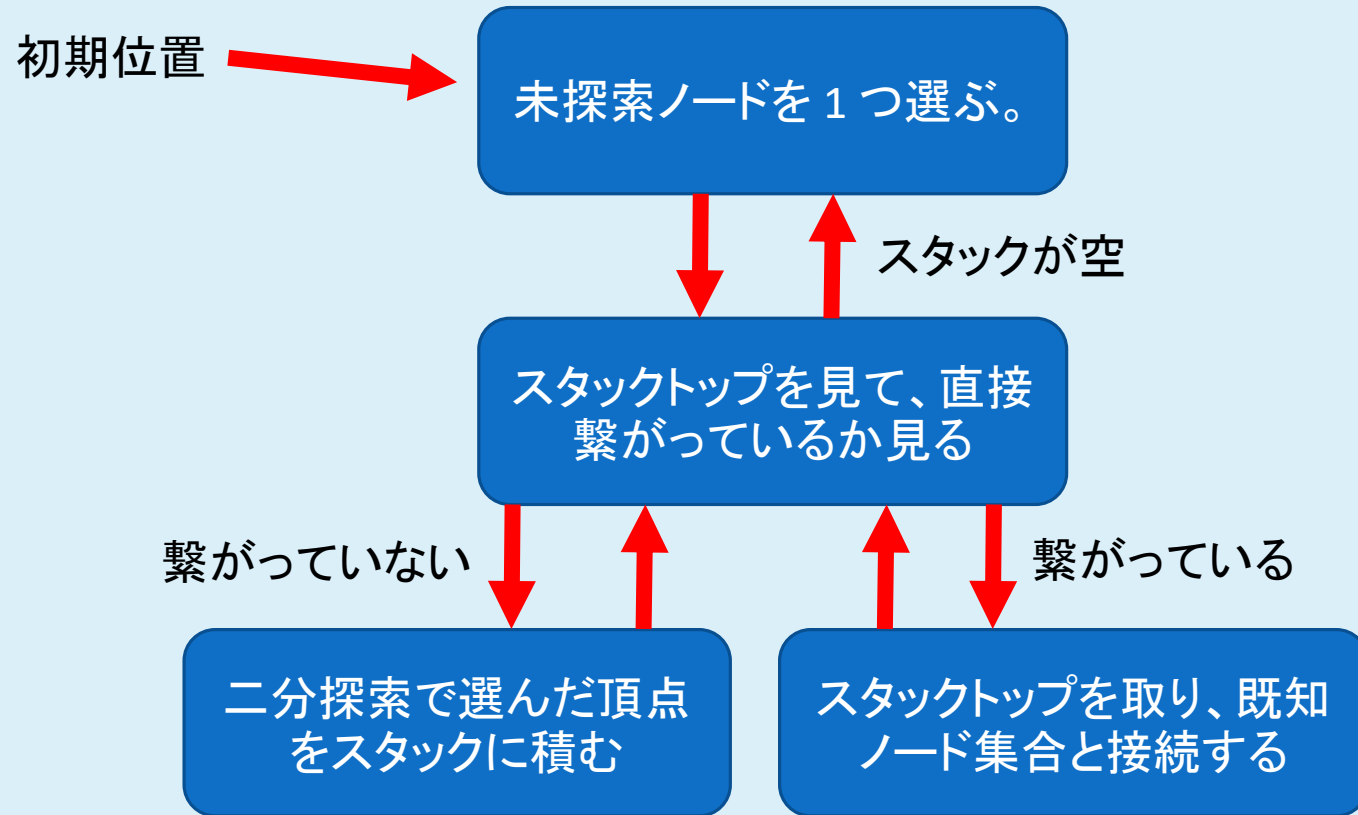
3. 既知ノードとの辺の接続関係を二分探索で決定する。

- $2N$ 回ほど二分探索をし、 N 回ほど 2 の最初の判定を行うので、全体で $(2\log N + 1)N \leq 32,200$ 回ほどで良く、OK。



小課題 4 (30 点) 解法

- 図にするとこのような形。



小課題 5 (23 点)

- 一般グラフ。

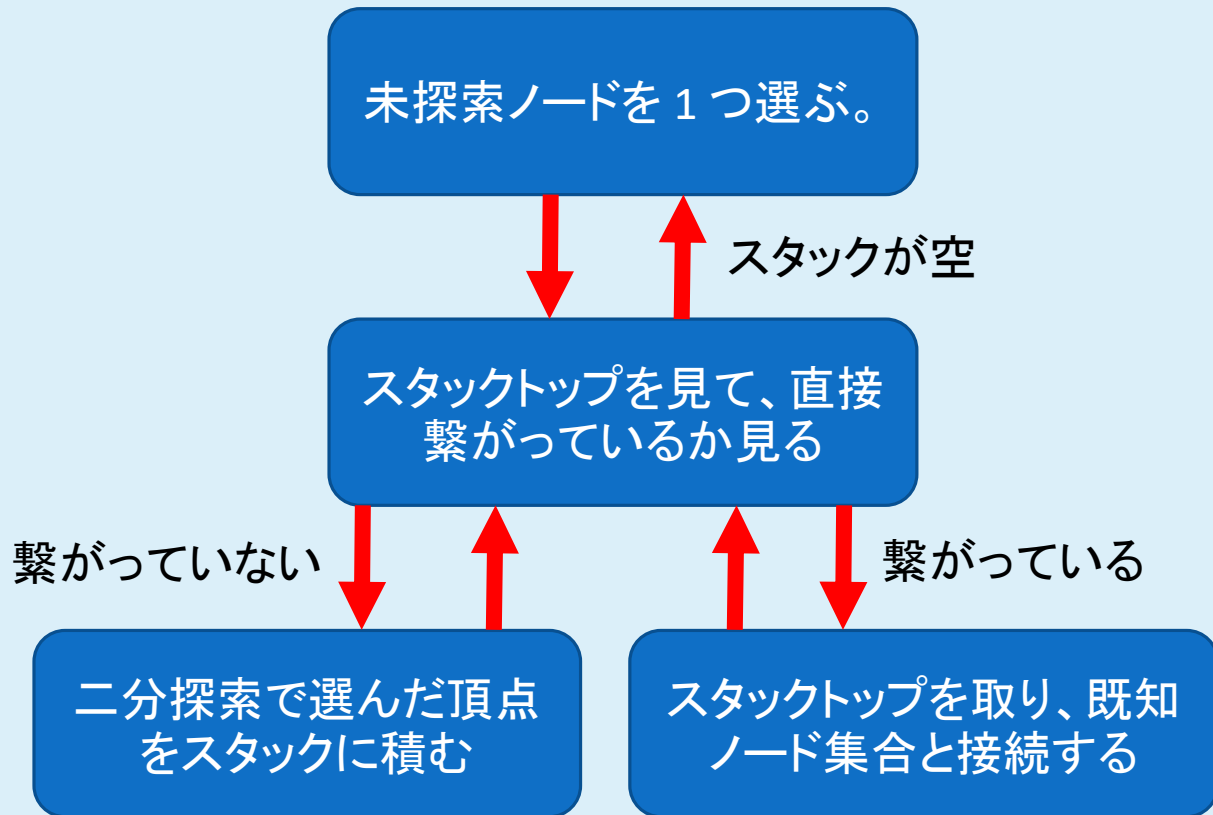
小課題 5 (23 点)

- 一般グラフ。

- 一見どうしようもなさそうですが、実は小課題 4 の解法を少しいじるだけで満点をとれます。

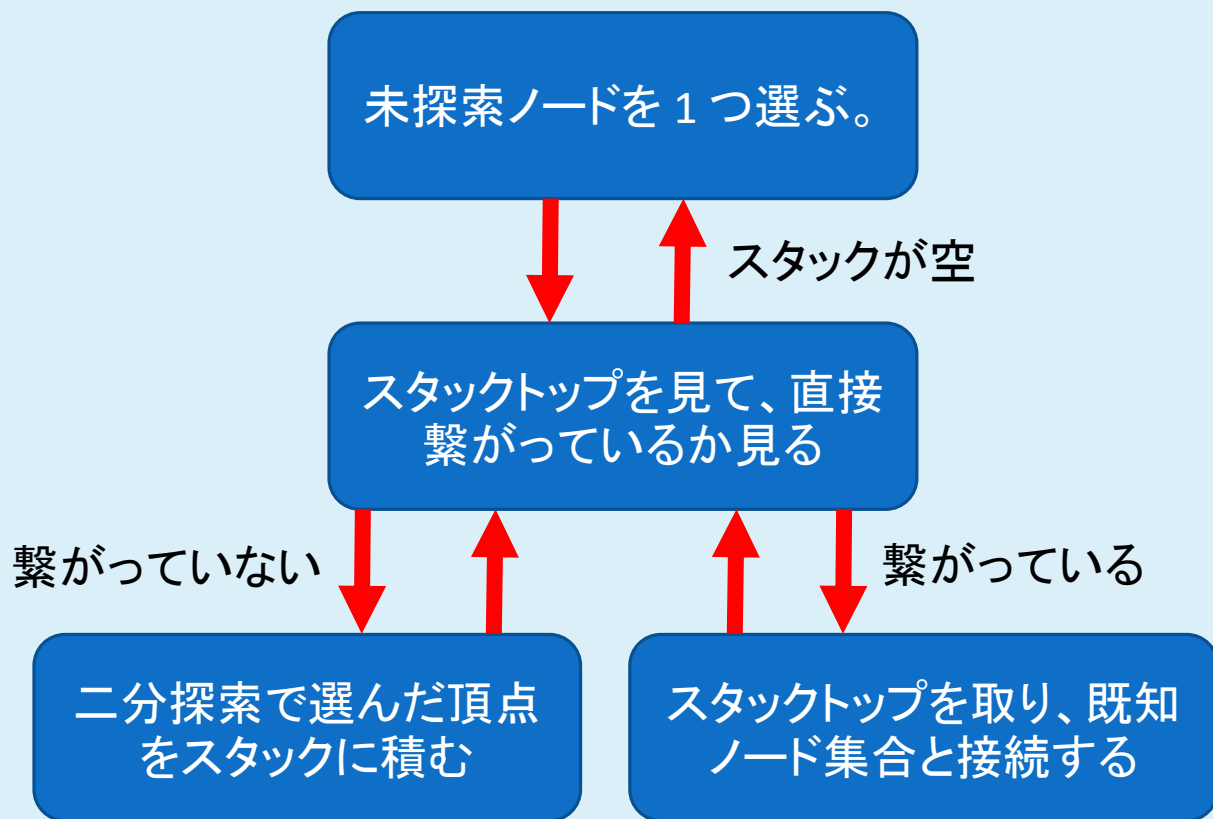
小課題 5 (23 点)

- 小課題 4 の解法を確認してみる。

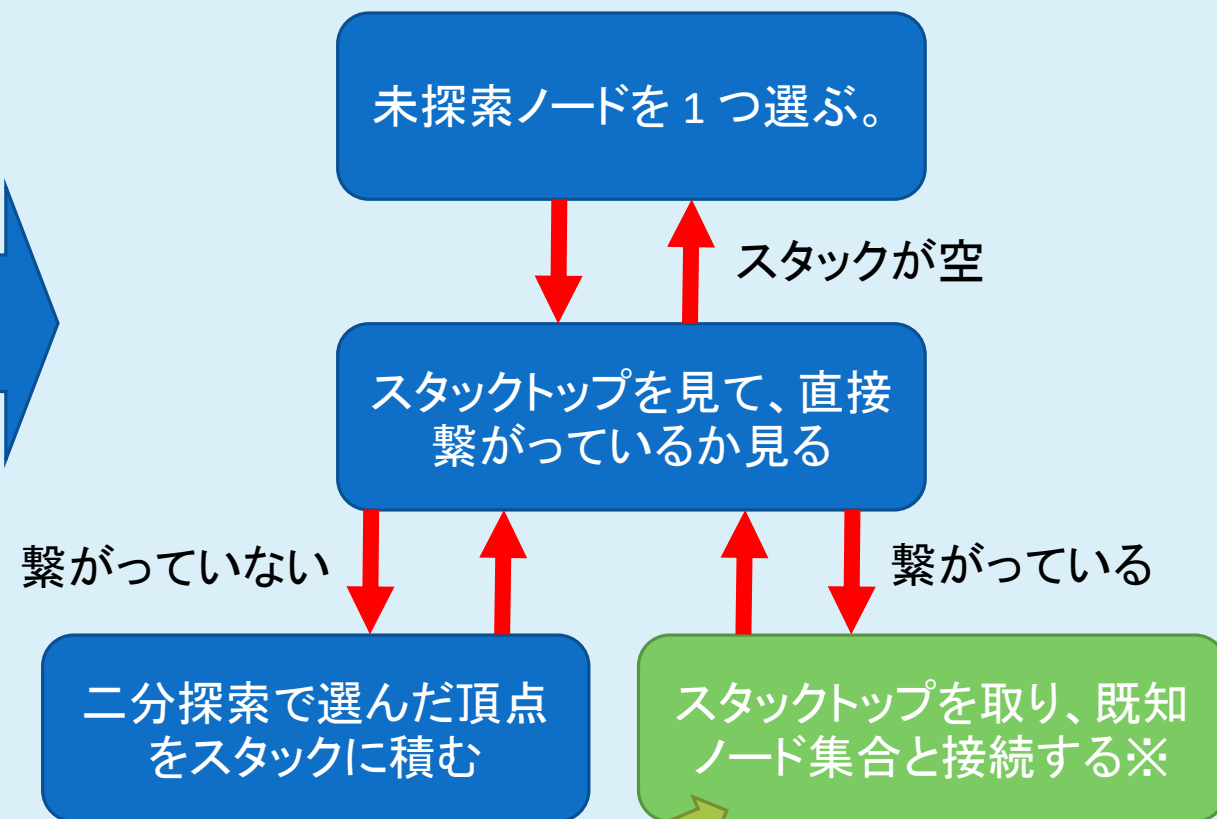


小課題 5 (23 点)

- 小課題 4 の解法を確認してみる。



- 小課題 5 の解法はほとんど同じ。



手を加えるのは主にここだけ！

小課題 5 (23 点)

- 確かに一般グラフでもスタック周りは同じように動きそうだけれど、スタックの中身はどうなるの？
- 木ならばパス上という性質が成り立っていたが、一般グラフだとどんな性質が成り立つの？

小課題 5 (23 点)

- 確かに一般グラフでもスタック周りは同じように動きそうだけれど、スタックの中身はどうなるの？
- 木ならばパス上という性質が成り立っていたが、一般グラフだとどんな性質が成り立つの？
- 実は下記のことが言えます：

命題：

スタックのどの要素についても、どのタイミングにおいても、それよりスタックの奥に置かれているいずれの頂点も使用せずに頂点 0 に行くことができる。

どういうことかという、スタックの中身を $[x_1, x_2, \dots, x_k]$ としたとき、 x_i ($1 \leq i \leq k$) が、頂点 $x_1, \dots, x_{i-1}$ のいずれをも使用せずに頂点 0 に到達可能であるという性質が常に成立するという主張。

小課題 5 (23 点)

命題:

スタックのどの要素についても、どのタイミングにおいても、それよりスタックの奥に置かれているいずれの頂点も使用せずに頂点 0 に行くことができる。

● 証明(概略):

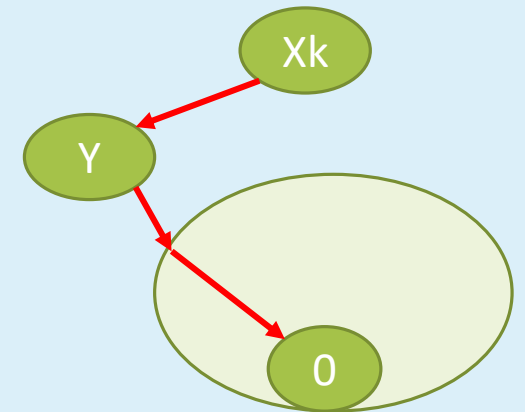
スタック内の要素は上から順に取られるので、一度スタックに入ったらそれより下の頂点集合は変化しない。→スタックに追加する際に性質が満たされていればOK。

今、スタックに $[X_1, X_2, \dots, X_k]$ が積まれているとして、新たに Y を積むことを考える。

Y が選ばれたということは、「 X から Y を経由して既知ノード集合に入り頂点 0 へ至る点素パスが存在する」ということである。($\because Y$ を選べなくなった瞬間に X から 0 へのパスがすべて消滅する)

このうち Y 以降を考えると、仮定より X_k より前の要素存在しないし、点素なので X_k も登場しない。

よって Y をスタックに入れても性質が壊れない！ □



小課題 5 (23 点)

- 以上のスタックを使うと何が嬉しいのか？

→ **スタックトップからは常に既知ノード集合に到達可能**である。

- スタックに追加する回数は高々 $N - 1$ 回なので、効率的にすべてのノードを既知ノード集合に加えることができるようになる！

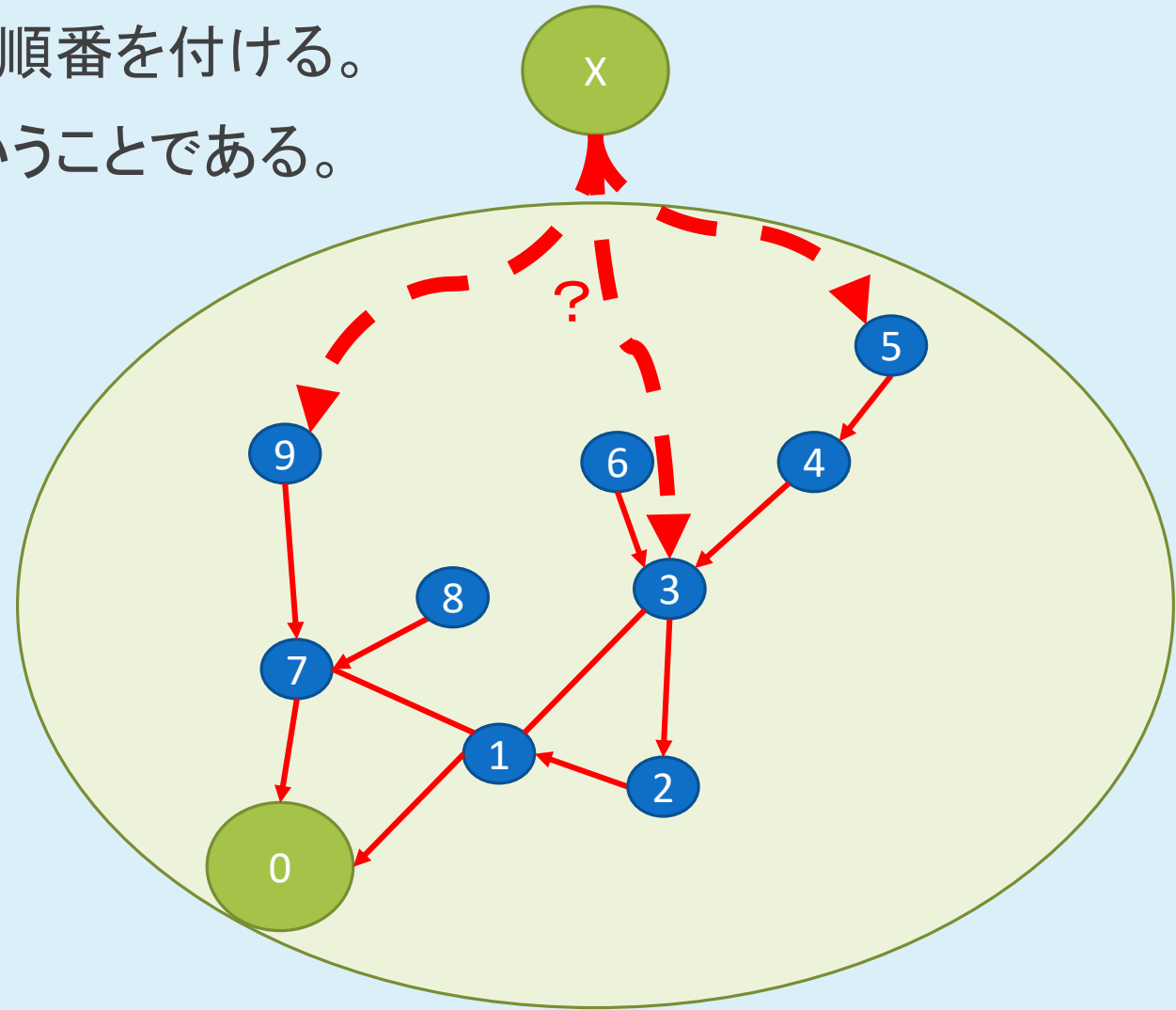
小課題 5 (23 点)

- さて、ここまでは既知ノード集合に接続する頂点の選び方である。
- では、実際に繋ぐときはどうすればいいのか？

→ 小課題 4 のやり方とほとんど同じである。

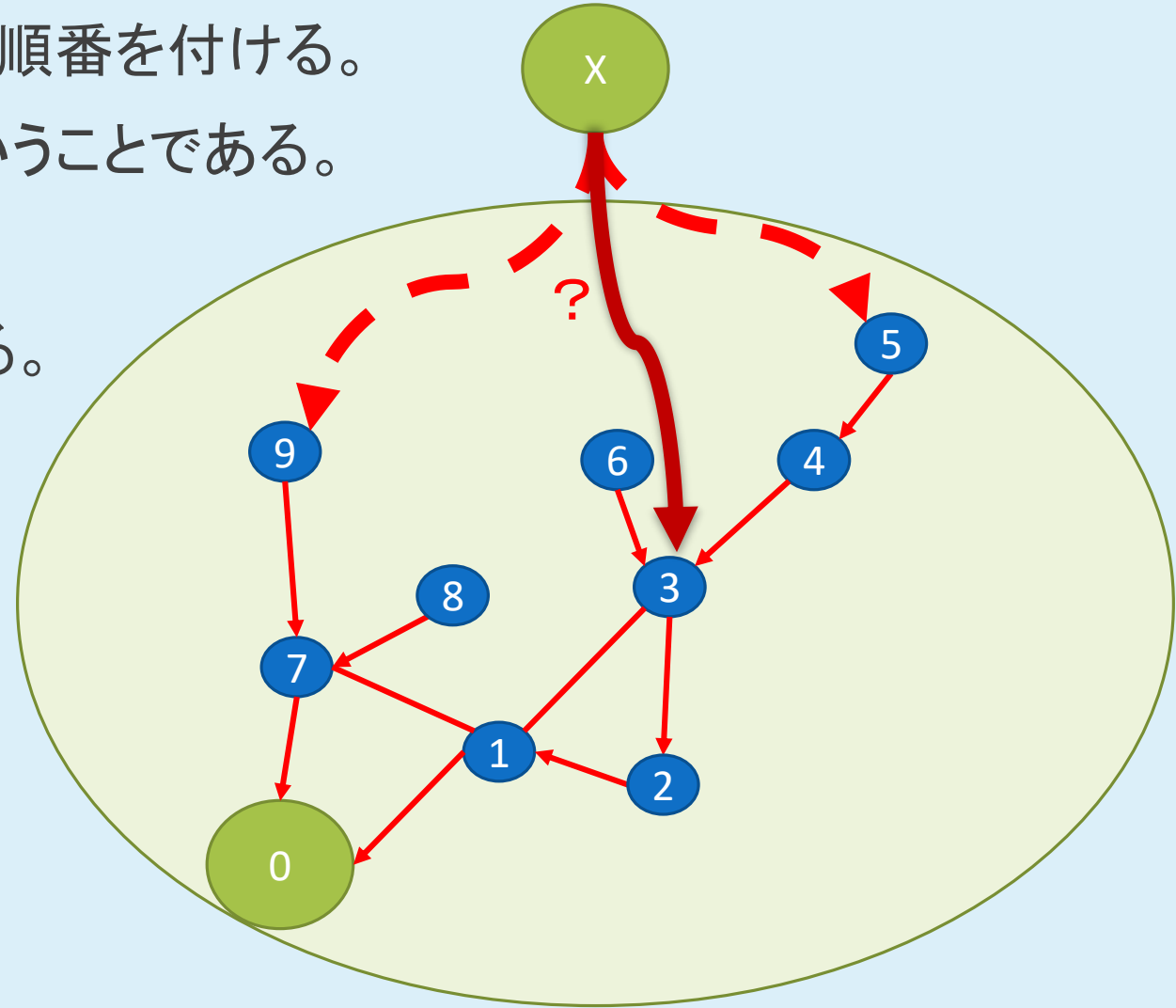
小課題 5 (23 点)

- 一般グラフに DFS 順 (BFS 順でも良い) で順番を付ける。
- 厄介なのが、辺が**複数本**伸びているということである。



小課題 5 (23 点)

- 一般グラフに DFS 順 (BFS 順でも良い) で順番を付ける。
- 厄介なのが、辺が**複数本**伸びているということである。
- **二分探索**すれば、辺を 1 本は特定できる。
(右の例だと 3 への辺が発見できる)



小課題 5 (23 点)

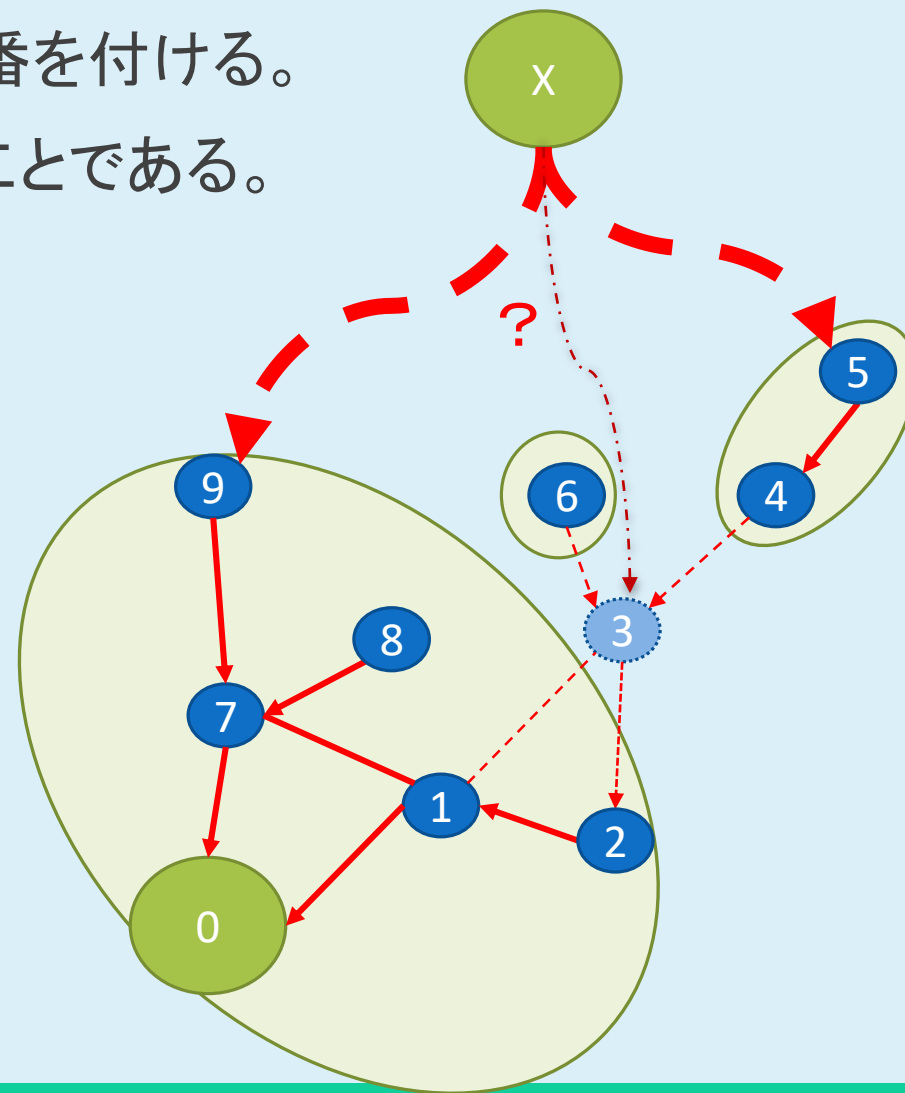
- 一般グラフに DFS 順 (BFS 順でも良い) で順番を付ける。
- 厄介なのが、辺が**複数本**伸びているということである。

- **二分探索**すれば、辺を 1 本は特定できる。

(右の例だと 3 への辺が発見できる)

- 残りの頂点は、発見した辺に関する頂点を除いた各連結成分について見ることで発見できる！

(構造は知っているので他の連結成分についても頂点 0 の役割を担う頂点を決定できる。)

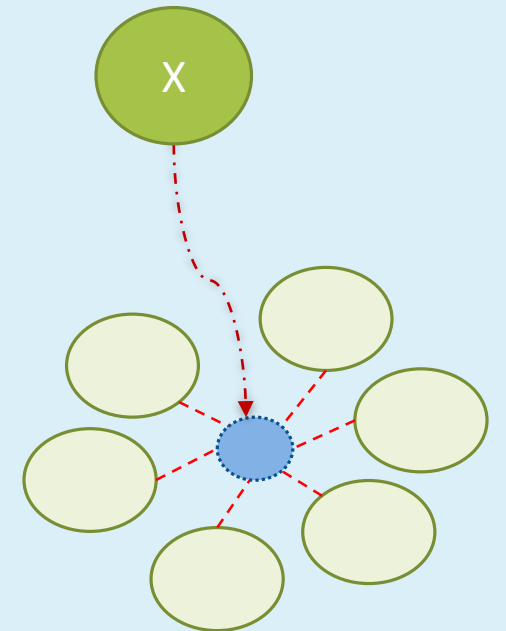


小課題 5 (23 点)

- しかし、増えた連結成分に等しい個数だけ追加でコストがかかってしまう。(大量に連結成分が増えると最悪なことになる)

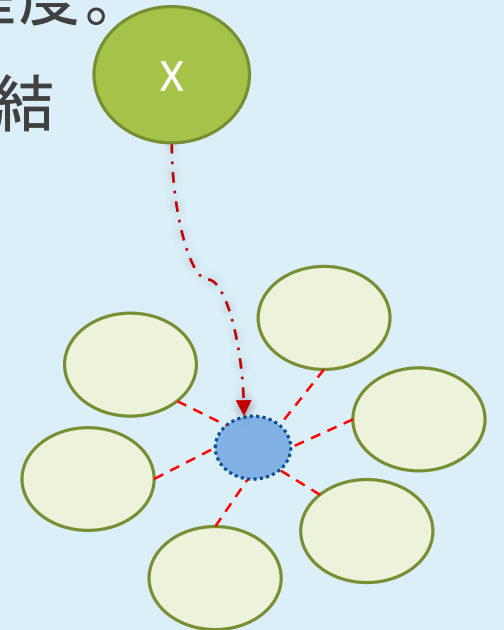
小課題 5 (23 点)

- しかし、増えた連結成分に等しい個数だけ追加でコストがかかってしまう。(大量に連結成分が増えると最悪なことになる)
- ところでどの頂点も**次数が 7 以下**だったので、追加で見なければならなくなる連結成分は**高々 6 個**である。



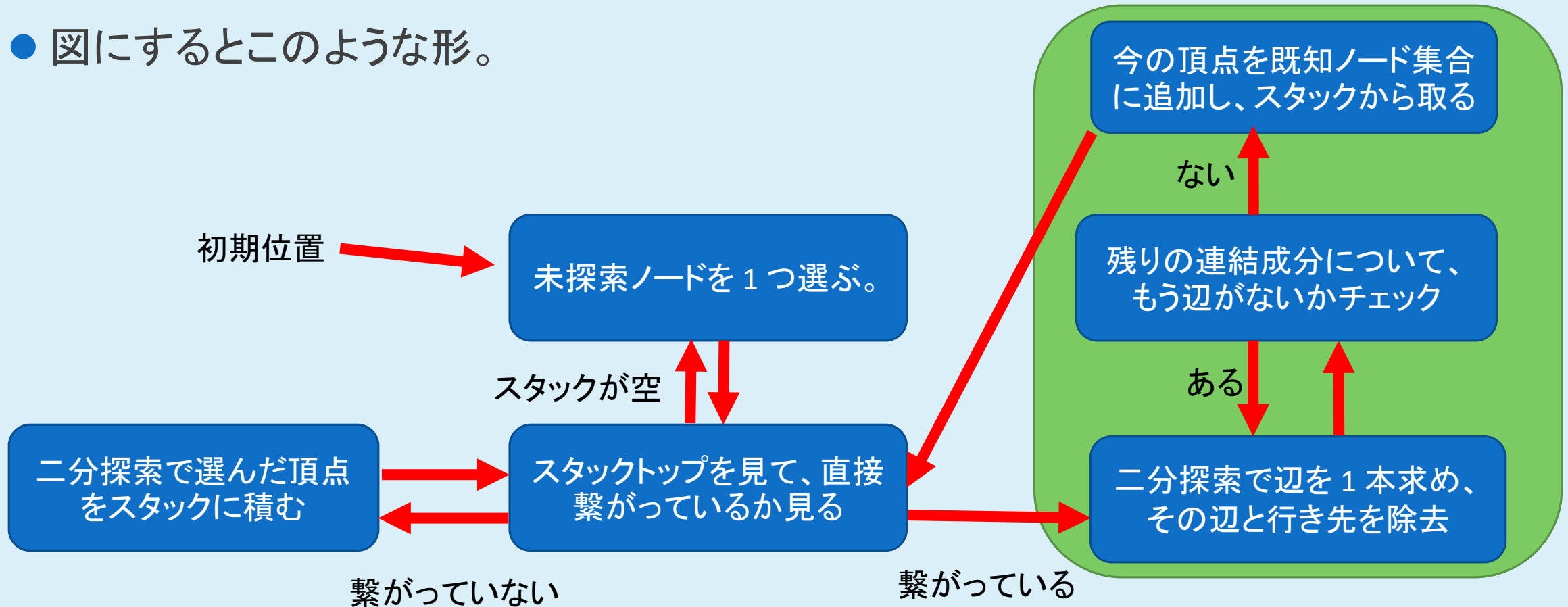
小課題 5 (23 点)

- しかし、増えた連結成分に等しい個数だけ追加でコストがかかってしまう。(大量に連結成分が増えると最悪なことになる)
- ところでどの頂点も**次数が 7 以下**だったので、追加で見なければならなくなる連結成分は**高々 6 個**である。
- ということは、辺の発見以外で要求される追加コストは **$6M$** 回程度。
($\because$ 辺があると言われるのが M 回で、辺がないと言われるのが連結成分の個数回。解析すると **$6M$** 回くらいになる。)



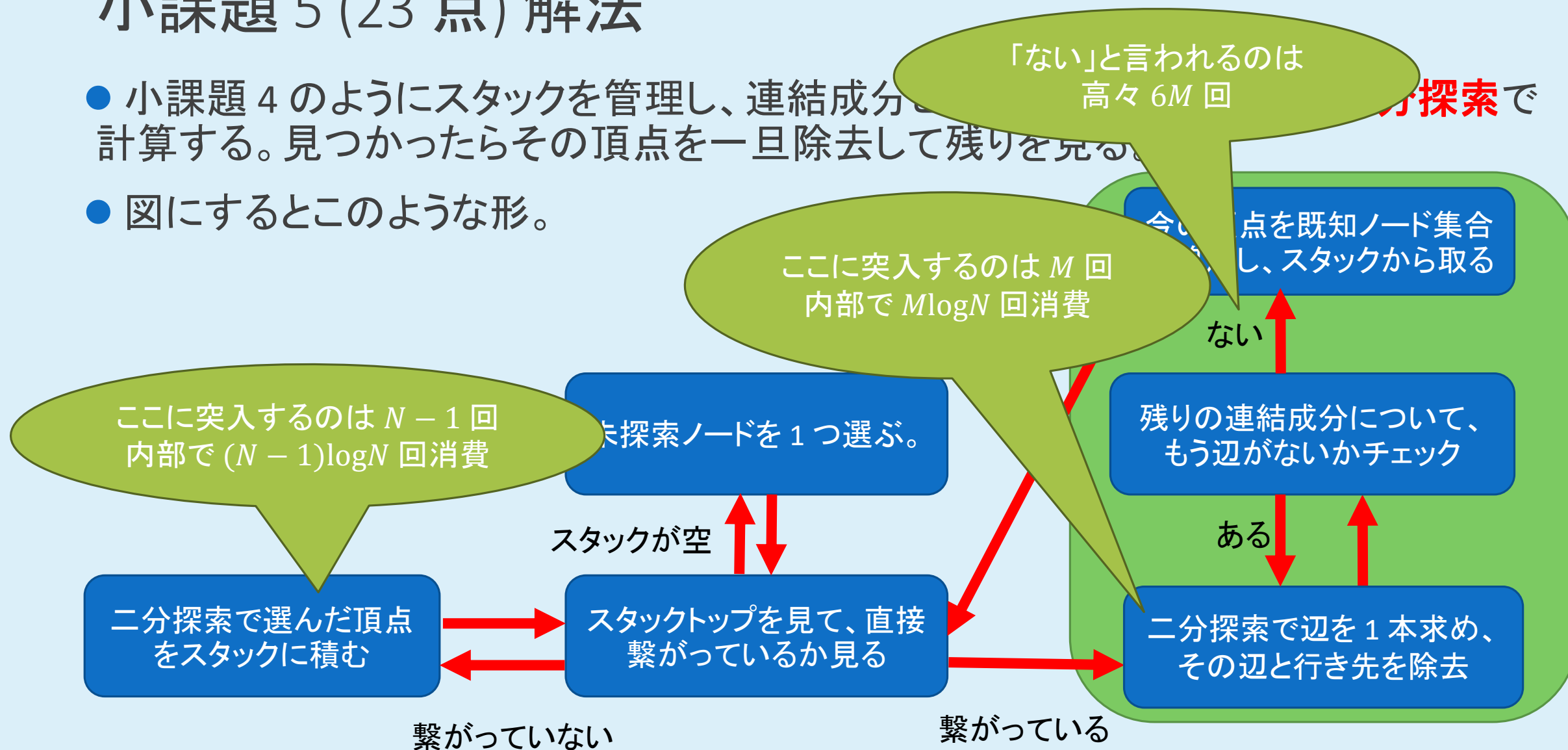
小課題 5 (23 点) 解法

- 小課題 4 のようにスタックを管理し、連結成分ごとに辺があるか判定し**二分探索**で計算する。見つかったらその頂点を一旦除去して残りを見る。
- 図にするとこのような形。



小課題 5 (23 点) 解法

- 小課題 4 のようにスタックを管理し、連結成分を計算する。見つかったらその頂点を一旦除去して残りを見る。
- 図にするとこのような形。



小

全体で $N(\log N + 1) + M(\log N + 7)$ 回以下。
 $N = 1,400, M = 1,500$ で 43,800 なので満点。

※これは最悪ケースで、現実的にはもっと少なく済む。

「ない」と言われるのは
高々 $6M$ 回

二分探索で

を元

突入するのは M 回
部で $M\log N$ 回消費

ここに突入するのは $N - 1$ 回
内部で $(N - 1)\log N$ 回消費

未探索ノードを 1 つ選ぶ。

スタックが空

二分探索で選んだ頂点
をスタックに積む

スタックトップを見て、直接
繋がっているか見る

繋がっていない

残りの連結成分について、
もう辺がないかチェック

ある

二分探索で辺を 1 本求め、
その辺と行き先を除去

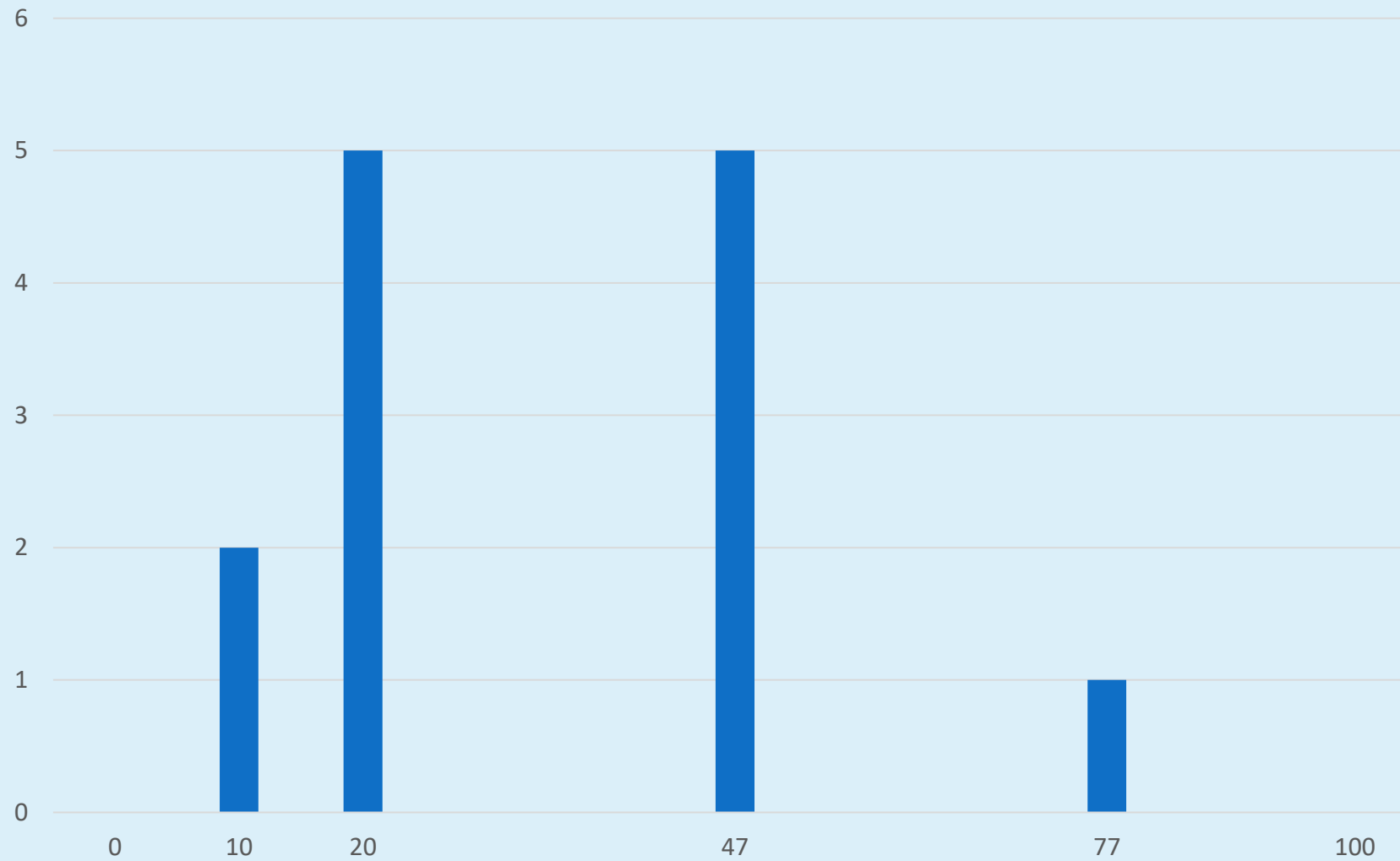
繋がっている

ない

点を既知ノード集合
し、スタックから取る



得点分布



※欠席1名を除く

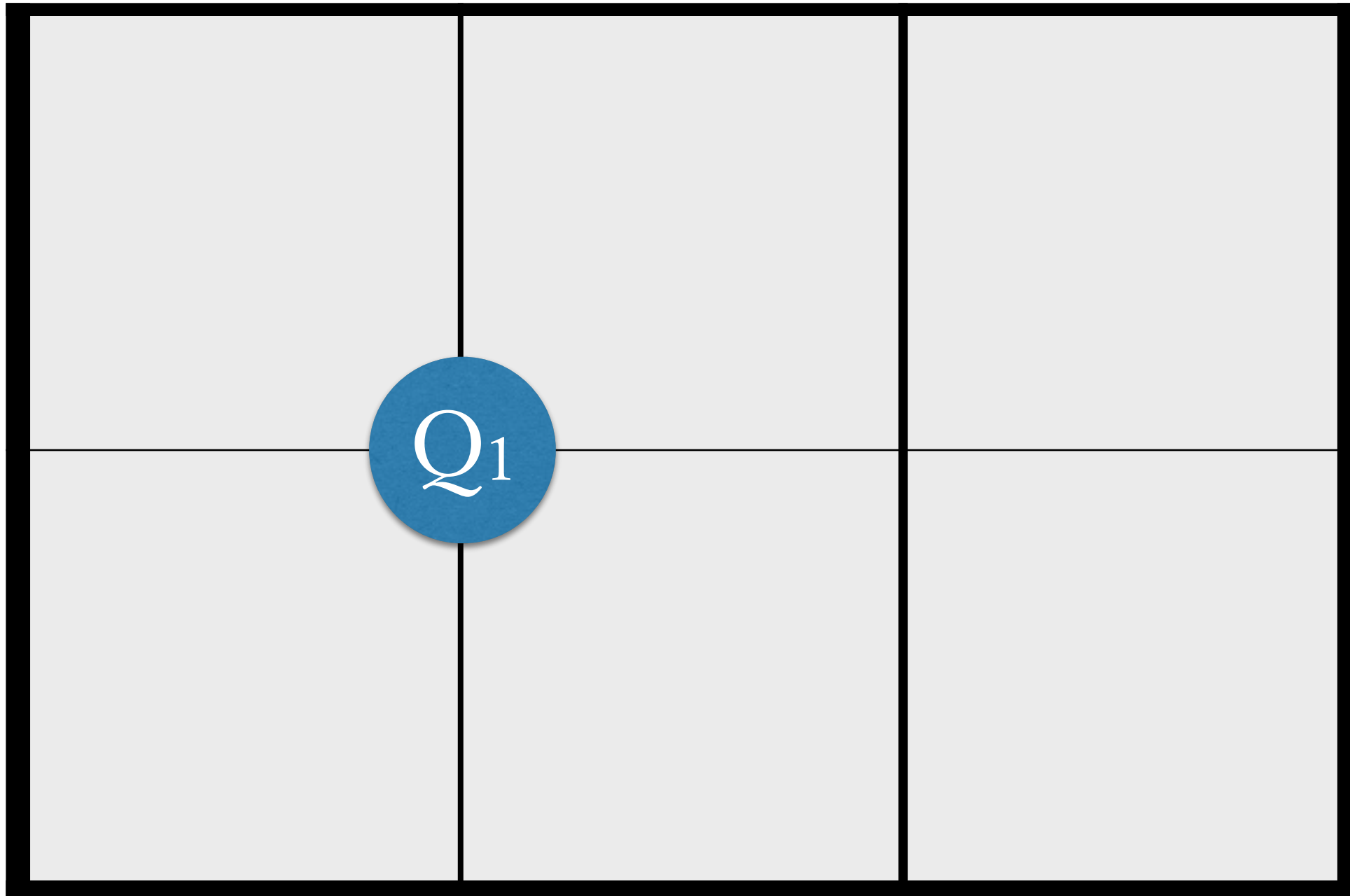
Abduction 2 解説

tozangezan

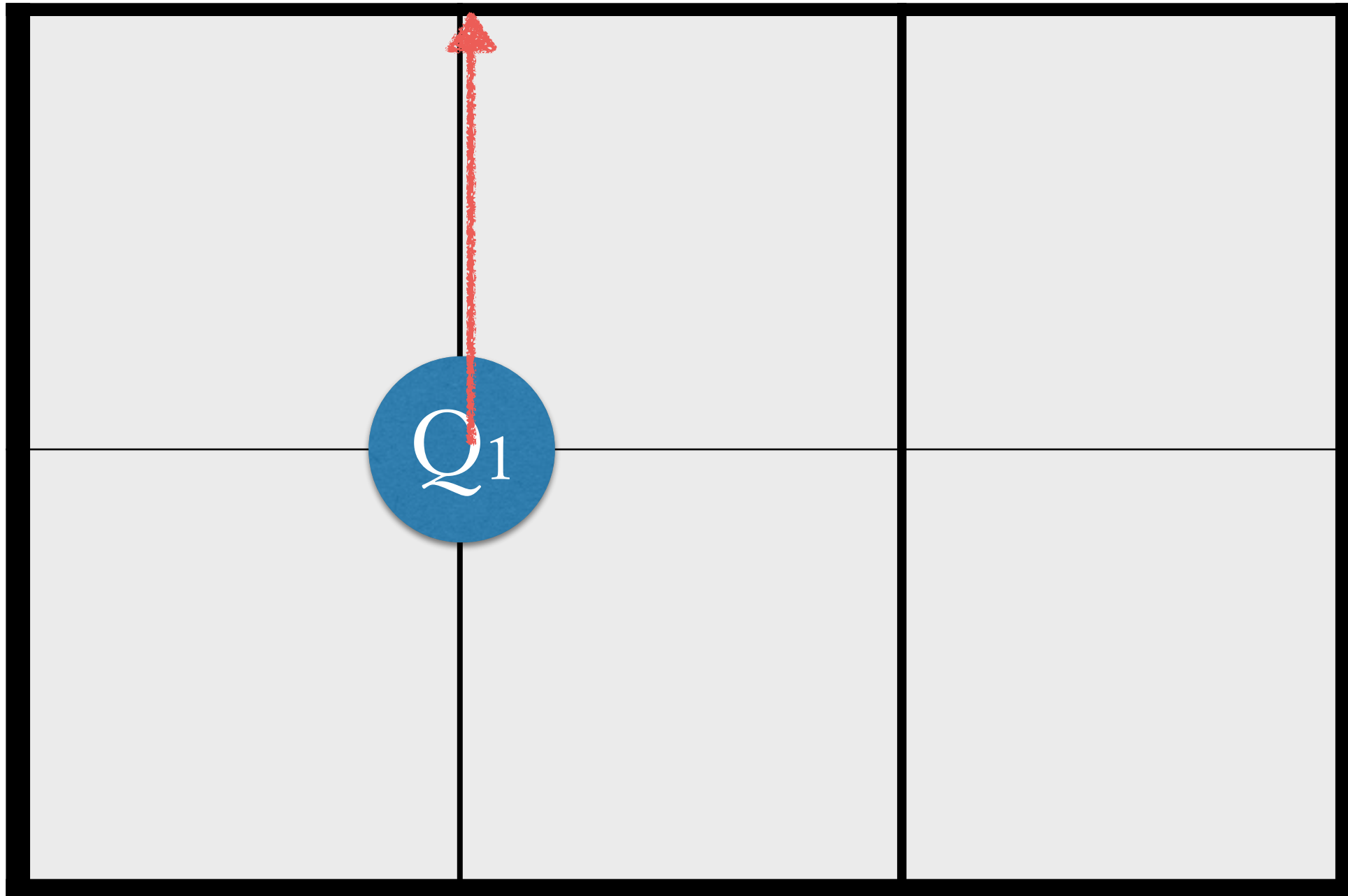
問題概要

- ・ $H * W$ のグリッドの辺上を、ある始点から移動を行う
- ・ 最初はどの方向にでも移動できる
- ・ 交差点に差し掛かった時、より人通りの多い道を選んで(曲がる/直進)する
- ・ Q 個の始点が与えられた時、移動可能な最長経路の長さを求めよ

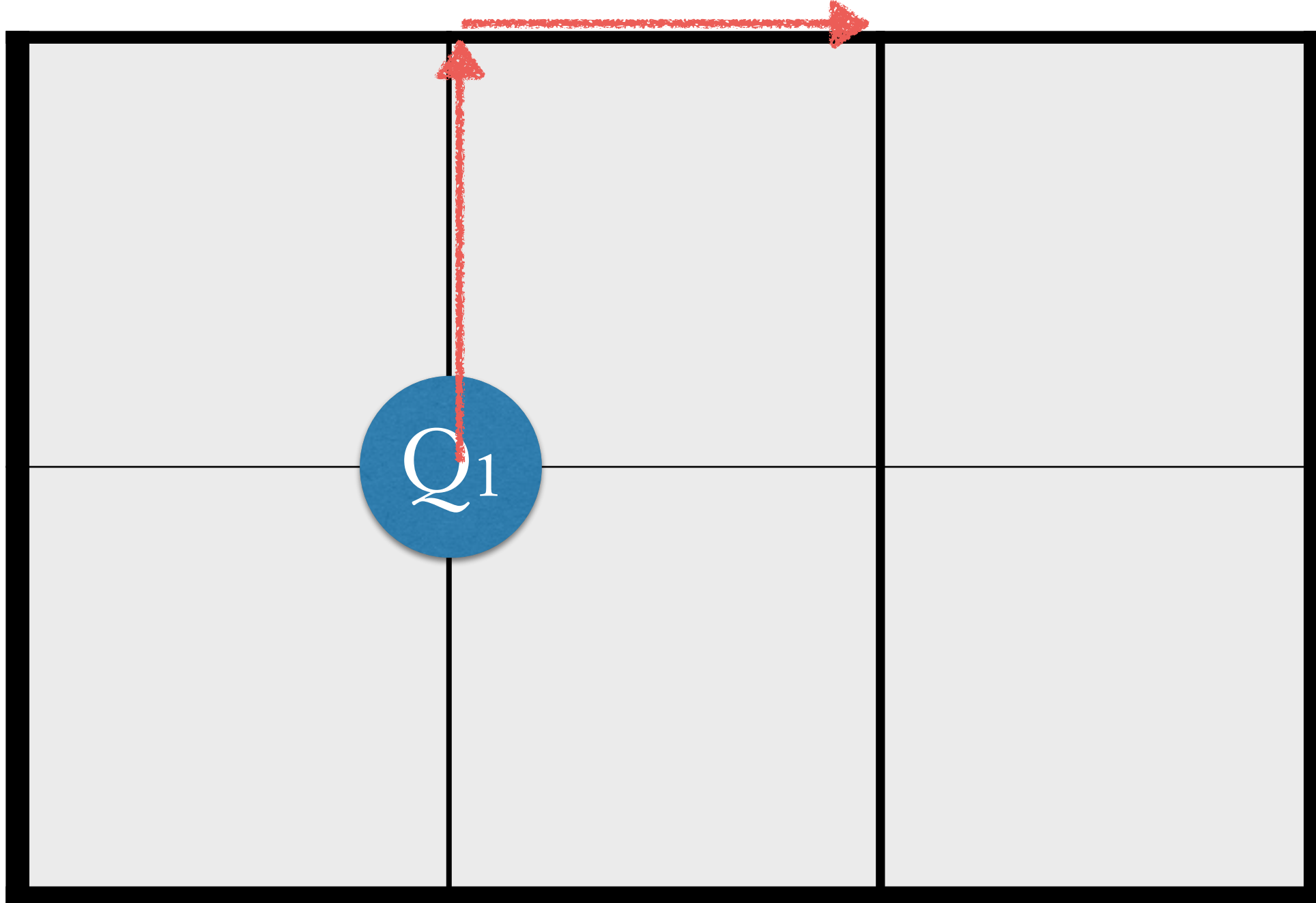
例



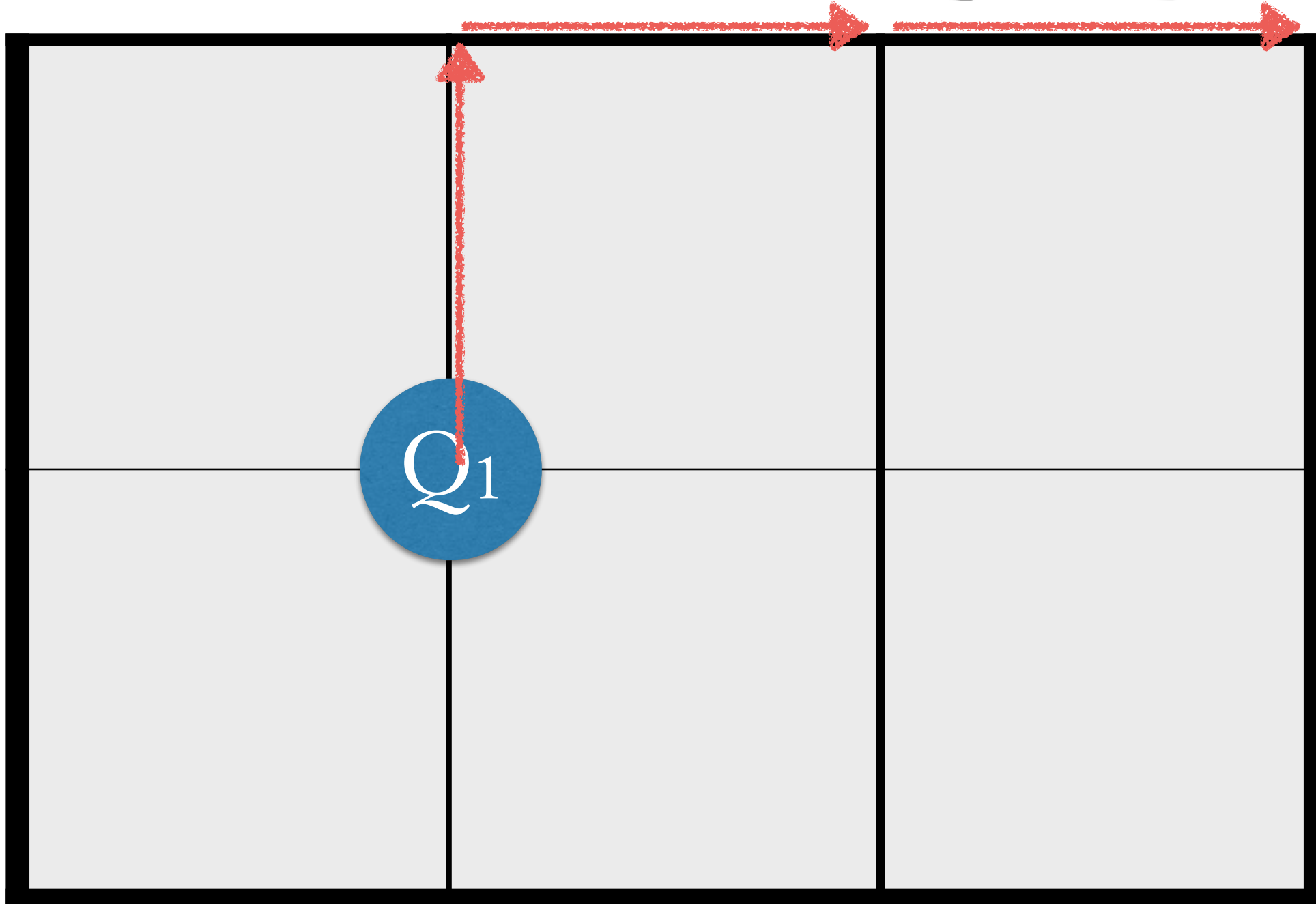
例



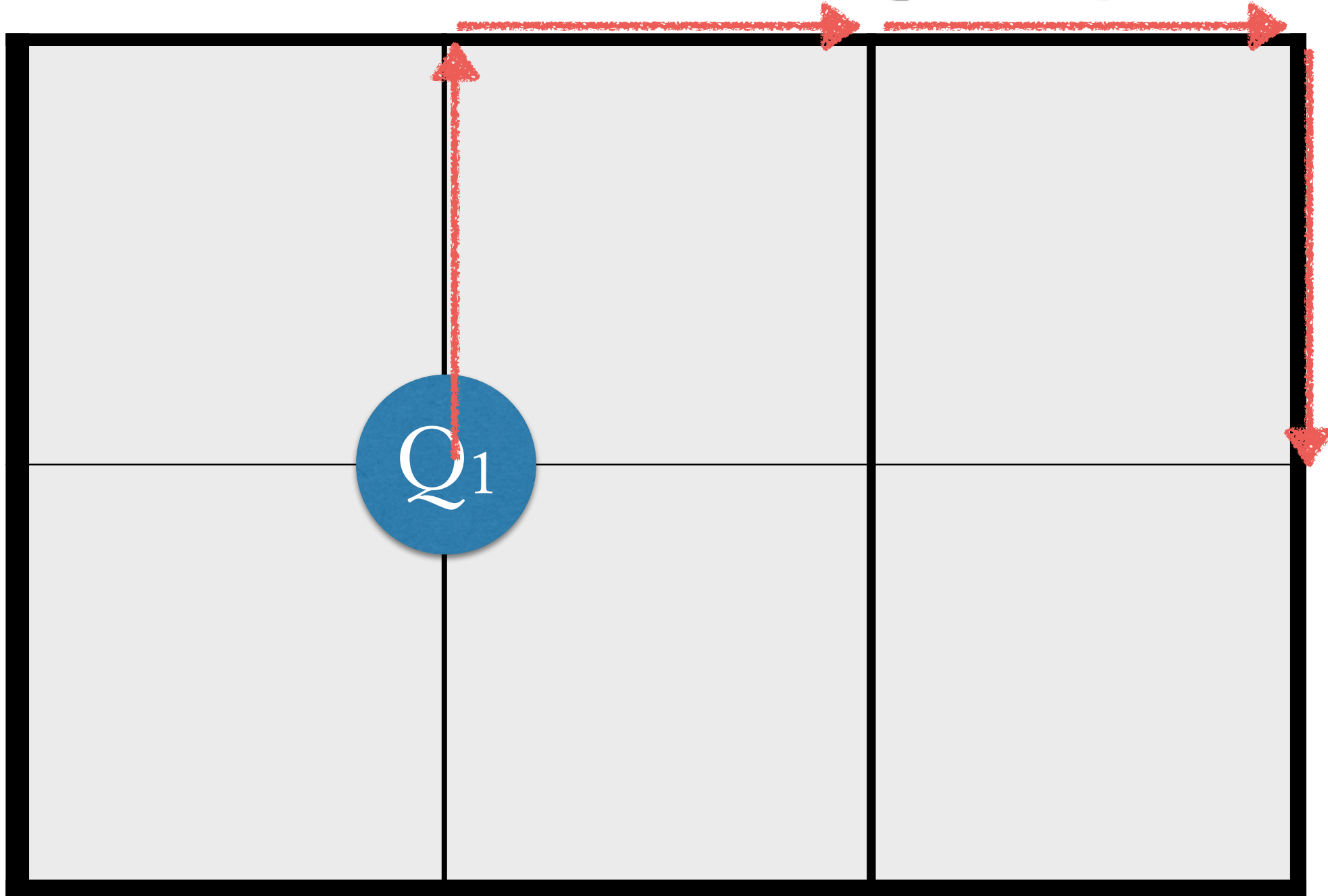
例



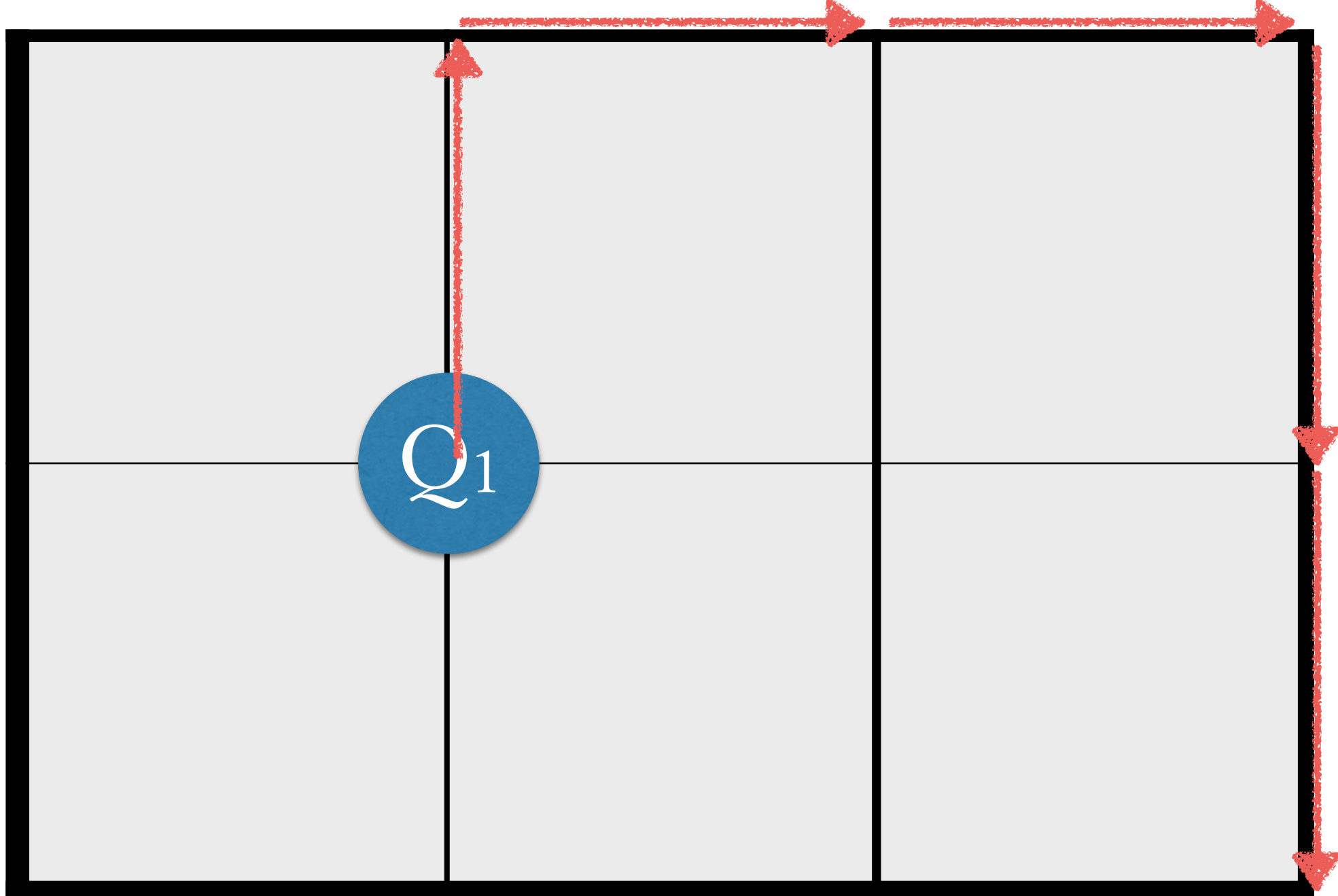
例



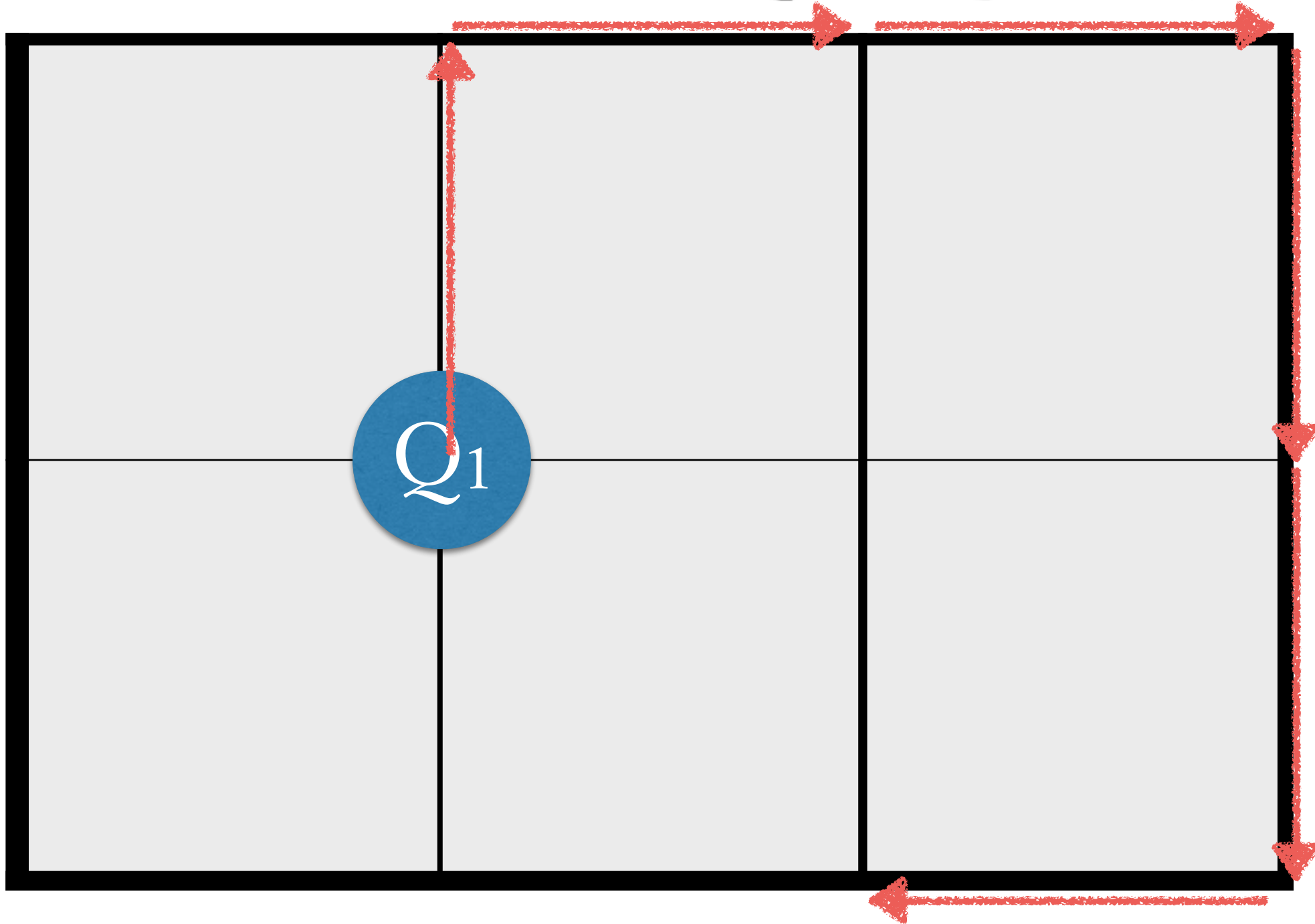
例



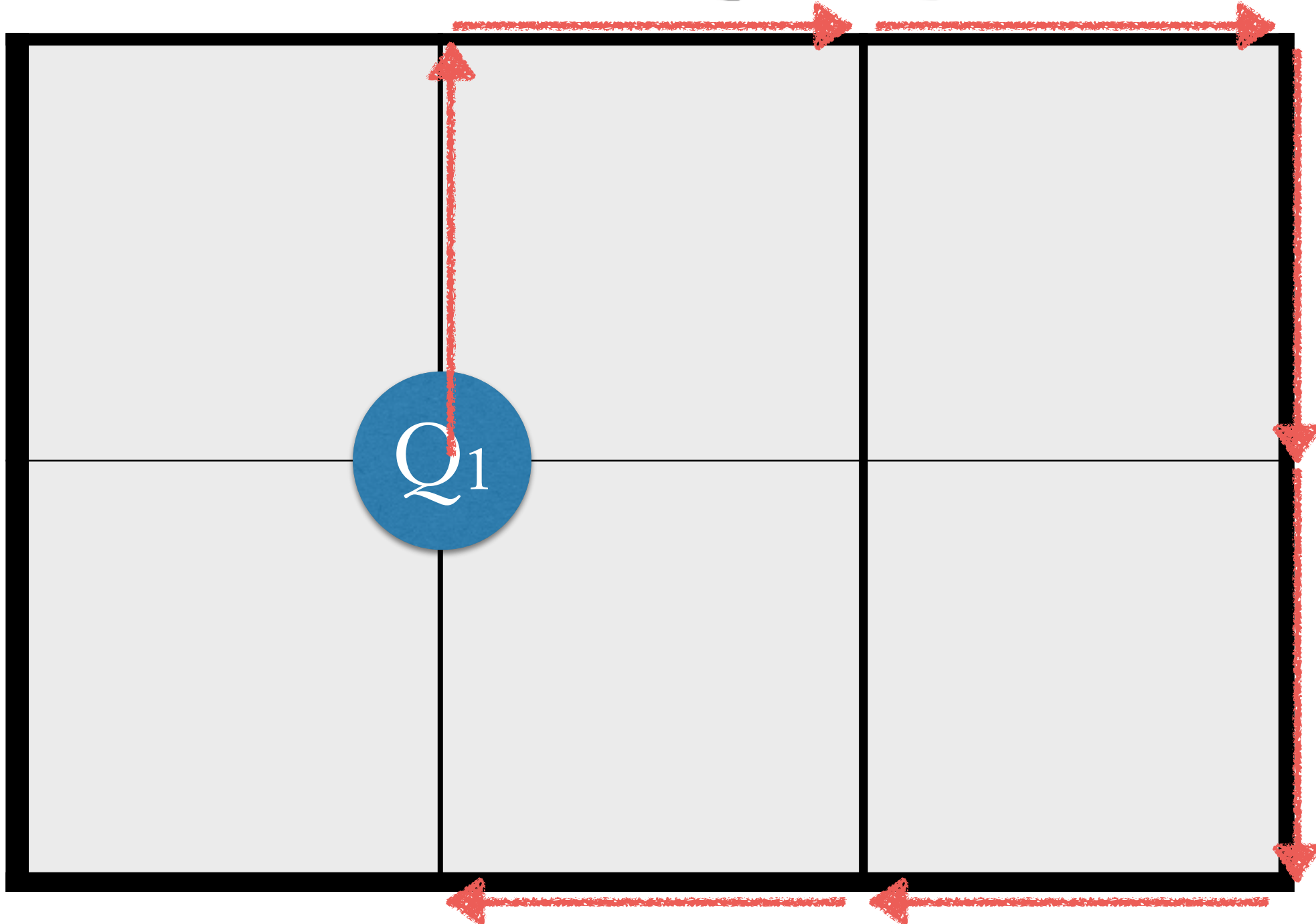
例



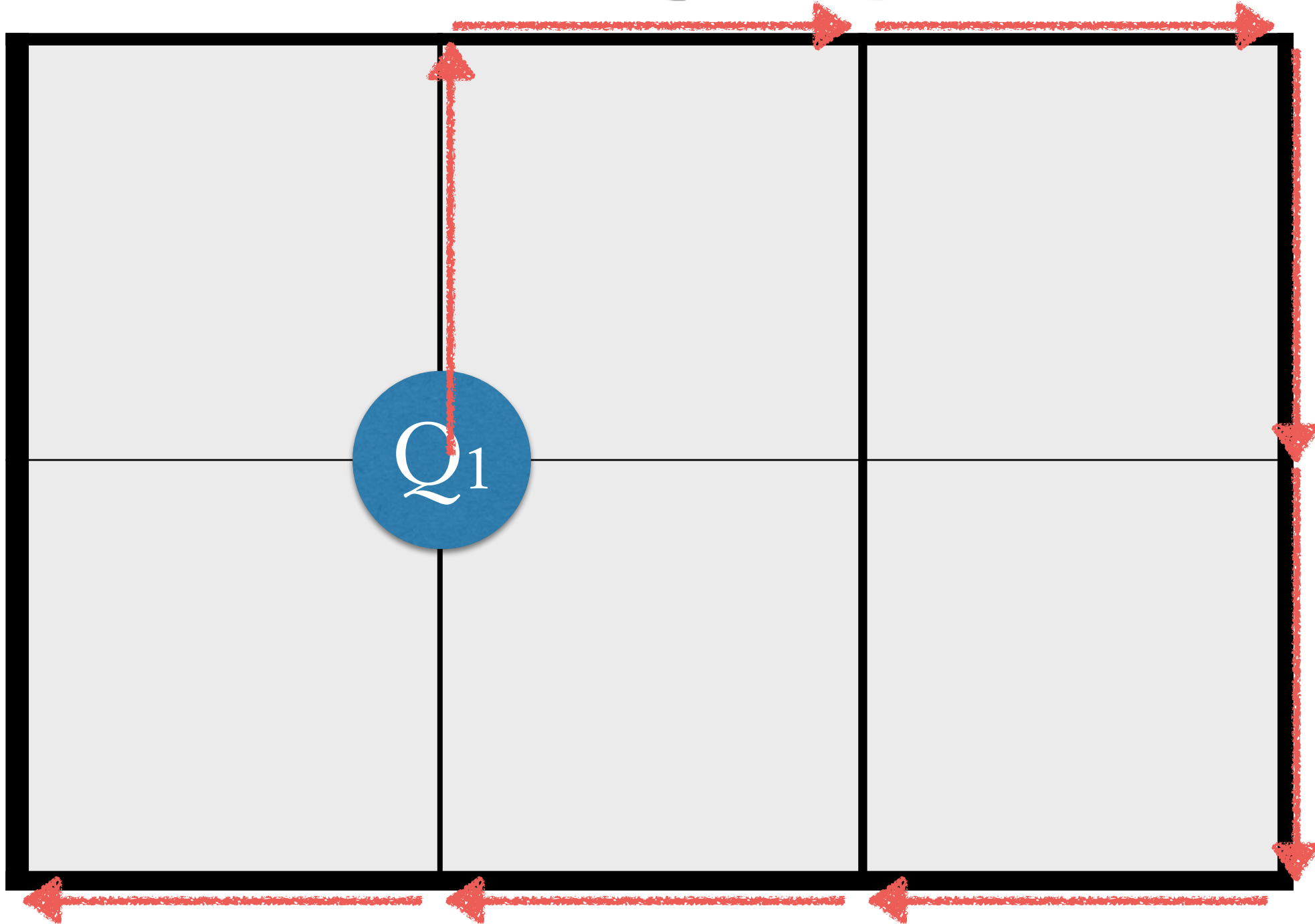
例



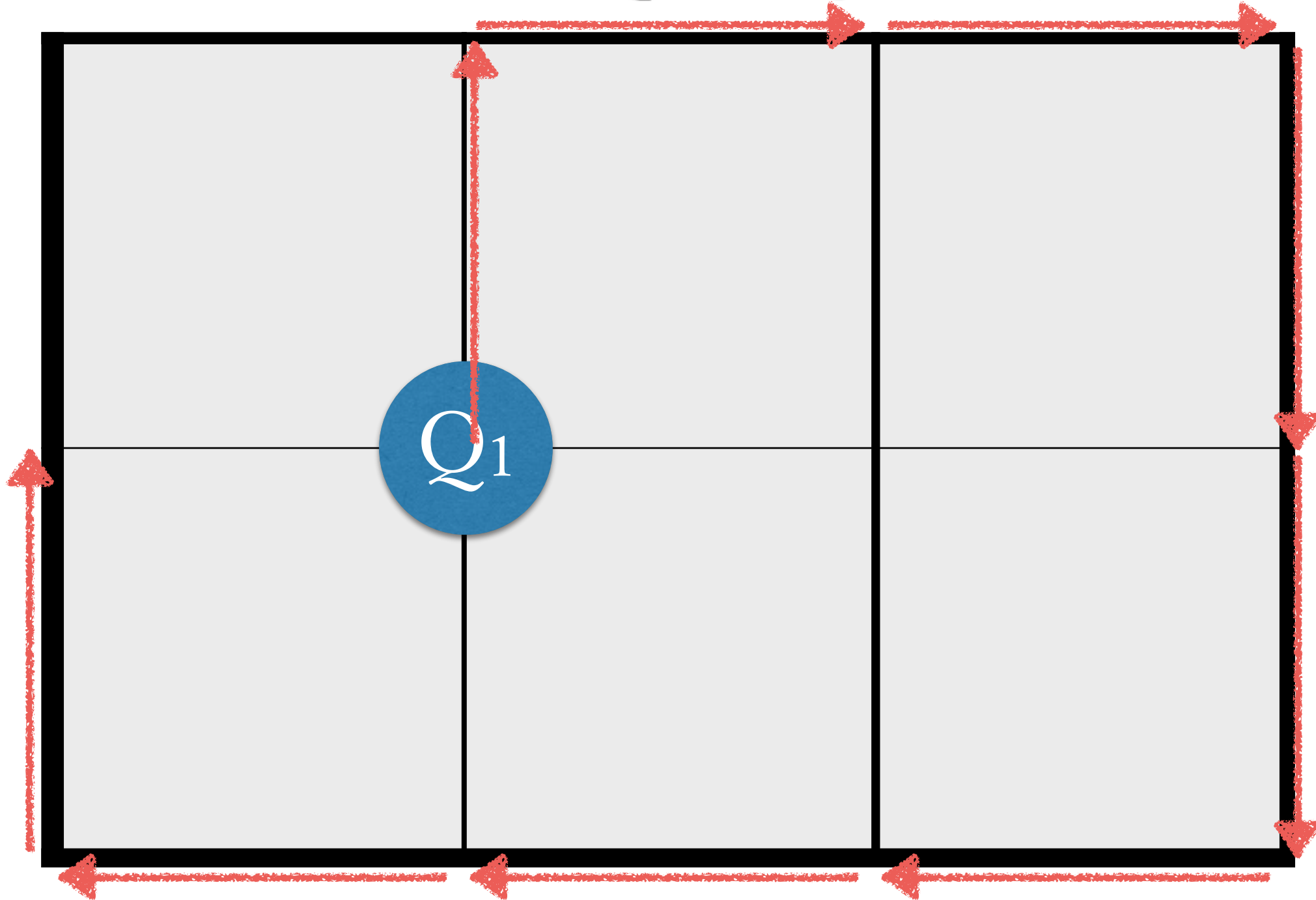
例



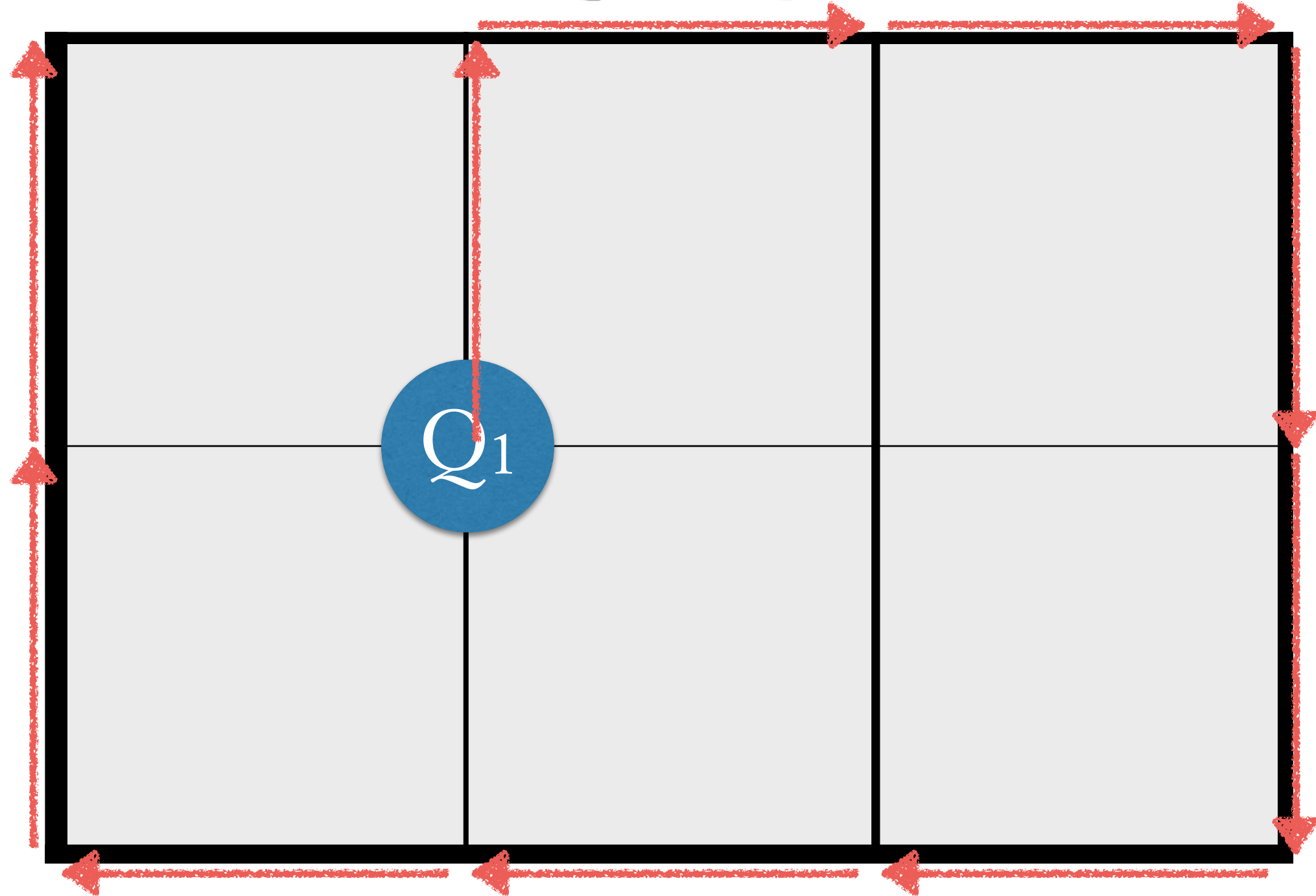
例



例

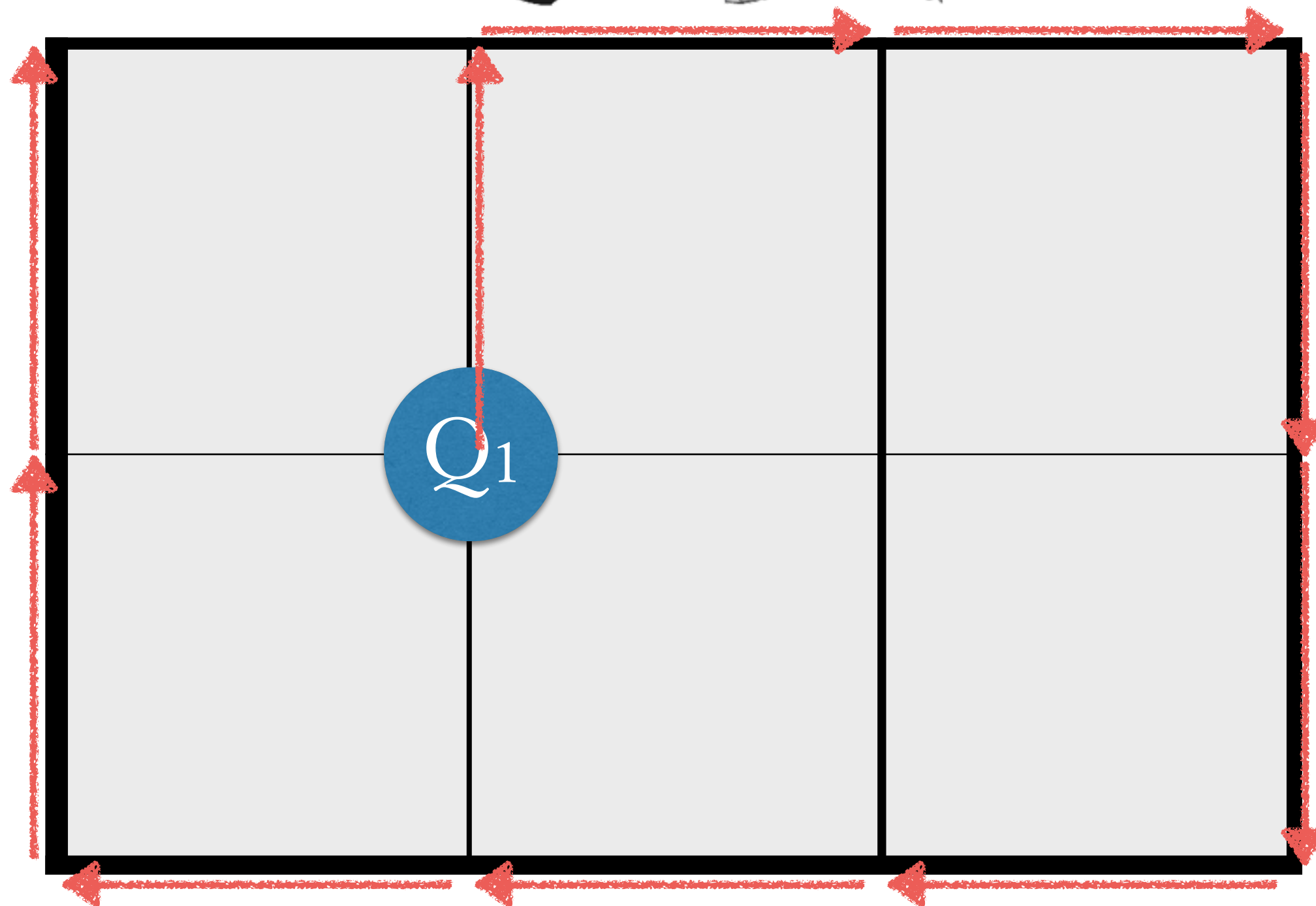


例



答え: 10

例



解法 1

- ・ DFSする
- ・ $\text{solve}(i,j,\text{方向})$: 交差点 (i,j) からある方向を向いて出
発したときの最大値
- ・ $\text{solve}(i,j,\text{方向}) = \max(\text{solve}(i',j',\text{方向}'_1), \text{solve}(i',j',\text{方向}'_2)) + 1$ 的な感じ
- ・ 行き止まりでは $\text{return } 0$;

何点取れるでしょうか？

13点

計算量？

- ・ 13点: $H, W \leq 8, Q = 1$
- ・ 分岐する回数は多めに見積もっても 16 回くらい
- ・ いつでも 2 通りに分岐できたとしても 216 通りくらい
- ・ 全通りを試しても余裕で間に合う

解法 2

- ・ 解法 1 をメモ化する
- ・ Q 回クエリがあるが、メモ化した値はクエリによって変わらないので、DP テーブルをクエリごとに初期化する必要はない

何点取れるでしょうか？

27点

計算量？

- ・ 27点: $H, W \leq 2,000$
- ・ 状態数が $O(HW)$ の DP となる
- ・ 遷移は $O(1)$ (隣の交差点に移動するだけ)
- ・ 計算量は $O(HW)$ なので間に合う

解法 3

- ・ 直進するだけの交差点をスキップ
- ・ $\text{dfs}(i,j,\text{方向})$ からは、その方向に進んで最初に曲がる場所を探し、曲がった先に対し $\text{dfs}(i',j',\text{方向}')$ を呼ぶ
- ・ どうやってスキップしよう？

Sparse Table

- ・ $a[i][j]$: $[i, i+2^j)$ における $A[i]$ の最大値
- ・ ↑の表は、ダブリングのようにして計算できる
- ・ ある点からどこまでいけるかも、これを j から大きい順に回していけば移動先が見つかる
- ・ segtree上で二分探索するよりも楽
- ・ 計算量は前処理 $O(N \log N)$ 、クエリ $O(\log N)$

何点取れるでしょうか？

何点取れるでしょうか？

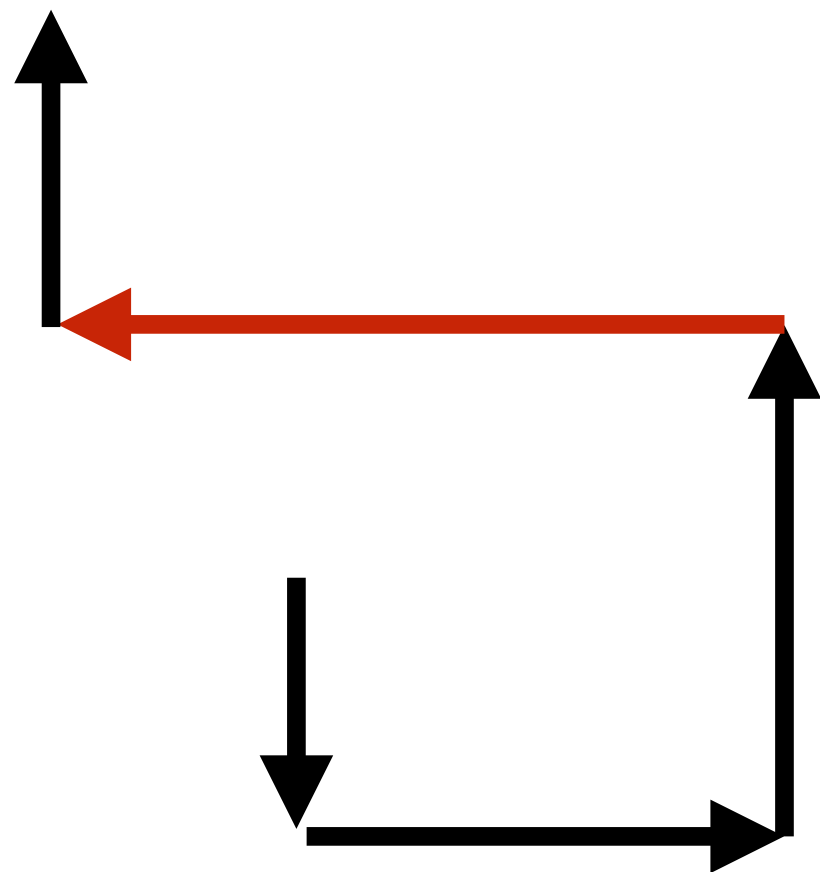
100点

計算量？

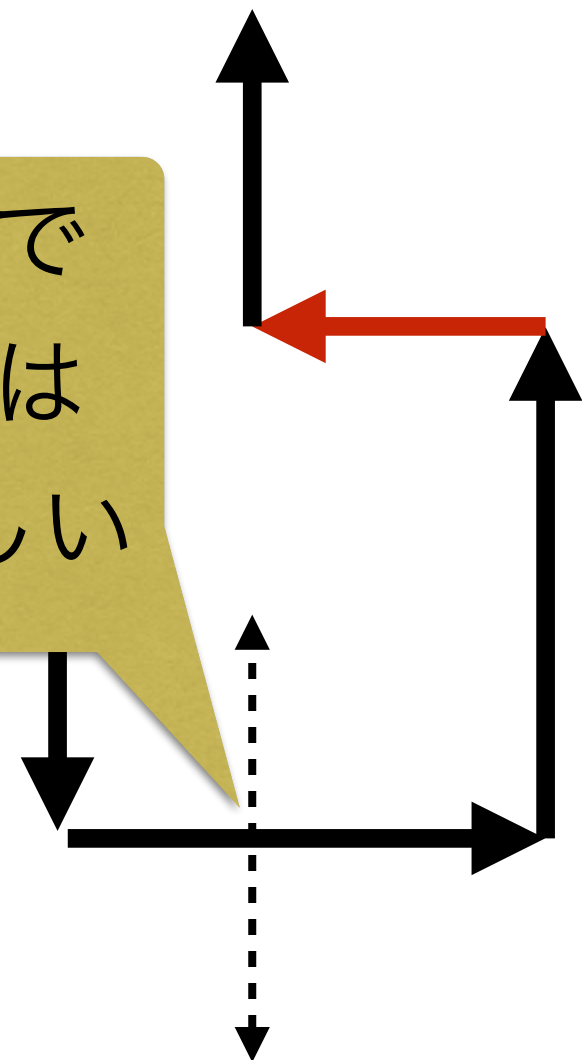
- ・ 計算量解析は自明ではない
- ・ とりあえず状態数はいくつだろう？

動き方

- ・ 右のような動き方はない



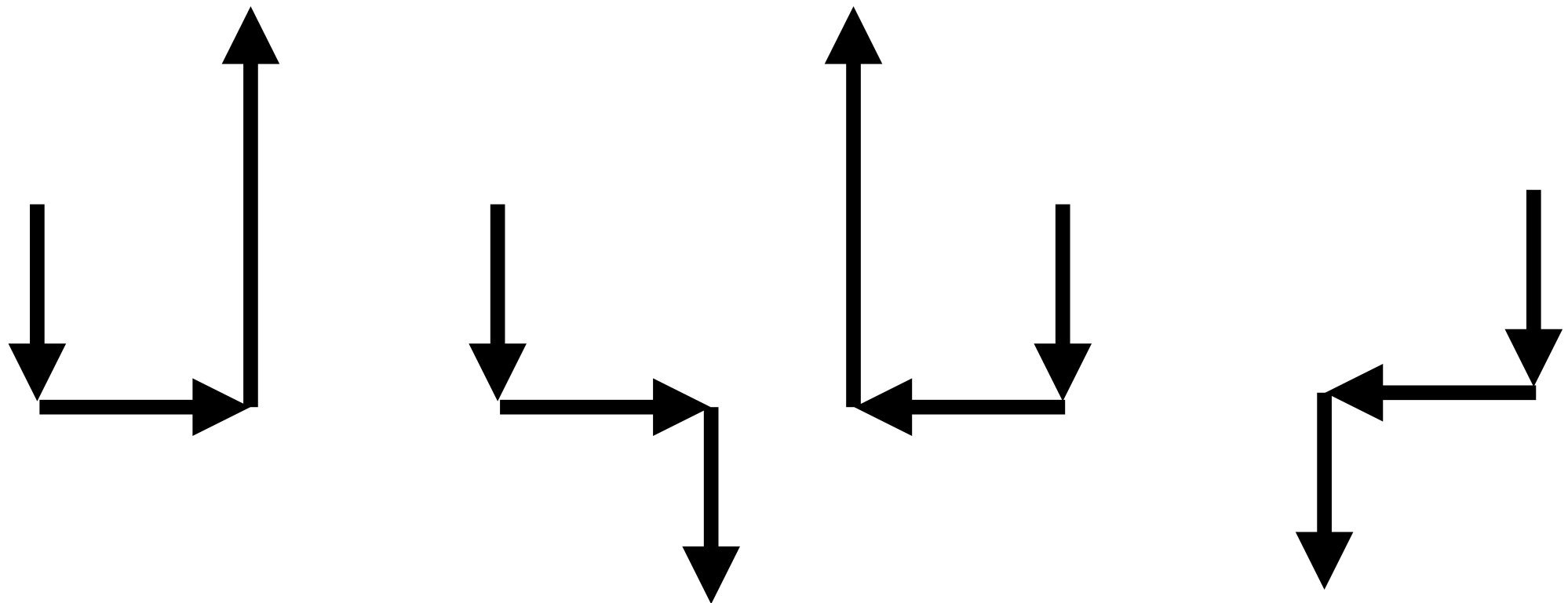
ここで
直進は
おかしい



状態数の解析

- ・ DFS で計算していく時、何回 DFS 回数と呼んだ内部での状態かを、「深さ」と言うことにする
- ・ 深さ i までの状態数は、いくらになるだろう？

例: 深さ 3

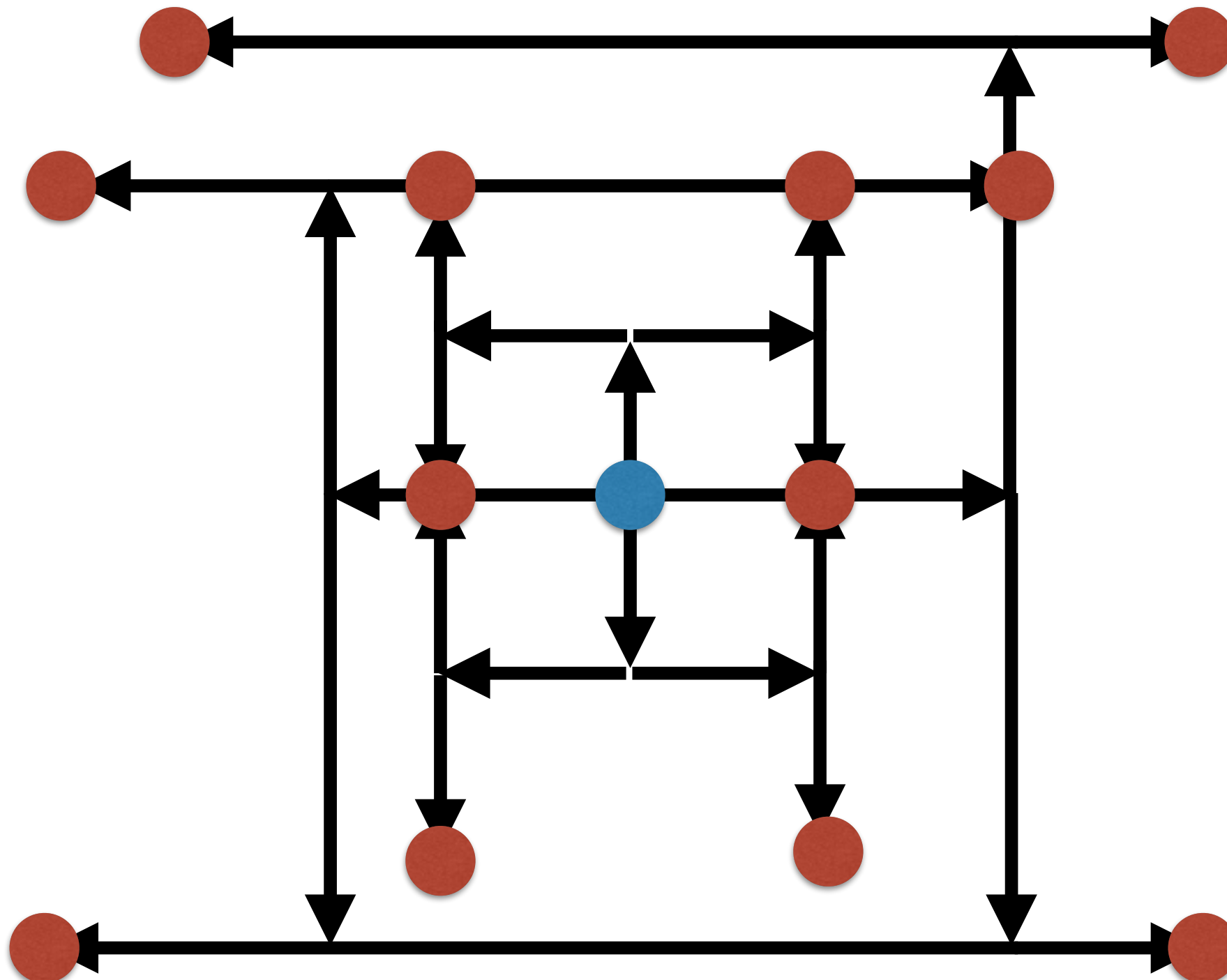


(向きを変えると 4 倍)

例: 深さ 3

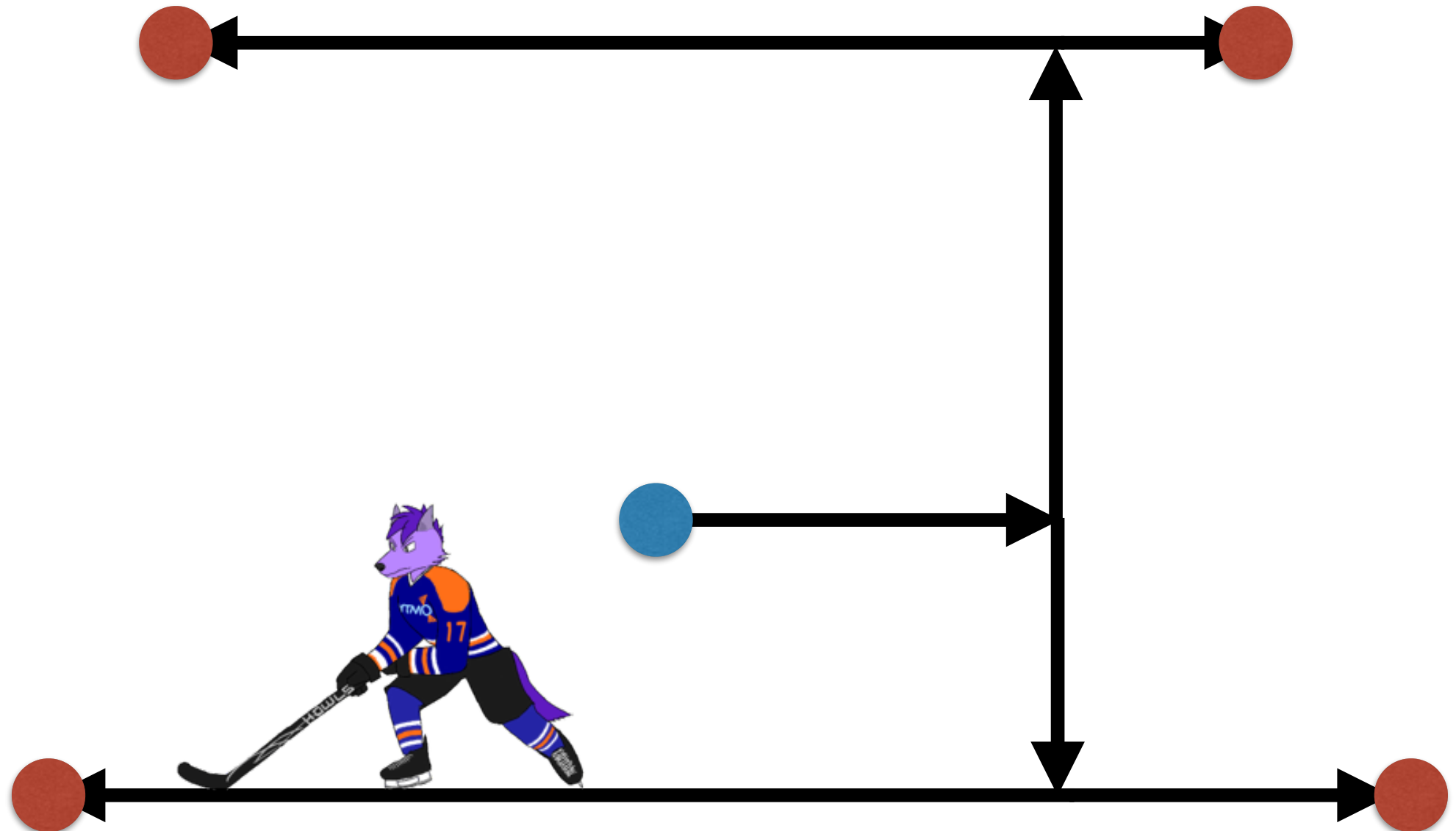
- ・ しかし、 前述のようにメモ化して計算しているの
で、途中の経路はどうでもよくて、最後何通りの場
所に到達できるかが問題
- ・ 行き先は全部で何通り？

例: 深さ 3

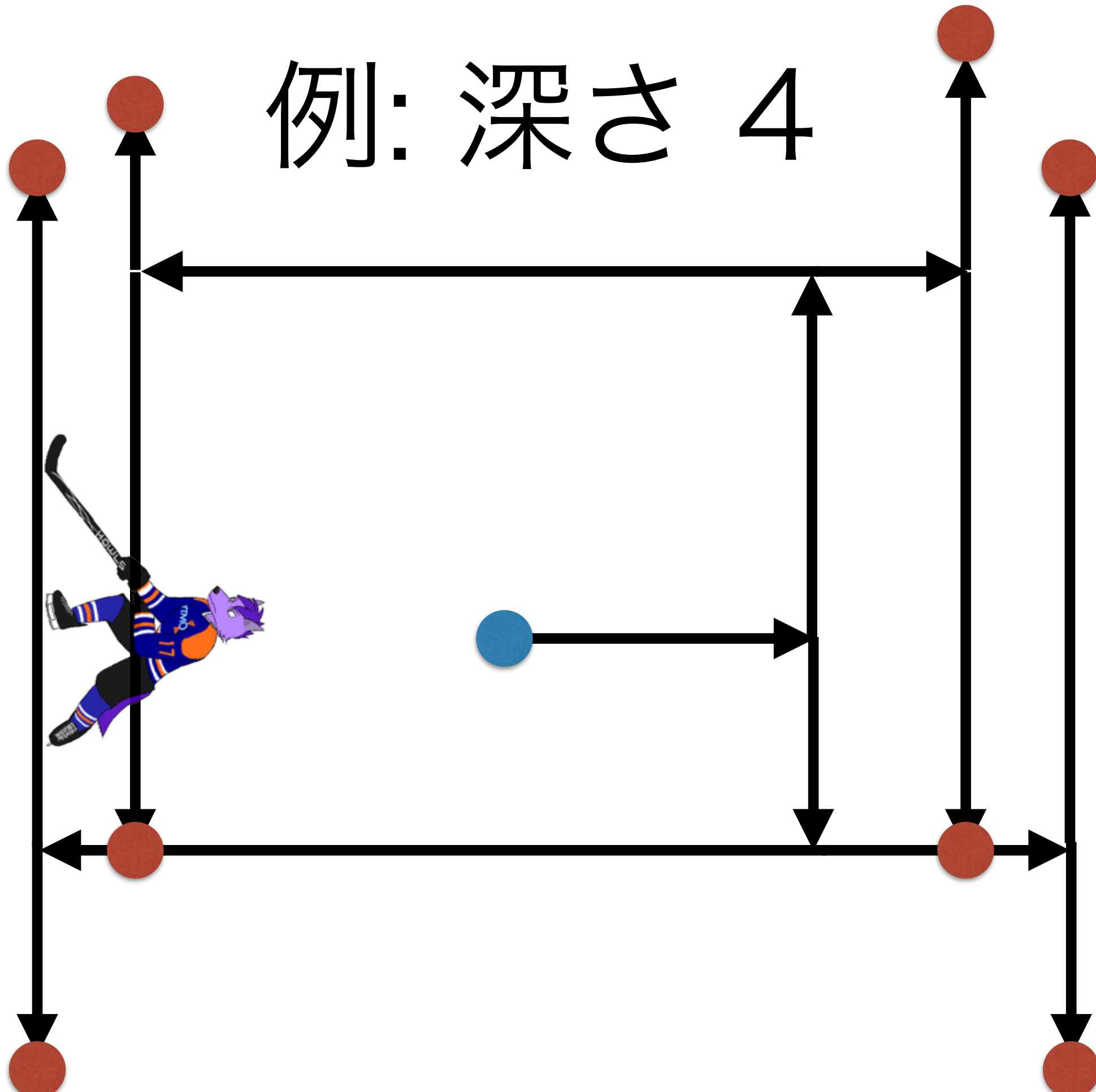


わけがわからないので

例: 深さ 3

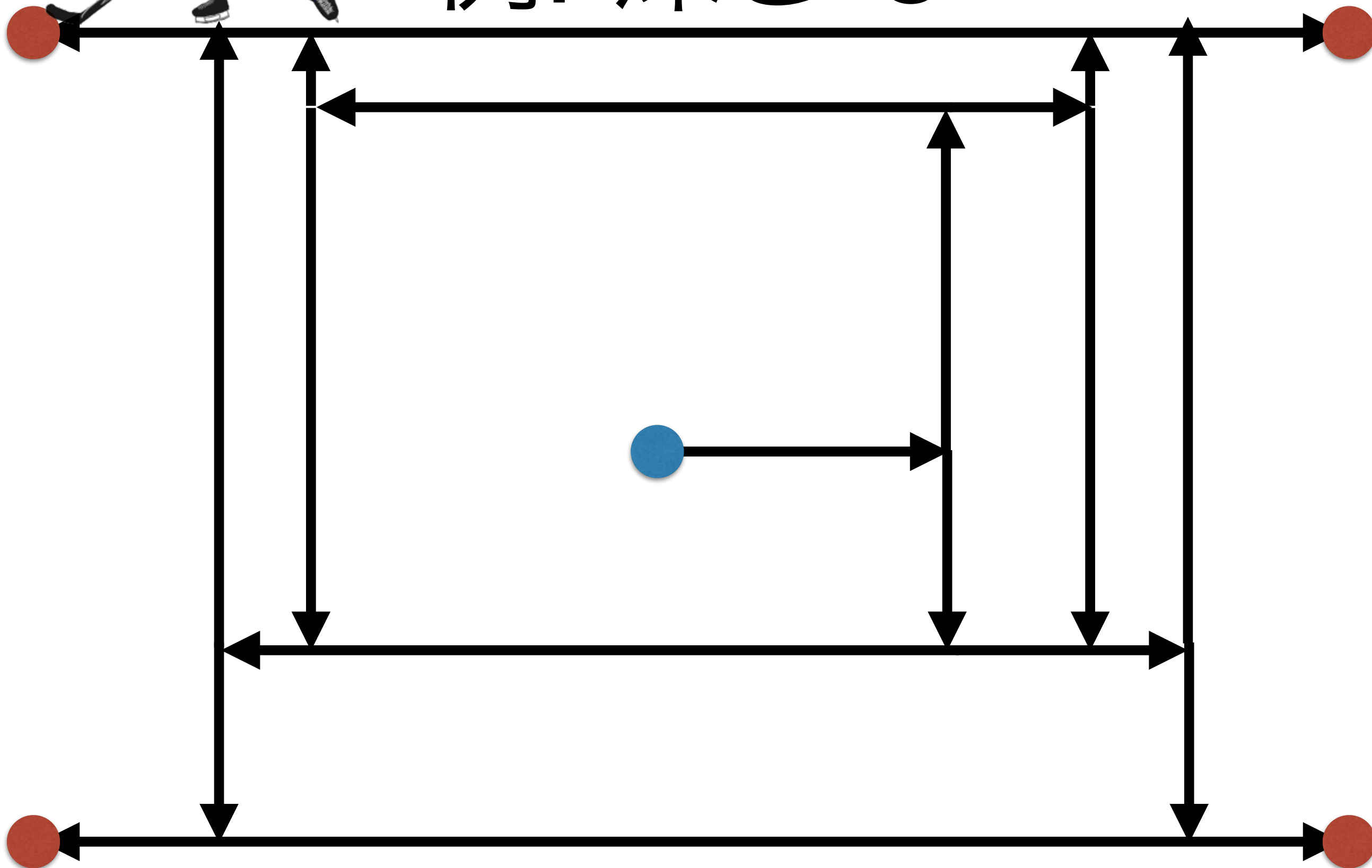


例: 深さ 4





例: 深さ 5



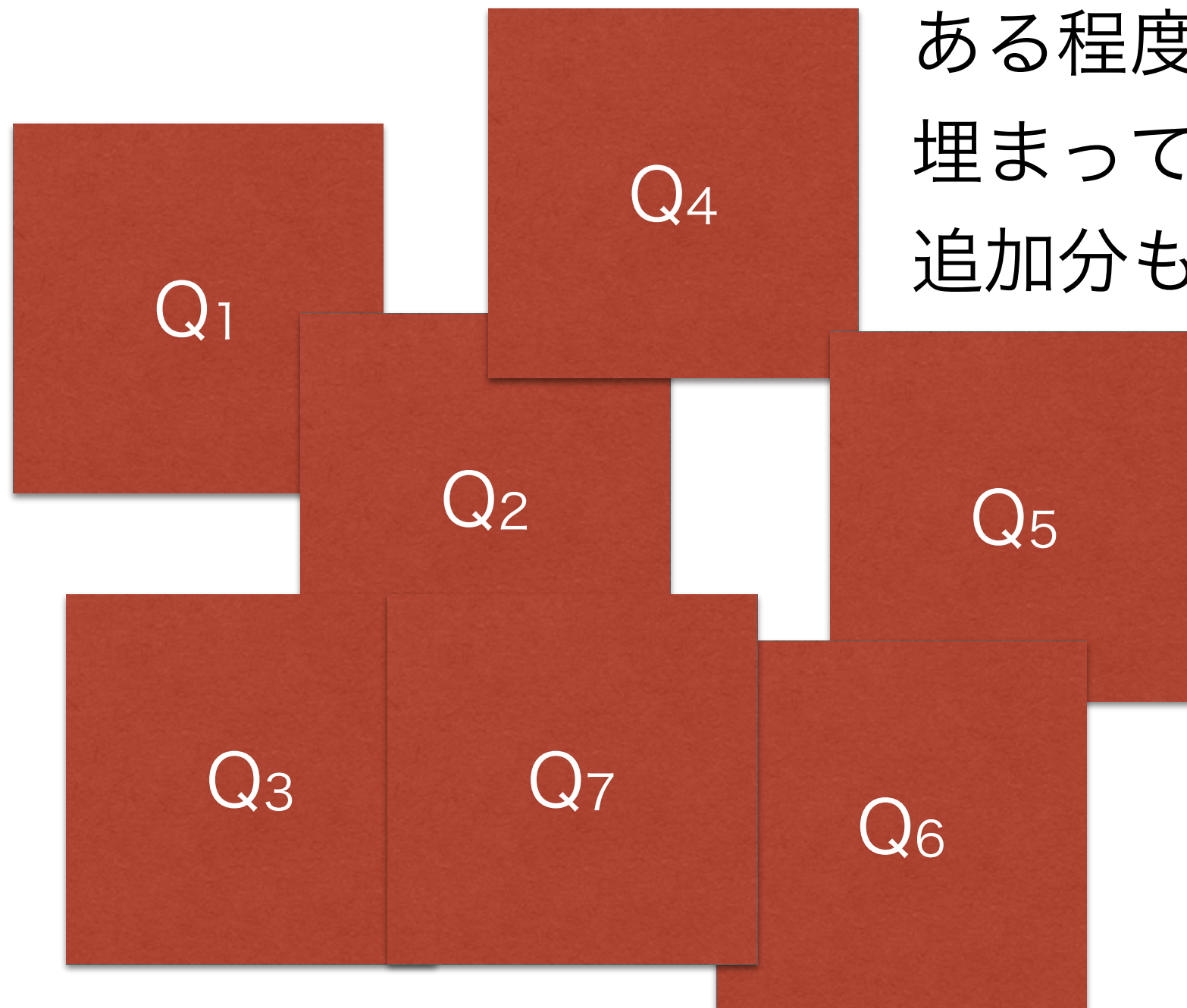
手で試して見ると？

- ・ 一見●は増えたりするように見えるが、結局 DP の中に埋もれて回収される
- ・ 全体的に見ると、ある点から移動し始めたときの、深さ $O(N)$ まで進めても状態数は $O(N)$

単純計算

- ・ クエリ(始点)は全部で Q 個
- ・ 深さは $O(N)$ ($=O(H+W)$) くらい
- ・ 全部で状態数は $O(NQ)$?

さらに拡大すると？



ある程度DPテーブルが
埋まっているので
追加分も少なくなる

後半の解析方法を変えよう

- ・ 途中からは見ていない面積がどう減っていくかで解析しよう
- ・ 深さ $N/\sqrt{Q}$ まで進めた後は、1ステップで面積 $N/\sqrt{Q}$ 以上減る ($1 \times N/\sqrt{Q}$ の長方形領域)
- ・ 残った領域が分断されすぎていないことにも注意

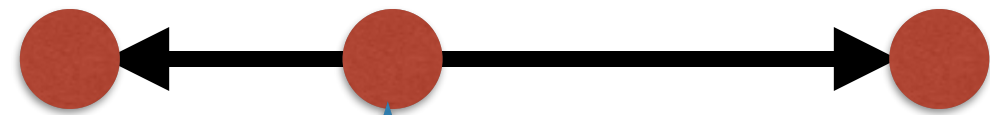
後半の状態数

- ・ 面積は全部で N^2

- ・ 1ステップで $N/\sqrt{Q}$ 減る

- ・ 必要なステップ数は $O(N\sqrt{Q})$

- ・ 同じ領域を変なところから何回も見るのは？
→ どうせ次のステップで埋もれる



どうせ次 return するだけ
オーダーに影響しない

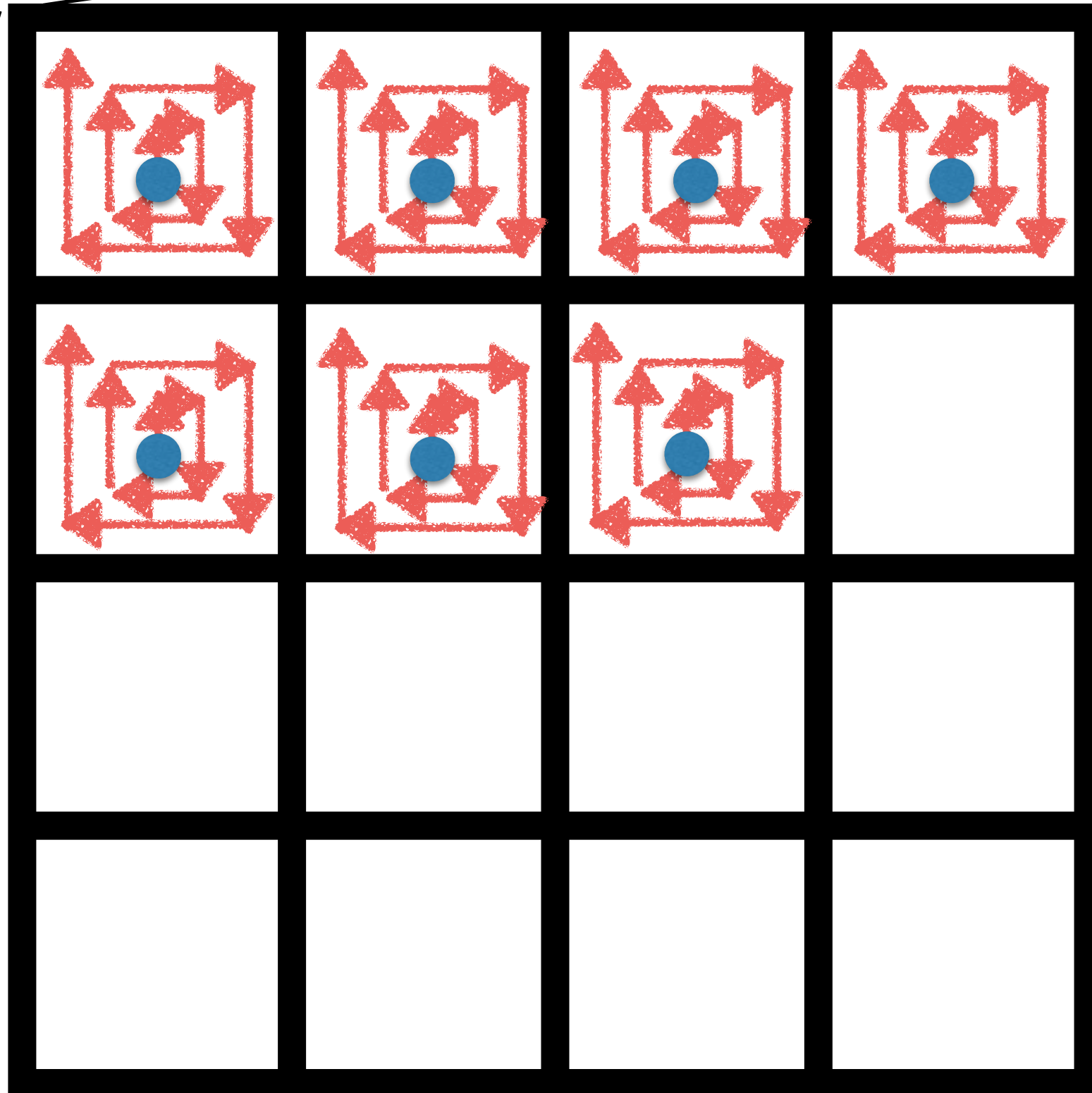
計算量まとめ

- ・ 状態数
 - ・ 前半: 各 $N/\sqrt{Q}$ ステップ $\times Q$ クエリ
→ $O(N\sqrt{Q})$
 - ・ 後半: 合計 $N^2 / (N/\sqrt{Q})$ すなわち $O(N\sqrt{Q})$
- ・ それぞれの状態
 - ・ メモ化(mapとか), Sparse Table: $O(\log N)$

状態数 $O(N\sqrt{Q})$ にかかるケース

$\sqrt{Q}$ 列

$\sqrt{Q}$ 行



余談

- ・ Sparse Tableの代わりにSegtreeを使う
 - 工夫して $O(\log N)$ → 大丈夫そう
 - 工夫せず $O(\log^2 N)$ → 怪しい
- ・ Sparse Tableの代わりにループを回す
 - 曲がる場所に到達したらループを抜ける
 - こうするとかなり軽い (合計 $O(N^2)$ 激軽)
→ 実はこれは大丈夫 (?)

得点分布

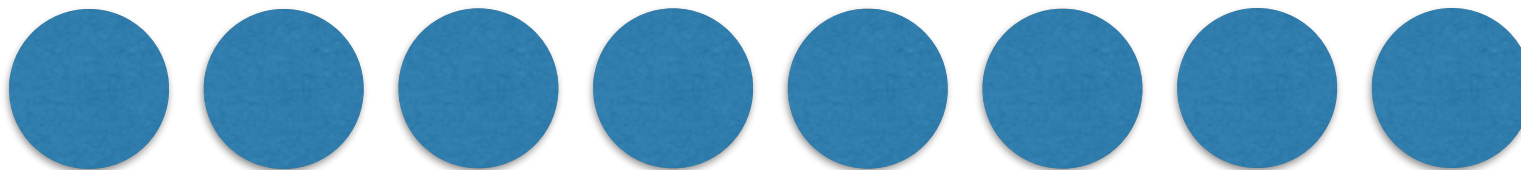
100



44



27



13



都市(City) 解説

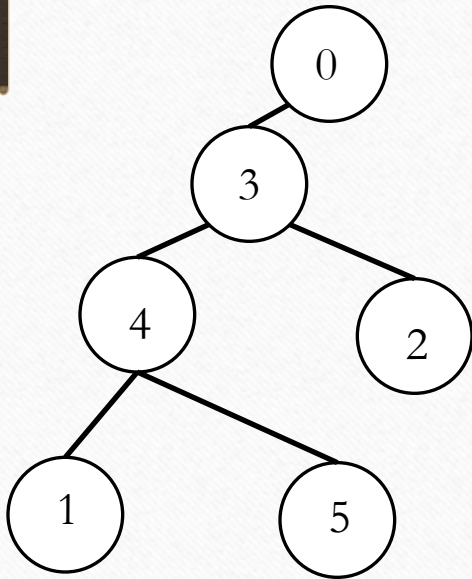
松崎照央

問題概要

- 頂点0が根の根付き木がもらえる(大きい)
- 木のある2つのノードの組について「その組は祖先-子孫か子孫-祖先かどちらでもないか」を調べる。
- **木の深さは18まで** (30点までは使いません)
- 頂点を符号化し、クエリの対象となる頂点の符号のみからクエリに答える。

例

- 各ノードを「自分の番号,頂点0が子孫かどうか,頂点1が子孫かどうか,頂点2が子孫かどうか,...」というように符号化してみる($N \leq 10$)



0: 0000 011111
1: 0001 000000
2: 0010 000000
3: 0011 011011
4: 0100 010001
5: 0101 000000

A 0101 000000
B 0011 011011
こたえ:0

本番で解く前に

- 符号化して復元するタイプの問題でした
- こういった形式の問題はなれてないとつらいかも・・・
- 過去問の似た形式の問題をやっておこう！

小課題1 8点

- $N \leq 10$, 符号の大きさ $\leq 2^{60} - 1$ つまり60ビット使える。
- 頂点が少ないし、送れる情報もいっぱい
- なんでもできるわけではないが、手法は様々でやさしい。

小課題1 実装例

- 自分の番号(4bit)と辺を隣接行列(45bit)で送る(ありうる辺の種類は $10C2$ 通りなので)
- 自分の番号(4bit)と、各頂点が祖先かどうか(10bit)、子孫かどうか(10bit)をすべて保持する
- 祖先か子孫片方だけでもOK
- 祖先頂点の番号全列挙(36bit)するのもOK

ここまで解けた人

- 8点 5人
- 8点超え 7人

小課題2

- $N \leq 250000$ (大変そう)
- 送る整数の最大値 L で得点が決まる。
- $L \geq 2^{38}$ で0点(かなしい)
- $L \leq 2^{28} - 1$ で満点(うれしい)
- 点数は L の大きさに徐々にな変わっていく。

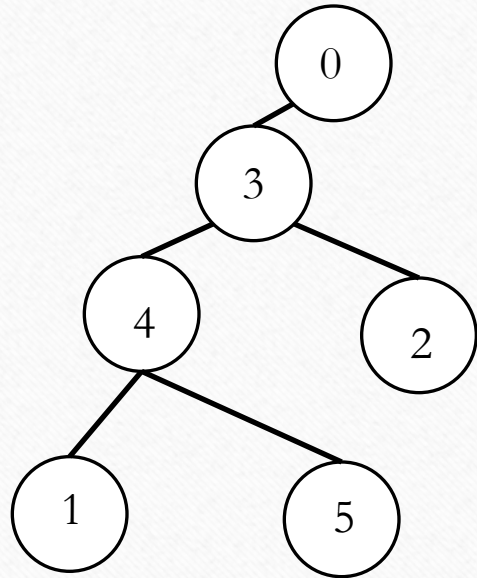
小課題2 (14点) or (10点)

- $L \leq 2^{36} - 1$ つまり36bit使える。
- 頂点も25万あるので、各頂点に対して何か持ったり全列挙などの解法は厳しそう
- 番号を1つ表すのに18bit必要なので、番号はたかだか2つだけしか表すことができない。(2つなら番号を持てる)

小課題2 Euler Tour

- Euler Tourで子孫を区間で扱えばよい。(頻出テク)
- Euler Tourとは？

Euler Tour



- 部分木を区間の左端と右端のindex2つで扱えるようになる(詳しくはググって)
 - 0の部分木 (0-5)
 - 1の部分木 (3-3)
 - 2の部分木 (5-5)
 - 3の部分木 (1-5)
 - 4の部分木 (2-4)
 - 5の部分木 (4-4)
- 本来の番号はもはや不要

0	3	4	1	5	2
---	---	---	---	---	---

小課題2 Euler Tour

- Euler Tourをして、各頂点に対して(部分木の区間の左端、右端)または(部分木の区間の左端、長さ)などを持てばよい
- 普通にやれば14点、帰りがけでも加算をしてしまうと10点

0	3	4	1	5	2
---	---	---	---	---	---

14点ツアー

0	3	4	1	1	5	5	4	2	2	3	0
---	---	---	---	---	---	---	---	---	---	---	---

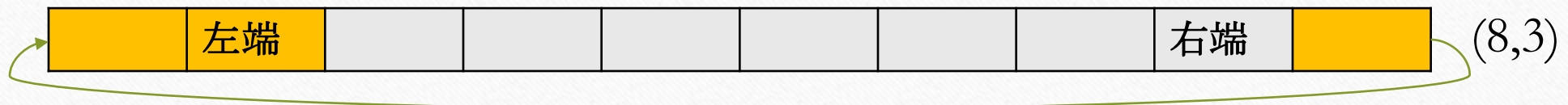
10点ツアー

小課題2 (22点)

- $L \leq 2^{35} - 1$ 1ビット節約で追加の8点を目指そう。
- 少し考察
- 区間の左の頂点は取りうる値が広いが、右は狭い
- 1番目の頂点 右端: $1 \sim N$ 長さ: $0 \sim (N - 2)$
- $N-1$ 番目の頂点 右端: $(N - 1)$ 長さ: 0
- 結構アンバランスで無駄がありそう

小課題2 (22点)

- 区間を円環(mod N)と見て、(区間の一端、端同士の差)を持つ(ただし差が短い方をもつ)



小課題2 (22点)

- 端同士の差は少なくとも一方は $2^{17} - 1$ 以下なので1bit節約
($2^{17} + 2^{17} = 2^{18} > 250000$ なので)
- おまけの8点ゲット
- しかし、この発想ではこれ以上の改善はできなさそう
- 他の方法もあるかも？

ここまでの得点分布

- 8点 5人
- 18点 0人
- 22点 4人
- 30点 3人
- 30点超 ???人

さらなる高みへ

- $L \leq 2^{28} - 1$ (もしくは $2^{28} \leq L \leq 2^{34} - 1$)
- ここからが本番です
- まずは考察をしましょう

木の深さについて

- たとえば「木の深さは高々18以下」という制約がありました。
- これ何の意味があるんでしょう？
- 各頂点の祖先は高々18個（子孫はたくさんありうる）
- つまり祖先－子孫の関係は今回は高々 $18 \cdot N$ 通り($O(N)$)
- ほかにも何かあるかも・・・(なくても解けます)

Euler Tourの方針は改善できるのか？

- まず左端は全通り存在する。
- (部分木の区間の左端、右端)という持ち方は厳しそう。(葉は全部右端になりうるので)
- (部分木の区間の左端、長さ)という持ち方はどうだろう・・・？

Euler Tourの区間の長さについて

- Euler Tourの区間の長さは頂点の子孫の数である(とする)
- 祖先－子孫の関係は高々 $18 \cdot N$ 通り ($O(N)$) (再掲)
- → 各頂点の子孫の数の総和は $18 \cdot N$ 以下になる。
- → 各頂点の子孫の数は平均すると18以下になる。

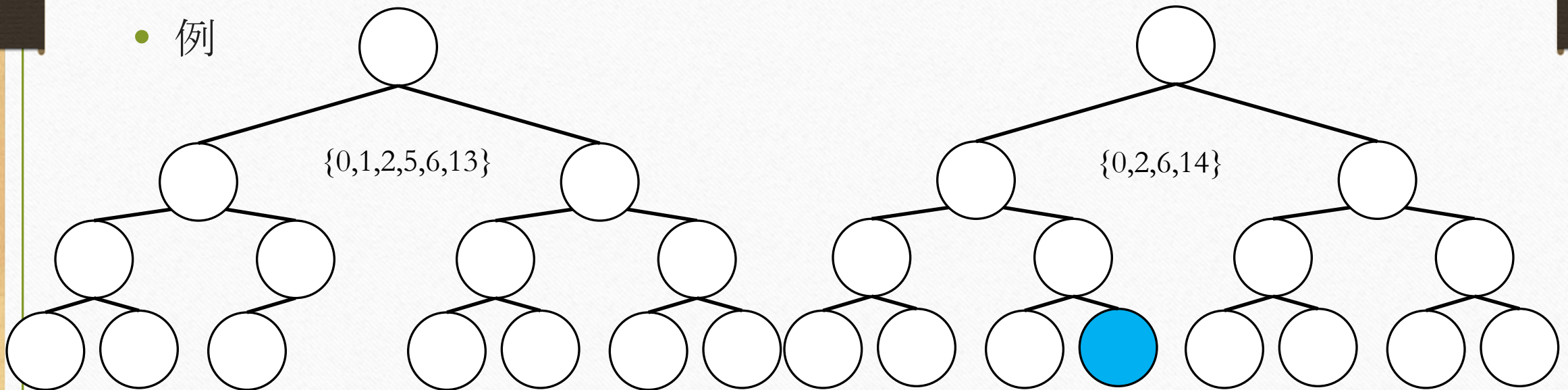
Euler Tourの区間の長さについて

- 区間の長さはたいてい短い
- なのに18ビットも使うのはもったいなくない？
- でもどんな長さがあるかはわからないからなあ
- 区間の長さを調整できればなあ

ダミーの頂点

- ダミーの頂点を挿入する(区間に空白を挿入する)ことで、区間の長さを変えることができる(取りうる値を制限することができる。)

- 例



ちょっとした注意

- (ソースコードでは区間の長さを変えるだけなので安心して)
- 区間の左端の番号の最大値が N より大きくなることに注意 (ダミーの頂点があるので)

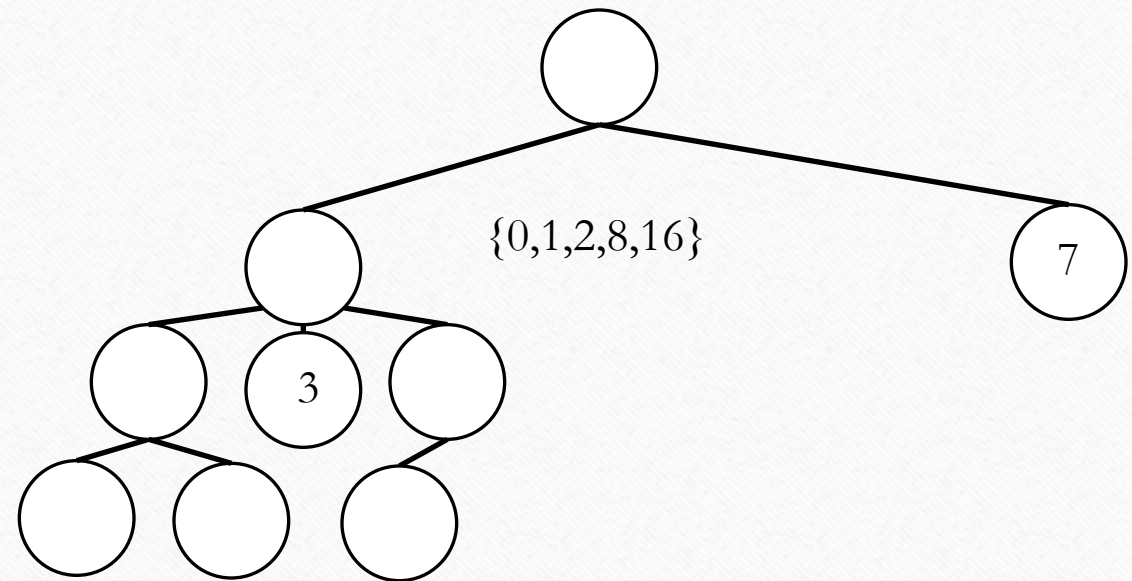
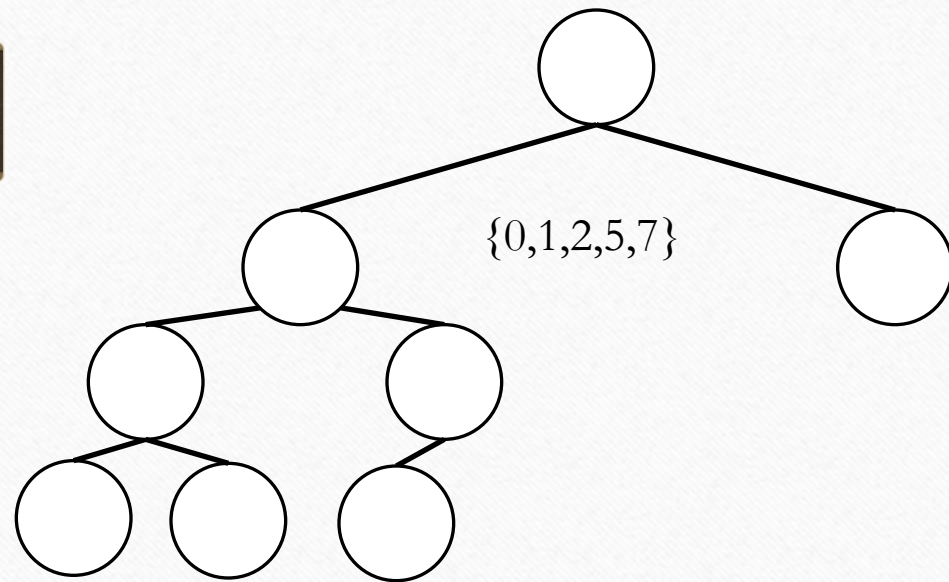
区間の長さを制限する

- 区間の長さとして取りうる値の数列 A_n を考える
- 区間の長さが取る値は、小さいほど頻繁に現れる
- 値が小さいときほど密、大きいときほど疎な数列とかが良さそう
- 等比数列とかがいいのでは？ (0も必要ですが)
- $A_n = 0, r^0, r^1, r^2, r^3, r^4, \dots$ (数列を整数に丸めれば r は実数でもよい)
- 他の数列でもよいかもしれませんが、ここでは等比数列を考えます

区間の長さを制限する

- 試しに区間の長さを取りうる値を $r = 2$ に制限してみる。(1,2,4,8,...)
- L はどのくらい大きくなるか？

ダミーの頂点の数は？



左端の番号の最大値は？

- 調整前の区間の長さ s は $(A_n < s \leq A_{n+1} = A_n r)$ である。
- → 1回の調整で、もとの長さのたかだか r 倍にしかない
- → 部分木の深さが d である頂点の区間の長さは、たかだか実際の長さの r^d 倍になる

左端の番号の最大値は？

- 左端の番号は最大 Nr^d ぐらいになる
- 区間の長さは $\log_r(Nr^d)$ 通りぐらい
- 全体でLはだいたい $Nr^d \log_r Nr^d = Nr^d (\log_r N + d)$
- $r=2$ では区間の左端が取りうる値が大きくなりすぎてかなしい

満点へ

- 実験したりプログラム書いて計算すると $r=1.05$ ぐらいで満点がとれることがわかります。(rの調整に失敗するとlogで計算される部分点になります)
- ここで注意、何も考えないと数列が $1, 1, 1, 1, \dots$ となってしまう(通るかも)
- 数列を $a_1 = 1, a_{n+1} = \max\{a_n * r, a_{n+1} + 1\}$ とすると解決。満点へ。
- 一見 $a_{n+1} = a_n + 1$ であるときにダミーの頂点の数の考察が成り立たない気がするが、 a_{n+1} ならダミー頂点の子は1つもないので大丈夫

得点分布

- 8点 5人
- 18点 0人
- 22点 4人
- 30点 3人
- 30点超 0人

情報オリンピック

2 0 1 7 年 春 合 宿

四 日 目

ド ラ ゴ ン 2

東 京 大 学

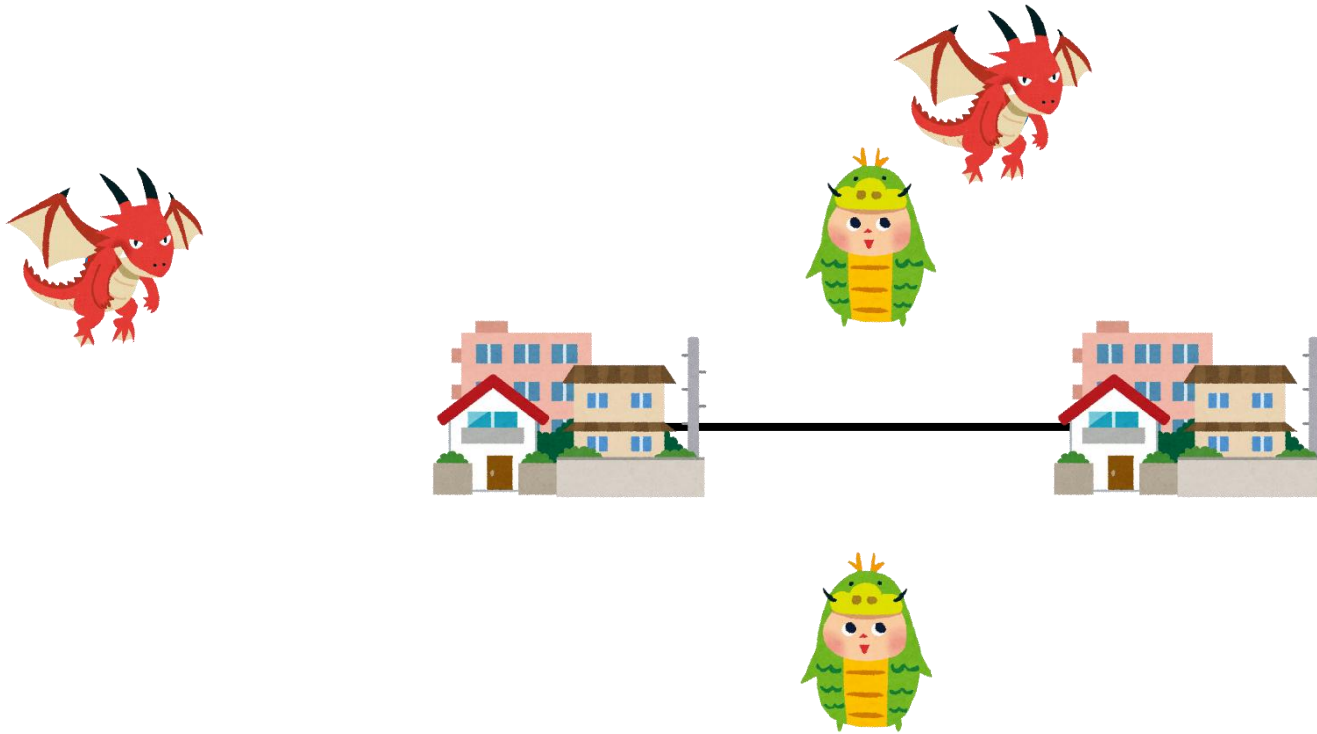
理 学 部 数 学 科 四 年

笠 浦 一 海

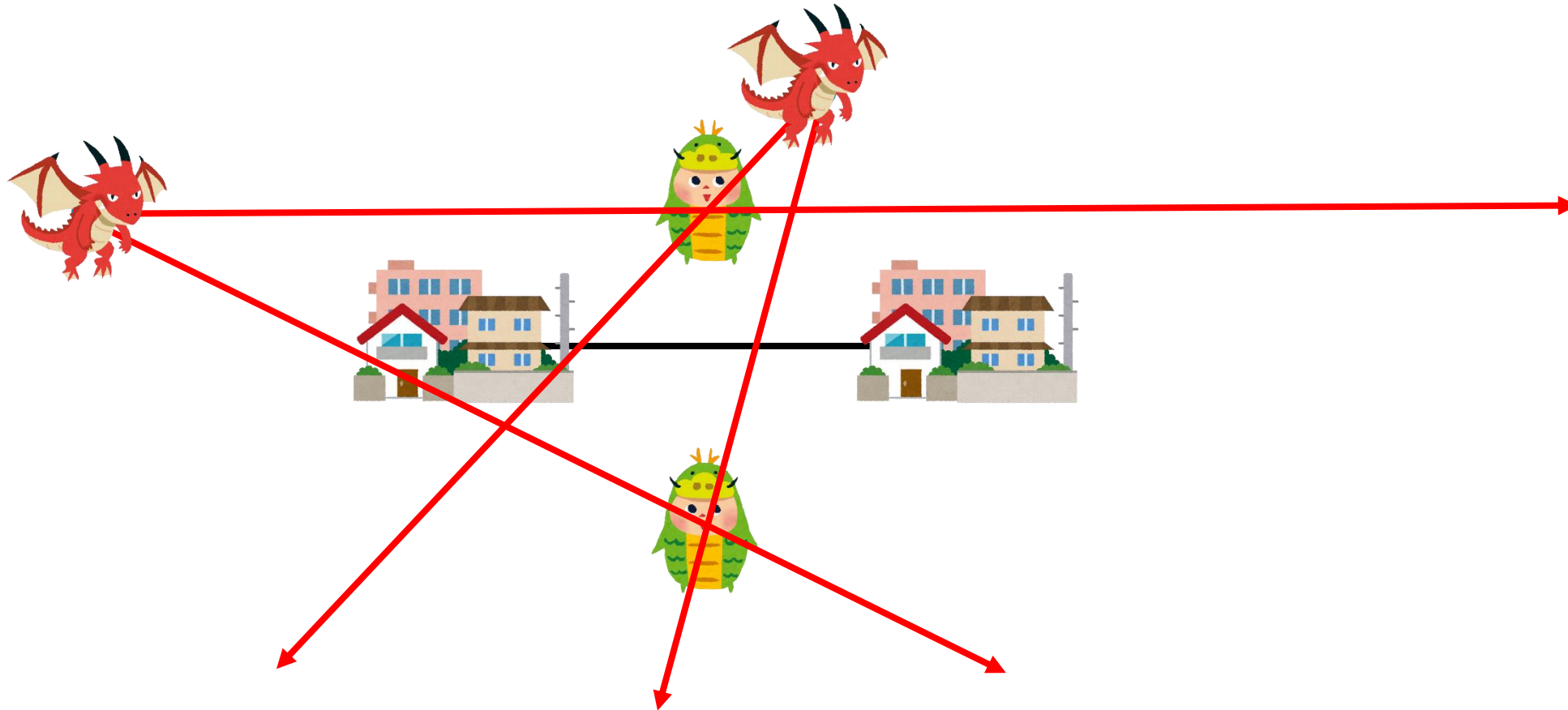
問題概要

- たくさんの点と一つの線分 XY が与えられる
- 点たちはいくつかの種類に分けられている
- 次のようなクエリに答えよ
- 「種類 a と種類 b が与えられるので、種類 a の点と種類 b の点の組 (A, B) で、半直線 AB が線分 XY と交わるようなものの個数を求めよ」

問題概要



問題概要



Subtask 1

Subtask 1

- $N \leq 3000$
- $O(N^2)$ で間に合う
- 点AとBについて、半直線ABが線分XYを交わるかどうかを $O(1)$ で判定できれば良い

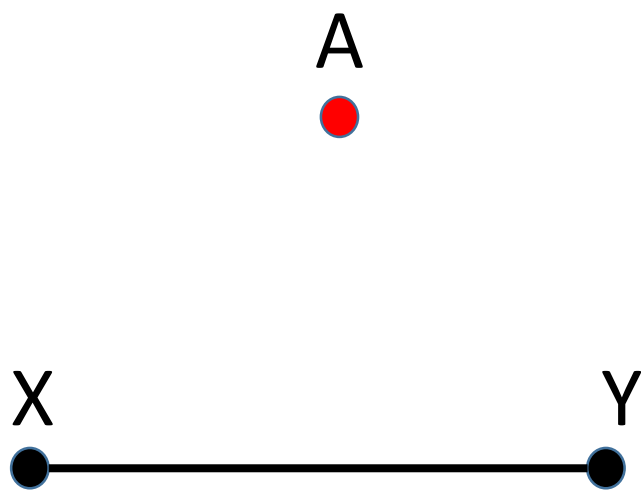
Subtask 1

- 方法1
- 直線AB, 直線XYの交点を計算して, 半直線AB, 線分XY上にあるか判定する
- もっとマシな方法はないか？

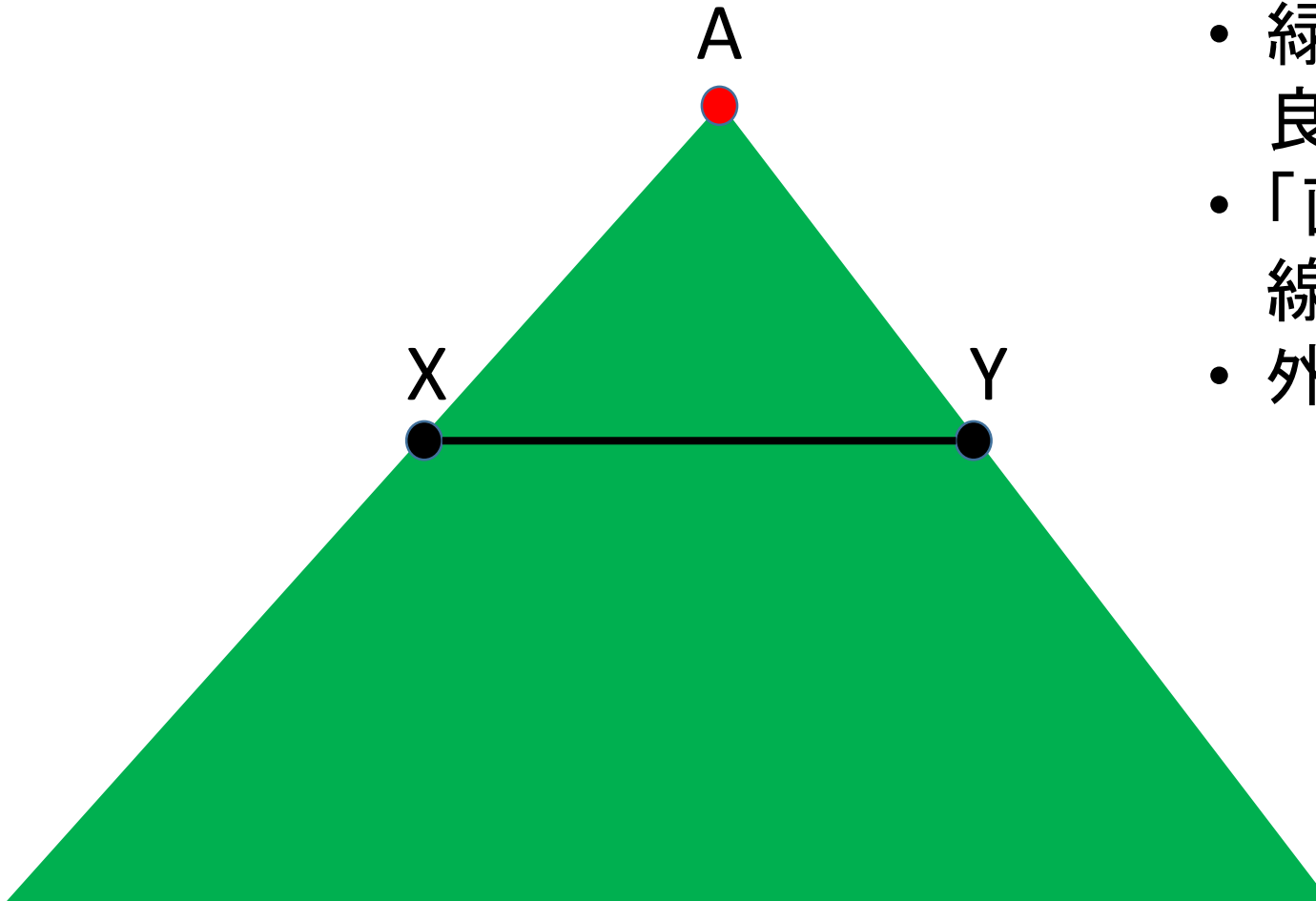
Subtask 1

- 方法2
- Aをfix
- Bがどの範囲にあるときに条件が満たされるか？

Subtask 1



Subtask 1



- 緑の三角形の領域にBが存在すれば良い
- 「直線XAについてYとBが同じ側 & 直線YAについてXとBが同じ側」ならよい
- 外積を使って計算できる

Subtask 1

- これでSubtask 1が解けた
- すべての(A, B)のペアについて条件を満たすか計算
- 時間 $O(N^2+Q)$

Subtask 1

- これでSubtask 1が解けた
- すべての(A, B)のペアについて条件を満たすか計算
- 時間 $O(N^2+Q)$
- 15点

Subtask 1

- 注意点

Subtask 1

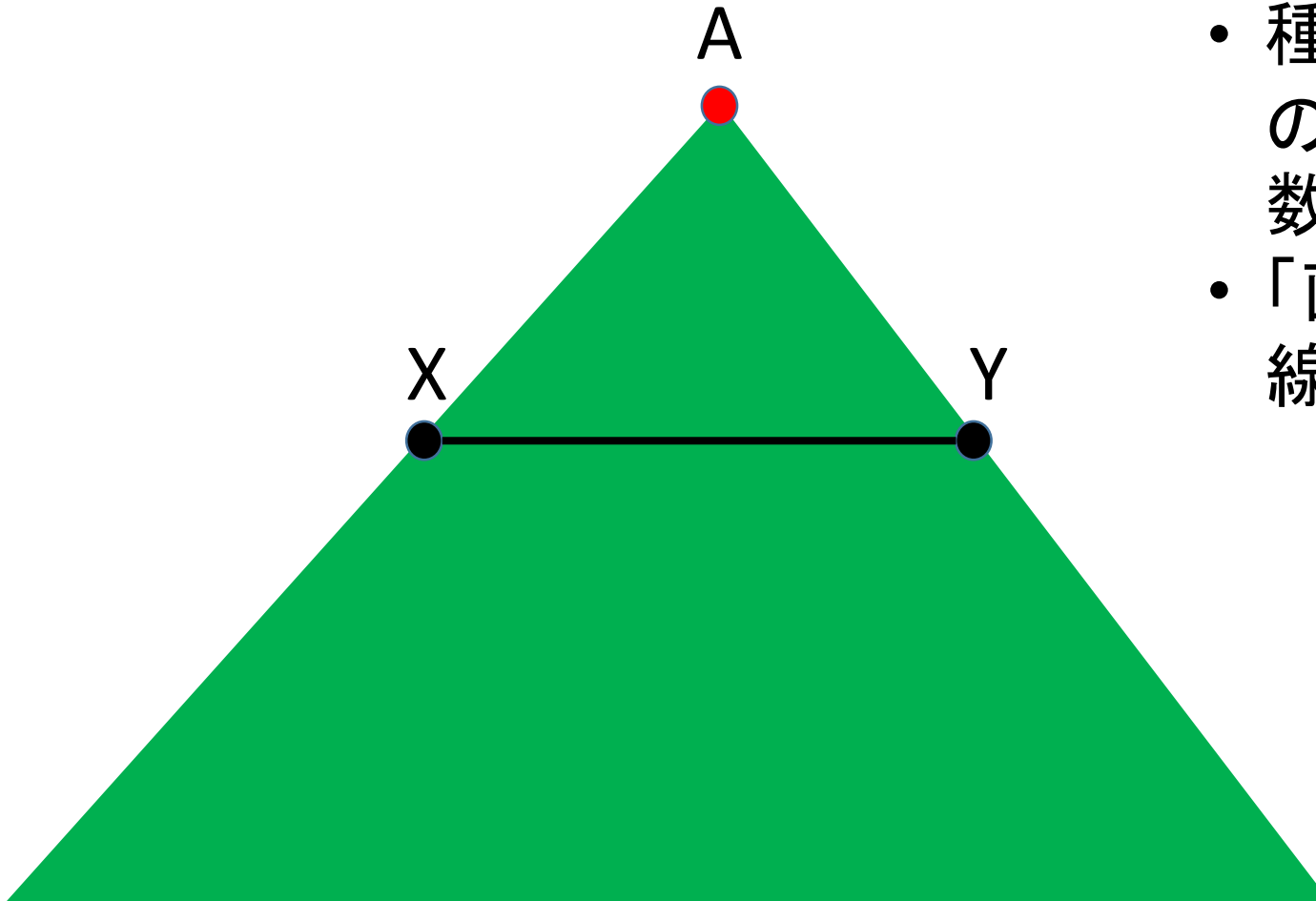
- 注意点
- 座標の値は 10^9 程度
- 外積の大きさは 10^{18} 程度
- →double だと精度が足りない
- →long long か long double を使えばOK

Subtask 2

Subtask 2

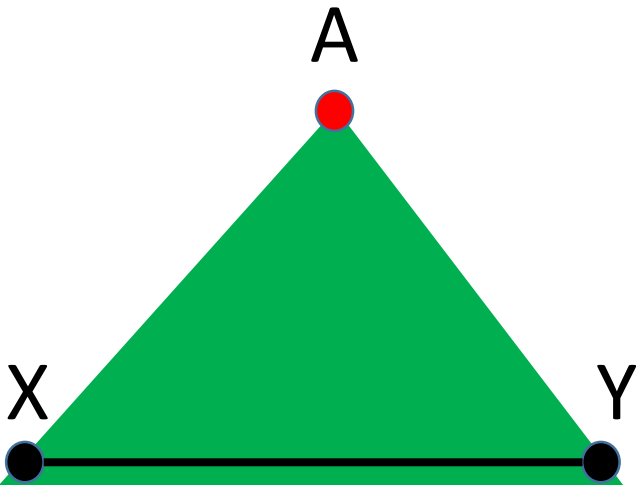
- $Q \leq 100$
- 各クエリについて $O(N)$ 程度で答えられれば良さそう
- 答えるクエリ (a, b) を fix して考える

Subtask 2



- 種類aの点Aのそれぞれについて、緑の三角形領域内の種類bの点の個数を調べれば良い
- 「直線XAについてYとBが同じ側 & 直線YAについてXとBが同じ側」

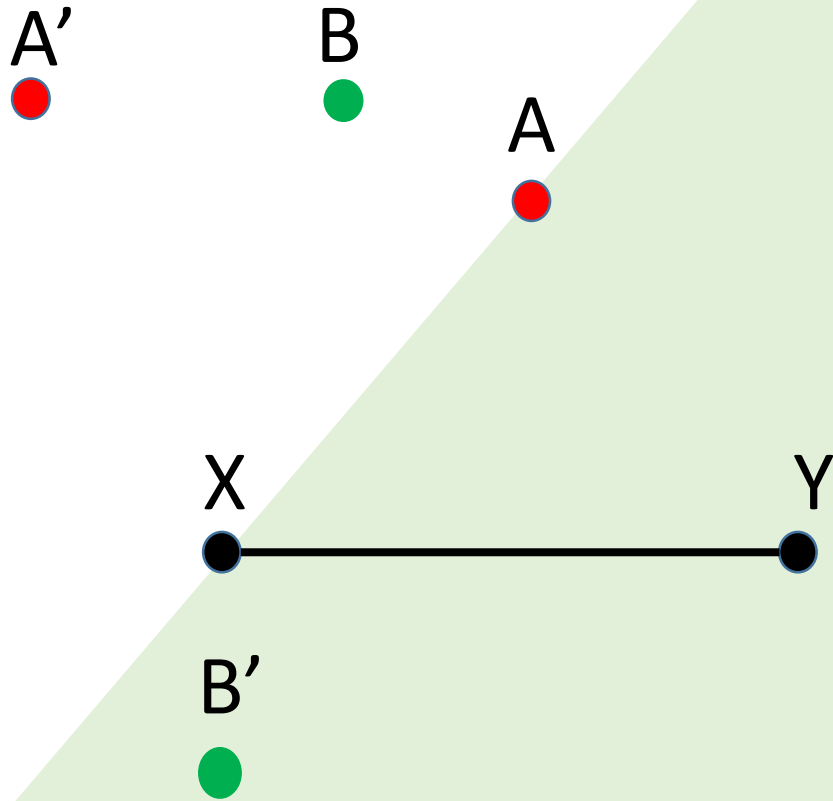
Subtask 2



フロー早い
けど微妙
にバグる

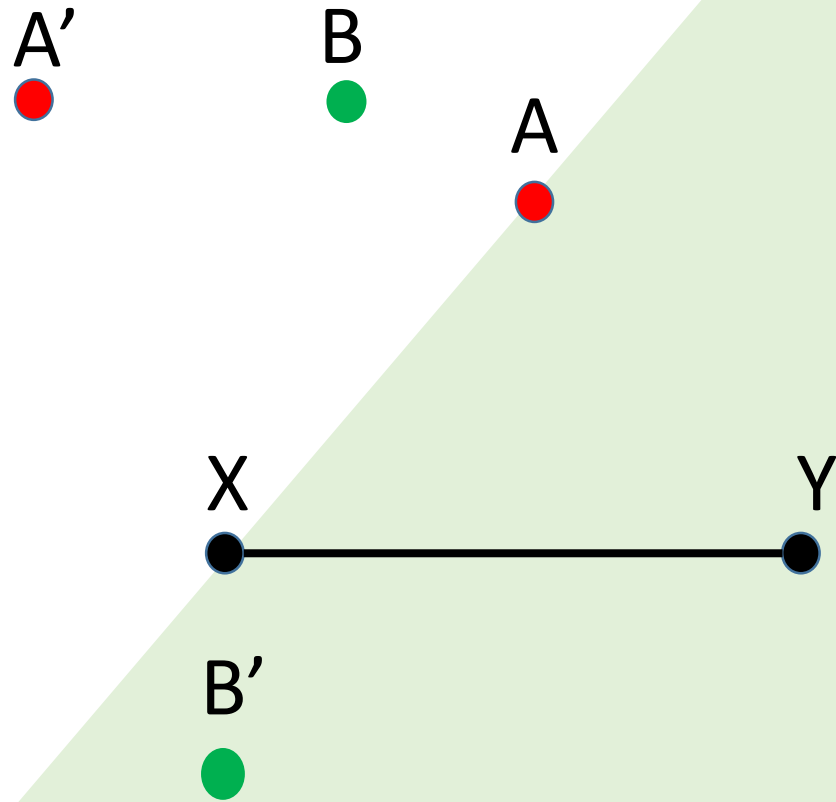
- はじめの条件はXからの直線の傾き、次の条件はYからの直線の傾きのみによる
- 条件はAが動くとき連続的に変化
- 平面走査

Subtask 2



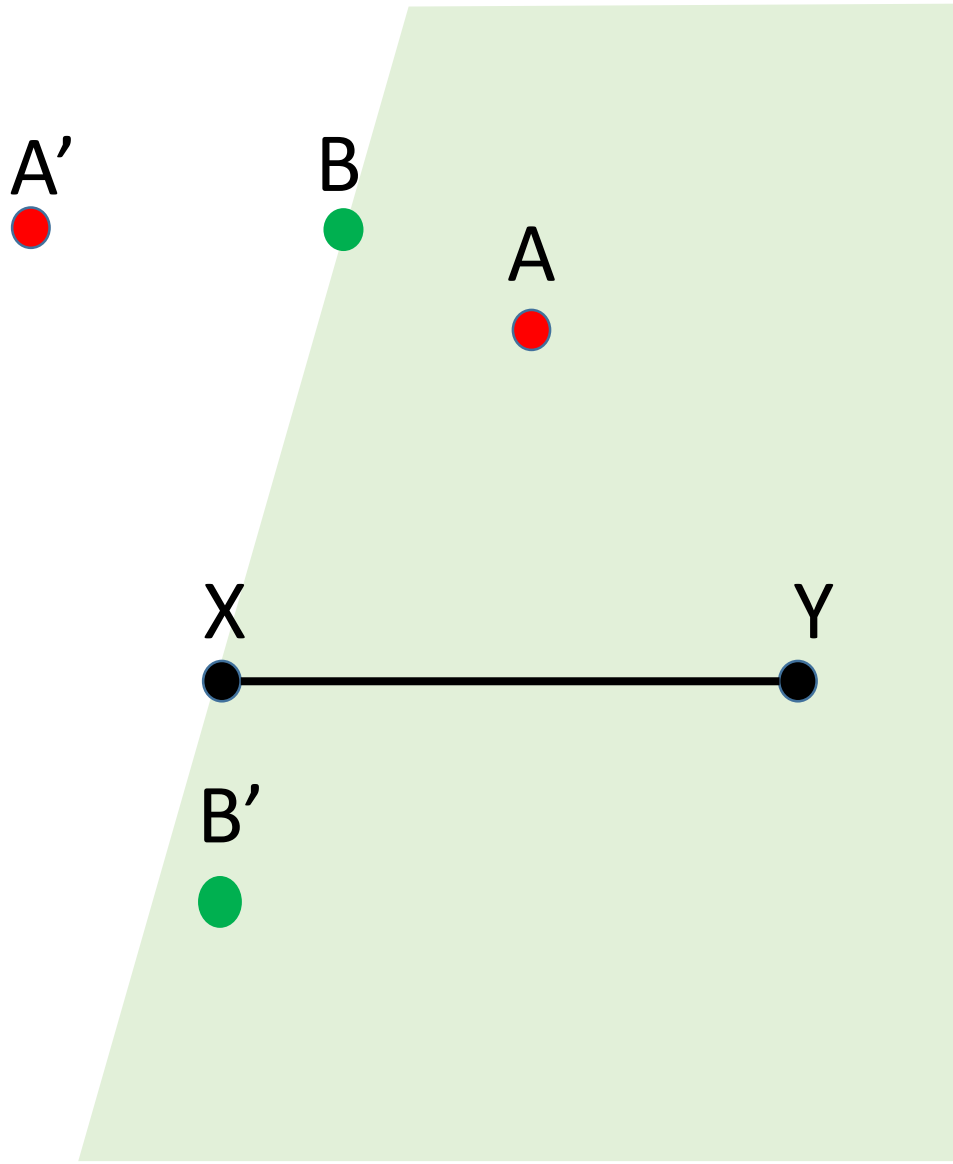
- 点を X からの方向でソート
- 直線 XA について Y と同じ側にある点の集合を管理しながら A を動かす

Subtask 2



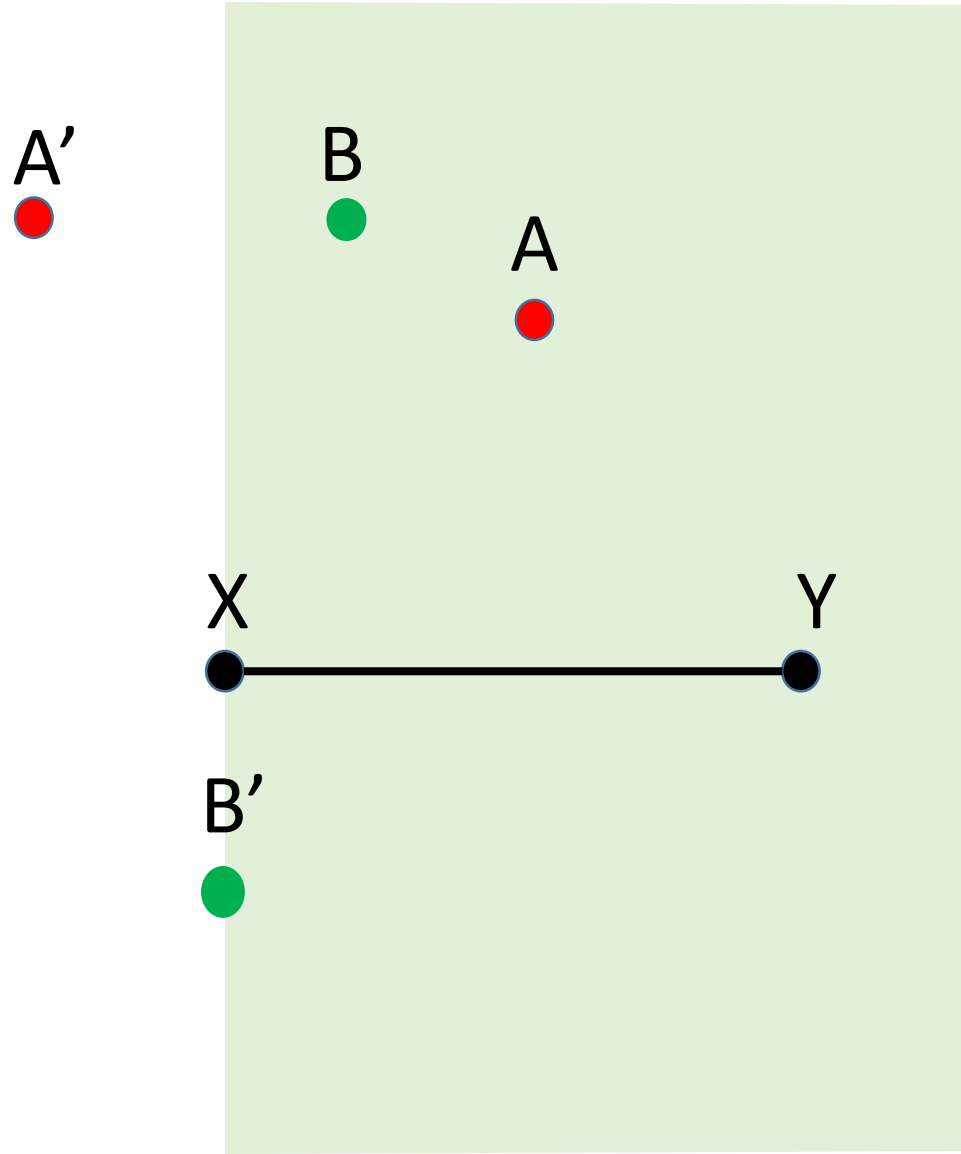
- 点をXからの方向でソート
- 直線XAについてYと同じ側にある点の集合を管理しながらAを動かす
- $\{B'\}$

Subtask 2



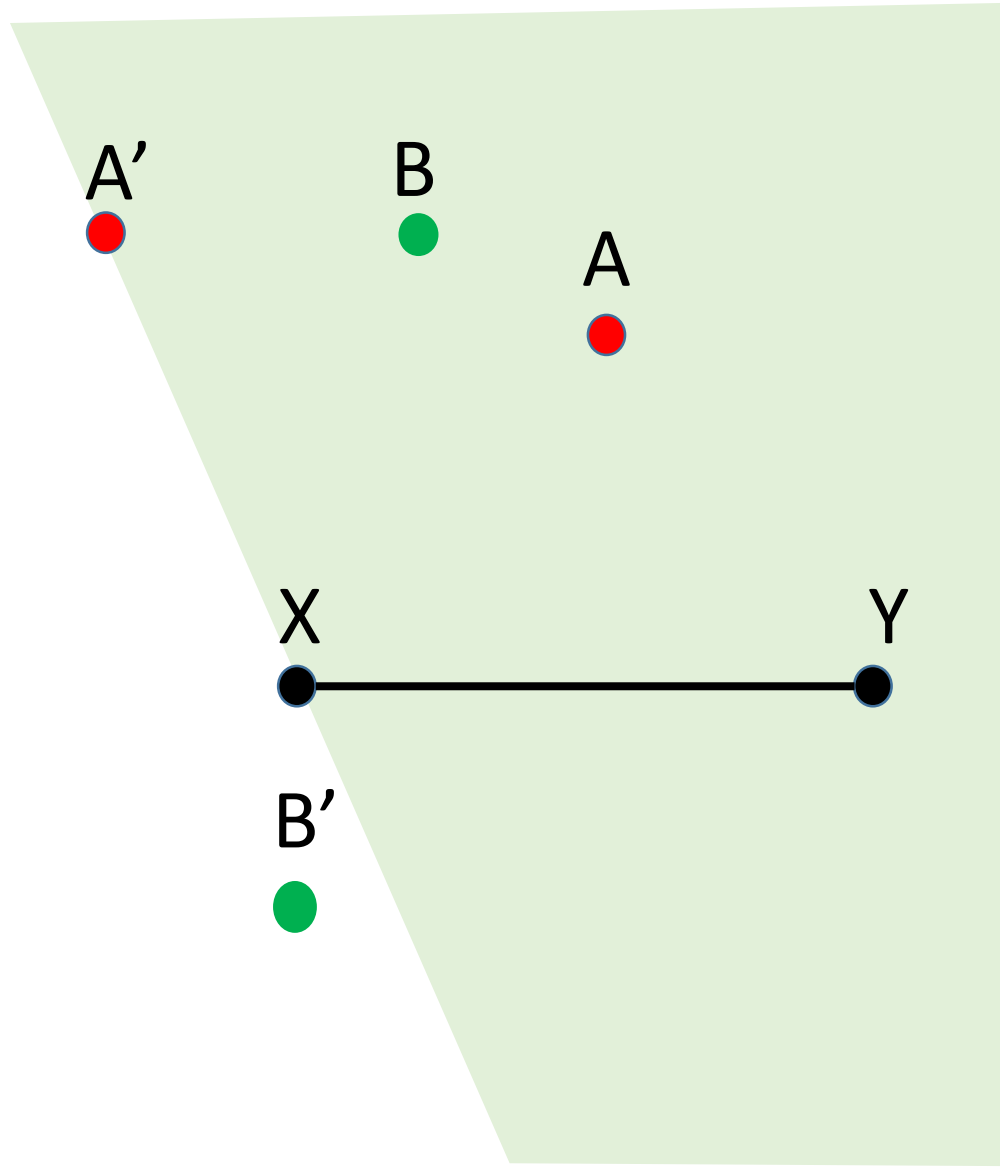
- 点をXからの方向でソート
- 直線XAについてYと同じ側にある点の集合を管理しながらAを動かす
- $\{B, B'\}$

Subtask 2



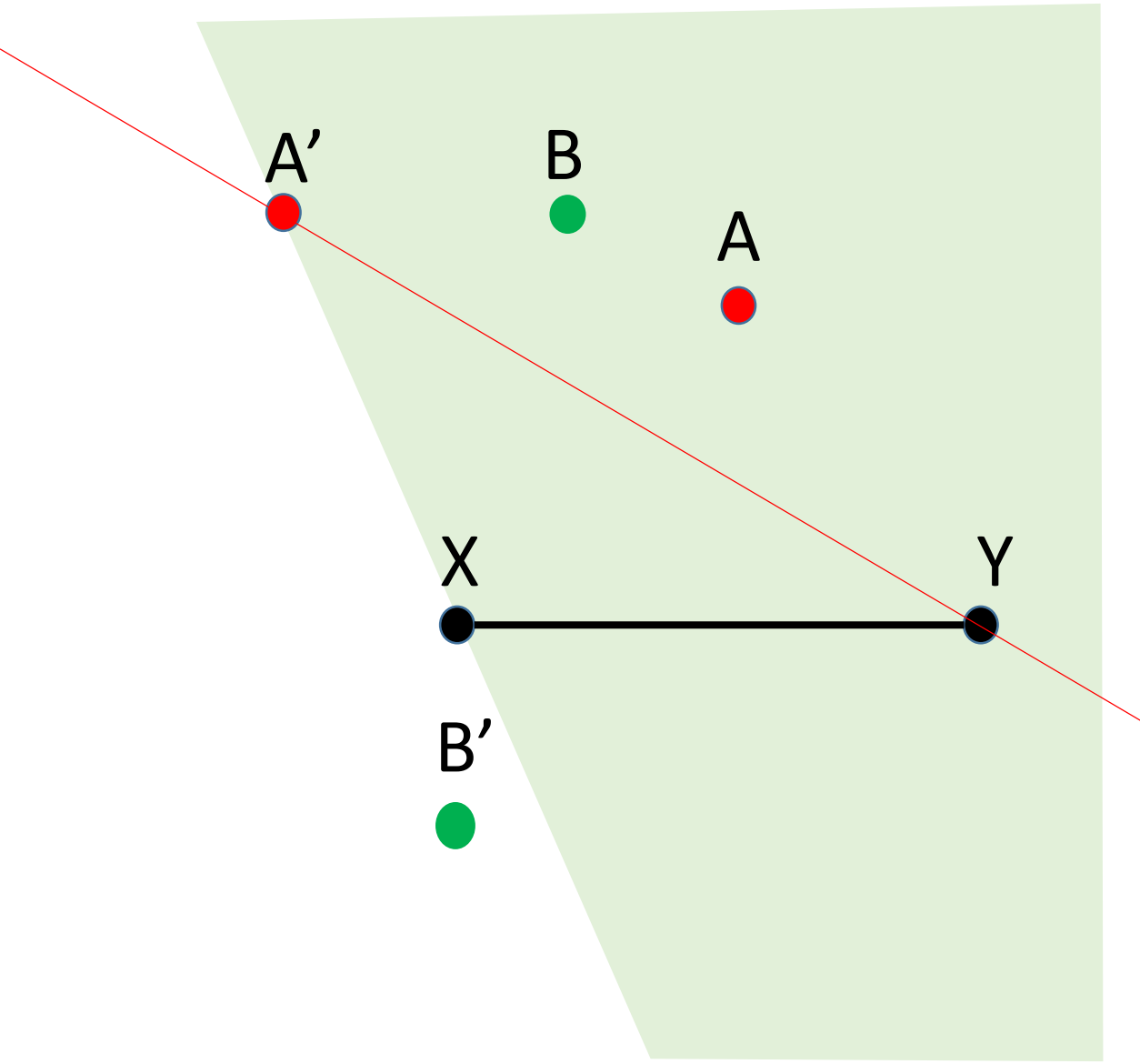
- 点をXからの方向でソート
- 直線XAについてYと同じ側にある点の集合を管理しながらAを動かす
- $\{B\}$

Subtask 2



- 点を X からの方向でソート
- 直線 XA について Y と同じ側にある点の集合を管理しながら A を動かす
- $\{B\}$

Subtask 2



- 点を X からの方向でソート
- 直線 XA について Y と同じ側にある点の集合を管理しながら A を動かす
- 種類 a の点に到達したら、集合の中で Y についての条件を満たすものを数える

Subtask 2

- 次のことができれば良い
- 集合に追加
- 集合から削除
- 集合のうちYからの方向がある範囲のものを数える

Subtask 2

- 次のことができれば良い
- 集合に追加
- 集合から削除
- 集合のうちYからの方向がある範囲のものを数える
- → 予め種類bの点をYからの方向でソートしておき、segment treeやBITで管理すれば良い

Subtask 2

- 一つのクエリに対して $O(N \log N)$ で解けた

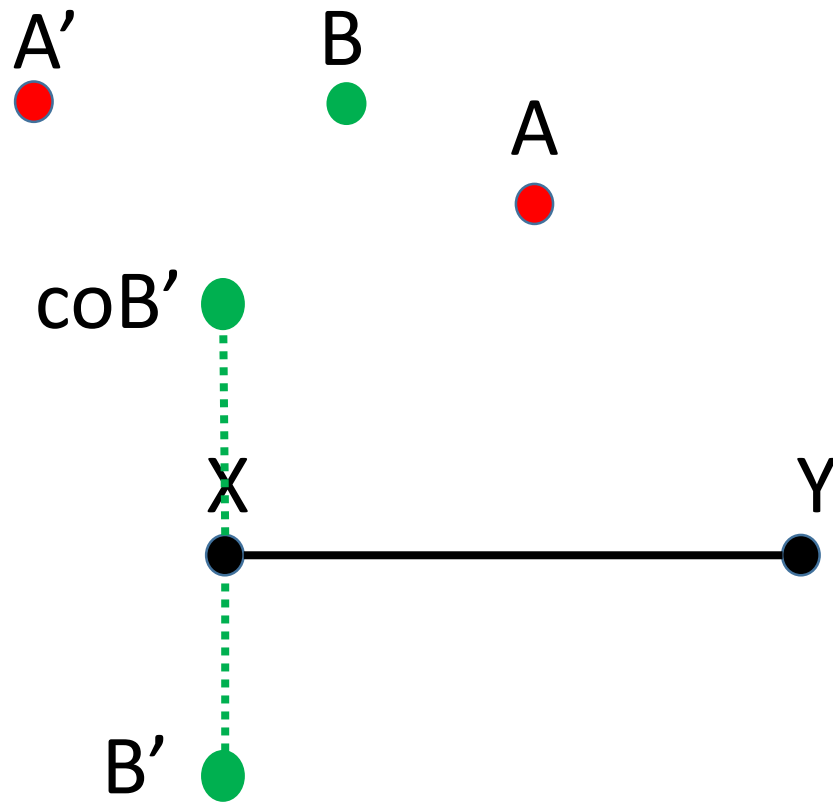
Subtask 2

- 一つのクエリに対して $O(N \log N)$ で解けた
- → 45点 or 60点
- (うまく実装すればまとめて Subtask 1 も解ける)

Subtask 2

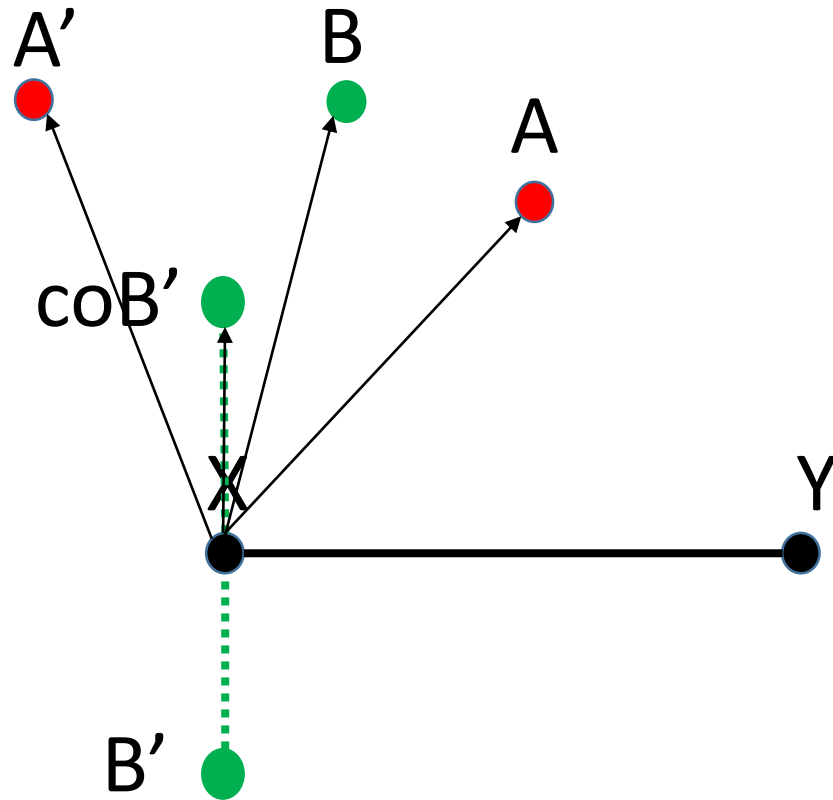
- 実装上の注意
- A, Bそれぞれについて直線XYのどちら側にあるかで場合分けするので面倒
- doubleは精度が足りない
- 誤差が怖いのでatan2で角度を計算しないほうがいい
- はじめに座標を変換して直線XYがX軸に一致するようにすると楽 (long doubleを使う場合)。傾きでソートできる

Subtask 2



- 模範的な実装
- はじめにすべての点を直線XYのどちらかの側に集める
- (XについてソートするときはXについての点対称、YについてソートするときはYについての点対称)

Subtask 2

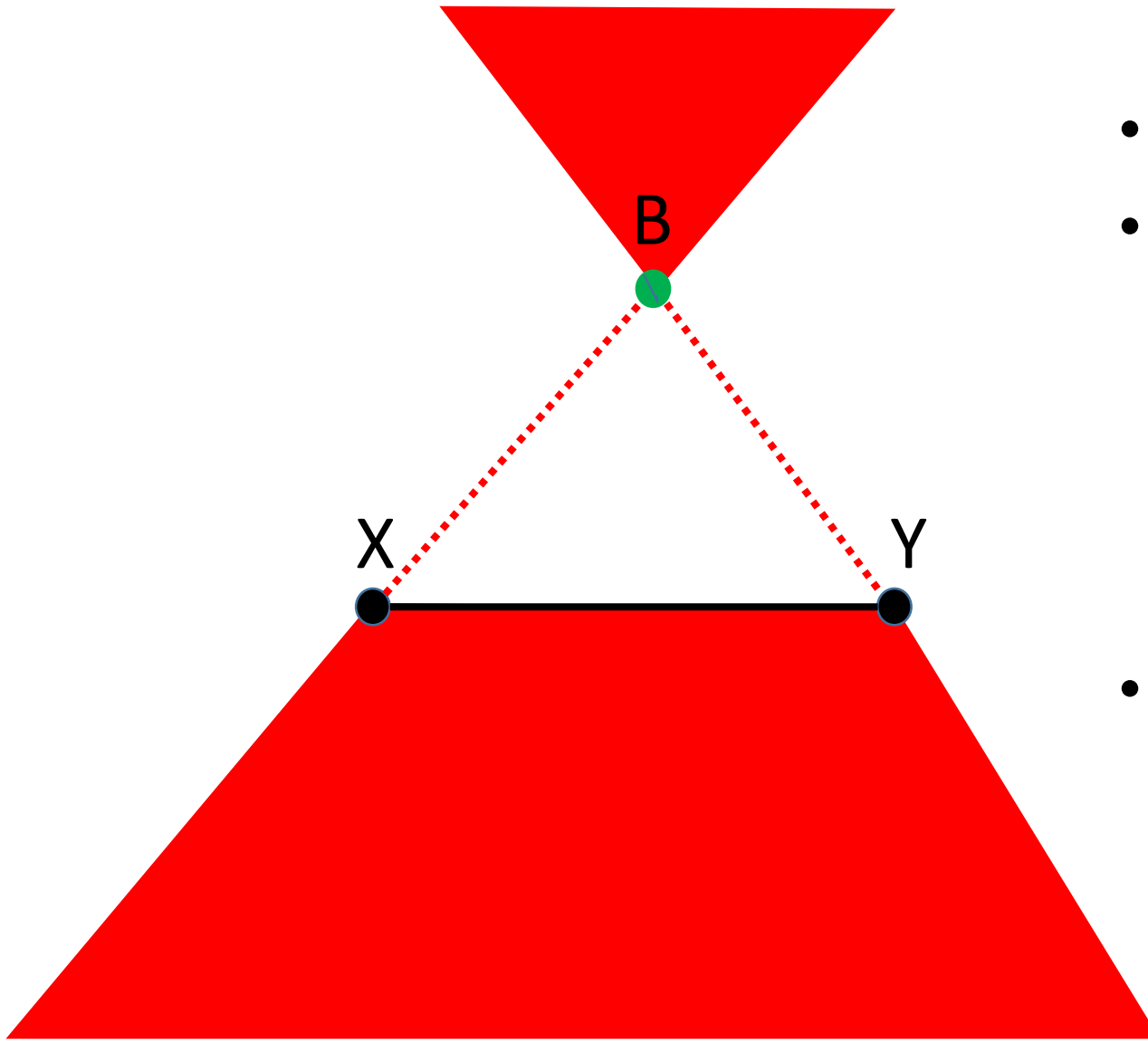


- 模範的な実装
- はじめにすべての点を直線 XY のどちらかの側に集める
- (X についてソートするときは X についての点対称、 Y についてソートするときは Y についての点対称)
- 外積により順序が定まる
- $P-X < Q-X \iff op(P-X, Q-X) > 0$

Subtask 2

- BとAの役割を交換しても解ける(伏線)
- Bをfixしたとき、Aがどの範囲にあれば条件を満たすか？

Subtask 2



- Aが赤の領域内にあればよい
- 「直線XYについてBと同じ側 & 直線XBについてYと異なる側 & 直線YBについてXと異なる側」または「直線XYについてBと異なる側 & 直線XBについてYと同じ側 & 直線YBについてXと同じ側」
- 同様に平面走査ができる

Subtask 3

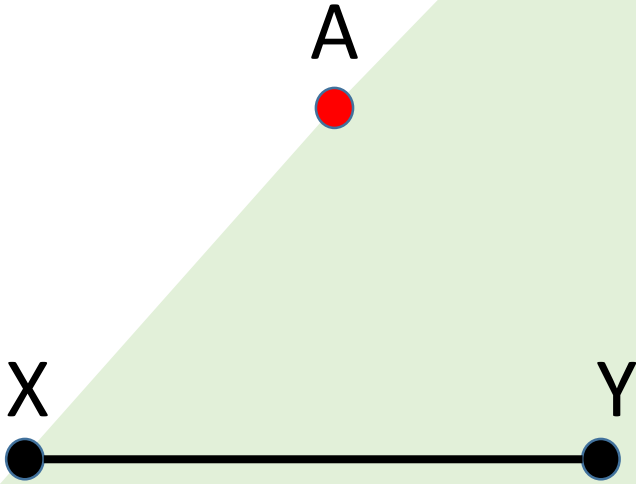
Subtask 3

- Qがでかい
- ひとつひとつのクエリに素早く答えるのは無理そう

Subtask 3

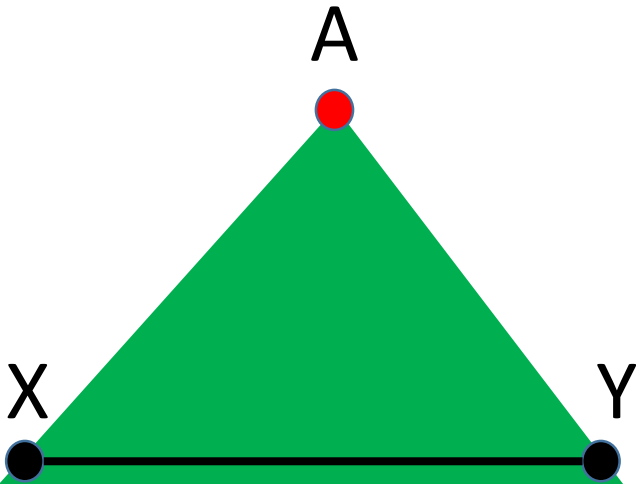
- Qがでかい
- ひとつひとつのクエリに素早く答えるのは無理そう
- クエリを先読みしてまとめて計算するのはできそう
- 予めクエリを全部読んで、適当に分類しておく
- Subtask 2の方法をまとめてやることを考える

Subtask 3



- 前述の平面走査をする
- ただし集合としてすべての種類の点を管理する

Subtask 3



フロー早い
けど微妙
にバグる

- 任意の種類の点Aについて、そこに達したら次のことを行う
- Aの種類をaとする
- (a, b) の形のクエリそれぞれについて、緑の三角形内の種類bの点の個数を調べる

Subtask 3

- 次のことができれば良い
- 集合に追加
- 集合から削除
- 集合のうちある種類でYからの方向がある範囲のものを数える

Subtask 3

- 次のことができれば良い
- 集合に追加
- 集合から削除
- 集合のうちある種類でYからの方向がある範囲のものを数える
- →予めすべての点を(種類、Yからの方向)でソートしておき、segment treeやBITで管理すれば良い

Subtask 3

- 各点について、
- 「(その点の種類、任意の種類)の形のクエリの数」 $\times O(\log N)$
- だけ時間がかかる
- 大体のケースで早い
- 点でもクエリでもたくさん登場する種類があるとやばい

Subtask 3

- 各点について、
- 「(その点の種類、任意の種類)の形のクエリの数」 $\times O(\log N)$
- だけ時間がかかる
- 大体のケースで早い
- 点でもクエリでもたくさん登場する種類があるとやばい
- →そういう種類だけ例外的に処理すればよさそう

Subtask 3

- 種類 a で、 (a,b) というクエリが vQ 個以上存在するものを重い種類、そうでない種類を軽い種類ということにする
- クエリ (a, b) で、種類 a が重いものを重いクエリ、そうでないものを軽いクエリということにする
- 軽いクエリについては前述の方法で計算する
- これは $O(NvQ \log N)$ でできる

Subtask 3

- 重い種類は高々 vQ 個しかない
- 種類 b について、 (a, b) の形の重いクエリは高々 vQ 個しかない
- 重いクエリについては、 B と A の役割を入れ替えて同様に計算する
- これも $O(NvQ \log N)$
- よって全体で $O(NvQ \log N)$

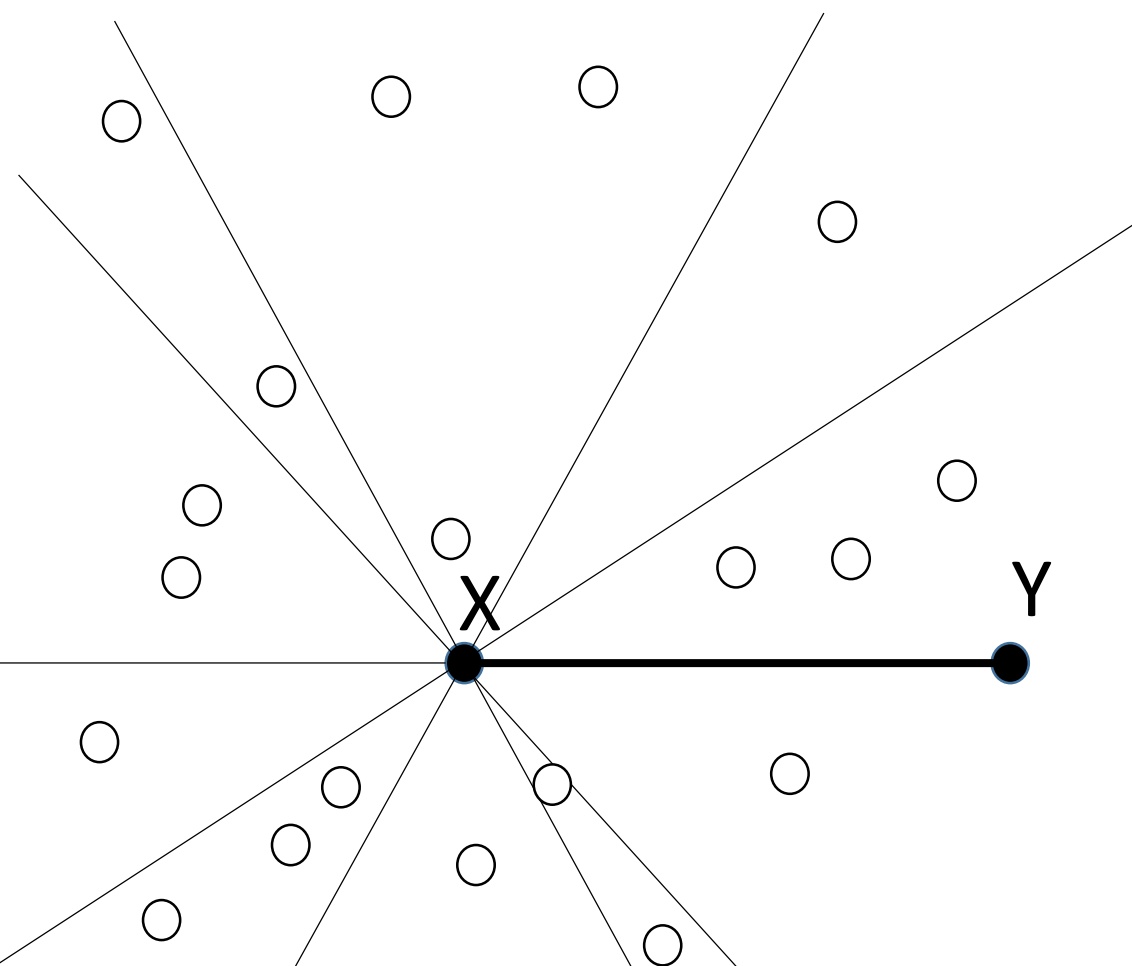
Subtask 3

- 重い種類は高々 vQ 個しかない
- 種類 b について、 (a, b) の形の重いクエリは高々 vQ 個しかない
- 重いクエリについては、 B と A の役割を入れ替えて同様に計算する
- これも $O(NvQ \log N)$
- よって全体で $O(NvQ \log N)$
- →満点！



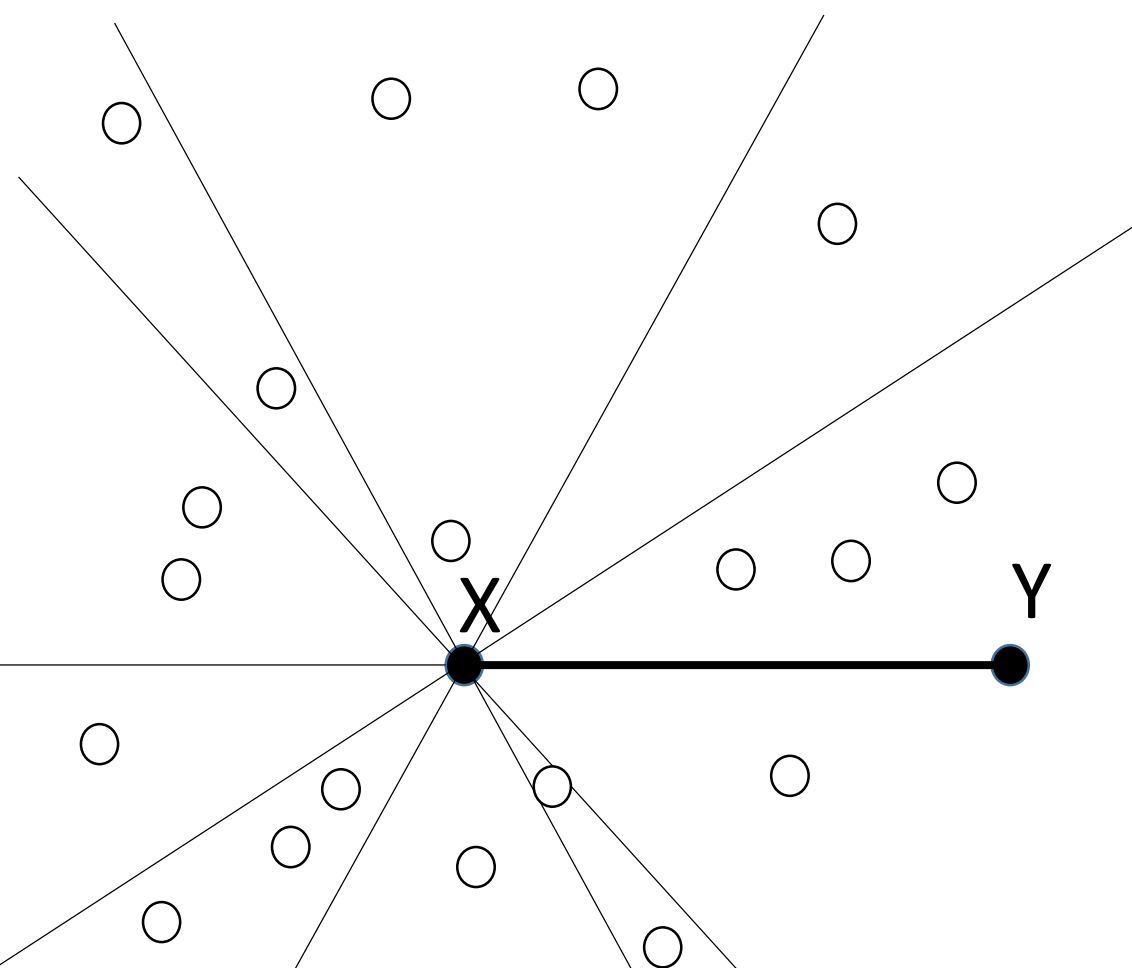
より良い解

より良い解



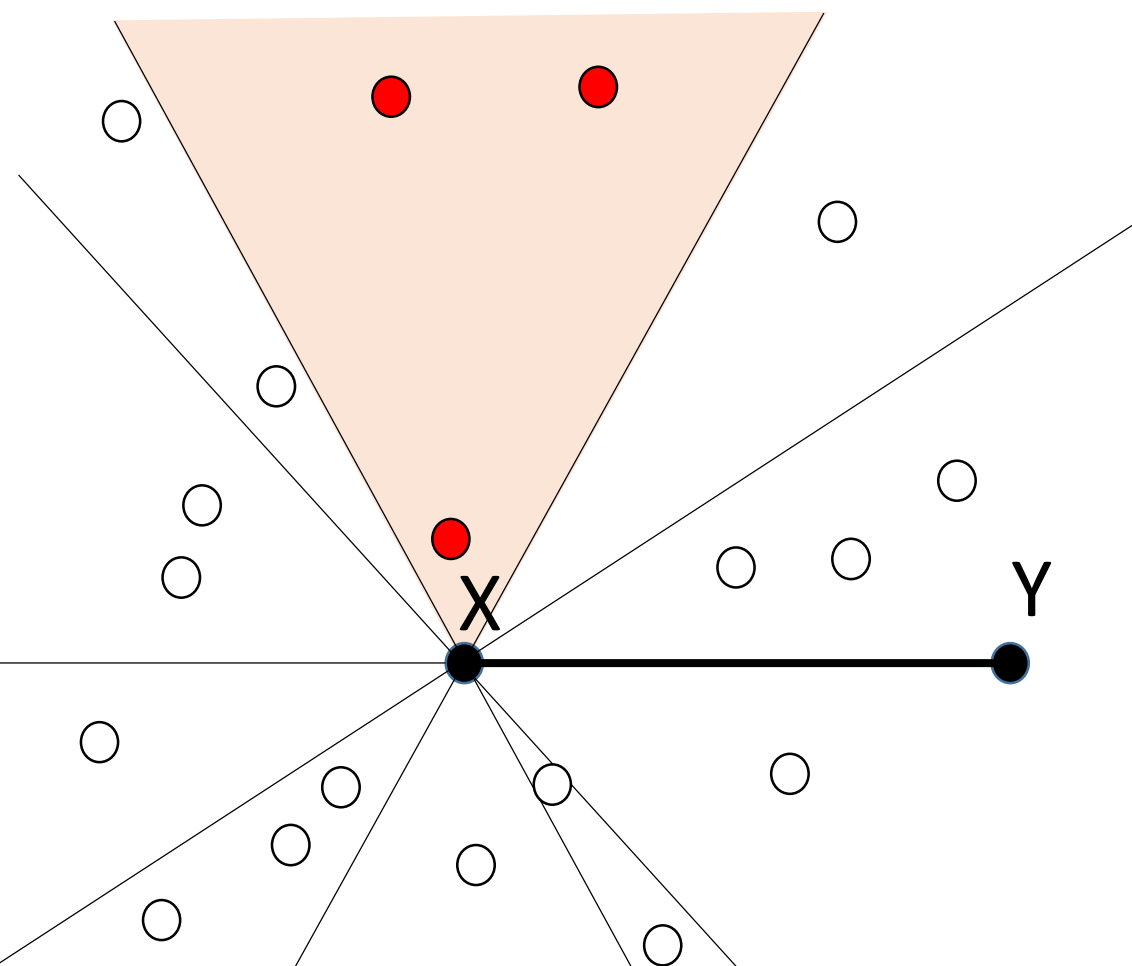
- 実は $O(N\sqrt{N}+N\sqrt{Q})$ で解ける
- 右図のようにXからの方向でソートして $\sqrt{N}$ 個ごとに区切る
- (点対称なふたつの領域は一つの区分とみなすことに注意)
- 図は $N=19$

より良い解



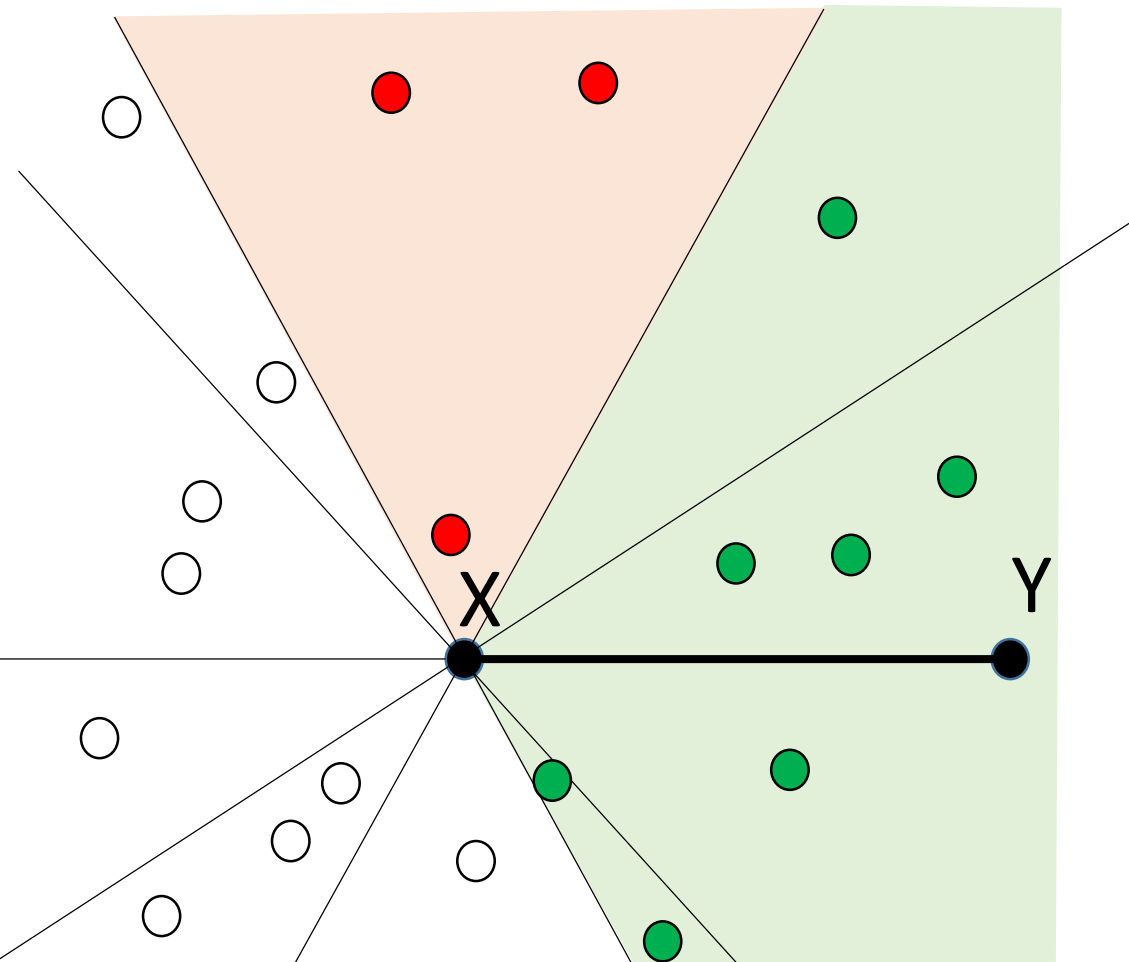
- 二点A, Bが同じ区分に属している場合と違う区分に属している場合に分ける
- 同じ場合
- 考えられる点の組が $O(N^2)$ しかない
- →全部試す

より良い解



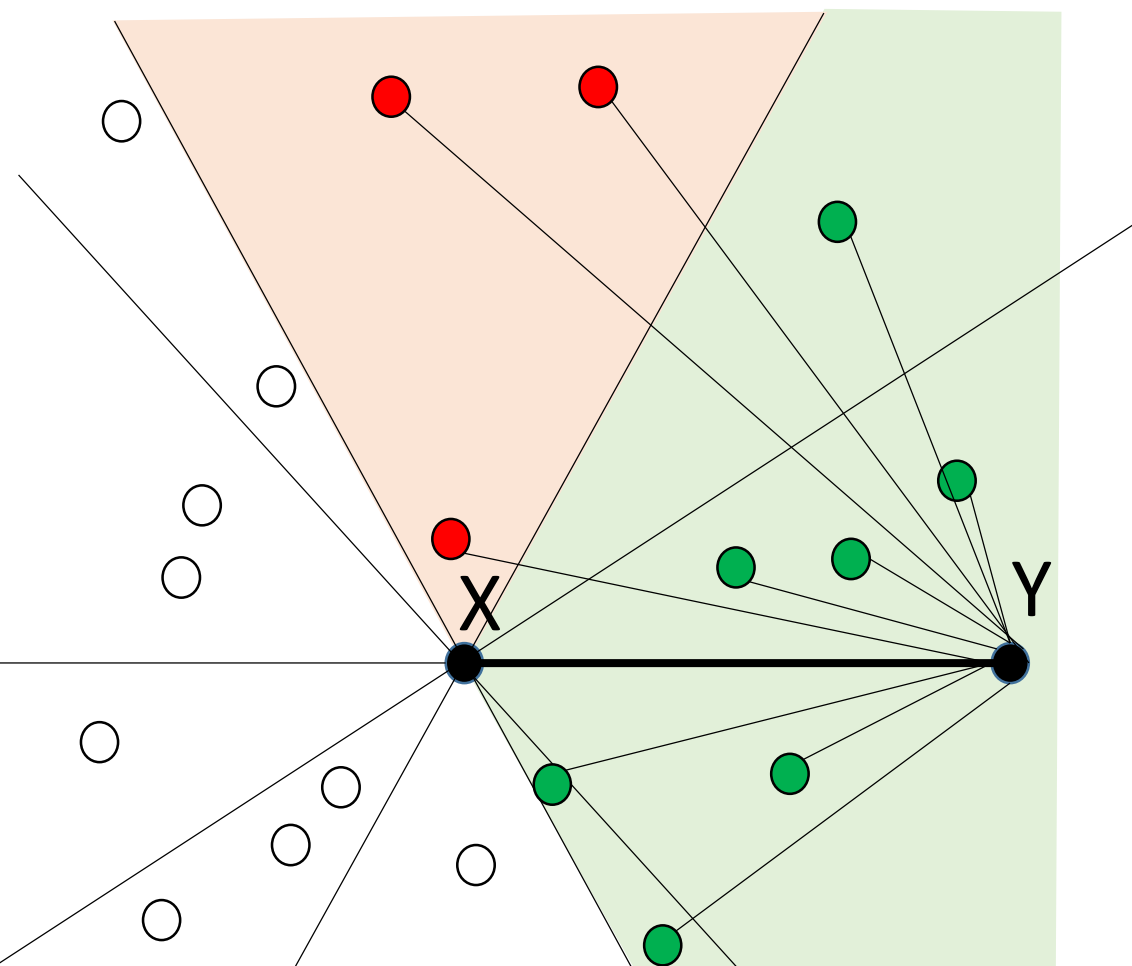
- 違う場合。
- 例えば右図の赤の領域にAがある場合を考える

より良い解



- 違う場合。
- 例えば右図の赤の領域にAがある場合を考える
- Bの候補として考えられるのは緑の領域の点
- これらの点ではXについての条件は自動的に満たされている

より良い解



- これらの点をYからの方向順に見ていけば良い
- 緑の点ではどの種類がいくつ出てきたかをカウントしておく
- 赤の点ではクエリを処理する
- 領域内の点についてまとめて計算できる

より良い解

- 一つのクエリに対し $Q(N \vee N)$ で計算できる

より良い解

- 一つのクエリに対し $O(N^2)$ で計算できる
- クエリをまとめて処理すれば $O(N^2 + NQ)$

より良い解

- 一つのクエリに対し $O(N\sqrt{N})$ で計算できる
- クエリをまとめて処理すれば $O(N\sqrt{N} + NQ)$
- →60点

より良い解

- 一つのクエリに対し $O(N\sqrt{N})$ で計算できる
- クエリをまとめて処理すれば $O(N\sqrt{N} + NQ)$
- $\rightarrow 60$ 点
- 前述の平方分割を行えば $O(N\sqrt{N} + N\sqrt{Q})$

より良い解

- 一つのクエリに対し $O(N\sqrt{N})$ で計算できる
- クエリをまとめて処理すれば $O(N\sqrt{N} + NQ)$
- →60点
- 前述の平方分割を行えば $O(N\sqrt{N} + N\sqrt{Q})$
- →満点



嘘解法

嘘解法

- Subtask 1の計算量を思い出してみる
- 時間計算量 $O(N^2)$
- $N \leq 30000$

嘘解法

- Subtask 1の計算量を思い出してみる
- 時間計算量 $O(N^2)$
- $N \leq 30000$
- →定数が軽ければ通る可能性が微粒子レベルで存在している.....?

嘘解法

- $M \times M$ 配列を用意するとMLE
- 点を種類ごとに分ける
- クエリ(a, b)に対し、種類aの点と種類bの点の組に対しすべて試す
- →同じクエリが来ないことから計算量 $O(N^2+Q)$ になる

嘘解法

- $M \times M$ 配列を用意するとMLE
- 点を種類ごとに分ける
- クエリ(a, b)に対し、種類aの点と種類bの点の組に対しすべて試す
- →同じクエリが来ないことから計算量 $O(N^2+Q)$ になる
- MLEが回避できた

嘘解法

- 一度の判定をできるだけ早く行いたい
- 前述のように外積を使うと最低でもlong longの掛け算が4回必要
- →厳しい？

嘘解法

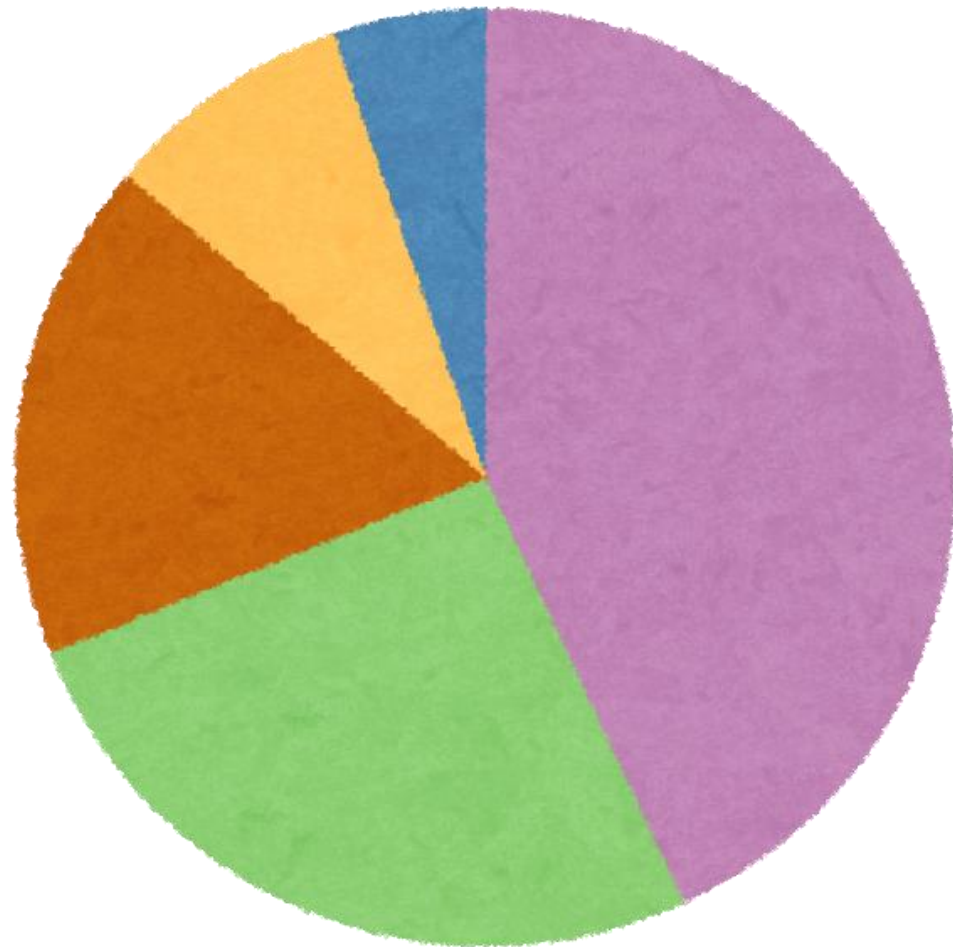
- 一度の判定をできるだけ早く行いたい
- 前述のように外積を使うと最低でもlong longの掛け算が4回必要
- →厳しい？
- 予め全点をXからの方向とYからの方向でソートしておく
- →数の比較だけで判定ができる

嘘解法

- 一度の判定をできるだけ早く行いたい
- 前述のように外積を使うと最低でもlong longの掛け算が4回必要
- → 厳しい？
- 予め全点をXからの方向とYからの方向でソートしておく
- → 数の比較だけで判定ができる
- → 満点？



得点分布



得点分布

